



Kubernetes Administration with RKE2

Workbook

Course ID: RKE201v2

Date: 2026-06-17

Rev: C

Proprietary Statement

Copyright © SUSE LLC. All rights reserved.

SUSE LLC, has intellectual property rights relating to technology embodied in the product that is described in this document.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of the publisher.

(C) SUSE LLC. All Rights Reserved. SUSE and the SUSE logo are registered trademarks of SUSE LLC in the United States and other countries. All third-party trademarks are the property of their respective owners.

Scope of Use

This SUSE Training Manual is intended for use by:

- Customers who have purchased an eLearning subscription
- Customers who attended training at a SUSE Authorized Training Partner
- SUSE Certified Instructors teaching courses as part of the SUSE Training Partner program
- SUSE Partners who qualify for access to SUSE Technical Training eLearning courses
- SUSE Employees as part of eLearning or self-study

Disclaimer

SUSE LLC, makes no representations or warranties with respect to the contents or use of this documentation, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose.

Further, SUSE LLC, reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes. Further, SUSE LLC, makes no representations or warranties with respect to any software, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, SUSE LLC, reserves the right to make changes to any and all parts of SUSE software, at any time, without any obligation to notify any person or entity of such changes.

Any products or technical information provided under this Agreement may be subject to U.S. export controls and the trade laws of other countries. You agree to comply with all export control regulations and to obtain any required licenses or classification to export, re-export or import deliverables. You agree not to export or re-export to entities on the current U.S. export exclusion lists or to any embargoed or terrorist countries as specified in the U.S. export laws. You agree to not use deliverables for prohibited nuclear, missile, or chemical biological weaponry end uses. SUSE assumes no responsibility for your failure to obtain any necessary export approvals.

This SUSE Training Manual is published solely to instruct students in the use of SUSE software. Although third-party application software packages may be used in SUSE training courses, this is for demonstration purposes only and shall not constitute an endorsement of any of these software applications.

Further, SUSE LLC does not represent itself as having any particular expertise in these application software packages and any use by students of the same shall be done at the student's own risk.

Table of Contents

Table of Contents.....	3
Document Conventions.....	6
Link to Documentation.....	6
Section 1: Course Introduction.....	7
Course Lab Environment Overview.....	8
Exercise 1.1 - Start Lab Environment VMs.....	9
Section 2: Introduction to containers and container orchestration.....	13
Exercise 2.1 - Run containers and manage container images.....	14
Section 3: Kubernetes Distributions by SUSE.....	21
Exercise 3.1 - Install RKE.....	22
Section 4: Basic Kubernetes Objects and Commands.....	31
Exercise 4.1 - Configure kubectl autocompletion.....	32
(End of Exercise).....	35
Exercise 4.2 - Display information about the RKE2 cluster using kubectl.....	36
(End of Exercise).....	38
Exercise 4.3 - Create a pod using the CLI and the inline help.....	39
(End of Exercise).....	43
Exercise 4.4 - Generate a YAML file using the --dry-run flag and create a Pod using the file.....	44
(End of Exercise).....	47
Exercise 4.5 - Understand the differences between kubectl apply -f (declarative) and kubectl create -f (imperative).....	48
(End of Exercise).....	51
Exercise 4.6 - Retrieve information using kubectl get and kubectl describe.....	52
Section 5: Running workloads in Kubernetes.....	56
Exercise 5.1 - Manage namespaces in Kubernetes.....	57
Exercise 5.2 - Deploy a Pod with an initContainer.....	64
Exercise 5.3 - Deploy a Pod with a sidecar Container.....	68
Exercise 5.4 - Manage Deployments in Kubernetes.....	72
Exercise 5.5 - Manage StatefulSets in Kubernetes.....	80
Exercise 5.6 - Manage DaemonSets in Kubernetes.....	85
Section 6: Configuration for Kubernetes Objects.....	89
Exercise 6.1 - Create ConfigMaps in Kubernetes.....	90
Exercise 6.2 - Use ConfigMaps in Pods.....	96
Exercise 6.3 - Create Secrets in Kubernetes.....	103
Exercise 6.4 - Injects sensitive information into Pods using Secrets.....	108

Section 7: Scheduling in Kubernetes.....	112
Exercise 7.1 - Schedule Pods using Node Names.....	113
Exercise 7.2 - Schedule Pods using Node Selectors.....	116
Exercise 7.3 - Schedule Pods using Node Affinity Rules.....	122
Exercise 7.4 - Manage Taints in Nodes.....	128
Exercise 7.5 - Schedule Pods into Tainted Nodes (Tolerations).....	133
Section 8: Resource Usage Control and Quality of Service (QoS).....	136
Exercise 8.1 - Define limits and requests for Pods.....	137
Exercise 8.2 - Quality of Service (QoS) classes and the eviction process.....	144
Exercise 8.3 - Define a LimitRange for a namespace.....	149
Exercise 8.4 - Define a ResourceQuota for a namespace.....	155
Section 9: Scaling in Kubernetes.....	161
Exercise 9.1 - Manually scale up and down a deployment.....	162
Exercise 9.2 - Automatically scale up and down a deployment using Horizontal Pod Autoscalers (HPA) objects.....	1
Exercise 9.3 - Scale a Deployment vertically and manually.....	1
Section 10: Cluster Networking and Services in Kubernetes.....	1
Exercise 10.1 - Deploy Services in Kubernetes.....	1
Exercise 10.2 - Expose an application using Ingress.....	1
Exercise 10.3 - Expose an application using Gateway API.....	1
Section 11: Storage in Kubernetes.....	1
Exercise 11.1 - Use an emptyDir volume to share files between containers.....	1
Exercise 11.2 - Provide Persistent Storage Using an NFS Volume (static provisioning)...	1
Exercise 11.3 - Deploy a Container Storage Interface (CSI) Driver.....	1
Exercise 11.4 - Provide Persistent Storage Using a Dynamically Provisioned NFS Persistent Volume (PV) (dynamic provisioning).....	1
Exercise 11.5 - Advanced Storage Operations: Clone.....	1
Section 12: Security in Kubernetes.....	1
Exercise 12.1 - Create a new Kubernetes user and ServiceAccount.....	1
Exercise 12.2 - Define and test RBAC permissions.....	1
Exercise 12.3 - Secure a multi-layer application using a NetworkPolicy.....	1
Exercise 12.4 - Secure a Pod using a securityContext.....	1
Section 13: Monitoring and Troubleshooting in Kubernetes.....	1
Exercise 13.1 - Use the Kubernetes Troubleshooting Toolbox.....	1
Exercise 13.2 - Deploy and diagnose a failing application.....	1
Exercise 13.3 - Diagnose and troubleshoot a Kubernetes Service.....	1
Section 14: Cluster Operations in Kubernetes.....	1
Exercise 14.1 - Node maintenance using Kubernetes tools.....	1
Exercise 14.2 - Update the RKE2 cluster to Kubernetes 1.34.XX.....	1
Exercise 14.3 - Back up and restore your RKE2 cluster.....	1
Section 15: Kubernetes Application Management with Helm.....	1

Exercise 15.1 - Add a repository to Helm.....	1
Exercise 15.2 - Deploy an application using Helm.....	1
Exercise 15.3 - Manage a Helm release lifecycle: upgrades and rollback.....	1
Section 16: Extending Kubernetes with Operators and CRDs.....	1
Exercise 16.1 - Create a Custom Resource Definition (CRD).....	1
Exercise 16.2 - Install and use a pre-built operator from operatorhub.io).....	1

Document Conventions

Annotations

BOLD_UPPERCASE_ITALIC

Indicates an “exercise variable” that you replace with another value.

bold monospace

Indicates file names and commands entered at the command line.

bold monospace

Indicates text from command-line sessions, including both input and output, as well as file content.

Callouts

In this manual you will find the below color-coded labels which indicate different types of information that add value to the tasks.

Explanation ▾

Note ▾

Tip ▾

Additional Information ▾

Warning ▾

Link to Documentation

The SUSE documentation for this product is available here:

<https://documentation.suse.com/>

Section 1: Course Introduction

Course Description and Objectives

This hands-on course provides the essential knowledge and practical skills required to install, configure, and manage production-grade Kubernetes, leveraging the Kubernetes solutions offered by SUSE.

The course is primarily designed for System administrators, DevOps engineers, Cloud and Platform engineers, and others who need an introduction to Kubernetes and Kubernetes administration, as well as an understanding of the different Kubernetes distributions provided by SUSE, their features, and benefits.

During the course, you will:

- Learn the most important concepts related to containers and Kubernetes
- Install Rancher Kubernetes Engine 2 (RKE2), one of the Kubernetes distributions by SUSE.
- Explore the most important Kubernetes cluster components and workload-related objects.
- Understand Kubernetes administration concepts, including scheduling, networking, storage, security, monitoring, and troubleshooting, as well as cluster operations.
- Deploy applications using Helm and CRDs.

This section also introduces the lab environment and its key components.

During the section, you will:

- Become familiar with the lab environment
- Get familiar with the role of each virtual machine in the lab.

Course Lab Environment Overview

Lab Environment Information

The lab environment consists of three virtual machines running on an isolated network:

- **RKE201-management:** This virtual machine (VM) provides services required for the Lab environment, such as DNS, NFS, and NTP. It must be up and running in order for the Lab environment to work as expected. It is also used as the entry point for the other VMs by providing a GUI.
 - 2vCPUs
 - 2GB RAM
 - 2 hard disks:
 - System hard disk: 40GB
 - NFS shared hard disk: 10GB
 - SUSE Linux Enterprise 16 installed.
 - IP address: 172.30.170.2
 - Hostname: management.example.com
- **RKE201-server:** The RKE2 server (control-plane) node is installed in this VM.
 - 2vCPUs
 - 6GB RAM
 - 1 hard disk
 - System hard disk: 40GB
 - IP address: 172.30.170.3
 - SUSE Linux Enterprise 16 installed.
 - Hostname: server.example.com
- **RKE201-agent:** The RKE2 agent (worker) node is installed in this VM.
 - 2vCPUs
 - 6GB RAM
 - 1 hard disk
 - System hard disk: 40GB
 - IP address: 172.30.170.4
 - SUSE Linux Enterprise 16 installed.
 - Hostname: agent.example.com

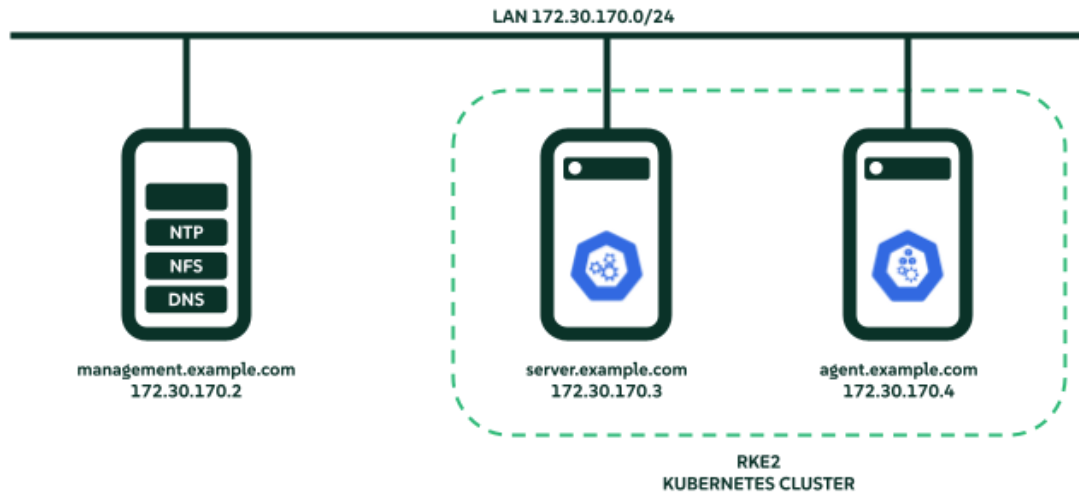
Required Minimum Product Version

This training course is based on Rancher Kubernetes Engine 2 (RKE2) v1.33, and during the course, the RKE2 Kubernetes cluster will be updated to version v1.34.

Lab Environment Hardware/Software Requirements

The Lab environment has been tested on a host with 16 cores, 64GB of RAM, and a 512GB hard disk. It requires at least 14GB of RAM and 135GB of hard disk space for the VMs.

Lab Environment Diagram



Networking:

- Isolated network
- DHCP service is enabled and provided by the management VM.
- DNS service is enabled and provided by the management VM.
- NTP service is enabled and provided by the management VM.
- Network address: 172.30.170.0/24
- DHCP range: 172.30.170.2-172.30.170.254

Exercise 1.1 - Start Lab Environment VMs

Description

In this exercise, you will use the Virt-Manager utility to start the lab VMs in the proper order and then log into the management workstation.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Start the course by turning on all the VMs in the Lab environment.
- Understand the relationship between the different VMs in the Lab environment.
- Log in to the management workstation and access the remaining VMs from there.

Exercise Overview

During the course, you will power on the Lab environment VMs in the right order. Once all the VMs are up and running, you will log into the management VM.

Finally, you will access the server and agent VMs using SSH to verify that the Lab environment is working correctly.

Task 1: Start the Lab Environment VMs

Step 1 ▾

On the **lab machine**, open the **Virtual Machine Manager** application. You will see all the VMs in the Lab environment listed:

- RKE201-management
- RKE201-server
- RKE201-agent

Explanation ▾

To open the **Virtual Machine Manager** application, you can choose between these two options:

- Open a terminal window and run the **virt-manager** CLI command.
- Open your application dashboard and select the application icon: 

Step 2 ▾

- Right-click on the **RKE201-management** VM and select **RUN** to start the virtual machine.
- Double-click on the **RKE201-management** VM to view its console.
- When the VM has finished the booting process, and you see the graphical login, close its console window. This VM provides several services to other VMs, and must always be up and running for the Lab environment to work properly.

Step 3 ▾

Note ▾

Once the management VM is ready, you can turn on the remaining VMs that use services provided by the management machine. To turn on these VMs, follow the same steps as before.

- Right-click on the **RKE201-server** VM and select **RUN** to start the virtual machine.
- Double-click on the **RKE201-server** VM to view its console.
- When the VM has finished the booting process, and you see the console login, close its console window.
- Repeat the same process for the **RKE201-agent** VM.

Task 2: Verify the Lab environment connections.**Step 1** ▾

With all the VMs up and running in the **Virtual Machine Manager**

- Double-click on the **RKE201-management** to view its console.
- Log in using the following credentials:
 - Username: **tux**
 - Password: **linux**

Step 2 ▾

- Open a terminal by selecting the terminal icon at the bottom
- Open an SSH connection to the server node using the following command: **ssh tux@server.example.com** Use **linux** as the password and select **yes** if prompted.
- Verify that the connection has been established and execute **exit** to return to the management terminal.
- Repeat the same process for the agent node, using the command **tux@agent.example.com**

Exercise Summary

In this exercise, you have started the Lab environment by turning on all the VMs in the proper order and have also verified that the connections between all the VMs are working as expected. You can now use the management VMs as an access point to the server and agent VMs.

(End of Exercise)

Section 2: Introduction to containers and container orchestration

Description

The way companies have deployed software applications has changed over time. Starting with a single bare-metal server, the technology supporting these deployments has evolved into Virtual Machines (VMs) and, in recent years, into containers and container orchestrators.

Before digging deeper into Kubernetes, the most relevant container orchestrator, this exercise shows how to manage container images and use them to run applications in containers.

Section Learning Objectives

- Present the concept of containerization: Understand the core principles of containers, including isolation, portability, and immutability. Describe how containers facilitate the shift from monolithic applications to microservices.
- Identify the need for container orchestration: Identify the challenges when managing containers at scale.
- Introduce Kubernetes: Describe Kubernetes and its historical context.
- Outline cluster architecture: Detail the responsibilities of the main elements for the different node types in a Kubernetes cluster (control plane node, worker node).

Section Topics

- Container fundamentals
- The need for orchestration
- Introduction to Kubernetes

Exercise 2.1 - Run containers and manage container images

Description

In this exercise, you will create containers and manage container images on the management virtual machine (VM) using Podman.

Podman is a daemonless container engine that manages OCI containers and pods. It also provides a command-line interface (CLI) compatible with the Docker CLI and is ideal for local development.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a container using a container image.
 - Understand the difference between a container and a container image.
 - Review the layers of a container image.
 - Manage containers and container images.
-

Task 1: Create a container**Step 1** ▾

Select and run the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

Once the graphical Login screen appears, use the following credentials to log into the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 2 ▾

Open a terminal in the management virtual machine and run the following command to display all the running containers:

command

```
podman ps
```

Explanation ▾

As you have not downloaded any container image or created any container yet, empty output is provided.

Example output:

```
tux@management:~> podman ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
```

Note ▾

The output is the same if you try to list all the containers, running or stopped, using the **--all** flag:

command

```
podman ps --all
```

Step 3 ▾

Use the following command to create a container that runs an Nginx web server and redirects the web server output to the 8080 port in the management virtual machine:

command

```
podman run -d --name webserver -p 8080:80/tcp nginx
```

Explanation ▾

This podman CLI command comprises the following elements:

- **run:** Specific podman subcommand to create and start a new container.
- **-d:** Short for --detach. When used, podman runs the container in background mode.
- **--name webserver:** Assigns the name webserver to the container.
- **-p 8080:80:** Maps the container port 80 to the management VM port 8080.
- **nginx:** Specifies the container image to be used. As no version is specified, **nginx:latest** is used.

Note ▾

Since this is your first time using the nginx image, it will be downloaded from an external container registry.

Step 4 ▾

Enter the first command again to display the list of running containers.

command

```
podman ps
```

Explanation ▾

Verify that the new container has been created and its status is **Up**. Notice that the port mapping information also displays:

Example output:

```
tux@management:~> podman ps
CONTAINER ID   IMAGE                                COMMAND
CREATED       STATUS      PORTS          NAMES
254b2b5f4e0e   docker.io/library/nginx:latest     nginx -g daemon o... 6
seconds ago   Up 6 seconds  0.0.0.0:8080->80/tcp  webserver
```


Step 5 ▾

Use the curl command to access the web server using the management VM URL and the assigned port. You will see the HTML of the default nginx web page:

command

```
curl http://management.example.com:8080
```

Task 2: Review the container information**Step 1** ▾

Enter the following command to list the available images in the management VM:

command

```
podman images
```

Explanation ▾

Only the **nginx:latest** image must appear in the container list:

Example output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
docker.io/library/nginx	latest	60adc2e137e7	9 days ago	155 MB

Step 2 ▾

Enter the following command to display the layers of the **nginx** image:

command

```
podman image tree docker.io/library/nginx:latest
```

Explanation ▾

Container images are composed of layers. These layers are stored and cached on your system when you download a new image. If any of those layers appear in a new image you want to use, you do not need to download them again. In this case, the image has seven layers.

Example output:

```
tux@management:~> podman image tree docker.io/library/nginx:latest
Image ID: 60adc2e137e7
Tags:    [docker.io/library/nginx:latest]
Size:    155.5MB
```

```
Image Layers
ID: 70a290c5e58b Size: 81.04MB
ID: 0cc07798e5e4 Size: 74.41MB
ID: a4bf3b0dc9cb Size: 3.584kB
ID: 988a68fe57bb Size: 4.608kB
ID: b27339978a35 Size: 2.56kB
ID: 9549fca913db Size: 5.12kB
ID: a2c34b24b51b Size: 7.168kB Top Layer of:
[docker.io/library/nginx:latest]
```

Step 3 ▾

You can access the container using the **podman exec** command:

command

```
podman exec -it webserver /bin/bash
```

Explanation ▾

This podman CLI commands comprises the following elements:

- **exec:** Specific podman subcommand to run a process inside a container.
- **-it:** Combines **-i** (interactive) and **-t** (TTY - allocate a pseudo terminal) flags. They are used together when you want to open a shell terminal in a container.
- **webserver:** Name of the target container.
- **/bin/bash:** Command you want to run on the container.

Example output:

```
tux@management:~> podman exec -it webserver /bin/bash
root@254b2b5f4e0e:/#
```

Note ▾

Notice that the shell prompt has changed.

Step 4 ▾

Enter the following command to identify the process with PID 1. You will see that it is the nginx application.

command

```
cat /proc/1/cmdline
```

Explanation ▾

Unlike typical Linux systems, where systemd is PID 1, containers can set a different process as PID 1. This is typically defined in the Dockerfile used to create those containers.

Additional Information ▾

You can find additional information about Dockerfile and how to build container images [here](#).

Step 5 ▾

Enter the following command to leave the container. You return to the shell prompt on the management host.

command

```
exit
```

Task 3: Manage the container image and delete the containers

Step 1 ▾

Stop the webserver container using the following command:

command

```
podman stop webserver
```

Step 2 ▾

List all the containers, running or stopped:

command

```
podman ps --all
```

Explanation ▾

The webserver container is no longer running and is not displayed with the **podman ps** command. You need to use the **--all** flag. The webserver container will display an **Exited** status.

Example output:

```
tux@management:~> podman ps --all
CONTAINER ID   IMAGE                                COMMAND
CREATED        STATUS          PORTS
NAMES
254b2b5f4e0e   docker.io/library/nginx:latest     nginx -g daemon o...
```

```
About an hour ago Exited (0) 5 seconds ago 0.0.0.0:8080→80/tcp  
webserver
```

Step 3 ▾

Remove the container using the following command:

command

```
podman rm webserver
```

Step 4 ▾

To return the workstation to its original state, remove the container image using the following command:

command

```
podman image rm docker.io/library/nginx
```

Exercise Summary

In this exercise, you used podman to create a new container using the nginx container image with port mapping to the management VM, which acted as the host for the container. You displayed all the layers of the image used by the container, and opened a shell in the container to recover information about the process holding PID 1 within it.

Finally, you exited the container, stopped and removed the container, and deleted the downloaded copy of the container image

(End of Exercise)

Section 3: Kubernetes Distributions by SUSE

Description

Kubernetes is the leading open-source container orchestrator, managed by the Cloud Native Computing Foundation (CNCF). Many companies offer specialized Kubernetes distributions based on the official project, modified to meet specific needs.

SUSE offers several Kubernetes distributions, including RKE2. RKE2 is a secure Kubernetes distribution optimized for highly regulated environments, such as banking and government.

During this exercise, you will install an RKE2 server (control-plane node) on one virtual machine (VM) and an RKE2 agent (worker node) on another VM in the Lab.

Section Learning Objectives

- Distinguish core features: Explain the differences between RKE2 and K3S.
- Execute a basic deployment: Install an RKE2 cluster using the tarball installation method.

Section Topics

- Distinguish core features: Explain the differences between RKE2 and K3S.
- Install RKE2

Exercise 3.1 - Install RKE

Description

In this exercise, you will install an RKE2 server (control-plane node) on one virtual machine (VM) and an RKE2 agent (worker node) on another.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Install RKE2 server and agent nodes using the tarball method.
 - Understand the different node roles within an RKE2 cluster.
 - Perform the RKE2 post-installation configuration steps.
 - Verify that the Kubernetes cluster is operational.
-

Task 1: Install a RKE2 server node (control plane node)

Step 1 ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and run the following command, using **linux** as the password when prompted.

command

```
ssh root@server
```

Explanation

The `ssh` command opens a shell session on the RKE201-server VM as the root user, because the RKE2 tarball installation requires root privileges.

Example output:

```
tux@management:~> ssh root@server.example.com
(root@server.example.com) Password:
Last login: Mon Dec  1 10:16:06 CET 2025 from 172.30.170.2 on ssh
...
Have a lot of fun...
server:~ #
```

Note

Note that the prompt has changed to display that you are on the **server** VM.

Step 4

Run the following commands to perform the RKE2 Tarball installation method:

commands

```
> curl -sL https://get.rke2.io --output ./install.sh
> chmod +x install.sh
> INSTALL_RKE2_CHANNEL=v1.33 INSTALL_RKE2_TYPE=server ./install.sh
```

Explanation

- **`curl -sL https://get.rke2.io --output ./install.sh`**: Downloads the installation script and saves it as `install.sh`.
- **`chmod +x install.sh`**: Adds execution permissions to the script.
- **`INSTALL_RKE2_CHANNEL=v1.33 INSTALL_RKE2_TYPE=server ./install.sh`**: Installs an RKE2 server (control-plane) node running Kubernetes version 1.33. **`INSTALL_RKE2_CHANNEL`** and **`INSTALL_RKE2_TYPE`** are environment variables used to set the node role and the Kubernetes version.

Warning

In those boxes containing more than one command, each command starts with `>` to clearly separate them.

Note

Later in this course, you will update your cluster to a newer version (v1.34).

Additional Information

The [official RKE2 installation documentation](#) provides more details on the Tarball installation process and other installation processes.

Step 5

Create the RKE2 server node configuration file, `/etc/rancher/rke2/config.yaml`, with the following content:

commands

```
> mkdir -p /etc/rancher/rke2/
> vim /etc/rancher/rke2/config.yaml
```

/etc/rancher/rke2/config.yaml

```
token: kubeadmincourse
tls-san:
  - server.example.com
cni: canal
ingress-controller: traefik
```

Explanation

- **mkdir -p /etc/rancher/rke2/:** Creates the RKE2 configuration directory.
- **vim /etc/rancher/rke2/config.yaml:** Opens the configuration file to add the configuration options.

The configuration parameters in the configuration file:

- **token:** Sets the shared secret (token) required for other nodes (server or agent) to join the cluster.
- **tls-san:** Adds **server.example.com** as Subject Alternative Name to the Kubernetes cluster certificate (kube-apiserver).
- **cni: canal:** Sets the network implementation for the Kubernetes cluster. You will learn more about Kubernetes and networking later in this course.
- **ingress-controller: traefik:** Sets the ingress controller for the Kubernetes cluster. You will also learn about Ingress later in this course.

Additional Information

The official [Server Configuration Reference](#) documentation provides more details on the server configuration parameters.

Step 6 ▾

Start the RKE2 server node by running the following command (Wait until the command finishes):

command

```
systemctl enable --now rke2-server.service
```

Warning ▾

If the command fails, wait a couple of minutes and try again. Labs run on nested virtualized environment and you might need to wait longer to have everything ready to the RKE2 service to be ready.

Step 7 ▾

Create a symbolic link for the **kubectl** (Kubernetes client) binary and the authentication information.

commands

```
> ln -s /var/lib/rancher/rke2/bin/kubectl /usr/local/bin/
> mkdir ~/.kube
> ln -s /etc/rancher/rke2/rke2.yaml ~/.kube/config
```

Explanation ▾

- **ln -s /var/lib/rancher/rke2/bin/kubectl /usr/local/bin/:** Creates the symbolic link to the Kubernetes client tool, **kubectl**, and adds the tool to the system path.
- **mkdir ~/.kube:** Creates the folder where **kubectl**, by default, looks for Kubernetes cluster authentication information.
- **vim /etc/rancher/rke2/config.yaml:** Opens the configuration file to add the configuration options.
- **ln -s /etc/rancher/rke2/rke2.yaml ~/.kube/config:** Links the generated authentication file to the default **kubectl** configuration directory.

Tip ▾

Kubectl, which acts as a Kubernetes client, uses the authentication information in the **~/.kube/config** to connect to the **kube-apiserver**, the entrypoint to the Kubernetes cluster. All the interactions with the cluster go through this server.

Step 8 ▾

Verify that the Kubernetes server (control-plane) node is already running and all the components are ready by running the following commands:

commands

```
> kubectl cluster-info
> kubectl get nodes
> kubectl get pods -A
```

Explanation ▾

- **kubectl cluster-info:** Display a brief description.
- **kubectl get nodes:** Lists all the nodes in the RKE2 Kubernetes cluster. Only the server node should appear in the list.
- **kubectl get pods -A:** List of the pods running in the cluster. Pods are the basic workload in a Kubernetes cluster.

Step 9 ▾

Access to the Kubernetes cluster from the **RKE201-management** VM, following these steps:

- Close the connection to the **RKE201-server** by running the **exit** command. Note that the terminal prompt should display again **tux@management:~>**
- Copy the **kubectl** Kubernetes client tool to the management machine by running the following command.

command

```
scp root@server:/usr/local/bin/kubectl ~/bin/
```

- Copy the authentication file to the **tux** account in the **management** VM by running the following commands:

commands

```
> mkdir -p ~/.kube
> scp root@server:/etc/rancher/rke2/rke2.yaml ~/.kube/config
```

- Edit the **~/.kube/config** file and replace this line:

```
server: https://127.0.0.1:6443
```

with this line:

```
server: https://172.30.170.3:6443
```

Explanation ▾

Since you installed the RKE2 cluster on the **RKE2-server** VM, the Kubernetes authentication information used the localhost IP address. Since **kubectl** now runs on the management node, you must update the configuration to point to the server's IP address (172.30.170.3).

Verify you can access the RKE2 cluster from the **RKE-management** VM by running **kubectl get nodes**.

Example output:

```
tux@management:~> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
server	Ready	control-plane,etcd,master	170m	v1.33.6+rke2r1

Task 2: Add an RKE2 agent node (worker node) to the RKE2 Kubernetes cluster

Step 1 ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

Once the graphical Login screen appears, use the following credentials to log into the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and run the following command, using **linux** as the password when prompted.

command

```
ssh root@agent
```

Explanation ▾

The ssh command opens a shell session on the **RKE201-agent** VM using the **root** user, as the RKE2 Tarball installation process requires the root user to succeed.

Example output:

```
tux@management:~> ssh root@agent.example.com
...
Last login: Mon Dec  1 12:27:55 CET 2025 from 172.30.170.2 on ssh
...
Have a lot of fun...
agent:~ #
```

Note ▾

Note that the prompt has changed to display that you are on the **agent** VM.

Step 4 ▾

Run the following commands to perform the RKE2 Tarball installation method:

commands

```
> curl -sL https://get.rke2.io --output ./install.sh
> chmod +x install.sh
> INSTALL_RKE2_CHANNEL=v1.33 INSTALL_RKE2_TYPE=agent ./install.sh
```

Explanation ▾

- **curl -sL https://get.rke2.io --output ./install.sh**: Downloads the installation script and stores the content in the install.sh file.
- **chmod +x install.sh**: Adds execution permissions to the script.
- **INSTALL_RKE2_CHANNEL=v1.33 INSTALL_RKE2_TYPE=agent ./install.sh**: Installs an RKE2 agent (worker) node running Kubernetes version 1.33. **INSTALL_RKE2_CHANNEL** and **INSTALL_RKE2_TYPE** are environment variables used to set the node role and the Kubernetes version.

Step 5 ▾

Create the RKE2 server node configuration file, **/etc/rancher/rke2/config.yaml**, with the following content:

commands

```
> mkdir -p /etc/rancher/rke2/
> vim /etc/rancher/rke2/config.yaml
```

/etc/rancher/rke2/config.yaml

```
server: https://172.30.170.3:9345
token: kubeadmincourse
```

Explanation

- **mkdir -p /etc/rancher/rke2/**: Creates the RKE2 configuration directory.
- **vim /etc/rancher/rke2/config.yaml**: Opens the configuration file to add the configuration options.

The configuration parameters in the configuration file:

- **server**: Points to the initial RKE2 server node installed in Task 1. The RKE2 server listens on port 9345 for new nodes to register.
- **token**: Authentication token. The value must match the value in the server configuration file.

Step 6

Start the RKE2 agent node by running the following command (Wait until the command finishes):

command

```
systemctl enable --now rke2-agent.service
```

Step 7

Run **exit** to close the SSH connection to the **RKE201-agent** VM and back to the **RKE201-management** VM, list all the nodes in the RKE2 cluster by running the following command:

command

```
kubectl get nodes
```

Example output:

```
agent:~ # exit
logout
Connection to agent.example.com closed.
tux@management:~> kubectl get nodes
NAME      STATUS    ROLES                  AGE   VERSION
agent     Ready     <none>                 77s   v1.33.6+rke2r1
server    Ready     control-plane,etcd,master 44m   v1.33.6+rke2r1
tux@management:~>
```

Explanation ▾

Back to the **RKE2-management** VM, the **kubectl** Kubernetes client tool was previously configured to access the cluster using the `~/.kube/config` file. No additional configuration is required. Run **kubectl get nodes** to verify that both nodes are listed:

- The server (control-plane) node that was added during **Task 1**.
- The agent (worker) node, which was installed and joined the cluster during **Task 2**.

Warning ▾

Pay attention to the **STATUS** column. Both of the nodes should appear as **Ready**.

Exercise Summary

In this exercise, you learned how to create an RKE2 Kubernetes cluster with two nodes: a server node that runs all the main Kubernetes components and an agent node, using the Tarball installation method for both.

You started by installing and configuring the RKE2 server on the RKE201-server VM. After testing the cluster and enabling access to it from the RKE201-management VM, you installed the RKE2 agent on the RKE201-agent VM, adding this new node to the cluster using the authentication token used to configure the server and the server's URL.

(End of Exercise)

Section 4: Basic Kubernetes Objects and Commands

Description

Kubernetes is a container orchestrator used to deploy applications at scale. Pods are the basic workload unit in Kubernetes, and the main Kubernetes object. You can create Pods and other Kubernetes objects using the `kubectl` client tool in two different ways:

- The Imperative way, using the `kubectl` subcommands and different arguments and options.
- The Declarative way, using YAML files and the ***kubectl apply*** command.

The learning curve for mastering all the subcommands and the YAML structure is steep. Still, this exercise provides several strategies to ease the process, shows how to manage pods in both imperative and declarative ways, and explains how to retrieve information about different Kubernetes objects in your cluster.

Section Learning Objectives

- Identify the main Kubernetes objects: Recognize and describe the structure and function of Namespaces, Pods, ReplicaSets, Deployments, Services, PV, PVC, StorageClass...
- Explain object relationships: Articulate the relationships between the different Kubernetes objects.
- Master imperative and declarative commands: Create objects in imperative and declarative ways.
- View and inspect resources: Use `kubectl get` and `kubectl describe` to retrieve information about Kubernetes objects.
- Improve your Kubernetes command-line efficiency: Configure auto-completion and aliasing, and learn how to use command-line commands to create YAML output and retrieve online help.

Section Topics

- Core Kubernetes objects and their relationships
- Create Kubernetes objects and retrieve information (imperative and declarative commands)
- Command-line efficiency tricks

Exercise 4.1 - Configure **kubectl** autocompletion

Description

In this exercise, you will configure your user to use autocompletion for the **kubectl** command, and also for the **k** command alias for **kubectl**. Once autocompletion is enabled and the alias for the command is in place, you will practice using autocompletion to speed up typing Kubernetes CLI commands.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Configure your user environment to enable autocompletion for the **kubectl** command
 - Add **k** as an alias for the **kubectl** command, and enable autocompletion also for the alias
 - Use autocompletion to speed up typing Kubernetes CLI commands.
-

Task 1: Configure autocompletion for the kubectl command**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and run the following commands to enable autocompletion for **kubectl**:

commands

```
> echo 'source <(kubectl completion bash)' >> ~/.bashrc
> source ~/.bashrc
```

Explanation ▾

The echo command adds the output of the kubectl completion command to your user environment file (~/.bashrc), and the source command reloads the file, so you can apply the changes without restarting the session.

Step 4 ▾

Test the autocompletion by following this command sequence (→ (TAB) stands for pressing the TAB key):

commands

```
kubectl g →(TAB)
kubectl get →(TAB)
kubectl get n →(TAB)
```

```
kubectl get nodes →(TAB)
kubectl get nodes s →(TAB)
```

Explanation ▾

Since autocompletion is already enabled, each time you press the → (TAB) key, the **kubectl** command provides all the possible subcommands, options, or Kubernetes objects relevant to your current command input.

- **kubectl g** → (TAB): Autocompletes to **kubectl get** as **get** is the only **kubectl** subcommand that starts with 'g'.
- **kubectl get** → (TAB): Displays all the Kubernetes objects you can retrieve information about.
- **kubectl get n** → (TAB): Displays all the Kubernetes objects whose name starts with n.
- **kubectl get nodes** → (TAB): Displays all the nodes in the RKE2 cluster.
- **kubectl get nodes s** → (TAB): Autocompletes to **kubectl get nodes server** as **server** is the only node starting with 's'.

Note ▾

You can use **kubectl** from the management VM because during the previous section you copied the cluster authentication file to your `~/.kube/config` and updated the cluster's URL to point to the server VM.

Task 2: Configure autocompletion for the 'k' alias

Step 1 ▾

Open a terminal in the management VM and run the following commands to enable autocompletion for **kubectl**:

commands

```
> echo 'alias k=kubectl' >> ~/.bashrc
> echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc
> source ~/.bashrc
```

Explanation ▾

The first **echo** command adds the 'k' alias for the **kubectl** command to your user environment file (`~/.bashrc`), the second **echo** command defines autocompletion for the alias, and the **source** command reloads the file, so you don't need to start the session again for those changes to be applied.

Step 2 ▾

Repeat **Task 1-Step 4**, replacing **kubect**l with **k**, the alias.

commands

```
k g →(TAB)
k get →(TAB)
k get n →(TAB)
k get nodes →(TAB)
k get nodes s →(TAB)
```

Warning

Since autocompletion is already set for both **kubect**l and **k**, you can use any of them. All the examples in this LAB manual use **kubect**l to avoid any confusion. I strongly recommend using the alias to speed up CLI typing.

Exercise Summary

In this exercise, you set Kubernetes autocompletion for the **kubect**l command and for the **k** alias for that command.

(End of Exercise)

Exercise 4.2 - Display information about the RKE2 cluster using kubectl

Description

Kubectl is a Kubernetes CLI client tool used to interact with Kubernetes clusters through the kube-apiserver component. In addition to creating Kubernetes objects, you can use the tool to get information about the whole cluster.

In this exercise, you will learn several kubectl subcommands to retrieve valuable information about your RKE2 Kubernetes cluster.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use kubectl to retrieve information about the Kubernetes cluster.
 - List all the nodes in the cluster and understand the role those nodes play within the cluster (control-plane, worker, etcd...)
 - Understand the authentication information in the ./kube/config file.
 - Discover the different APIs and shortnames for the cluster resources using the kubectl api-resources command.
-

Task 1: Display information about your cluster

Step 1 ▾

Run **kubectl cluster-info** to display information about the cluster.

Example output:

```
tux@management:~> kubectl cluster-info
Kubernetes control plane is running at https://server.example.com:6443
CoreDNS is running at
https://server.example.com:6443/api/v1/namespaces/kube-system/services
/rke2-coredns-rke2-coredns:udp-53/proxy
```

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.

Explanation ▾

The command returns information about the server (control-plane node) and the DNS service in the cluster. The role of the DNS in the cluster is presented later in the course.

Step 2 ▾

Run the following commands to retrieve detailed information about the cluster and its status:

commands

```
> mkdir ~/rke2-info
> kubectl cluster-info dump --output-directory=/home/tux/rke2-info/ \
> rke2-cluster.json
```

Explanation ▾

- The **dump** option makes the **cluster-info** command return detailed information about the cluster nodes, all Kubernetes objects in the current and kube-system namespaces, and the logs for all pods in the cluster. By default, the output is formatted in json. Namespaces are treated in detail later in this course.
- The **--output-directory=/home/tux/reke-inf/** option causes the cluster-info command to produce organized output in the specified directory, dividing the information into namespaces.

Step 3 ▾

Review the **rke2-info** folder and its contents using vim, a text editor, or Linux CLI commands.

Example output:

```
tux@management:~/rke2-info> ls -l
total 24
drwxr-xr-x. 1 tux tux   216 Dec  4 16:56 default
drwxr-xr-x. 1 tux tux  1668 Dec  4 16:56 kube-system
-rw-r--r--. 1 tux tux 23174 Dec  4 16:56 nodes.json
```

Task 2: Display information about the nodes in the cluster

Step 1 ▾

Run **kubectl get nodes** to display the nodes in the **RKE2** cluster:

Example output:

```
tux@management:~/rek2-info> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
agent	Ready	<none>	3d3h	v1.33.6+rke2r1
server	Ready	control-plane,etcd,master	3d4h	v1.33.6+rke2r1

Exercise Summary

In this exercise, you retrieved information about the RKE2 cluster using the simplest version of **kubectl cluster-info** and the **dump** option to get a full report. Later, you used **kubectl get nodes** to get information about the nodes in the cluster.

(End of Exercise)

Exercise 4.3 - Create a pod using the CLI and the inline help

Description

Kubectl is the tool that you, as a Kubernetes administrator, use to manage and configure Kubernetes objects such as pods, deployments, services, configmaps, or secrets.

If you work using an imperative approach, getting familiar with all the options for `kubectl` and subcommands can be overwhelming at the beginning; however, there is a flag, the `--help` flag, that provides a summary of the most important options for the commands and also examples that you can use as a starting point to create objects.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use the `--help` to retrieve help information and examples.
 - Create a Pod using an example displayed using the `--help` command
-

Task 1: Use the --help to get inline help for kubectl subcommands**Step 1** ▾

Run the following command to get inline help about **kubectl create**:

command

```
kubectl create --help
```

Explanation ▾

Adding the --help flag to any kubectl subcommand displays a list of examples and a description of the most essential options for the subcommand.

Step 2 ▾

Review the output from the previous command and identify the different Kubernetes objects that can be directly created in imperative mode using the **kubectl create** command.

Example output:

```
tux@management:~> kubectl create --help
Create a resource from a file or from stdin.
...
Available Commands:
  clusterrole           Create a cluster role
  clusterrolebinding    Create a cluster role binding for a particular cluster role
  configmap             Create a config map from a local file, directory or literal value
  cronjob               Create a cron job with the specified name
  deployment            Create a deployment with the specified name
  ingress               Create an ingress with the specified name
  job                   Create a job with the specified name
  namespace             Create a namespace with the specified name
  poddisruptionbudget   Create a pod disruption budget with the specified name
  priorityclass         Create a priority class with the specified name
  quota                 Create a quota with the specified name
  role                  Create a role with single rule
  rolebinding           Create a role binding for a particular role or cluster role
  secret               Create a secret using a specified subcommand
  service               Create a service using a specified subcommand
  serviceaccount        Create a service account with the specified name
  token                Request a service account token
...
```

Step 3 ▾

Run the following command to get inline help about **kubectl scale**:

command

```
kubectl scale --help
```

Step 4 ▾

Review the output from the previous command and identify the different examples for the **kubectl scale** subcommand provided by the inline help:

Example output:

```
tux@management:~> kubectl scale --help
...
Examples:
  # Scale a replica set named 'foo' to 3
  kubectl scale --replicas=3 rs/foo

  # Scale a resource identified by type and name specified in "foo.yaml" to 3
  kubectl scale --replicas=3 -f foo.yaml

  # If the deployment named mysql's current size is 2, scale mysql to 3
  kubectl scale --current-replicas=2 --replicas=3 deployment/mysql

  # Scale multiple replication controllers
  kubectl scale --replicas=5 rc/example1 rc/example2 rc/example3

  # Scale stateful set named 'web' to 3
  kubectl scale --replicas=3 statefulset/web
...
```

Tip

Examples provided by the `--help` can frequently be used as a starting point or for testing purposes.

Task 2: Create a pod using one of the examples displayed in the inline help**Step 1**

As shown earlier, pods are not among the Kubernetes objects that can be created with the **kubectl create** command. To make a pod with an imperative CLI command, use **kubectl run**. Run the following command, review all the options, and copy the first example that appears:

command

```
kubectl run --help
```

Example output:

```
tux@management:~> kubectl run --help
Create and run a particular image in a pod.

Examples:
  # Start a nginx pod
  kubectl run nginx --image=nginx
...
```

Step 2

Create a pod that deploys an nginx server using the previously copied example:

command

```
kubectl run nginx --image=nginx
```

Explanation ▾

These are the elements in the command:

- **kubectl run:** The command to create the pod.
- **nginx:** The name of the running nginx instance.
- **--image nginx:** The name of the container image. As no version is set, **nginx** stands for **nginx:latest**

Step 2 ▾

Check that the pod is running using the following command:

command

```
kubectl get pods
```

Explanation ▾**Example output:**

```
tux@management:~> kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           11s
```

The **STATUS** column displays a **Running** value; therefore, the Pod has been created without any problem.

Task 3: Delete the nginx pod**Step 1** ▾

Delete the nginx pod using the following command:

command

```
kubectl delete pod nginx
```

Explanation ▾

To delete any Kubernetes object from the CLI, use the **kubectl delete** command. In the previous command:

- **pod:** It's the type of object you want to delete in this scenario. It could be any other Kubernetes object type, such as Deployments, Services, or Replicasets.
- **nginx:** It's the name of the pod to be deleted.

Step 2 ▾

Check that the pod no longer exists by running the following command:

command

```
kubectl get pods
```

Example output:

```
ttux@management:~> kubectl delete pod nginx
pod "nginx" deleted
tux@management:~> kubectl get pods
No resources found in default namespace.
```

Note ▾

The last line in the output highlights that, for the moment, you are working in the default namespace. Namespaces are presented later in the course.

Exercise Summary

In this exercise, you learned how to use the `--help` option to retrieve additional information and examples for the `kubectl` command and subcommands. Later, you use one of the examples provided by the help to create a Pod from the command line.

(End of Exercise)

Exercise 4.4 - Generate a YAML file using the `--dry-run` flag and create a Pod using the file

Description

Working in a declarative way to create Kubernetes objects requires remembering object properties and handling YAML syntax. Constantly visiting the Kubernetes official documentation, frequently switching context to look for those properties you don't know or remember, and writing YAML files from scratch can take more time and effort than necessary.

However, using the `--dry-run` flag when creating objects with the **kubectl** command allows you to create a perfectly formatted YAML file. This is quite an advantage.

This workflow combines an imperative and a declarative approach. You use imperative commands to generate YAML files that you modify or adapt as needed, then use the declarative command **kubectl apply -f** to create the Kubernetes objects.

In this exercise, you will learn how to use `--dry-run` flag, which, in conjunction with the previously presented `--help` flag, can speed up how you work with Kubernetes, as an administrator.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use the **kubectl --dry-run** flag to generate a YAML file you can use as the foundation to create Kubernetes objects.
 - Learn how to speed up your Kubernetes workflow using `--dry-run` and `--help` all together.
 - Create **Kubernetes** objects using the declarative way and YAML files created using imperative commands.
-

Task 1: Create a YAML description for a Pod using the --dry-run flag**Step 1** ▾

Run the following **kubectl run** command using the **--dry-run=client** flag and export the output to a YAML file without creating any Kubernetes object:

command

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > mypod.yaml
```

Explanation ▾

- **kubectl run nginx --image:nginx** : This is the same command used previously to create a pod.
- **--dry-run=client** : When used, the **--dry-run** flag with the **client** value, the command prints the desired object without sending any request to the kube-apiserver. No object is created.
- **-o yaml > mypod.yaml** : Sets YAML as the output format and saves the output in the mypod.yaml file.

Step 2 ▾

Review the content of the **mypod.yaml** using the **cat mypod.yaml** Linux command:

Example output (mypod.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Explanation ▾

The YAML file created using the **--dry-run=client** option has the four most common sections to create objects:

```
apiVersion: v1
```

- The **apiVersion** defines the api used to create the object. If you are building the YAML from scratch and you don't know the value for this property, use the **kubectl api-resources** command that provides these values and additional information, such as the shortname for the resource and whether the resource is namespaced or not.

kind: Pod

- The **kind** property defines the type of object you want to create. A pod in this exercise.

```
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
```

- The **metadata** section contains information that describes the pod, such as array of labels to categorize the pod and the name.

```
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

- The **spec** section describes the desired state or behavior for the pod. It contains properties such as the arrays of containers in the pod, with the image and name used for each one, **nginx** for both in this scenario.

Note ▾

The **status: {}** is not a required property for creating the Pod. This section is filled up with content once the pod is up and running.

Additional Information ▾

For a complete reference on all the Kubernetes objects' properties, visit the [Kubectl API Reference](#) section on the [Kubernetes official documentation](#) site.

Step 3 ▾

Create the pod declaratively using the YAML file and the following command:

command

```
kubectl apply -f mypod.yaml
```

Explanation ▾

kubectl apply is the command used to create Kubernetes objects in a declarative way, where you specify the desired state of the pod. The **-f** flag specifies the file used to create the objects.

Step 4 ▾

Verify that the pod is up and running using the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~> kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           12s
```

Exercise Summary

In this exercise, you used a mixed approach, imperative and declarative, to create Kubernetes objects. You started by creating the YAML file for a Pod from an imperative command with the `--dry-run=client` flag, and redirecting the YAML output to a file. Later, you created the Pod declaratively with the `kubectl apply -f` command.

(End of Exercise)

Exercise 4.5 - Understand the differences between *kubectl apply -f* (declarative) and *kubectl create -f* (imperative)

Description

In the previous exercise, you created Kubernetes objects declaratively using a YAML file with the desired state and the **kubectl apply -f** command. Other CLI commands, such as **kubectl create -f**, can take a YAML file as input and are imperative.

In this exercise, you will use both commands, **kubectl apply -f** and **kubectl create -f**, to achieve a better understanding of declarative commands that use the desired state and YAML files to create Kubernetes objects, and imperative commands where you merely provide the steps you need to perform.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Dive deeper into the differences between declarative and imperative commands.
-

Task 1: Modify the desired state for the nginx pod**Step 1** ▾

You left the **nginx** pod up and running at the end of the previous exercise. Now you will change the desired state. First, modify the **mypod.yaml** to contain the following content:

mypod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx:1.28.0
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Explanation ▾

The only change in the file is the image used for the pod. The new version uses **nginx:1.28.0**

Tip ▾

Changes to an existing object can also be performed using the imperative **kubectl edit** command. This command opens an editor with the current YAML state for the object; you can make changes, but those changes don't modify the YAML file, so you won't be able to track the changes if you are using a version control system.

Warning ▾

When updating the desired state of a pod, not all of the pod's properties or fields can be updated. If you try to modify one of those fields, an *"....field is immutable..."* error appears, and you would need to delete and recreate the Kubernetes object.

Step 2 ▾

Apply the new desired state using the following command:

command

```
kubectl apply -f mypod.yaml
```

Step 3 ▾

Retrieve a YAML description for the running pod with the following command and review the output to verify the container is using the new container image:

command

```
kubectl get pods nginx -o yaml
```

Explanation ▾

Adding the **-o yaml** flag to the **kubectl get** commands displays a complete description of the objects in YAML format. You can redirect this output to a file for later reuse if needed.

Step 4 ▾

Delete the **nginx** pod using the following command:

command

```
kubectl delete -f mypod.yaml
```

Tip ▾

You can also delete the pod using the imperative **kubectl delete pod nginx** command.

Task 2: Use the **kubectl create -f** to create pods

Step 1 ▾

Even when the **kubectl create** command doesn't allow you to create pods using something like **kubectl create pod**, you can add the **-f** flag to use a YAML file to create Kubernetes objects.

Reuse the **mypod.yaml** with the **nginx:1.28.0** image and create the pod using the following command:

command

```
kubectl create -f mypod.yaml
```

Step 2 ▾

Modify the **mypod.yaml** file and use the following content:

Explanation ▾

mypod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
labels:
```

```

    run: nginx
    name: nginx
spec:
  containers:
  - image: nginx:1.27.0
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}

```

The only change in the file is the image used for the pod. The new version uses **nginx:1.27.0**

Step 3 ▾

Try to recreate the pod using the image using the following command:

command

```
kubectl create -f mypod.yaml
```

Explanation ▾

Example output:

```

tux@management:~> kubectl create -f mypod.yaml
Error from server (AlreadyExists): error when creating "mypod.yaml":
pods "nginx" already exists

```

As `kubectl create` is not declarative, the Kubernetes reconciliation loop does not apply here, and Kubernetes does not modify the cluster state to match the new desired state for the pod. The **`kubectl create`** command is an imperative command, and it just runs the steps you order. As you tried to create an object with the same name of an existing one, it failed.

Exercise Summary

In this exercise, you learned how declarative commands are used by Kubernetes to match desired states with current states for Kubernetes objects, while imperative commands are used to tell Kubernetes exactly what actions to perform to create or modify objects step-by-step.

(End of Exercise)

Exercise 4.6 - Retrieve information using *kubectl get* and *kubectl describe*

Description

In a Kubernetes cluster, you can deploy many applications that will create many Kubernetes objects of different types. As the number of Kubernetes objects increases, you might not remember all the information about them, and the likelihood of having issues goes up. Being able to retrieve information about the objects helps in these situations.

In this lesson, you will learn about the `kubectl` commands used to retrieve information on Kubernetes objects.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Retrieve information about Kubernetes objects using **`kubectl get`** and its different options.
 - Retrieve detailed information about Kubernetes objects using **`kubectl describe`**.
-

Task 1: Use `kubectl get` to retrieve information from Kubernetes objects

Step 1 ▾

Retrieving information about Kubernetes objects in your cluster is one of the most common tasks as a Kubernetes administrator. The basic command for this operation is **`kubectl get`**, which can be used in many different ways, some of them are displayed below:

commands

```
> kubectl get pods
> kubectl get pods nginx
> kubectl get pods nginx -o yaml
> kubectl get pods nginx -o wide
> kubectl get pods nginx --show-labels
> kubectl get pods nginx -o wide --show-labels
```

Explanation ▾

- **`kubectl get pods`**: Returns basic information about all the pods in the current namespace. The default namespace for us.
- **`kubectl get pods nginx`**: Returns basic information about the **nginx** pod. Information such as the status, the age, and the number of containers or restarts.
- **`kubectl get pods nginx -o yaml`**: Returns the complete YAML description to recreate the pod.
- **`kubectl get pods nginx -o wide`**: Adds additional information about the **nginx** pod. Information such as the IP address or the node where the pod is running.
- **`kubectl get pods nginx --show-labels`**: Adds the labels defined in the metadata section of the **nginx** pod to the basic information.
- **`kubectl get pods nginx -o wide --show-labels`**: Combines and displays all the information of the two previous examples.

Example output:

```
tux@management:~> kubectl get pods nginx -o wide --show-labels
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE
READINESS GATES   LABELS
nginx     1/1     Running   0           56m   10.42.1.12    agent     <none>
run=nginx
```

Additional Information ▾

`kubectl get` has many options; you can define selectors, filters, and even select and format the information you want about the different Kubernetes objects.

Note ▾

In the **`kubectl get pods`** command, you can replace **pods** with any other resource type to retrieve information about that resource type. Resources such as Services or Deployments.

Task 2: Use `kubectl describe` to retrieve detailed information about Kubernetes objects and related events

Step 1 ▾

The **`kubectl describe`** command provides more structured and detailed information about the Kubernetes objects in your cluster. It also adds information about volumes, events, annotations, or tolerations. Use the following command to retrieve that kind of information from the **`nginx`** pod.

command

```
kubectl describe pods nginx
```

Example output:

```
tux@management:~> kubectl describe pod nginx
Name:          nginx
Namespace:     default
Priority:       0
Service Account: default
Node:          agent/172.30.170.4
Start Time:    Fri, 05 Dec 2025 10:07:49 +0100
Labels:        run=nginx
Annotations:    cni.projectcalico.org/containerID:
e03f078d27db6771d8a3948ac3fef9355dba93521a85384d72b55f6c071845c4
cni.projectcalico.org/podIP: 10.42.1.12/32
cni.projectcalico.org/podIPs: 10.42.1.12/32
Status:        Running
IP:            10.42.1.12
IPs:
  IP: 10.42.1.12
Containers:
  nginx:
    Container ID:  containerd://53182655aa46ac51e08952a35c93cf91ab504a721a7cb2974cfb3001804012bb
    Image:         nginx:1.28.0
    Image ID:      docker.io/library/nginx@sha256:3048d7a6bad5fblala2516249e570b85645a00411bad6b3126983577f040bb5f
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Fri, 05 Dec 2025 10:07:50 +0100
    Ready:         True
    Restart Count:  0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vkqnv (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers         True
  Initialized                       True
  Ready                             True
  ContainersReady                   True
  PodScheduled                      True
Volumes:
  kube-api-access-vkqnv:
```

```

Type:                Projected (a volume that contains injected data from multiple sources)
TokenExpirationSeconds: 3607
ConfigMapName:       kube-root-ca.crt
Optional:            false
DownwardAPI:         true
QoS Class:           BestEffort
Node-Selectors:       <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:              <none>

```

Note ▾

In the **kubectl describe pods** command, you can replace **pods** with any other resource type to retrieve information about that resource type. Resources such as Services or Deployments

Step 2 ▾

Delete the **nginx** pod running the following command:

command

```
kubectl delete pod nginx
```

Exercise Summary

In this exercise, you learned how to use **kubectl get** and its various flags to retrieve information about Kubernetes objects in different formats, including additional data such as labels. Later, you used the **kubectl describe** command to recover detailed information about a Kubernetes object.

(End of Exercise)

Section 5: Running workloads in Kubernetes

Description

Building on the Pod concepts introduced previously, this section delves deeper into Pod lifecycle management, introduces init and sidecar containers, and presents examples of the different Kubernetes objects you can create to manage Pods for real-world applications.

Section Learning Objectives

- Present the namespace concept and how to select the working namespace
- Understand Pod Abstraction: Recognize Pods as the workload unit and the need for higher-level concepts to manage Pod lifecycles.
- Manage stateless applications: Use Deployments to manage stateless applications.
- Handle stateful applications: Use StatefulSets to deploy them.
- Deploy cluster-wide applications: Use DaemonSets to ensure a single copy of a Pod runs on every node in the cluster.
- Execute Batch Processes: Differentiate between Jobs (one-time tasks) and CronJobs(scheduled tasks) and deploy them effectively.

Section Topics

- Namespaces to group Kubernetes objects
- Workloads and the relationship with pods
- ReplicaSets and Deployments for stateless applications
- StatefulSets for stateful applications
- DaemonSets to deploy applications on every cluster node
- Batch processes using Jobs and CronJobs

Exercise 5.1 - Manage namespaces in Kubernetes

Description

Namespaces are a basic Kubernetes object. They help to group and isolate Kubernetes resources in a cluster.

In this exercise, you will learn how to create, use, and delete namespaces and how to set a namespace as the default working namespace.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a namespace
 - Create pods within the fresh namespace
 - Retrieve the name of the working namespace
 - Change the default working namespace
 - Delete namespace
-

Task 1: Create a namespace**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select **RKE201-management** from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the other VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the other VMs via SSH.

Step 3 ▾

Open a terminal in the management VM and run the following commands to list all the available namespaces:

command

```
kubectl get ns
```

Explanation ▾

The **kubectl get ns** command lists all namespaces in the cluster. This command uses **ns** as a shorthand for **namespace**.

Example output:

```
tux@management:~> kubectl get ns
NAME                STATUS    AGE
default             Active   33m
kube-node-lease     Active   33m
kube-public         Active   33m
kube-system         Active   33m
```

Step 4 ▾

Create a new namespace by running the following command:

command

```
kubectl create ns test
```

Explanation ▾

The **kubectl create ns** command creates a new namespace (ns) with the name at the end (**test**).

Step 5 ▾

Run the following command to verify that the new namespace has been successfully created:

command

```
kubectl get ns
```

Example output:

```
Example output:
tux@management:~> kubectl get ns
NAME                STATUS    AGE
default             Active    33m
kube-node-lease     Active    33m
kube-public         Active    33m
kube-system         Active    33m
test                Active    10s
```

Task 2: Create pods within the new namespace**Step 1** ▾

Create a new pod in the new namespace by running the following command:

command

```
kubectl run pod-in-test-ns -n test --image=httpd
```

Explanation ▾

- **kubectl run:** kubectl command that imperatively creates a pod.
- **pod-in-test-ns:** Name for the new pod.
- **-n test:** Working namespace for the kubectl command. Sets the namespace where the pod is created. If the **-n** flag is omitted, the command defaults to the current working namespace.
- **-image=httpd:** Image used for the container. Httpd is the name for the official Apache web server image.

Step 2 ▾

Verify that the pod was successfully created in the test namespace by running the following command:

command

```
kubectl get pods -n test
```

Explanation ▾

- **kubectl get pods:** Retrieves the list of pods.
- **-n test:** Working namespace. Set the namespace for retrieving the list of pods from. If missed, the command will be executed in the default namespace.

Example output:

```
tux@management:~> kubectl get pods -n test
NAME          READY   STATUS    RESTARTS   AGE
pod-in-test-ns 1/1     Running   0           126m
tux@management:~> kubectl get pods
No resources found in default namespace.
```

Task 3: Set the new namespace as the working namespace**Step 1** ▾

Change the working namespace by running the following command:

command

```
kubectl config set-context $(kubectl config current-context) \
--namespace=test
```

Explanation ▾

- **kubectl config set-context:** Sets the context for the **kubectl** commands. A context is a combination of a Kubernetes cluster, a user, and a namespace. The **~/.kube/config** file stores authentication information about the different clusters and contexts used by **kubectl**. When y
- **\$(kubectl config current-context):** Retrieves information about the current context
- **--namespace=test:** Set the **test** namespace as the working namespace.

Step 2 ▾

Run the following to display the current context and its namespace:

command

```
kubectl config get-contexts
```

Example output:

```
tux@management:~> kubectl config get-contexts
CURRENT   NAME      CLUSTER   AUTHINFO   NAMESPACE
*          default   default   default    test
```

Explanation ▾

- **kubectl config get-contexts:** Lists all the contexts in the `~/.kube/config` file. The current context begins with the `*` character and show the working namespace at the end.

Step 3 ▾

List the pods for the working namespace by running the following command:

command

```
kubectl get pods
```

Explanation ▾

- **kubectl get pods:** Retrieves the list of pods. As the working namespace is the **test** namespace, the **pod-in-test-ns** appears in the list.

Example output:

```
tux@management:~> kubectl get pods -n test
NAME             READY   STATUS    RESTARTS   AGE
pod-in-test-ns   1/1     Running   0           126m
```

Step 4 ▾

Set the working namespace again to the default namespace:

command

```
kubectl config set-context $(kubectl config current-context) \
--namespace=default
```

Step 5 ▾

Delete the **test** namespace by running the following command:

command

```
kubectl delete ns test
```

Warning ▾

Deleting a namespace removes all the Kubernetes objects within it. This Kubernetes operation must be performed carefully. The **pod-in-test-ns**, previously created, is deleted when the test namespace is removed.

Task 4: Install kubectl and kubens to manage contexts and namespaces

Step 1 ▾

Switching between contexts and namespaces using **kubectl config** can be cumbersome. However, Kubernetes administrators usually install **kubectx** and **kubens**, two open source tools widely used by the Kubernetes community.

To install **kubectx** and **kubens** run the following commands:

commands

```
> sudo git clone https://github.com/ahmetb/kubectx /opt/kubectx
> sudo ln -s /opt/kubectx/kubectx /usr/local/bin/kubectx
> sudo ln -s /opt/kubectx/kubens /usr/local/bin/kubens
> sudo chmod +x /usr/local/bin/kubectx /usr/local/bin/kubens
```

Explanation ▾

- **sudo git clone https://github.com/ahmetb/kubectx /opt/kubectx** : Clones the tools repository.
- **sudo ln -s /opt/kubectx/kubectx /usr/local/bin/kubectx** and **sudo ln -s /opt/kubectx/kubens /usr/local/bin/kubens** : Adds the tools to the path by creating two symbolic links.
- **sudo chmod +x /usr/local/bin/kubectx /usr/local/bin/kubens** : Make the tools executable.

Step 2 ▾

Test the **kubens**, a tool for easily switching between namespaces, by running the following commands:

commands

```
> kubens
> kubens -c
> kubens kube-system
```

Explanation ▾

- **kubens**: Lists all the namespaces in the current Kubernetes clusters (context).
- **kubens -c**: Displays the working namespace.
- **kubens kube-system**: Sets the **kube-system** namespace as the working namespace. The **-n** flag is no longer needed to retrieve information or to create new Kubernetes objects in the namespace.

Example output:

```
tux@management:~> kubens
default
kube-node-lease
kube-public
kube-system
tux@management:~> kubens -c
default
tux@management:~> kubens kube-system
Context "default" modified.
Active namespace is "kube-system".
```

Exercise Summary

In this exercise, you learned how to manage namespaces in Kubernetes, set the working namespace for `kubectl`, and install third-party tools to make it easier to switch between namespaces or contexts.

(End of Exercise)

Exercise 5.2 - Deploy a Pod with an initContainer

Description

Kubernetes Pods can contain multiple containers. Those containers can be application containers, init containers, or sidecar containers. Init containers run tasks before application containers start, typically for initial configuration. Once the tasks are completed, application containers start.

In this exercise, you will learn how to create init containers and verify that application containers do not start until init containers have finished.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create init containers
 - Understand use cases for init containers.
 - Use init containers and verify their behavior.
-

Task 1: Create a Pod with an Init container

Step 1 ▾

Pods with more than one container can not be created using CLI commands. You will use a Kubernetes manifest to deploy a pod using an init container. Review the YAML manifest by running the following command:

command

```
cat ~/course_files/RKE201v2/labs/manifests/pod-examples/pod-with-init.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-init
spec:
  initContainers:
  - name: asset-downloader
    image: busybox
    command:
    - sh
    - -c
    - |
      echo "Downloading web assets..."
      for i in 1 2 3 4 5; do
        echo "Downloading package $i of 5..."
        sleep 5
      done
      echo "Download completed."
      touch /workdir/ready.txt
    volumeMounts:
    - name: assets
      mountPath: /workdir
  containers:
  - name: web-server
    image: nginx
    volumeMounts:
    - name: assets
      mountPath: /usr/share/nginx/html
  volumes:
  - name: assets
    emptyDir: {}
```

Explanation ▾

The **spec** section for the Pod containers has two subsections: The **initContainers** section and the **containers** section.

- The **containers** section contains the **web-server** container, which uses the **nginx** web server image.
- The **initContainers** section contains the **asset-downloader** container. This container:
 - Uses the **busybox** image.
 - Runs a shell script that runs for 25 seconds and simulates downloading assets needed by the web server.
 - When this process completes, the **web-server** container starts.
- The **asset-downloader** and the **web-server** container use the **assets** volume to share files. The volume, using **volumeMounts**, is mounted in **/workDir** for the Init containers and in the **/usr/share/nginx/html** folder for the **webserver** containers. Volumes and storage for Kubernetes are covered later in this course.

Step 2 ▾

In a separate terminal, run the following command to watch the running pods:

command

```
watch kubectl get pods
```

Step 3 ▾

Change the working directory to the folder containing the examples and create the pod by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples/
> kubectl apply -f pod-with-init.yaml
```

Explanation ▾

In the terminal running the **watch** command, you will see a pod named **pod-with-init**. The STATUS column will initially show **Init:0/1** while the init container is running. When the init container completes, the STATUS will change to **Running**, and the READY column will show **1/1**.

Step 4 ▾

Run the following command to display more detailed information about the **pod-with-init** pod:

command

```
kubectl describe pod pod-with-init
```

Note ▾

Notice the information about the containers in the **Init Containers:** and **Containers:** sections in the output.

Step 5 ▾

Run the following command to delete the **pod-with-init** Pod:

command

```
kubectl delete pod pod-with-init
```

Explanation ▾

The command removes the pod imperatively. You could also use **kubectl delete -f pod-with-init.yaml**.

Exercise Summary

In this exercise, you learned how to create an init container in a Pod, verified that the task in this container is executed before the application containers, and discovered a typical use case for init containers: downloading required assets for the application container before the application container starts.

(End of Exercise)

Exercise 5.3 - Deploy a Pod with a sidecar Container

Description

In this exercise, you will explore multi-container Pods by creating a Pod with a sidecar container. Sidecar containers provide additional secondary or infrastructure services within the Pod.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create sidecar containers
 - Understand use cases for sidecar containers.
 - Use sidecar containers and verify their behavior.
-

Task 1: Create a Pod with a sidecar container

Step 1 ▾

Review the YAML manifest for the Pod using a sidecar container by running the following command:

command

```
cat ~/course_files/RKE201v2/labs/manifests/pod-examples/pod-with-sidecar.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server-sidecar
spec:
  containers:
    - name: web-server
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - name: html-content
          mountPath: /usr/share/nginx/html
          readOnly: true

    - name: content-sync
      image: busybox
      command: ["sh", "-c"]
      args:
        - |
          while true; do
            echo "<html><body><h1>Updated on: $(date)</h1></body></html>"
          > /data/index.html
            echo "Synchronized"
            sleep 10;
          done
      volumeMounts:
        - name: html-content
          mountPath: /data

  volumes:
    - name: html-content
      emptyDir: {}
```

Explanation ▾

The **spec** section for the Pod containers has two container definitions in the **containers** section:

- The **web server container** uses the **nginx** web server image.

- The **content-sync** container:
 - Uses the **busybox** image.
 - Runs a shell script that updates the web server content every 10 seconds. In a production environment, a sidecar container could sync content from a Git repository at a set interval. It is a typical use case for sidecar containers: separation of responsibilities. The **web-server** container serves the web, and the **content-sync** container keeps the content updated.
 - This container runs as long as the Pod runs.
- The **content-sync** and the **web-server** container use the **html-content** volume to share files.

Step 2 ▾

Change the working directory to the folder containing the examples and create the pod by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples/
> kubectl apply -f pod-with-sidecar.yaml
```

Step 3 ▾

Run the following command to verify that the Pods have been successfully created:

command

```
kubectl get pods web-server-sidecar
```

Explanation ▾

The output displays 2/2 in the READY column, meaning that both containers in the Pod are running correctly.

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples> k
get pods web-server-sidecar
```

NAME	READY	STATUS	RESTARTS	AGE
web-server-sidecar	2/2	Running	0	83s

Step 4 ▾

Run the following command to display more detailed information about the **web-server-sidecar**:

command

```
kubectl describe pod web-server-sidecar
```

Note ▾

Notice the information about both containers in the **Containers:** section in the output.

Step 5 ▾

Run the following command to delete the **pod-with-init** Pod:

command

```
kubectl delete -f pod-with-sidecar
```

Explanation ▾

The command removes the pod declaratively. You could also use **kubectl delete pod web-server-sidecar**

Exercise Summary

In this exercise, you learned how to add a sidecar container in a Pod, verified that the sidecar container runs as long as the application container runs, and discovered a typical use case for sidecar containers: synchronize the content served by application containers.

(End of Exercise)

Exercise 5.4 - Manage Deployments in Kubernetes

Description

Self-healing, scaling, rollouts, and rollbacks are container orchestrator capabilities offered by Kubernetes. Previously, you deployed Pods as standalone units, which lack these orchestration capabilities. Moving forward, we will introduce the resources and objects provided by Kubernetes to achieve these objectives.

In this exercise, you will learn to manage stateless applications using Deployments.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Manage stateless applications in Kubernetes using Deployments
 - Understand the relationship between Deployments, Replicas, and Pods.
 - Use the `kubectl rollout` command to manage the update lifecycle, including rollbacks.
-

Task 1: Create a deployment using a mixed imperative and declarative approach**Step 1** ▾

Run the following command to get inline help on creating a Kubernetes deployment:

command

```
kubectl create deployment --help
```

Step 2 ▾

Copy the third example provided by the previous command and use the example to create a YAML file for the deployment:

command

```
kubectl create deployment my-dep --image=nginx --replicas=3 \
  --dry-run=client -o yaml > my-deployment.yaml
```

Explanation ▾

The previous command:

- Creates a Kubernetes deployment named **my-dep**.
- **--replicas=3**: Sets the number of replicas to 3. The command will create 3 pods managed by a replicaset.
- **--image=nginx**: Uses the nginx container image for all the replicas.
- **--dry-run=client**: Runs the command without sending the request to the api server. The command does not affect the cluster.
- **-o yaml > my-deployment.yaml**: Redirects the outputs to a file in YAML format.

Step 3 ▾

Review the content of the **my-deployment.yaml** file by running **cat my-deployment.yaml**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: my-dep
    name: my-dep
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-dep
  strategy: {}
  template:
    metadata:
```

```

    creationTimestamp: null
    labels:
      app: my-dep
  spec:
    containers:
      - image: nginx
        name: nginx
        resources: {}
  status: {}

```

Explanation ▾

- **apiVersion: apps/v1:** This field has a new value. For pods, the value is **v1**.
- **replicas: 3:** Sets the number of replicas to 3.
- **selector:** This property selects the pods belonging to the deployment. The label in the selector **app**, **my-dep**, must match the label in the pod template definition.
- **strategy: {}:** The **strategy** used by the deployment for the update processes. Since it is empty, the default strategy, rolling update, is used. The result would be the same if used in this way:

```

spec:
...
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
...

```

- **template:** It contains a typical pod definition with the **metadata** and **spec** sections.

Step 4 ▾

Create the deployment by running the following command:

command

```
kubectl apply -f my-deployment.yaml
```

Explanation ▾

This command creates the deployment declaratively using the YAML file.

Step 5 ▾

Retrieve information about the Kubernetes objects created by the deployment by running this command:

command

```
kubectl get all
```

Explanation ▾

This is the first time in the course that you use this command. This command does not return all Kubernetes objects; it only returns information about pods, services, deployments, replicaset, statefulsets, daemonsets, jobs, and cronjobs. This is why it is quite helpful for this section.

Example output:

NAME	READY	STATUS	RESTARTS	AGE
pod/my-dep-7565875b9c-n2zdt	1/1	Running	0	16s
pod/my-dep-7565875b9c-r7xkc	1/1	Running	0	16s
pod/my-dep-7565875b9c-x6jd5	1/1	Running	0	16s
pod/web-server-sidecar	2/2	Running	0	119m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP
AGE				3h55m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/my-dep	3/3	3	3	16s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/my-dep-7565875b9c	3	3	3	16s

Note ▾

Pay attention to the naming convention for deployments, replicaset, and pods:

- **my-dep:** It is the name of the deployment. Deployments manage replicaset.
- **my-dep-7565875b9c:** It is the name of the replicaset. It starts with the name of the deployment it belongs to. Replicaset manage the pods and keep their number stable.
- **my-dep-7565875b9c-XXXXX:** The name of the pods starts with the name of the replica owning them, and adds additional IDs for each pod.

Warning ▾

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Task 2: Verify how replicaset keep the number of pods stable

Step 1 ▾

In a separate terminal, run the following command to watch the running pods:

command

```
watch kubectl get pods
```

Step 2 ▾

Delete two replicas in the deployment by running a command similar to the one below, and watch the ReplicaSet spin up two new replicas to keep the number of replicas as specified in the manifest:

command

```
kubectl delete pods my-dep-7565875b9c-n2zdt my-dep-7565875b9c-r7xkc
```

Warning ▾

Replace the exact pod names with the names on your computer.

Task 3: Manage the deployment lifecycle

Step 1 ▾

In a separate terminal, run the following command to watch the running pods:

command

```
watch kubectl get pods
```

Step 2 ▾

Edit the running deployment by running this command:

command

```
kubectl edit deployments.apps my-dep
```

Explanation ▾

This is the first time in the course that you use the **kubectl edit** command. This command displays the YAML for an existing Kubernetes object in the default editor (vim). Once the content displays, you can modify the object as desired.

Warning ▾

The **kubectl edit** command is imperative and does not modify the YAML file used to create the deployment.

Step 3 ▾

Use the editor and replace the image for the deployment in the template section. Use the following line and save :

```
- image: nginx:1.28.0
```

Explanation ▾

Once you save the file, in the terminal running the watch command, you will see pods in the deployment replaced one by one with pods using the new image version. The deployment is being updated using the default strategy: Rolling update. You can use one of the following commands to display the image used by the deployment:

command

```
kubectl get deploy my-dep -o wide
kubectl describe deploy my-dep
```

Step 4 ▾

Review the deployment lifecycle using the following commands :

commands

```
kubectl rollout status deployment my-dep
kubectl rollout history deployment my-dep
kubectl rollout history deployment my-dep -o yaml
```

Explanation ▾

The **kubectl rollout** command helps to manage the update lifecycle, including rollbacks.

- **kubectl rollout status deployment my-dep:** Displays the status of the last update process for the deployment.
- **kubectl rollout history deployment my-dep:** Displays the deployment history. In our case, it has two revisions: the original and the one using nginx:1.28.0
- **kubectl rollout history deployment my-dep -o yaml:** Display the YAML for each revision.

Example output:

```
tux@management:~> kubectl rollout status deployment my-dep
deployment "my-dep" successfully rolled out
tux@management:~> kubectl rollout history deployment my-dep
deployment.apps/my-dep
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

Step 5 ▾

Use **kubectl edit** again and replace the image for the deployment with the wrong image below and save:

```
- image: nginx:1.50.0
```

Explanation ▾

As you are using a wrong image for the new deployment revision, if you watch the terminal running the **watch** command, you can see how a new container appears and fails, stopping the update lifecycle.

If you run **kubectl rollout status deployment my-dep** you see how the update process gets stuck in ***Waiting for deployment "my-dep" rollout to finish: 1 out of 3 new replicas have been updated...*** and, if you run **kubectl rollout history deployment my-dep**, three revisions display.

Step 6 ▾

Rollback the last update in the deployment by running the following command:

command

```
kubectl rollout undo deployment my-dep --to-revision=2
```

Explanation ▾

The **kubectl rollout undo** command allows you to rollback your deployment to a previous revision.

- **--to-revision=2**: Performs a rollback to revision two. This flag is optional. If not used, the deployment reverts to the latest revision, which, in the exercise, is also revision number 2.

Use **kubectl get** or **kubectl describe** to display the deployment's version, as explained earlier. It must be **nginx:1.28.0**.

If you review the deployment history using **kubectl rollout history**, you can realize how the older revision has disappeared, and now it display 4 revisions in the history.

Example output:

```
tux@management:~> kubectl rollout history deployment my-dep
deployment.apps/my-dep
REVISION  CHANGE-CAUSE
1          <none>
```

```
3      <none>
4      <none>
```

Step 7 ▾

Delete the deployment imperatively by running the :

command

```
kubectl delete deployments.apps my-dep
```

Additional Information ▾

Review the Kubernetes official documentation for additional information on [Deployments](#) and the [kubectl rollout](#) command.

Exercise Summary

In this exercise, you have learned how to deploy stateless applications using Kubernetes Deployments, understood the relationship between Deployments, Replicasets, and Pods, and managed Deployments' update lifecycle using the `kubectl rollout` command.

(End of Exercise)

Exercise 5.5 - Manage StatefulSets in Kubernetes

Description

Deployments are for stateless applications that do not store information between requests. However, stateful applications remember the history of interactions. Kubernetes provides stateful sets for stateful applications, and pods managed by a statefulset maintain an identity.

In this exercise, you will learn how to create a StatefulSet and understand how pods within the StatefulSet keep their identity even when they are being replaced.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Stateful set.
 - Understand the relationship between statefulsets, replicaset, and pods.
 - Realize how the pods managed by a StatefulSet keep their identity.
-

Task 1: Review a YAML manifest for a StatefulSet**Step 1** ▾

Run the following command to display the manifest for the StatefulSet used in this exercise:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat redis-statefulset.yaml
```

Example output:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis-node
spec:
  updateStrategy:
    rollingUpdate:
      maxUnavailable: 1
    type: RollingUpdate
  replicas: 5
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7.0-alpine
          command: ["redis-server", "--appendonly", "yes"]
          ports:
            - containerPort: 6379
              name: redis
```

Explanation ▾

YAML manifests for statefulsets are quite similar to deployment manifests. Except for the **kind** property and the **updateStrategy** syntax, the manifest is similar to a Deployment. The statefulset uses a rolling update strategy, which is also the default. Providing permanent storage for them would add new changes, but Kubernetes storage is covered later in this course.

Step 2 ▾

In a separate terminal, run the following command to watch the running pods:

command

```
watch kubectl get pods
```

Step 3 ▾

Create the statefulset by running the following command:

command

```
kubectl apply -f redis-statefulset.yaml
```

Step 4 ▾

Review the Kubernetes objects created for the StatefulSet by running the following command :

command

```
kubectl get all
```

Example output:

NAME	READY	STATUS	RESTARTS	AGE
pod/redis-node-0	1/1	Running	0	25m
pod/redis-node-1	1/1	Running	0	25m
pod/redis-node-2	1/1	Running	0	25m
pod/redis-node-3	1/1	Running	0	25m
pod/redis-node-4	1/1	Running	0	25m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP

NAME	READY	AGE
statefulset.apps/redis-node	5/5	25m

Explanation ▾

StatefulSets do not create ReplicaSets. Instead, they manage Pods directly, assigning them a sticky identity (ordered). As we have 5 replicas, pods to build their name, add a number from 0 to 5 to the statefulset's name.

Task 2: Remove pods from the statefulset**Step 1** ▾

Run the following command to delete pods 0, 1, and 2, and switch to the terminal running the watch command, and verify that the StatefulSet replaces the pods while maintaining identity.

command

```
kubectl delete pod redis-node-0 redis-node-1 redis-node-2
```

Task 3: Update the statefulset and change the image used by the pods**Step 1** ▾

Edit the YAML manifest for the statefulset and replace the image used by its pods. Use ***redis:8.4-alpine*** instead of ***redis:7.0-alpine***.

Step 2 ▾

Run the following command to apply the changes, and switch to the terminal running the watch command, and verify that the statefulset replaces the pods while maintaining identity.

command

```
kubectl apply -f redis-statefulset.yaml
```

Note ▾

By default, statefulsets, as deployments, use the rolling upgrade strategy.

Step 3 ▾

Review the statefulset history by running the following command:

command

```
kubectl rollout history statefulset redis-node
```

Explanation ▾

The **kubectl rollout** command works with statefulsets as it does with deployments. Using that command, you can manage the update application lifecycle and roll back changes if needed.

Example output:

```
tux@management:~> kubectl rollout history statefulset redis-node
statefulset.apps/redis-node
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

Note ▾

Using **kubectl describe pod redis-node-0**, you can verify that the StatefulSet is using the new image for the pods.

Step 4 ▾

Delete the StatefulSet by running the following imperative command:

command

```
kubectl delete statefulset redis-node
```

Exercise Summary

In this exercise, you have learned how to create stateful applications in Kubernetes using StatefulSets and realized the differences and similarities between Deployments and StatefulSets.

(End of Exercise)

Exercise 5.6 - Manage DaemonSets in Kubernetes

Description

Daemonsets are another way to manage pods in Kubernetes. A DaemonSet ensures that a copy of the defined Pod runs on all (or a selected subset) of nodes in the cluster.

In this exercise, you will create and manage daemonsets for a typical use case: Log collectors.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create DaemonSets in your Kubernetes cluster.
 - Understand how DaemonSets are deployed in the cluster.
 - Discover a typical use case for DaemonSets: Log collectors.
-

Task 1: Review a YAML manifest for a DaemonSet

Step 1 ▾

Run the following command to display the manifest for the used in this exercise:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat fluentd-daemonset.yaml
```

Example output:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
  namespace: kube-system
spec:
  selector:
    matchLabels:
      name: collector
  template:
    metadata:
      labels:
        name: collector
    spec:
      containers:
        - name: fluentd
          image: fluent/fluentd:v1.16-debian-1
          volumeMounts:
            - name: varlog
              mountPath: /var/log
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

Explanation ▾

This YAML manifest for the DaemonSet is similar to manifests for Deployments and StatefulSets. However, you don't have any information about replicas or strategies, since only one replica is started on each node, and if the replica fails, it is replaced.

The YAML also defines a **hostPath** volume, which is mounted to the container via **volumeMounts**. These properties map the *host's /var/log folder* to the *container's /var/log folder*.

Since the **fluentd** container is a log collector, the **namespace: kube-system** is defined in the metadata section and is used to run the pods in that namespace.

Step 2 ▾

Create the daemon by running the following command:

command

```
kubectl apply -f fluentd-daemonset.yaml
```

Step 3 ▾

Display the node where the Pods for the DaemonSet have been deployed using the following command:

command

```
kubectl get pods -n kube-system -o wide
```

Explanation ▾

- **-o wide:** Displays additional information for the Pods, as the node where the Pods are running.
- **-n kube-system:** Displays only Pods running in the **kube-system** namespace.

This command must display two **log-collector-XXXX** Pods. Since the cluster has two nodes (agent and server), one Pod must run in each cluster.

Task 3: Update the DaemonSet and change the image used by the pods**Step 1** ▾

Edit the YAML manifest for the DaemonSet and replace the image used by its Pods. Use **fluent/fluentd:v1.19-debian-1** instead of **fluent/fluentd:v1.16-debian-1**.

Step 2 ▾

Run the following command to apply the changes:

command

```
kubectl apply -f fluentd-daemonset.yaml
```

Step 3 ▾

Review the DaemonSet history by running the following command:

command

```
kubectl rollout history daemonset -n kube-system log-collector
```

Explanation ▾

The **kubectl rollout** command works with DaemonSets as it does with deployments and StatefulSets. Using that command, you can manage the update application lifecycle and roll back changes if needed.

Example output:

```
tux@management:~> kubectl rollout history daemonset -n kube-system
log-collector
daemonset.apps/log-collector
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

Note ▾

Using **kubectl describe pod -n kube-system log-collector-XXXX**, you can verify that the DaemonSet is using the new image for the pods.

Step 4 ▾

Delete the DaemonSet by running the following imperative command:

command

```
kubectl delete daemonset -n kube-system log-collector
```

Exercise Summary

In this exercise, you have learned how to use DaemonSets to deploy applications across all the cluster nodes, and realized the differences and similarities between Deployments, StatefulSets and DaemonSets.

(End of Exercise)

Section 6: Configuration for Kubernetes Objects

Description

Real-world applications require different decouple applications and their configurations. A tight coupling between them can be risky from a security perspective and inefficient. Kubernetes provides two objects for decoupling applications and their configuration: ConfigMaps and Secrets.

Section Learning Objectives

- Understand decoupling: Separate configuration from code to improve portability and security.
- Manage ConfigMaps.
- Manage Secrets.
- Inject configuration data: Integrate ConfigMaps into pods using different methods.
- Inject sensitive data: Integrate Secrets into pods using different methods.
- Analyze security: Explain the limitations of Secrets security, where base64 is not encryption, and the need for external solutions.

Section Topics

- Configuration decoupling.
- Managing configurations: ConfigMaps.
- Managing sensitive information: Secrets.
- Security considerations.

Exercise 6.1 - Create ConfigMaps in Kubernetes

Description

ConfigMaps allow you to decouple applications from their configuration. This facilitates adding parameters for different environments (testing, staging, production) without rebuilding container images.

In this exercise, you will learn how to use different options for creating ConfigMaps in Kubernetes.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create ConfigMaps using the **kubecttl create** command.
 - Create ConfigMaps retrieving configuration values and parameters from a file.
 - Inspect ConfigMaps once they are created.
-

Task 1: Creating a ConfigMap using literals

Step 1 ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select RKE201-management from the VM list in Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

The RKE201-management VM runs services for the other Lab VMs and must remain running. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and use the following command to create a ConfigMap:

command

```
kubectl create configmap db-configuration \
  --from-literal=DB_TIMEOUT=60s --from-literal=DB_MAX_CONNECTIONS=100 \
  --from-literal=DB_IDLE_TIMEOUT=5m
```

Explanation ▾

- **kubectl create configmap db-configuration:** Creates a ConfigMap named *db-configuration* from the command line.
- **--from-literal=DB_TIMEOUT=60s:** Creates an entry in the ConfigMap using *DB_TIMEOUT* as the key and *60s* as the value.
- **--from-literal=DB_MAX_CONNECTIONS=100:** Creates an entry in the ConfigMap using *DB_MAX_CONNECTIONS* as the key and *100* as the value.
- **--from-literal=DB_IDLE_TIMEOUT=5m:** Creates an entry in the ConfigMap using *DB_IDLE_TIMEOUT* as the key and *5m* as the value.

Step 4 ▾

Use **kubectl describe** to display a detailed description of the ConfigMap:

command

```
kubectl describe cm db-configuration
```

Example output:

```
tux@management:~> kubectl describe cm db-configuration
Name:          db-configuration
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
DB_MAX_CONNECTIONS:
----
100

DB_TIMEOUT:
----
60s

DB_IDLE_TIMEOUT:
----
5m

BinaryData
=====
```

Explanation ▾

kubectl describe provides detailed information about Kubernetes objects. For ConfigMaps, keys and values display under the Data section.

Task 2: Create a ConfigMap using the dry-run=client flag

Step 1 ▾

Use the following command to create the YAML file for the ConfigMap:

command

```
kubectl create cm api-configuration --from-literal=LOG_LEVEL=4 \
--from-literal=MAX_RETRIES=10 \
--from-literal=URL=https://myapiserver/v2 --dry-run=client -o yaml \
> my-apiconf-cm.yaml
```

Explanation ▾

- **kubectl create cm api-configuration:** Creates a configmap named *api-configuration* from the command line. It use **cm** as the shorthand for ConfigMap.

- **--from-literal=LOG_LEVEL=4**, **--from-literal=MAX_RETRIES=10**, and **--from-literal=URL=https://myapiserver/v2**: Creates three entries in the ConfigMap.
- **--dry-run=client -o yaml > my-apiconf-cm.yaml**: Redirects the YAML output to the yaml file without creating the ConfigMap.

Step 2 ▾

Display and review the YAML using the following command:

command

```
cat my-apiconf-cm.yaml
```

Expected output:

```
apiVersion: v1
data:
  LOG_LEVEL: "4"
  MAX_RETRIES: "10"
  URL: https://myapiserver/v2
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: api-configuration
```

Explanation ▾

Keys and values from the literals are displayed in the **data** section.

Step 3 ▾

Create the ConfigMap by running the following command:

command

```
kubectl apply -f my-apiconf-cm.yaml
```

Step 4 ▾

Retrieve all the ConfigMaps in the working namespace by running the following command:

command

```
kubectl get cm
```

Expected output:

```
tux@management:~> kubectl get cm
NAME          DATA   AGE
```

api-configuration	3	28s
db-configuration	3	44m
kube-root-ca.crt	1	3d

Task 2: Create a ConfigMap using a file

Step 1 ▾

Review the file that will be used to create the ConfigMap by running the following commands:

commands

```
cd ~/course_files/RKE201v2/labs/miscellaneous
cat myapp.conf
```

Expected output:

```
server {
    listen 80;
    server_name myapp.com;
    location / {
        proxy_pass http://localhost:8080;
        add_header X-Config-Source "Kubernetes-ConfigMap";
    }
}
```

Step 2 ▾

Use the following command to create the YAML file for the ConfigMap:

command

```
kubectl create cm myapp-conf --from-file=./myapp.conf \
--dry-run=client -o yaml > my-appconf-cm.yaml
```

Step 3 ▾

Display and review the YAML using the following command:

command

```
cat my-appconf-cm.yaml
```

Expected output:

```
apiVersion: v1
data:
  myapp.conf: "server {\n    listen 80;\n    server_name myapp.com;\n  location\n    / {\n        proxy_pass http://localhost:8080;\n        add_header\nX-Config-Source
```

```

    \ "Kubernetes-ConfigMap\";\n    }\n}    \n"
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: myapp-conf

```

Explanation ▾

The ConfigMap YAML contains a single key named after the filename (myapp.conf), with the file contents as the value.

Step 4 ▾

Create the ConfigMap by running the following command:

command

```
kubectl apply -f my-appconf-cm.yaml
```

Step 5 ▾

Retrieve all the ConfigMaps in the working namespace by running the following command:

command

```
kubectl get cm
```

Expected output:

```

tux@management:~/course_files/RKE201v2/labs/miscellaneous> kubectl get
cm
NAME                               DATA  AGE
api-configuration                   3      28m
db-configuration                    3      72m
kube-root-ca.crt                   1      3d1h
myapp-conf                         1      13s

```

Exercise Summary

In this exercise, you have created ConfigMaps using literal values from the command line, using a YAML file, and using file content.

(End of Exercise)

Exercise 6.2 - Use ConfigMaps in Pods

Description

Once ConfigMaps have been created, they can be injected and used in pods in several different ways. In this exercise, you will inject ConfigMap values as environment variables and use volumes to load ConfigMaps into Pods.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Load a single value in a ConfigMap as a Pod environment variable.
 - Load all values in a ConfigMap as environment variables.
 - Use volumes to load the values of a ConfigMap.
-

Task 1: Load a single ConfigMap entry as an environment variable in a Pod.**Step 1** ▾

Run the following commands to review the Pod definition:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat configmap-single-value.yaml
```

Example output

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-single-value-pod
spec:
  containers:
    - name: demo
      image: alpine
      command: ["sleep", "3600"]
      env:
        - name: TIMEOUT
          valueFrom:
            configMapKeyRef:
              name: db-configuration
              key: DB_TIMEOUT
```

Explanation ▾

This YAML describes a Pod that:

- Uses **alpine** as the container image.
- Runs the **sleep 3600** command once the container starts.
- Configures the container environment so that:
 - **TIMEOUT**: Name for the Pod's environment variable.
 - **valueFrom**: Defines where the value for the variable is coming from.
 - **configMapKeyRef**: Refers to a ConfigMap and a key to retrieve values from them. In the example, it refers to the **db-configuration** Configmap and the **DB_TIMEOUT** key. The value for that key is stored in the Pod's **TIMEOUT** environment variable.

Step 2 ▾

Create the Pod using the ConfigMap by running the following command:

command

```
kubectl apply -f configmap-single-value.yaml
```

Step 3 ▾

Open a shell terminal in the container by running the following command:

command

```
kubectl exec -it configmap-single-value-pod -- sh
```

Explanation ▾

This is the first time we have used the **kubectl exec** command in this course. This command directly executes commands in a pod or container.

- **-it**: It used to provide interactivity.
- **configmap-single-value-pod**: The name of the Pod.
- **-sh**: Launches a shell.

Note ▾

Note the shell prompt change, indicating you are running commands inside the container.

Step 4 ▾

Display the value for the **TIMEOUT** environment variable by running the following command:

command

```
echo $TIMEOUT
```

Example output:

```
/ # echo $TIMEOUT
60s
/ #
```

Step 5 ▾

Run **exit** inside the container and delete the Pod by running the following command:

command

```
kubectl delete -f configmap-single-value.yaml
```

Example output:

```
/ # exit
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl delete -f configmap-single-value.yaml
pod "configmap-single-value-pod" deleted
```

Task 2: Load a complete ConfigMap as a collection of environment values in a Pod

Step 1 ▾

Run the following commands to review the Pod definition:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat configmap-complete.yaml
```

Example output

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-complete-pod
spec:
  containers:
    - name: app
      command: ["sleep", "3600"]
      image: busybox
      envFrom:
        - configMapRef:
            name: api-configuration
```

Explanation ▾

The previous YAML describes a Pod with the following characteristics:

- Use **busybox** as the container image.
- Runs the **sleep 3600** command once the container starts.
- Populates the environment by loading all key-value pairs from a ConfigMap:
 - **envFrom:** Defines where values for the variables are coming from.
 - **configMapRef:** Refers to a ConfigMap. All key-value pairs in the ConfigMap **api-configuration** are loaded as environment variables in the Pod.

Step 2 ▾

Create the Pod using the ConfigMap by running the following command:

command

```
kubectl apply -f configmap-complete.yaml
```

Step 3 ▾

Open a shell terminal in the container by running the following command:

command

```
kubectl exec -it configmap-complete-pod -- sh
```

Step 4 ▾

Display all the environment variables by running the following command:

command

```
env
```

Example output:

```
/ # env
...
LOG_LEVEL=4
HOSTNAME=configmap-complete
URL=https://myapiserver/v2
...
MAX_RETRIES=10
...
/ #
```

Explanation ▾

All the **api-configuration** ConfigMap key-value pairs (**LOG_LEVEL**, **URL**, **MAX_RETRIES**) are stored as environment variables in the Pod.

Step 5 ▾

Run **exit** inside the container and delete the Pod by running the following command:

command

```
kubectl delete -f configmap-complete.yaml
```

Task 3: Load a complete ConfigMap as a volume in a Pod

Step 1 ▾

Run the following commands to review the Pod definition:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat configmap-volume.yaml
```

Example output

```
apiVersion: v1
kind: Pod
```

```

metadata:
  name: configmap-volume-pod
spec:
  containers:
    - name: myapp
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/nginx/conf.d
  volumes:
    - name: config-volume
      configMap:
        name: myapp-conf

```

Explanation ▾

The previous YAML describes a Pod with the following characteristics:

- Use **nginx** as the container image.
- Creates a volume named **config-volume** that:
 - Uses the **myapp-conf** ConfigMap as the volume source. Each key-value entry in the ConfigMap becomes a file when mounted. The ConfigMap has only a key-value pair, which uses **myapp.conf** as the name, so only a file will be created.
 - **volumeMounts** mounts the volume in the **/etc/nginx/conf.d** folder.

Step 2 ▾

Create the Pod using the ConfigMap by running the following command:

command

```
kubectl apply -f configmap-volume.yaml
```

Step 3 ▾

Open a shell terminal in the container by running the following command:

command

```
kubectl exec -it configmap-volume-pod -- cat \
/etc/nginx/conf.d/myapp.conf
```

Explanation ▾

This **kubectl exec** command does not launch a shell, it just runs the **cat** command required to display the content of the file.

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl exec -it configmap-volume-pod -- cat
/etc/nginx/conf.d/myapp.conf
server {
    listen 80;
    server_name myapp.com;
    location / {
        proxy_pass http://localhost:8080;
        add_header X-Config-Source "Kubernetes-ConfigMap";
    }
}
```

Step 5 ▾

Delete the Pod by running the following command:

command

```
kubectl delete -f configmap-volume.yaml
```

Exercise Summary

In this exercise, you used ConfigMap values in Pods: either loading a key-value pair or all key-value pairs as environment variables, or mounting a ConfigMap as a volume.

(End of Exercise)

Exercise 6.3 - Create Secrets in Kubernetes

Description

While ConfigMaps provide configuration options, they should not be used to provide confidential information to Pods. Kubernetes provides secrets for injecting confidential information into Pods. In this exercise, you will learn how to create secrets in Kubernetes and how to inject those secrets into pods as environment variables or volumes.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create basic-authentication and TLS secrets.
 - Inject the basic-authentication secret information as environment variables in a Pod.
 - Use the TLS secret using volumes in a Pod.
-

Task 1: Creates a Secret of type 'generic' for basic authentication.**Step 1** ▾

Run the following command to create a basic authentication secret:

command

```
kubectl create secret generic wp-secret \
--from-literal=username=admin --from-literal=password=admin \
--dry-run=client -o yaml > wp-secret.yaml
```

Explanation ▾

- ***kubectl create secret generic wp-secret:*** Creates a basic authentication secret (generic).
- ***--from-literal=username=admin*** and ***--from-literal=password=admin:*** Define two key-value pairs for the secret.
- ***--dry-run=client -o yaml > wp-secret.yaml:*** Redirects the YAML output to the ***wp-secret.yaml*** file without creating it.

Step 2 ▾

Display the content of the YAML file by running the following command:

command

```
cat wp-secret.yaml
```

Example output:

```
apiVersion: v1
data:
  password: YWRtaW4=
  username: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: null
  name: wp-secret
```

Explanation ▾

Values for the ***password*** and ***username*** keys are base64-encoded but not encrypted. Run the following command to unveil the secret:

command

```
echo "YWRtaW4=" | base64 -d | xargs
```

Example output:

```
tux@management:~> echo "YWRtaW4=" | base64 -d | xargs
```



```
admin
```

Step 3 ▾

Run the following command to create the secret:

command

```
kubectl apply -f wp-secret.yaml
```

Step 4 ▾

Retrieve detailed information about the secret using the following command:

command

```
kubectl describe secret wp-secret
```

Example output:

```
tux@management:~> kubectl describe secrets wp-secret
Name:          wp-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  Opaque

Data
====
password:  5 bytes
username:  5 bytes
```

Explanation ▾

kubectl describe does not display the secret value. The Type appears as **Opaque**, even though it contains basic authentication data. Basic authentication is a convenience name for opaque secrets, which is the default value for a secret type.

Task 2: Create a TLS Secret in Kubernetes.

Step 1 ▾

TLS secrets in Kubernetes require the **type** property and the certificate and key files in the data section. You can create a YAML file for a TLS secret using the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/certs
> kubectl create secret tls wp-tls-secret --cert=./server.crt \
  --key=./server.key --dry-run=client -o yaml > wp-tls-secret.yaml
```

Step 2 ▾

Display the content of the YAML file by running the following command:

command

```
cat wp-tls-secret.yaml
```

Example output:

```
apiVersion: v1
data:
  tls.crt: LS0tLS1CRUdJTiBDRV...
  tls.key: LS0tLS1CRUdJTiBQUk...
kind: Secret
metadata:
  creationTimestamp: null
  name: wp-tls-secret
type: kubernetes.io/tls
```

Explanation ▾

Key-value pairs for TLS secrets are always ***tls.crt*** and ***tls.key***. As TLS is not the default type for secrets, the ***type: kubernetes.io/tls*** property is required.

Step 3 ▾

Run the following command to create the secret:

command

```
kubectl apply -f wp-tls-secret.yaml
```

Step 4 ▾

Retrieve detailed information about the secret using the following command:

command

```
kubectl describe secret wp-tls-secret
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/certs> kubectl describe
secret wp-tls-secret
Name:          wp-tls-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  kubernetes.io/tls
```

Data

====

tls.key: 3268 bytes**tls.crt:** 1826 bytes**Additional Information ▾**

In addition to basic authentication and TLS secrets, Kubernetes provides other secret types. You can learn more about them and the required properties on [the Kubernetes official documentation site](#).

Exercise Summary

In this exercise, you created two secrets: a basic authentication secret and a TLS secret. For both of them, you used a mixed approach, creating the YAML file on the command line and later using it to create the secrets declaratively. These secrets are used in the next exercise.

(End of Exercise)

Exercise 6.4 - Injects sensitive information into Pods using Secrets

Description

Creating and using secrets is very similar to creating and using ConfigMaps. Once created, Secrets can be used as environment variables or injected via volumes.

In this exercise, you will inject the previously created secrets into a Pod using a WordPress image. Values in the basic authentication secrets will be available as environment variables, and the TLS secret will be mounted as a volume.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Inject secrets as environment variables in a Pod definition.
 - Use volumes to inject secrets into Pods.
-

Step 1 ▾

Run the following commands to review the Pod definition:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat secrets.yaml
```

Example output

```
apiVersion: v1
kind: Pod
metadata:
  name: wp-secret-volume-pod
spec:
  containers:
  - name: wordpress
    image: wordpress
    env:
    - name: WORDPRESS_DB_USER
      valueFrom:
        secretKeyRef:
          name: wp-secret
          key: username
    - name: WORDPRESS_DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: wp-secret
          key: password
    volumeMounts:
    - name: secrets-volume
      mountPath: "/etc/ssl/certs/wordpress"
      readOnly: true
  volumes:
  - name: secrets-volume
    secret:
      secretName: wp-tls-secret
```

Explanation ▾

The previous YAML describes a Pod with the following characteristics:

- Use **wordpress** as the container image.
- Create two environment variables in the container:
 - **WORDPRESS_DB_USER** loads (**secretKeyRef**) the value for the **username** key in the **wp-secret** secret (admin).
 - **WORDPRESS_DB_PASSWORD** loads (**secretKeyRef**) the value for the **password** key in the **wp-secret** secret (admin).
- Creates a volume named **secret-volume** that:

- Uses the ***wp-tls-secret*** Secret as the volume source. Each key-value entry in the Secret becomes a file when mounted (***tls.crt***, ***tls.key***).
- **volumenMounts** mounts the volume in the ***/etc/ssl/certs/wordpress*** folder.

Step 2 ▾

Create the Pod using the secrets by running the following command:

command

```
kubectl apply -f secrets.yaml
```

Step 3 ▾

Run the following commands and verify the containers is using the secrets as described in the pod definition:

commands

```
> kubectl exec -it wp-secret-volume-pod -- env
> kubectl exec -it wp-secret-volume-pod -- ls -la /etc/ssl/certs/wordpress
```

Explanation ▾

- **kubectl exec -it wp-secret-volume-pod -- env:** List all the environment variables in the container. ***WORDPRESS_DB_USER*** and ***WORDPRESS_DB_PASSWORD*** should display.
- **kubectl exec -it wp-secret-volume-pod -- ls -la /etc/ssl/certs/wordpress:** List all the files in the folder where the TLS secret was mounted.

Example output:

```
tux@management:~> kubectl exec -it wp-secret-volume-pod -- env
...
WORDPRESS_DB_USER=admin
WORDPRESS_DB_PASSWORD=admin
...

tux@management:~> kubectl exec -it wp-secret-volume-pod -- ls -la
/etc/ssl/certs/wordpress
total 0
drwxrwxrwt 3 root root 120 Dec 22 15:46 .
drwxr-xr-x 1 root root  18 Dec 22 15:47 ..
drwxr-xr-x 2 root root  80 Dec 22 15:46
..2025_12_22_15_46_48.1125962805
lrwxrwxrwx 1 root root  32 Dec 22 15:46 ..data →
..2025_12_22_15_46_48.1125962805
lrwxrwxrwx 1 root root  14 Dec 22 15:46 tls.crt → ..data/tls.crt
lrwxrwxrwx 1 root root  14 Dec 22 15:46 tls.key → ..data/tls.key
```

Step 5 ▾

Delete the Pod by running the following command:

command

```
kubectl delete -f secrets.yaml
```

Exercise Summary

During this exercise, you used the previously created secrets to inject confidential information into a Pod as environment variables and as a volume mounted in a folder in the container. Note that creating and using ConfigMaps and Secrets are similar processes.

(End of Exercise)

Section 7: Scheduling in Kubernetes

Description

During the previous sections, you ran stand-alone pods or other Kubernetes objects that manage pods, such as Deployments, StatefulSets, or DaemonSets. Sometimes those pods ran on one node, and sometimes on any other node, but we ignored why or how that happened.

In this section, you will dive deeper into scheduling, the process within a Kubernetes cluster that assigns a Pod to a Node.

Section Learning Objectives

- Understand the Kubernetes scheduler: Learn how the Kubernetes scheduler decides where to place newly created Pods.
- Apply Node selectors: Use basic Node selection methods to restrict Pod execution to nodes.
- Use Node affinity or Pod affinity and anti-affinity rules to group or separate Pods.
- Set Taints and Tolerations: Apply Taints to repel Pods in Nodes and Tolerations on Pods to allow the placement of Pods on tainted nodes.

Section Topics

- The scheduling process.
- Basic Node selection.
- Affinity and Anti-Affinity rules.
- Taints and Tolerations.

Exercise 7.1 - Schedule Pods using Node Names

Description

The scheduling process in Kubernetes has three phases: filtering, scoring, and finally, assigning the Pod to the Node with the highest rank. However, Kubernetes administrators can bypass the scheduling process and directly assign a Pod to a node.

In this exercise, you will learn how to bypass the Kubernetes scheduler and directly assign a Pod to a Node using the Node's name.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Pod and directly schedule it to a Node by name.
-

Task 1: Directly schedule a Pod into a Node using the Node's name**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select RKE201-management from the VM list in Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

The RKE201-management VM runs services for the other Lab VMs and must remain running. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and review the YAML file used for this task by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-nodename.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: pod-nodename
    name: pod-nodename
spec:
  nodeName: agent
  containers:
  - image: nginx
    name: pod-nodename
```

Explanation ▾

The **pod-nodename** definition:

- Uses an **nginx** image.

- Will be scheduled on the **agent** Node because the **nodeName: agent** property is defined in the Pod's **spec**.

Step 4 ▾

Create the Pod by running the following command

command

```
kubectl apply -f pod-nodename.yaml
```

Step 5 ▾

Retrieve information about the recently created Pod by running:

command

```
kubectl get pods pod-nodename -o wide
```

Example output:

```
tux@management:~> kubectl get pods pod-nodename -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP           NODE   NOMINATED NODE   READINESS GATES
pod-nodename        1/1     Running   0           115s  10.42.1.45   agent  <none>           <none>
```

Explanation ▾

The **-o wide** flag adds additional information to the Pod list, such as the **NODE** column. Note that the **pod-nodename** Pod has been scheduled and is running on the agent Node, as requested via the **nodeName** property.

Step 6 ▾

Delete the Pod by running the following command:

command

```
kubectl delete -f pod-nodename.yaml
```

Exercise Summary

In this exercise, you utilized the **nodeName** property in a Pod definition to bypass the Kubernetes scheduler and directly assign Pods to specific Nodes by name.

(End of Exercise)

Exercise 7.2 - Schedule Pods using Node Selectors

Description

Using the **nodeName** property to bypass the Kubernetes scheduler and directly assign Pods to Nodes has some serious disadvantages:

- Tight coupling. If the Node set by nodeName fails, the Pod will always fail.
- Portability. Different environments, such as development, testing, or production, may have different Node names, so you will need different Pod definitions, with different values for the **nodeName** property, depending on the environment.
- Flexibility. Scheduling by Node name is a rigid rule; a specific Node must be selected as the target.

Kubernetes provides additional mechanisms for scheduling to address those challenges. In this exercise, you will schedule Pods on Nodes using the **nodeSelector** property and add or remove labels on Nodes.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Add and remove labels to Nodes
 - Understand how to use the **nodeSelector** property to schedule Pods in Nodes.
 - Discover the meaning of the **Pending** status in Pods.
-

Task 1: Schedule a Pod using the nodeSelector property

Step 1 ▾

Review the YAML file used for this task by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-nodeselector.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nodeselector
spec:
  containers:
    - name: machine-learning-workload
      image: alpine:latest
      command: ["sh", "-c", "echo 'Processing ML data'; sleep 3600"]
  nodeSelector:
    hardware: "gpu"
    disktype: "ssd"
```

Explanation ▾

The **pod-nodeselector** definition:

- Uses an **alpine:latest** image.
- As the **alpine:latest** image does nothing, the Pod simulates a Machine Learning (ML) task by running commands that display a message and sleep.
- Uses **nodeSelector** under the **spec** property to select the Nodes where the Pod will be scheduled. Kubernetes will try to run this Pod on Nodes that match both labels: **hardware: "gpu"** and **disktype: "ssd"**.

Step 2 ▾

Use the following command to create the Pod:

command

```
kubectl apply -f pod-nodeselector.yaml
```

Step 3 ▾

Retrieve information about the pod using the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod-nodeselector	0/1	Pending	0	55s

Explanation ▾

The **pod-nodeselector** gets stuck in a **Pending** status, but the kubectl get command does not provide a reason.

Step 4 ▾

Retrieve information about the Pod using the following command:

command

```
kubectl describe pod pod-nodeselector
```

Example output:

```
...
Events:
  Type    Reason             Age          From
  Message
  ----    -
  Warning  FailedScheduling  3m2s (x4 over 18m)  default-scheduler
0/2 nodes are available: 2 node(s) didn't match Pod's node
affinity/selector. preemption: 0/2 nodes are available: 2 Preemption
is not helpful for scheduling
```

Explanation ▾

At the end of the **kubectl describe** command output, the **Events** information section displays the reason: None of the two Nodes available in the cluster matches the selector requirements, so Kubernetes fails to schedule the Pod.

Task 2: Add the required labels to the agent Node

Step 1 ▾

In a real-world scenario, the previous Pod would continue in Pending status unless new Nodes with the required features (labels) were added. For the sake of this exercise, you will simulate that by adding new labels to the agent Node. Add the **hardware: "gpu"** label to the agent Node by running the following command:

command

```
kubectl label node agent hardware=gpu
```

Step 2 ▾

Retrieve information about the Pods by running the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
pod-nodeselector    0/1     Pending   0           37m
```

Explanation ▾

The Pod remains in a **Pending** STATUS because the **nodeSelector** property defined two labels, **hardware: "gpu"** and **disktype: "ssd"**, and you only added the first one.

Step 3 ▾

Add the **disktype: "ssd"** label to the agent Node by running the following command:

command

```
kubectl label node agent disktype=ssd
```

Step 4 ▾

Verify that the Pod is running using the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod-nodeselector	1/1	Running	0	43m

Explanation

Once the **agent** Node has the required properties (labels), the scheduler can place the Pod in the Node. You can review the Node labels using the following command:

command

```
kubectl get nodes agent --show-labels
```

Example output:

```
tux@management:~> kubectl get nodes agent --show-labels
```

NAME	STATUS	ROLES	AGE	VERSION	LABELS
agent	Ready	<none>	4dlh	v1.33.6+rke2r1	beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=rke2,be ta.kubernetes.io/os=linux,disktype=ssd,hardware=gpu,kubernetes.io/arch =amd64,kubernetes.io/hostname=agent,kubernetes.io/os=linux,node.kubern etes.io/instance-type=rke2

Step 5

Return the cluster to its original state by running the following commands

commands

```
> kubectl delete pod pod-nodeselector
> kubectl label node agent hardware- disktype-
```

Explanation

- **kubectl delete pod pod-nodeselector:** Removes the Pod imperatively.
- **Kubectl label node agent hardware- disktype- :** Removes both labels from the **agent** Node.

Exercise Summary

In this exercise, you configured Pod scheduling using the **nodeSelector** property. You observed how mismatched labels result in a **Pending** status and resolved the issue by applying the required labels to the target Node, successfully binding the Pod.

(End of Exercise)

Exercise 7.3 - Schedule Pods using Node Affinity Rules

Description

The **nodeSelector** property is also a hard rule for selecting Nodes where to schedule Pods. The Nodes must have all the required labels for the Pods to be scheduled on them. Kubernetes provides an additional method: NodeAffinity. This method is more flexible and allows selecting Nodes using powerful expressions and complex conditions.

In this exercise, you will schedule a Pod using hard and soft NodeAffinity rules.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create hard and soft NodeAffinity rules within a Pod definition.
 - Understand the differences, in terms of scheduling, between hard and soft NodeAffinity rules.
 - Review how to manage labels for Nodes.
-

Task 1: Schedule a Pod using NodeAffinity rules.**Step 1** ▾

Review the YAML file used for this task by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-nodeaffinity.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nodeaffinity
spec:
  containers:
    - name: machine-learning-workload
      image: alpine
      command: ["sh", "-c", "echo 'Processing ML data'; sleep 3600"]
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: hardware
                operator: In
                values:
                  - gpu
                  - arm64
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
```

Explanation ▾

The **pod-nodeaffinity** Pod definition is very similar to the Pod in the previous exercise:

- Uses an **alpine:latest** image.
- As the **alpine:latest** image does nothing, the Pod simulates a Machine Learning (ML) task by running commands that display a message and sleep.
- Uses **nodeAffinity** under the **affinity** property in the **spec** section to select the Nodes where the Pod will be scheduled. The **nodeAffinity** property has two rules.
 - A hard rule:

```

requiredDuringSchedulingIgnoredDuringExecution:
  nodeSelectorTerms:
  - matchExpressions:
    - key: hardware
      operator: In
      values:
      - gpu
      - arm64

```

The ***requiredDuringSchedulingIgnoredDuringExecution:*** means that the rule is a hard one, so Nodes must match it for the Pod to be scheduled on them. The rule is a complex expression that requires the ***hardware*** label to have one of the values ***In*** the list: ***gpu*** or ***arm64***.

- A soft rule:

```

preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 100
    preference:
      matchExpressions:
      - key: disktype
        operator: In
        values:
        - ssd

```

The ***preferredDuringSchedulingIgnoredDuringExecution:*** means that the rule is soft-weighted. It is not required, and if a Node matches the rule, the ***weight*** is added to the Node's score during the scheduling process. The rule looks for Nodes with the ***disktype: "ssd"*** label.

Step 2 ▾

Use the following command to create the Pod:

command

```
kubectl apply -f pod-nodeaffinity.yaml
```

Step 3 ▾

Retrieve information about the Pod using the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
pod-nodeaffinity    0/1     Pending   0           6s
```

Explanation ▾

The **pod-nodeaffinity** Pod gets stuck in a **Pending** status, but the `kubectl get` command does not provide a reason.

Step 4 ▾

Retrieve information about the pod using the following command:

command

```
kubectl describe pod pod-nodeaffinity
```

Example output:

```
...
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Warning         FailedScheduling    46s   default-scheduler    0/2 nodes are
available: 2 node(s) didn't match Pod's node affinity/selector.
preemption: 0/2 nodes are available: 2 Preemption is not helpful for
scheduling.
```

Explanation ▾

At the end of the `kubectl describe` command output, the **Events** information section displays the reason: None of the two Nodes available in the cluster matches the selector requirements, so Kubernetes fails to schedule the Pod.

Task 2: Add the required labels to the server Node

Step 1 ▾

Add the **hardware: "arm64"** label to the server Node by running the following command:

command

```
kubectl label node server hardware=arm64
```

Step 2 ▾

Retrieve information about the Pods by running the following command:

command

```
kubectl get pods -o wide
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples> kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
pod-nodeaffinity	1/1	Running	0	38s	10.42.0.43	server	<none>

Explanation ▾

Since the **disktype: "ssd"** was a soft requirement, the Pod can now be scheduled on the server Node, which is the only one that matches the hard requirement.

Step 3 ▾

Return the cluster to its original state by running the following commands

commands

```
> kubectl delete pod pod-nodeaffinity
> kubectl label node server hardware-
```

Explanation ▾

- **kubectl delete pod pod-nodeaffinity:** Removes the Pod imperatively.
- **Kubectl label node agent hardware- :** Removes the **hardware** label from the server Node.

Additional Information ▾

Visit the [official Kubernetes documentation site](#) for information on NodeAffinity, PodAffinity, and PodAntiAffinity, and on how to use different operators to build complex hard and soft rules.

Exercise Summary

In this exercise, you implemented NodeAffinity to define both hard (required) and soft (preferred) rules for scheduling Pods on Nodes. You analyzed how these rules influence Pod placement and verified successful scheduling by modifying Node labels to match the affinity criteria.

(End of Exercise)

Exercise 7.4 - Manage Taints in Nodes

Description

In the previous exercises, you discovered properties such as `nodeName`, `nodeSelect`, and `nodeAffinity` for assigning Pods to Nodes. With those mechanisms, you described how to attract Pods to Nodes.

Taints are different; they are applied to Nodes and define how Pods are repelled from being scheduled on them. Think of Taints as locks: Pods cannot be scheduled on Nodes with a specific lock unless they have the matching key (Toleration)..

In this exercise, you will look for Taints inside information about Nodes and learn how to add Taints to Nodes.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Understand the role of Taints in Node.
 - Review information about Nodes to verify if they have Taints defined.
 - Add Taints to Nodes.
-

Task 1: Review Taint information in Node descriptions

Step 1 ▾

Retrieve detailed information about the **server** node in the cluster by running the following command:

command

```
kubectl describe node server
```

Example output:

```
...  
Taints:                <none>  
...
```

Explanation ▾

Within the Node description, you can find an empty line for the Taints; therefore, the **server** Node is not repelling any Pod.

Step 2 ▾

Retrieve detailed information about the **agent** Node in the cluster by running the following command:

command

```
kubectl describe node agent
```

Example output:

```
...  
Taints:                <none>  
...
```

Explanation ▾

Taints are also empty for the **agent** Node.

Task 2: Add a Taint to the server Node

Step 1 ▾

As a Kubernetes administrator, let's say you want to avoid running workloads on the server Node unless they are critical. Only the Kubernetes main components (api-server, scheduler, etcd, controller-manager) will remain running on that Node, but no additional workload will be allowed.

To achieve this target, you must add a Taint to the **server** (control-plane) Node. Run the following command to add a Taint

command

```
kubectl taint nodes server priority=critical:NoSchedule
```

Explanation ▾

This command creates a taint in the server Node:

- **priority:** Represents the key for the Taints.
- **critical:** It's the value for the key.
- **NoSchedule:** Defines the Taint effect. It means that no Pod will be scheduled on the **server** Node unless it has the key (Toleration) for the Taint, unless the Pod tolerates the Taint.

Step 2 ▾

Retrieve detailed information about the **server** Node by running the following command:

command

```
kubectl describe node server
```

Example output:

```
...
Labels:          beta.kubernetes.io/arch=amd64
                  beta.kubernetes.io/instance-type=rke2
                  beta.kubernetes.io/os=linux
                  kubernetes.io/arch=amd64
                  kubernetes.io/hostname=server
                  kubernetes.io/os=linux
                  node-role.kubernetes.io/control-plane=true
                  node-role.kubernetes.io/etcd=true
                  node-role.kubernetes.io/master=true
                  node.kubernetes.io/instance-type=rke2
...
Taints:          priority=critical:NoSchedule
...
```

Explanation ▾

The **server** Node now displays a Taint. Pay attention to the Node's label. You will use **node-role.kubernetes.io/control-plane=true** as the nodeSelector when scheduling Pods on the server.

Step 3 ▾

Review the YAML manifest ***pod-servernode-without-toleration.yaml*** by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-servernode-without-toleration.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: pod-server-no-toleration
    name: pod-server-no-toleration
spec:
  nodeSelector:
    node-role.kubernetes.io/control-plane: "true"
  containers:
  - image: nginx
    name: webserver
```

Explanation ▾

The ***pod-server-no-toleration*** Pod:

- Use the ***nginx*** images.
- Uses ***nodeSelector*** and tries to be assigned to Nodes with the ***node-role.kubernetes.io/control-plane: "true"*** label. In our cluster, only the server Node has this label.

Step 4 ▾

Use the following command to create the Pod:

command

```
kubectl apply -f pod-servernode-without-toleration.yaml
```

Step 3 ▾

Retrieve information about the pod using the following command:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod-server-no-toleration	0/1	Pending	0	11s

Explanation ▾

The **pod-server-no-toleration** Pod gets stuck in a **Pending** status, but the `kubectl get` command does not provide a reason.

Step 5 ▾

Retrieve information about the Pod using the following command:

command

```
kubectl describe pod pod-server-no-toleration
```

Example output:

```
...
Events:
  Type            Reason              Age   From                  Message
  ----            -
  Warning         FailedScheduling    104s  default-scheduler    0/2 nodes are
available: 1 node(s) didn't match Pod's node affinity/selector, 1
node(s) had intolerated taint {priority: critical}. preemption: 0/2
nodes are available: 2 Preemption is not helpful for scheduling.
```

Explanation ▾

At the end of the `kubectl describe` command output, the **Events** information section displays the reason: **1 node(s) had intolerated taint {priority: critical}**, so Kubernetes fails to schedule the Pod in the only Pod with the label required by **nodeSelector**.

Step 6 ▾

Use the following command to remove the Pod:

command

```
kubectl delete -f pod-servernode-without-toleration.yaml
```

Exercise Summary

In this exercise, you inspected Node specifications to identify existing Taints. You applied a restrictive Taint to a Node and verified its effect by attempting to schedule a non-tolerant Pod, confirming that the Taint successfully repelled the workload.

(End of Exercise)

Exercise 7.5 - Schedule Pods into Tainted Nodes (Tolerations)

Description

Pods need keys (Toleration) for the locks (Taints) on Nodes so they can be scheduled to run on them. In the previous exercise, Kubernetes failed to schedule a Pod in a Node because of the Taint you added. In this exercise, you will schedule a Pod in a tainted Node using a Toleration.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use Tolerations for tainted nodes.
 - Remove Taints from Nodes
-

Step 1 ▾

Review the YAML manifest *pod-servernode-with-toleration.yaml* by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-servernode-with-toleration.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: pod-server-with-toleration
    name: pod-server-with-toleration
spec:
  nodeSelector:
    node-role.kubernetes.io/control-plane: "true"
  containers:
  - image: nginx
    name: webserver
  tolerations:
  - key: "priority"
    operator: "Equal"
    value: "critical"
    effect: "NoSchedule"
```

Explanation ▾

The *pod-server-with-toleration* Pod:

- Use the *nginx* images.
- Uses **nodeSelector** and tries to be assigned to Nodes with the **node-role.kubernetes.io/control-plane: "true"** label. In our cluster, only the server node has this label.
- Defines a Toleration for a **priority=critical:NoSchedule**. The key, value, and effect in the Toleration match the key, value, and effect of the Taint.

Additional Information ▾

Review the [Kubernetes official documentation site](#) for more information about Taints, Tolerations, and operators for them.

Step 2 ▾

Use the following command to create the Pod:

command

```
kubectl apply -f pod-servernode-with-toleration.yaml
```

Step 3 ▾

Retrieve information about the pod using the following command:

command

```
kubectl get pods -o wide
```

Example output:

```
tux@management:~> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
NOMINATED NODE   READINESS GATES
pod-server-with-toleration   1/1     Running   0           14s   10.42.0.45   server
<none>                <none>
```

Explanation ▾

The **pod-server-with-toleration** Pod was scheduled on the **server** node because it tolerates the Taint.

Step 4 ▾

Use the following commands to remove the Pod and the Taint:

commands

```
kubectl delete -f pod-servernode-with-toleration.yaml
kubectl taint node server priority=critical:NoSchedule-
```

Explanation ▾

To remove a Taint from a Node uses the same command you used to add the Taint with a - character after the Taints.

Exercise Summary

In this exercise, you configured a Pod with a Toleration to bypass a Node's Taint restriction. You verified that the Pod was successfully scheduled on the previously restricted Node, demonstrating how to permit specific workloads on tainted infrastructure.

(End of Exercise)

Section 8: Resource Usage Control and Quality of Service (QoS)

Description

This course has not yet covered Pod resource consumption. Resources in a Kubernetes cluster are finite; however, users can define how many resources Pods use, and Kubernetes administrators can set constraints at the Pod or namespace level. Those definitions and constraints influence the Kubernetes scheduling process

Section Learning Objectives

- Implement Requests and Limits for Pods.
- Relate resource demands to the scheduling process.
- Determine the Quality of Service (QoS) for a Pod.
- Understand eviction policy.
- Enforce namespace boundaries: ResourceQuotas and LimitRanges

Section Topics

- Resources in Kubernetes.
- Resources, scheduling, and Quality of Service (QoS).
- Controlling defaults and boundaries.

Exercise 8.1 - Define limits and requests for Pods.

Description

When users define a Pod, as a stand-alone Kubernetes object or as part of larger Kubernetes entities, such as ReplicaSets or Deployments, they can set the minimum and maximum resource (CPU or Memory) usage. Requests define minimum values, while limits define maximum thresholds.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Define Pods using requests and limits.
 - Understand how requests influence Kubernetes scheduling.
 - Demonstrate the consequences of a Pod overcoming a defined limit.
-

Task 1: Create a Pod with a high request**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select RKE201-management from the VM list in Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

The RKE201-management VM runs services for the other Lab VMs and must remain running. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and review the YAML manifest for a Pod with a large memory request by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-with-high-requests.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-high-requests
  labels:
    app: resource-test
spec:
  containers:
  - name: web-server
    image: nginx
    resources:
      requests:
        memory: 32Gi
```

Explanation ▾

- The YAML manifest defines a Pod named ***pod-with-high-requests***.

- The Pod has a single container, **web-server**, that uses an **nginx** container image.
- The Pod manifest requests 32 GB of RAM. To be scheduled, Kubernetes must find a Node that can provide that amount of memory.

Step 4 ▾

Create the Pod using the previous Pod definition by running the following command:

command

```
kubectl apply -f pod-with-high-requests.yaml
```

Step 5 ▾

Retrieve information about the Pod using the following command:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod-with-high-requests	0/1	Pending	0	39s

The Pod cannot be scheduled and remains in a **Pending** state.

Step 6 ▾

Retrieve detailed information about the Pod using the following command:

command

```
kubectl describe pod pod-with-high-requests
```

Explanation ▾

Example output:

```
...
...
Events:
```

Type	Reason	Age	From
Message			
----	-----	----	----

```
Warning FailedScheduling 96s (x4 over 16m) default-scheduler 0/2
nodes are available: 2 Insufficient memory. preemption: 0/2 nodes are
available: 2 No preemption victims found for incoming pod.
```

The ***kubectl describe*** command provides additional information. The Events section explains the scheduling failure: none of the nodes have enough memory to meet the Pod's memory request.

Step 7 ▾

Remove the Pod by running the following command:

command

```
kubectl delete -f pod-with-high-requests.yaml
```

Task 2: Modify the YAML manifest to set a request that can be scheduled**Step 1** ▾

Edit the ***pod-with-high-requests.yaml*** manifest and change the request to set a 512 megabytes of RAM as the minimum required for the Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-high-requests
  labels:
    app: resource-test
spec:
  containers:
    - name: web-server
      image: nginx
      resources:
        requests:
          memory: "512Mi"
```

Step 2 ▾

Create the Pod using the previous Pod definition by running the following command:

command

```
kubectl apply -f pod-with-high-requests.yaml
```

Step 3 ▾

Retrieve information about the Pod using the following command:

command

```
kubectl get pods
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
pod-with-high-requests	1/1	Running	0	11s

Because the nodes have sufficient capacity for this request, Kubernetes schedules the Pod, and it finally runs.

Step 4 ▾

Remove the Pod by running the following command:

command

```
kubectl delete -f pod-with-high-requests.yaml
```

Task 3: Exceeding Pod limits**Step 1** ▾

Review the YAML manifest for a Pod with requests and limits by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-with-requests-and-limits.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-requests-and-limits
  labels:
    app: pod-with-requests-and-limits
spec:
  containers:
  - name: web-server
    image: nginx
    resources:
      requests:
        memory: "1Mi"
        cpu: "100m"
      limits:
        memory: "2Mi"
        cpu: "200m"
```

Explanation

- The YAML manifest defines a Pod named **pod-with-requests-and-limits**
- The Pod has a single container, **web-server**, that uses an **nginx** container image.
- The Pod definition sets requests and limits for both CPU and memory. The limits are low to demonstrate what happens when a Pod exceeds its memory limits.

Step 2

Create the Pod using the previous Pod definition by running the following command:

command

```
kubectl apply -f pod-with-requests-and-limits.yaml
```

Step 3

Retrieve information about the Pod using the following command:

command

```
kubectl get pods
```

Explanation

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS	GATES	
pod-with-requests-and-limits	1/1	Running	0	2m11s	
10.42.1.5	agent	<none>	<none>		

The Pod is scheduled and **Running**.

Step 4

In a separate terminal, run the following command to watch the running pods:

command

```
watch kubectl get pods
```

Step 5

The new terminal, running the **watch** command, displays how the Pod fails and restarts in an endless loop, displaying errors in the **STATUS** column, such as **CrashLoopBackOff** or **OOMKilled**.

Example output:

```
Every 2.0s: kubectl get pods
NAME                                READY   STATUS    RESTARTS
AGE
pod-with-requests-and-limits        0/1     OOMKilled  2 (22s ago)  42s
```

Explanation ▾

The container in the Pod runs an nginx web server. This application needs more memory than the memory limits set in the Pod definition. When the memory used by the application exceeds memory limits, the Pod is terminated.

Note ▾

Memory is a non-compressible resource; Kubernetes will terminate any Pod that exceeds its memory limit. However, CPU usage is a compressible resource; if a Pod requests more CPU than it is allowed, Kubernetes makes it wait.

Step 6 ▾

Finish the **watch** command, close the terminal where it was running, and finally delete the Pod by running the following command:

command

```
kubectl delete -f pod-with-requests-and-limits.yaml
```

Exercise Summary

In this exercise, you have verified how resource requests, the minimum amount of resources needed by a Pod, have an influence on the Kubernetes scheduling process and how exceeding limits impacts Pods.

(End of Exercise)

Exercise 8.2 - Quality of Service (QoS) classes and the eviction process.

Description

Resource requests and limits determine how Kubernetes evicts Pods when a node is under pressure. In this exercise, you will learn how to retrieve the QoS of running Pods, and you will verify how Kubernetes uses QoS classes to decide which Pods to evict.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Pod for each QoS class.
 - Recover the QoS information from running Pods.
-

Task 1: Create a Pod for every Quality of Service (QoS) class**Step 1** ▾

Review the YAML manifest for a Pod that, when running, will be assigned to the Guaranteed QoS class:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-guaranteed-qos.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-guaranteed
spec:
  containers:
  - name: redis
    image: redis
    resources:
      limits:
        memory: "200Mi"
        cpu: "100m"
      requests:
        memory: "200Mi"
        cpu: "100m"
```

Explanation ▾

The YAML manifest sets limits and requests for both the CPU and memory, and limits and requests are equal, so the Pod, when running, belongs to the Guaranteed QoS class.

Step 2 ▾

Review the YAML manifest for a Pod that, when running, will be assigned to the Burstable QoS class:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-burstable-qos.yaml
```

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-burstable
spec:
```

```

containers:
- name: nginx
  image: nginx
  resources:
    requests:
      memory: "100Mi"
      cpu: "50m"
    limits:
      memory: "500Mi"

```

Explanation ▾

The YAML manifest does not match the criteria for the Guaranteed QoS, but at least one of the containers has a request or limit, so the Pod, when running, belongs to the Burstable QoS class.

Step 3 ▾

Review the YAML manifest for a Pod that, when running, will be assigned to the BestEffort QoS class:

commands

```

> cd ~/course_files/RKE201v2/labs/manifests/pod-examples
> cat pod-besteffort-qos.yaml

```

Example output:

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-besteffort
spec:
  containers:
  - name: busybox
    image: busybox
    command:
    - "sleep"
    - "3600"

```

Explanation ▾

The YAML manifest does not set any request or limits, so the Pod, when running, belongs to the BestEffort QoS class.

Step 4 ▾

Create the Pods by running the following commands:

commands

```
> kubectl apply -f pod-guaranteed-qos.yaml
> kubectl apply -f pod-burstable-qos.yaml
> kubectl apply -f pod-besteffort-qos.yaml
```

Step 5

Execute the following command to verify that all the Pods are running:

command

```
kubectl get pods
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
pod-besteffort      1/1     Running   0           20s
pod-burstable       1/1     Running   0           25s
pod-guaranteed      1/1     Running   0           32s
```

Task 2: Recover the QoS information from running Pods**Step 1**

Recover the QoS class for the running Pods in the working namespace by running the following command:

command

```
kubectl get pods -o=custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/pod-examples>
kubectl get pods
-o=custom-columns=NAME:.metadata.name,QOS:.status.qosClass
NAME                QOS
pod-besteffort      BestEffort
pod-burstable       Burstable
pod-guaranteed      Guaranteed
```

The **-o=custom-column** and **-o jsonpath** flags allow users to tailor the output of the `kubectl get` command and display only the required information. The example above displays two columns (NAME and QOS) with information about the Pods' names and qosClasses. Information about the QoS is under the *status* property as this information is assigned once the Pods are running.

When a Node is under pressure, Kubernetes can evict Pods depending on the Quality of Service (QoS). **BestEffort** pods are terminated first because they have no reserved resources, followed by **Burstable** Pods that exceed their requested memory, while **Guaranteed** Pods are protected until the node has no other choice but to kill them to save the operating system.

Additional Information ▾

Review the [official Kubernetes documentation site](#) for information on those flags.

Step 2 ▾

Delete the Pods and leave the cluster in a clean state by running the following command:
command:

```
kubectrl delete pods pod-besteffort pod-burstable pod-guaranteed
```

Exercise Summary

In this exercise, you created a Pod for each Quality of Service (QoS) class and learned how to recover this information from running Pods using a new **kubectrl** flag: **-o=custom-columns**.

(End of Exercise)

Exercise 8.3 - Define a LimitRange for a namespace.

Description

Relying on users to define resource settings is a 'good-faith' approach that becomes difficult to manage as the cluster scale.. Kubernetes provides LimitRanges to set default values, default maximums, and default minimums for all Pods at the namespace level.

In this exercise, you will learn how to create a LimitRange, understand the values within its definition, and verify how a LimitRange enforces requests and limits in a namespace.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create LimitRange Kubernetes objects to define requests and limits values in a namespace.
 - Verify how LimitRanges are applied.
-

Task 1: Create a LimitRange for the lab-resources namespace**Step 1** ▾

Create the **lab-resources** namespace by running the following command:

command

```
kubectl create namespace lab-resources
```

Step 2 ▾

Verify that the namespace has been created by running the following command:

command

```
kubectl get ns
```

Explanation ▾

The command retrieves the namespace list using **ns** as the shortname for namespace.

Example output

```
tux@management:~> kubectl get ns
NAME                STATUS   AGE
default             Active   13h
kube-node-lease     Active   13h
kube-public         Active   13h
kube-system         Active   13h
lab-resources       Active   3s
```

Step 3 ▾

Set the **lab-resources** namespace as the working namespace by running one of the following commands:

commands

```
> kubectl config set-context $(kubectl config current-context) \
  --namespace=lab-resources
> kubens lab-resources
```

Step 4 ▾

Review the YAML manifest for the LimitRange::

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control
> cat limit-range-for-lab-resources-ns.yaml
```

Example output:

```

apiVersion: v1
kind: LimitRange
metadata:
  name: my-limit-range
spec:
  limits:
  - default:
      cpu: "300m"
      memory: "256Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
    max:
      cpu: "500m"
      memory: "512Mi"
    min:
      cpu: "50m"
      memory: "64Mi"
  type: Container

```

Explanation ▾

In this YAML manifest for the LimitRange:

- The **default** section defines default limits for Pods that have no limits in their definitions.
- The **defaultRequest** section sets default requests for Pods that have no requests defined.
- The **max** and **min** values define the limits range:
 - CPU limits must range from 50m to 500m.
 - Memory requests must range from 64Mi to 512Mi.

Step 5 ▾

Create the LimitRange and retrieve information about it once it is created by running the following commands:

commands

```

> kubectl apply -f limit-range-for-lab-resources-ns.yaml
> kubectl get limitranges
> kubectl describe limitranges my-limit-range

```

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl apply -f limit-range-for-lab-resources-ns.yaml
limitrange/my-limit-range created
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl get limitranges

```

```

NAME                CREATED AT
my-limit-range      2026-01-12T08:38:32Z
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl describe limitranges my-limit-range
Name:                my-limit-range
Namespace:           lab-resources
Type                 Resource  Min    Max    Default Request  Default Limit  Max
Limit/Request Ratio
-----
Container            memory    64Mi   512Mi  128Mi             256Mi           -
Container            cpu       50m    500m   100m              300m           -

```

Task 2: Create Pods to verify how the LimitRange is applied

Step 1 ▾

Create a Pod without requests and limits imperatively by running the following command:
command

```
kubectl run simple-pod --image=nginx
```

Step 2 ▾

Retrieve information about Pod limits and requests by running the following command:
command

```
kubectl get pods simple-pod -o=jsonpath='{.spec.containers[0].resources}{"\n"}'
```

Example output:

```
{"limits":{"cpu":"300m","memory":"256Mi"},"requests":{"cpu":"100m","memory":"128Mi"}}
```

Explanation ▾

Even when requests and limits were not set, the Pod still has them defined because of the LimitRange in the namespace.

Step 3 ▾

Review the YAML manifest for a Pod that exceeds the CPU limits for the *lab-resources* namespace by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control
> cat pod-exceeding-limits.yaml
```

Example output:

```
apiVersion: v1
```



```

kind: Pod
metadata:
  name: pod-exceeding-limits
spec:
  containers:
  - name: web-server
    image: nginx
    resources:
      requests:
        cpu: "1"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"

```

Explanation ▾

The YAML manifest defines a Pod requesting 1 CPU, which exceeds the limits set in the LimitRange for the working namespace.

Step 4 ▾

Create the Pods by running the following command:

command

```
kubectl apply -f pod-exceeding-limits.yaml
```

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/resource-control
> kubectl apply -f pod-exceeding-limits.yaml
Error from server (Forbidden): error when creating
"pod-exceeding-limits.yaml": pods "pod-exceeding-limits" is forbidden:
maximum cpu usage per Container is 500m, but limit is 1

```

Explanation ▾

Because the CPU maximum request defined in the LimitRange is 500m, a Pod with a 1 CPU limit and request cannot be created.

Step 5 ▾

Run the following commands to clean the cluster:

commands

```

> kubectl delete pods simple-pod
> kubectl delete limitranges my-limit-range

```

Exercise Summary

In this exercise, you defined a Kubernetes **LimitRange** object to establish default, maximum, and minimum values for Pod requests and limits. Additionally, you verified how these values are applied and enforced when running Pods.

(End of Exercise)

Exercise 8.4 -Define a ResourceQuota for a namespace.

Description

LimitRanges apply to standalone Pods in a namespace. However, with LimitRange, you cannot set global limits for a namespace by aggregating the resources used by all Pods within that namespace. For these tasks, Kubernetes offers ResourceQuotas, which also allow you to constrain the number of objects in a namespace.

In this exercise, you will learn how to create a ResourceQuota, understand the values within its definition, and verify how a ResourceQuota defines constraints in the working namespace.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create ResourceQuotas Kubernetes objects to define aggregated requests, limits, and the number of objects in a namespace.
 - Verify how ResourceQuotas are applied.
-

Task 1: Create a ResourceQuota for the lab-resource namespace**Step 1** ▾

Review the YAML manifest for the ResourceQuota:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control
> cat resource-quota-for-lab-resources-ns.yaml
```

Example output:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: my-resource-quota
spec:
  hard:
    requests.cpu: "500m"
    requests.memory: "512Mi"
    limits.cpu: "1"
    limits.memory: "4Gi"
    pods: "10"
```

Explanation ▾

In this YAML manifest for the ResourceQuota object:

- ***requests.cpu: "500m"***: The total sum of CPU requests from all Pods cannot exceed 500 millicores (0.5 CPU).
- ***requests.memory: "512Mi"***: The total sum of memory requests cannot exceed 512 Megabytes.
- ***limits.cpu: "1"***: The total sum of CPU limits cannot exceed 1 full CPU (1000m).
- ***limits.memory: "4Gi"***: The total sum of memory limits cannot exceed 4 Gigabytes.
- ***pods: "10"***: This is a Count Quota. It limits the total number of Pods in this namespace to 10, regardless of size.

Step 2 ▾

Create the ResourceQuota and retrieve information about it once it is created by running the following commands:

commands

```
> kubectl apply -f resource-quota-for-lab-resources-ns.yaml
> kubectl get resourcequotas
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl apply -f resource-quota-for-lab-resources-ns.yaml
resourcequota/my-resource-quota created
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl get resourcequotas my-resource-quota
```

NAME	REQUEST	AGE
my-resource-quota	pods: 1/10, requests.cpu: 1/500m, requests.memory: 256Mi/512Mi limits.cpu: 1/1, limits.memory: 512Mi/4Gi	8s

Task 2: Verify how the ResourceQuota is enforcing aggregated constraints**Step 1** ▾

Try to create a Deployment with 15 replicas imperatively, which exceeds the total number of Pods, by running the following command:

command

```
kubectl create deployment new-deployment --image=nginx --replicas=15
```

Step 2 ▾

Retrieve information about the Deployment by running the following command:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl get all
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/new-deployment	0/15	0	0	2m42s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/new-deployment-6c84b9b6cc	15	0	0	2m42s

Explanation ▾

As shown in the output, the Deployment and ReplicaSet are created, but no new Pods are ready. Retrieve information about the ReplicaSet using the following command to understand why:

command

```
kubectl describe rs new-deployment-6c84b9b6cc
```

Example output:

```
...
Warning FailedCreate 2m5s (x8 over 7m30s) replicaset-controller (combined
from similar events): Error creating: pods "new-deployment-6c84b9b6cc-lb457"
is forbidden: failed quota: my-resource-quota: must specify limits.cpu for:
nginx; limits.memory for: nginx
```

Warning

When using ResourceQuotas in a namespace to constrain the aggregated CPU or memory usage, Pods defined in the namespace must include limits and requests.

Warning

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 3

Review the YAML manifest for the following Deployment:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control
> cat deployment-for-lab-resources-ns.yaml
```

Example output:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-quota-infractor
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
```

```
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "256Mi"
        cpu: "100m"
      limits:
        memory: "512Mi"
        cpu: "200m"
```

Step 4 ▾

Create the Deployment by running the following command:

command

```
kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 5 ▾

Retrieve information about the created objects by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/app-quota-infractor-6655b7dcdc-grhg9  1/1     Running   0           9s
pod/app-quota-infractor-6655b7dcdc-r4nhp  1/1     Running   0           9s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/app-quota-infractor  2/4     2             2           9s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/app-quota-infractor-6655b7dcdc  4         2         2       9s
```

Explanation ▾

Even after the Deployment is created, the number of Pods remains stuck at 2. Run the following command on the ReplicaSet to understand the reason (replace the ReplicaSet name with the one displayed in your cluster):

command

```
kubectl describe rs app-quota-infractor-6655b7dcdc
```

Example output:

```
...
Warning FailedCreate      70s (x8 over 4m17s) replicaset-controller
(combined from similar events): Error creating: pods
"app-quota-infractor-6655b7dcdc-5rvzw" is forbidden: exceeded quota:
my-resource-quota, requested: requests.memory=256Mi, used:
requests.memory=512Mi, limited: requests.memory=512Mi
```

The ResourceQuota defined an aggregated memory request of 512Mi for the Pods. As long as the Deployment set four replicas, each with a 256Mi memory request, only two replicas were created, and an **“exceeded quota”** message appeared while trying to create the third one.

Step 6 ▾

Run the following commands to clean up the cluster and reset the working namespace to default:

commands

```
> kubectl delete deployment app-quota-infractor new-deployment
> kubectl delete resourcequotas my-resource-quota
> kubectl delete namespace lab-resources
> kubectl config set-context $(kubectl config current-context) \
  --namespace=default
```

Exercise Summary

In this exercise, you defined a Kubernetes **ResourceQuota** object to set aggregated resource usage constraints for a namespace. Additionally, you verified how these values are applied and enforced when trying to run applications with more than one Pod.

(End of Exercise)

Section 9: Scaling in Kubernetes

Description

Scaling is a core capability of container orchestrators like Kubernetes. When discussing scaling, a Kubernetes administrator can consider two perspectives: application scaling and infrastructure scaling. Application scaling applies to the workload running in the Kubernetes cluster, while infrastructure scaling applies to the Kubernetes cluster itself.

Section Learning Objectives

- Understand the different scaling processes in Kubernetes.
- Perform manual application scaling: Manually scale a workload up or down using imperative or declarative commands.
- Implement HPA: Create and configure an HPA resource to automatically scale a deployment based on resource utilization.
- Understand Vertical Scaling for application workloads: Differentiate between HPA and VPA.
- Scaling an RKE2 Kubernetes cluster.

Section Topics

- Scaling in Kubernetes.
- Horizontal workload scaling in Kubernetes.
- Vertical workload scaling in Kubernetes.
- Scale an RKE2 Kubernetes cluster.

Exercise 9.1 - Manually scale up and down a deployment

Description

Horizontal scaling involves adjusting the number of Pods used by an application to meet demand. In this exercise, you will learn how to perform scaling from the CLI using the ***kubectrl scale*** or ***kubectrl edit*** commands, or simply by modifying a YAML file.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Horizontally scale an application by running ***kubectrl scale***.
 - Horizontally scale an application by running ***kubectrl edit***.
 - Horizontally scale an application by modifying its YAML manifest.
-

Task 1: Scale a Deployment up using kubectl scale**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select RKE201-management from the VM list in Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

The RKE201-management VM runs services for the other Lab VMs and must remain running. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and create a Deployment imperatively by running the following command:

command

```
kubectl create deployment deployment-to-scale --image=nginx --replicas=4
```

Step 4 ▾

Verify that the Deployment and all its elements (ReplicaSet and Pods) have been created by running the following command:

command

```
kubectl get all
```

Example output:

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-to-scale-6c4bf75867-8vlg5	1/1	Running	0	91s
pod/deployment-to-scale-6c4bf75867-d86gn	1/1	Running	0	91s
pod/deployment-to-scale-6c4bf75867-p5ctq	1/1	Running	0	91s
pod/deployment-to-scale-6c4bf75867-xjgln	1/1	Running	0	91s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	20h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment-to-scale	4/4	4	4	91s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-to-scale-6c4bf75867	4	4	4	91s

Warning

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 5

Scale up the Deployment manually and increase the number of replicas to 8 by running the following command:

command

```
kubectl scale deployment deployment-to-scale --replicas=8
```

Explanation

The ***kubectl scale*** command horizontally scales workloads (Deployments/ReplicaSet/StatefulSets) by changing the number of managed Pods.

Step 6

Verify that the Deployment now has 8 replicas by running::

command

```
kubectl get all
```

Example output:

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-to-scale-6c4bf75867-8vlg5	1/1	Running	0	7mls
pod/deployment-to-scale-6c4bf75867-d86gn	1/1	Running	0	7mls
pod/deployment-to-scale-6c4bf75867-p5ctq	1/1	Running	0	7mls
pod/deployment-to-scale-6c4bf75867-s52gf	1/1	Running	0	2m10s
pod/deployment-to-scale-6c4bf75867-ssrq9	1/1	Running	0	2m10s
pod/deployment-to-scale-6c4bf75867-vjqrr	1/1	Running	0	2m10s
pod/deployment-to-scale-6c4bf75867-xjgln	1/1	Running	0	7mls
pod/deployment-to-scale-6c4bf75867-xlqmd	1/1	Running	0	2m10s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	20h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment-to-scale	8/8	8	8	7mls

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-to-scale-6c4bf75867	8	8	8	7mls

```
replicaset.apps/deployment-to-scale-6c4bf75867    4    4    4    91s
```

Task 2: Scale a Deployment using kubectl edit

Step 1 ▾

Run the following command, which opens the current status of the previously created deployment in your system's default text editor:

command

```
kubectl edit deployment deployment-to-scale
```

Example output:

```
...
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2026-01-12T15:06:31Z"
  generation: 2
  labels:
    app: deployment-to-scale
  name: deployment-to-scale
  namespace: default
  resourceVersion: "132362"
  uid: 3b15fbf2-d340-4658-8ff8-82affea3e258
spec:
  progressDeadlineSeconds: 600
  replicas: 8
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: deployment-to-scale
  strategy:
...

```

Step 2 ▾

Change the replicas value to 4, then save and exit the editor to apply the changes.

Step 3 ▾

Verify that the Deployment now has 4 replicas again by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-to-scale-6c4bf75867-8vlg5	1/1	Running	0	16m
pod/deployment-to-scale-6c4bf75867-s52gf	1/1	Running	0	11m
pod/deployment-to-scale-6c4bf75867-ssrq9	1/1	Running	0	11m
pod/deployment-to-scale-6c4bf75867-xlqmd	1/1	Running	0	11m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	20h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment-to-scale	4/4	4	4	16m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-to-scale-6c4bf75867	4	4	4	16m

Step 4

Finally, finish the task and remove the Deployment:

command

```
kubectl delete deployment deployment-to-scale
```

Task 3: Scale up and down a Deployment using kubectl apply -f (Declarative approach)**Step 1**

Create a Deployment using the YAML manifest used in the previous section by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control/
> kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 2

Verify that the Deployment has 4 replicas by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/app-quota-infractor-6655b7dcdc-6c277	1/1	Running	0	56s
pod/app-quota-infractor-6655b7dcdc-9wgvf	1/1	Running	0	56s
pod/app-quota-infractor-6655b7dcdc-d7n5r	1/1	Running	0	56s
pod/app-quota-infractor-6655b7dcdc-rpzgr	1/1	Running	0	56s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	20h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/app-quota-infractor	4/4	4	4	56s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-quota-infractor-6655b7dcdc	4	4	4	56s

Step 3

Use your preferred text editor to modify the YAML file and scale up the Deployment by setting the number of replicas in the Deployment to 6. Apply the new manifest by running the following command:

command

```
kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 4

Verify that the Deployment has 6 replicas by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/app-quota-infractor-6655b7dcdc-6c277	1/1	Running	0	22m
pod/app-quota-infractor-6655b7dcdc-9wgvf	1/1	Running	0	22m
pod/app-quota-infractor-6655b7dcdc-crpgp	1/1	Running	0	2s
pod/app-quota-infractor-6655b7dcdc-d7n5r	1/1	Running	0	22m
pod/app-quota-infractor-6655b7dcdc-rpzgr	1/1	Running	0	22m
pod/app-quota-infractor-6655b7dcdc-s7xhz	1/1	Running	0	2s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	21h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/app-quota-infractor	6/6	6	6	22m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-quota-infractor-6655b7dcdc	6	6	6	22m

Step 5

Scale down the Deployment. Change back the number of replicas back to 4 in the YAML manifest and apply the manifest again by running:

command

```
kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 6

Verify that the Deployment has 4 replicas again by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/app-quota-infractor-6655b7dcdc-6c277	1/1	Running	0	25m
pod/app-quota-infractor-6655b7dcdc-9wgvf	1/1	Running	0	25m
pod/app-quota-infractor-6655b7dcdc-d7n5r	1/1	Running	0	25m
pod/app-quota-infractor-6655b7dcdc-rpzgr	1/1	Running	0	25m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	21h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/app-quota-infractor	4/4	4	4	25m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-quota-infractor-6655b7dcdc	4	4	4	25m

Step 7 ▾

Finally, finish the task and remove the Deployment by running the following Declarative command:

command

```
kubectl delete -f deployment-for-lab-resources-ns.yaml
```

Exercise Summary

In this exercise, you have manually scaled up and down a Deployment using the **kubectl scale** and **kubectl edit** commands, and you have also manually scaled up and down a Deployment using a declarative approach, a YAML manifest, and the **kubectl apply -f** command.

(End of Exercise)

Exercise 9.2 - Automatically scale up and down a deployment using Horizontal Pod Autoscalers (HPA) objects

Description

Manual scaling requires continuous monitoring of Pod metrics to adjust replica counts based on actual demand. Kubernetes provides the HorizontalPodAutoscaler (HPA) objects to automatically scale depending on resource usage thresholds defined for Pods in the workload.

In this exercise, you will create a HorizontalPodAutoscaler (HPA) for a Deployment and see it scale up live.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create an HPA object that targets a workload (Deployment).
 - Demonstrate how Kubernetes automatically scales a Deployment as workload demand increases.
-

Task 1: Verify if the metrics server is running in the cluster**Step 1**

Autoscaling using HPA objects requires the metrics server to be installed in your Kubernetes cluster. Unlike many other Kubernetes distributions, RKE2 installs the metrics server by default. Use the following command to verify that the metrics-server is running properly:

command

```
kubectl get pods -A
```

Example output:

```
tux@management:~> kubectl get pods -A
NAMESPACE      NAME                                     READY
STATUS         RESTARTS   AGE
...
kube-system    rke2-metrics-server-5699475d99-l92zw    1/1
Running        1 (9h ago)  21h
...
```

Explanation

The ***kubectl get pods -A*** command displays the Pods in all (-A) namespaces. The Pod running the metrics server appears on the list with a name similar to the one above: (***rke2-metrics-server-5699475d99-l92zw***). This Pod has a ***Running*** status with 1 out of 1 containers ready. Therefore, the metrics server can be considered to be running in a healthy state and ready to be used by other Kubernetes resources, such as HPA objects.

Task 2: Create the required objects to demonstrate autoscaling and watch live how Kubernetes autoscales a Deployment**Step 1**

Review the YAML manifest for the Deployment that will be autoscaled by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/scaling/
> cat deployment-to-be-autoscaled.yaml
```

Example output:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-to-be-autoscaled
spec:
```

```

selector:
  matchLabels:
    app: deployment-autoscale
replicas: 1
template:
  metadata:
    labels:
      app: deployment-autoscale
  spec:
    containers:
    - name: busybox
      image: busybox
      command: ["/bin/sh", "-c", "shasum /dev/zero"]
      resources:
        requests:
          cpu: "100m"
        limits:
          cpu: "200m"

```

Explanation ▾

The YAML manifest defines a Deployment:

- It is named ***deployment-to-be-autoscaled***
- Uses one replica from the ***busybox*** container image.
- Defines a CPU-intensive command to be executed once the container starts.
- Sets CPU requests and limits. These are set to relatively low values to trigger autoscaling quickly for demonstration purposes.

Step 2 ▾

Review the YAML manifest for the HPA object that will track the resource usage in the Deployment and scale the Deployment up and down according to specific conditions:

commands

```

> cd ~/course_files/RKE201v2/labs/manifests/scaling/
> cat hpa-object.yaml

```

Example output:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-object
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: deployment-to-be-autoscaled
  minReplicas: 1
  maxReplicas: 4
  metrics:

```

```
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
```

The YAML manifest for the HPA object:

- Sets the previous Deployment (*deployment-to-be-autoscaled*) as the target for autoscaling using the *scaleTargetRef*.
- Sets the minimum and the maximum number of replicas for the Deployment (1 and 4, respectively).
- Defines an average CPU threshold of 50% to scale up the Deployment.

Step 3 ▾

Create the Deployment and the HPA object by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/scaling/
> kubectl apply -f hpa-object.yaml
> kubectl apply -f deployment-to-be-autoscaled.yaml
> kubectl get hpa -w
```

Explanation ▾

After creating the HPA object and the Deployment the *kubectl get hpa -w* (same as *watch kubectl get hpa*), you will see live how new Pods are created while the load increases:

Example output:

```
tux@management:~> kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: <unknown>/50%	1	4	1	114s
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: <unknown>/50%	1	4	1	2m
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 200%/50%	1	4	1	2m15s
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 201%/50%	1	4	4	2m30s
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 201%/50%	1	4	4	2m45s
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 200%/50%	1	4	4	3m
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 200%/50%	1	4	4	4m45s
hpa-object	Deployment/deployment-to-be-autoscaled	cpu: 200%/50%	1	4	4	8m27s

Note ▾

Alternatively, you can use ***kubectrl get pods -w*** or ***watch kubectrl get pods*** to see how the pods are created.

Step 4 ▾

Stop the watch command by pressing CTRL+C and return the cluster to the previous state by running the following commands:

commands

```
> kubectrl delete -f hpa-object.yaml  
> kubectrl delete -f deployment-to-be-autoscaled.yaml
```

Exercise Summary

In this exercise, you created a HorizontalPodAutoScaler (HPA) object that targets a Deployment to see how the Deployment scales up as the auto-generated load in the first replica increases.

(End of Exercise)

Exercise 9.3 - Scale a Deployment vertically and manually

Description

Kubernetes can also scale workloads vertically. Horizontal scaling changes the number of Pods (replicas), whereas vertical scaling adjusts the resources (CPU, memory) allocated to each Pod.

Scaling workload vertically and automatically is possible using VerticalAutoScaler objects. However, those objects are Kubernetes add-ons that are not included by default in RKE2, so for the Labs only manual vertical scaling will be considered. Refer to the [official Kubernetes documentation](#) for information on VPA objects and their installation.

In this exercise, you will vertically scale up a Deployment by changing the resources used by the Pods in the Deployment in the YAML manifest.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Vertically scale a Deployment by changing its YAML manifest and applying the changes.
-

Task 1: Create a Deployment

Step 1 ▾

Review the YAML for the Deployment by running the following commands

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/resource-control/  
> cat deployment-for-lab-resources-ns.yaml
```

Example output:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: app-quota-infractor  
spec:  
  replicas: 4  
  selector:  
    matchLabels:  
      app: web  
  template:  
    metadata:  
      labels:  
        app: web  
    spec:  
      containers:  
      - name: nginx  
        image: nginx  
        resources:  
          requests:  
            memory: "256Mi"  
            cpu: "100m"  
          limits:  
            memory: "512Mi"  
            cpu: "200m"
```

Explanation ▾

As the Deployment, which was already used in the previous section, has requests and limits, it is a good option for demonstrating vertical scaling.

Step 2 ▾

Create the Deployment by using the following command:

command

```
kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 3 ▾

Run the following command to verify that the Deployment and the related workloads (ReplicaSet and Pods) have been successfully created:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/app-quota-infractor-6655b7dcdc-9fv95	1/1	Running	0	7m38s
pod/app-quota-infractor-6655b7dcdc-b8qv6	1/1	Running	0	7m38s
pod/app-quota-infractor-6655b7dcdc-dsx7c	1/1	Running	0	7m38s
pod/app-quota-infractor-6655b7dcdc-gdfv6	1/1	Running	0	7m38s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	25h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/app-quota-infractor	4/4	4	4	7m38s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/app-quota-infractor-6655b7dcdc	4	4	4	7m38s

Step 4 ▾

Check the resources assigned to all the Pods by running the following command:

command

```
kubectl get pods \
-o custom-columns=NAME:.metadata.name,REQUEST:.spec.containers[0].resources.requests
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl get pods -o
custom-columns=NAME:.metadata.name,REQUEST:.spec.containers[0].resources.requests
```

NAME	REQUEST
app-quota-infractor-6655b7dcdc-9fv95	map[cpu:100m memory:256Mi]
app-quota-infractor-6655b7dcdc-b8qv6	map[cpu:100m memory:256Mi]
app-quota-infractor-6655b7dcdc-dsx7c	map[cpu:100m memory:256Mi]
app-quota-infractor-6655b7dcdc-gdfv6	map[cpu:100m memory:256Mi]

Explanation ▾

All the Pods display the requests as defined in the YAML file.

Task 2: Vertically scale up the Deployment by adding more resources

Step 1 ▾

Open a new terminal and watch the Pods by running the following command:

command


```
kubectl get pods -w
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control> k get
pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
app-quota-infractor-6655b7dcdc-9fv95	1/1	Running	0	21m
app-quota-infractor-6655b7dcdc-b8qvk	1/1	Running	0	21m
app-quota-infractor-6655b7dcdc-dsx7c	1/1	Running	0	21m
app-quota-infractor-6655b7dcdc-gdfv6	1/1	Running	0	21m

Step 2 ▾

In the primary terminal, edit the **deployment-for-lab-resources-ns.yaml** and increase the request as displayed below:

Example out:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-quota-infractor
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          requests:
            memory: "512Mi"
            cpu: "100m"
          limits:
            memory: "1024Mi"
            cpu: "200m"
```

Step 3 ▾

Update and vertically scale the Deployment by using the following command:

command

```
kubectl apply -f deployment-for-lab-resources-ns.yaml
```

Step 4 ▾

Switch to the terminal running the command to watch the Pods and see what happens in real time. Pods with the original resource specifications are replaced by pods with the updated configuration.

Warning ▾

Vertical scaling always involves replacing existing Pods with new ones that use the updated resource assignments.

Step 5 ▾

Check the resources assigned to all the Pods by running the following command:

command

```
kubectl get pods \
-o custom-columns=NAME:.metadata.name,REQUEST:.spec.containers[0].resources.requests
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/resource-control>
kubectl get pods -o
custom-columns=NAME:.metadata.name,REQUEST:.spec.containers[0].resources.requests
NAME                                REQUEST
app-quota-infractor-68c9dd957d-fnrwq  map[cpu:100m memory:512Mi]
app-quota-infractor-68c9dd957d-nl596  map[cpu:100m memory:512Mi]
app-quota-infractor-68c9dd957d-r8rc5  map[cpu:100m memory:512Mi]
app-quota-infractor-68c9dd957d-rjwb6  map[cpu:100m memory:512Mi]
```

Explanation ▾

All the Pods display the requests with the new values defined in the YAML file.

Step 6 ▾

Stop the watch command by pressing CTRL+C and return the cluster to the previous state by running the following commands:

command

```
kubectl delete -f deployment-for-lab-resources-ns.yaml
```

Exercise Summary

In this exercise, you created a Deployment and manually scaled the Pods vertically by changing the resource assignments in it. Once you applied the new resource definition, you watched live how Pods with the old resource assignment are replaced by new Pods with the new values.

(End of Exercise)

Section 10: Cluster Networking and Services in Kubernetes

Description

Applications running in a Kubernetes cluster can have many components and elements. Those components must be able to communicate with each other even after deployment. The Kubernetes Networking Model enables cross-cluster communication and provides additional Kubernetes entities for exposing applications to the external world.

Section Learning Objectives

- Explain the fundamentals of the Kubernetes Networking Model and the Container Network Interface(CNI).
- Justify the need for Services to abstract Pod access and address the ephemeral nature of Pods.
- Differentiate the main Service Types: ClusterIP, NodePort, and LoadBalancer.
- Configure Ingress and understand Gateway API for external access, routing, and traffic management.

Section Topics

- The Kubernetes Networking Model.
- Services in Kubernetes.
- External Service Exposure (Ingress and Gateway API).

Exercise 10.1 - Deploy Services in Kubernetes

Description

Services and a built-in DNS server are the Kubernetes mechanisms for providing a stable networking interface and DNS resolution for Pods running in the Kubernetes cluster, which are ephemeral and change IP addresses when recreated or replaced.

In this exercise, you will start by reviewing some of the most important concepts related to the Kubernetes Networking Model, and later you will create and test different types of Services to provide a stable network environment for your applications.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Retrieve network information from your Pods.
 - Verify how Pod IP addresses change when Pods are restarted or recreated.
 - Create and test a Cluster IP Service for a Kubernetes Deployment.
 - Create and test a NodePort Service for a Kubernetes Deployment.
-

Task 1: Understand why Services are needed**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select RKE201-management from the VM list in Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

The RKE201-management VM runs services for the other Lab VMs and must remain running. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and create a Deployment imperatively by running the following command:

command

```
kubectl create deployment web-server --image=httpd:2.2 --replicas=3
```

Explanation ▾

The command deploys an application with three replicas and uses the **http:2.2** container image, which is an Apache web server image.

Step 4 ▾

Use the following command to retrieve information about the Deployment and the underlying ReplicaSets and Pods:

command

```
kubectl get all -o wide
```

Explanation ▾

The **-o wide** flag displays additional information about Kubernetes objects, such as Pod IP addresses

Example output:

```
tux@management:~> kubectl get all -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
pod/web-server-5cd6f496dc-cnd84	1/1	Running	0	9m54s	10.42.0.55
pod/web-server-5cd6f496dc-fd86m	1/1	Running	0	9m54s	10.42.1.53
pod/web-server-5cd6f496dc-hcq5k	1/1	Running	0	9m54s	10.42.1.52

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	4d21h	<none>

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS
deployment.apps/web-server	3/3	3	3	9m54s	httpd

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS
replicaset.apps/web-server-5cd6f496dc	3	3	3	9m54s	httpd

Warning

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 5

Use one of the Pods' IP addresses and verify whether you can reach the web server or not by running a curl command:

command

```
curl --max-time 10 http://10.42.0.55
```

Explanation

The previous command, which tries to reach the web server and get an HTTP response, fails with a timeout error:

Example output:

```
tux@management:~> curl --max-time 10 http://10.42.0.55
curl: (28) Connection timed out after 10002 milliseconds
```

Why does the command timeout? The answer is simple:

- You are connecting to the cluster from the **RKE2-management** VM using the **kubectl** CLI client, with the authentication information in the **~/.kube/config** file.
- The **RKE2-management** VM is not part of the Kubernetes cluster, so you cannot access Pods by IP address, as these IPs are internal to the cluster.

Step 6

Open a shell in the server node by running the following command (Use tux/linux as username/password when prompted):

command

```
ssh root@server
```

Step 7 ▾

Try to reach the same Pod again from the Server node:

command

```
curl --max-time 10 http://10.42.0.55
```

Explanation ▾**Example output:**

```
server:~ # curl --max-time 10 http://10.42.0.55
<html><body><h1>It works!</h1></body></html>server:~ #
```

Now, when the **curl** command is executed from a cluster node, it receives a proper HTTP response from one of the Deployment Pods.

Note ▾

If you try the same **curl** command from the agent Node, it will also work.

Task 2: Change the Deployment image**Step 1** ▾

Still from the server node (server should display at the terminal prompt), update the image used by the Deployment using the following command:

command

```
kubectl set image deployment/web-server *=httpd:2.4.60
```

Explanation ▾

The previous **kubectl set image** command replaces the images in all (*) the containers of the **web-server** Deployment setting **httpd:2.40.60** as the new container image.

Additional Information ▾

You could also have used **kubectl edit deployment web-server** and changed the container image using your default editor.

Step 2 ▾

Use the following command to retrieve information about the Deployment and the underlying ReplicaSets and Pods:

command

```
kubectl get all -o wide
```

Explanation**Example output:**

```
server:~ # kubectl get all -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/web-server-9cfbfb549-b87hl	1/1	Running	0	19m	10.42.1.55	agent
pod/web-server-9cfbfb549-jbzs5	1/1	Running	0	19m	10.42.0.56	server
pod/web-server-9cfbfb549-p4c9x	1/1	Running	0	19m	10.42.1.56	agent

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	4d23h	<none>

NAME	IMAGES	SELECTOR	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS
deployment.apps/web-server			3/3	3	3	119m	httpd

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS
replicaset.apps/web-server-9cfbfb549	3	3	3	19m	httpd

As a result of changing the container image, Pods within the Deployment have been replaced, and now they have different IP addresses and different names

Step 3

Try to connect to one of the *web-server* Pods using the previous IP.

command

```
curl --max-time 10 http://10.42.0.55
```

Explanation**Example output:**

```
server:~ # curl --max-time 10 http://10.42.0.55
curl: (28) Connection timed out after 10002 milliseconds
```

Why does the command time out now? Even though the command is executed on the server node in the Kubernetes cluster, the previous IP is no longer used by any container. Pods in the Deployment have been replaced and assigned a new IP.

Task 3: Create a ClusterIP Service for a Deployment**Step 1**

Kubernetes Services solve the issue of ephemeral IP addresses and provide a stable network interface for applications. There are several types of Services, and in this task, you will create a ClusterIP Service. ClusterIP services provide an internal IP to reach the

workloads selected by the Service. This internal IP remains even when the underlying IPs change.

Still from the shell in the server Node, run the following command to create a YAML manifest for a Service for the **web-server** deployment.

command

```
kubectl expose deployment web-server --port=8080 --target-port=80 \
  --type=ClusterIP --name=web-service --dry-run=client \
  -o yaml > web-service.yaml
```

Explanation ▾

- **kubectl expose:** Imperative command to create a Service.
- **deployment web-server:** Sets the target workloads for the Pod. In this example, the target workload is a Deployment, but you can also choose among others, such as Pods and ReplicaSets.
- **--port=8080:** Port offered by the Service. Traffic arriving at this Port is routed to the targetPort in the containers.
- **--target-port=80:** Port in the Pods to access the web server application. If this attribute is missing, the value equals the **--port** flag value.
- **--type=ClusterIP:** Sets the value for the Service Type. **ClusterIP** is also the default value if the flag is not present.
- **--name=web-service:** Sets the name for the Service.
- **--dry-run=client:** When used, the **--dry-run** flag with the **client** value, the command prints the desired object without sending any request to the kube-apiserver. No object is created.
- **-o yaml > web-service.yaml:** Sets YAML as the output format and saves the output as a Service manifest in the web-service.yaml file.

Step 2 ▾

Review the previously created YAML manifest for the Service by running the following command:

command

```
cat web-service.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Service
metadata:
```

```

creationTimestamp: null
labels:
  app: web-server
  name: web-service
spec:
  ports:
  - port: 8080
    protocol: TCP
    targetPort: 80
  selector:
    app: web-server
  type: ClusterIP
status:
  loadBalancer: {}

```

The YAML manifest reflects all the parameters in the CLI command. However, there is a detail that is different:

- In the **kubectl expose** command, you defined the Service selecting the target Pods using the Deployment name. The Service will load balance and redirect traffic to all the Pods within the **web-server** Deployment.
- The YAML manifest, under the **spec** property, displays the **selector** section. The selector identifies target Pods for the Service using the label **app: web-server**. This label was assigned by default to the Pods when you created the Deployment from the CLI.

Warning

If the Service does not properly select the Pods, it will not route traffic to them. You will learn later how to verify that a Service has target Pods attached.

Additional Information

You can also use the **--selector** flag for the **kubectl expose** command. Refer to the [official Kubernetes documentation](#) for detailed information.

Step 3

Show labels for the Deployment Pods by running the following command:

command

```
kubectl get pods --show-labels
```

Explanation

Example output:

```

server:~ # kubectl get pods --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS

```

```

web-server-9cfbfb549-b87hl 1/1 Running 0 5h19m app=web-server,...
web-server-9cfbfb549-jbzs5 1/1 Running 0 5h19m app=web-server,...
web-server-9cfbfb549-p4c9x 1/1 Running 0 5h19m app=web-server,...

```

Step 4 ▾

Edit the **web-service.yaml** file and remove the **selector** section. The file must display as below.

Example output:

```

apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: web-server
    name: web-service
spec:
  ports:
    - port: 8080
      protocol: TCP
      targetPort: 80
  type: ClusterIP
status:
  loadBalancer: {}

```

Step 5 ▾

Create the ClusterIP Service declaratively by running the following command:

command

```
kubectl apply -f web-service.yaml
```

Step 6 ▾

Retrieve information about the Service using **kubectl get**:

command

```
kubectl get svc web-service
```

Explanation ▾

Example output:

```

server:~ # kubectl get svc web-service
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
web-service   ClusterIP   10.43.56.252 <none>        8080/TCP   3m49s

```

The ***kubectl get svc*** command provides the following important information for the Service:

- The name: ***web-service***
- The Service Type: **ClusterIP**
- The Cluster-IP: **10.43.56.63**. This is the stable internal IP offered by Kubernetes for accessing the Deployment Pods, even when their IP addresses change.
- **EXTERNAL-IP** has no value since the Service type is **ClusterIP**, which is used for internal traffic only.

Step 7 ▾

Try to reach the web server using the following command:

command

```
curl --max-time 10 http://10.43.56.252:8080
```

Explanation ▾

Example output:

```
server:~ # curl --max-time 10 http://10.43.56.252:8080
curl: (7) Failed to connect to 10.43.56.252 port 8080 after 0 ms:
Could not connect to server
```

Even when the Service has an IP, and you are sending the request from one of the cluster nodes, the curl command fails. Move to the next step to identify the problem and its solution.

Step 8 ▾

Use ***kubectl describe*** to retrieve additional information about the Service:

command

```
kubectl describe svc web-service
```

Explanation ▾

Example output:

```
Name:                web-service
Namespace:           default
Labels:              app=web-server
Annotations:         <none>
Selector:            <none>
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.43.56.252
```

```

IPs:                10.43.56.252
Port:               <unset> 8080/TCP
TargetPort:        80/TCP
Endpoints:         <none>
Session Affinity:   None
Internal Traffic Policy: Cluster
Events:            <none>

```

As you removed the selector from the YAML manifest, the Service does not know where to route the traffic, and **Endpoints** displays empty.

Step 9 ▾

Edit the **web-service.yaml** file and add back the **selector** section:

Example output:

```

apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: web-server
  name: web-service
spec:
  ports:
    - port: 8080
      protocol: TCP
      targetPort: 80
  selector:
    app: web-server
  type: ClusterIP
status:
  loadBalancer: {}

```

Apply changes by running the following command:

command

```
kubectl apply -f web-service.yaml
```

Try to reach the web server again using the

command

```
curl --max-time 10 http://10.43.56.252:8080
```

Explanation ▾

Example output:

```
server:~ # curl --max-time 10 http://10.43.56.252:8080
<html><body><h1>It works!</h1></body></html>
```

As you fixed the selector, the Service is now working properly and routing traffic to the Pods within the Deployment. You can verify that by running the **kubectl describe** command:

command

```
kubectl describe svc web-service
```

Example output:

```
server:~ # kubectl describe svc web-server
Name:                web-server
Namespace:           default
Labels:              app=web-server
Annotations:         <none>
Selector:            app=web-server
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.43.56.252
IPs:                 10.43.56.252
Port:                <unset> 8080/TCP
TargetPort:          80/TCP
Endpoints:           10.42.0.56:80,10.42.1.56:80,10.42.1.55:80
Session Affinity:    None
Internal Traffic Policy: Cluster
Events:              <none>
```

Now the Service information displays the selector and three endpoints, one for each Pod in the Deployment.

Step 10 ▾

Update the Deployment image by running the following command:

command

```
kubectl set image deployment/web-server *=httpd:2.4.66
```

Step 11 ▾

Use the following command to retrieve information about the Pods, Deployment, and Service:

command

```
kubectl get all -o wide
```

Explanation ▾

Example output:

```
server:~ # kubectl get all -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
pod/web-server-84db5fb4d7-dznbt	1/1	Running	0	15m	10.42.0.63	server
pod/web-server-84db5fb4d7-jg62r	1/1	Running	0	15m	10.42.1.66	agent
pod/web-server-84db5fb4d7-ws4vb	1/1	Running	0	14m	10.42.1.67	agent

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	5d15h
service/web-service	ClusterIP	10.43.56.252	<none>	8080/TCP	

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINER
deployment.apps/web-server	3/3	3	3	18h	httpd

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS
replicaset.apps/web-server-84db5fb4d7	3	3	3	15m	httpd

Since the Deployment was updated with a new container image:

- Pod IP addresses have changed.
- Service IP remains the same. Services provides an stable network interface even when managed workloads change their IP addresses.

Step 12 ▾

Close the SSH connection to the server node by running the **exit** command, and try to reach the service again from the **RKE2-management** VM:

command

```
curl --max-time 10 http://10.43.56.252:8080
```

Explanation ▾

Example output:

```
tux@management:~> curl --max-time 10 http://10.43.56.252:8080
curl: (28) Connection timed out after 10002 milliseconds
```

From the management terminal, the Service can still not be reached. ClusterIP services make Services available via an internal IP address, and this internal IP can not be reached from the **RKE2-management** VM, which does not belong to the cluster. It can only interact with the cluster using the **kubectl** command.

Task 4: Verify DNS resolution in your Kubernetes cluster

Step 1 ▾

Review the cluster components by running the following command:

command

```
kubectl get pods -n kube-system
```

Explanation**Example output:**

```
tux@management:~> kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
cloud-controller-manager-server	1/1	Running	11 (53m ago)	5d15h
etcd-server	1/1	Running	5	5d15h
helm-install-rke2-canal-46t9p	0/1	Completed	0	5d15h
helm-install-rke2-coredns-dtw9w	0/1	Completed	0	5d15h
helm-install-rke2-metrics-server-fh4q7	0/1	Completed	0	5d15h
helm-install-rke2-runtimeclasses-gn92t	0/1	Completed	0	5d15h
helm-install-rke2-snapshot-controller-crd-8p5v8	0/1	Completed	0	5d15h
helm-install-rke2-snapshot-controller-wjftq	0/1	Completed	2	5d15h
helm-install-rke2-traefik-crd-gp4kl	0/1	Completed	0	5d15h
helm-install-rke2-traefik-gw2fv	0/1	Completed	2	5d15h
kube-apiserver-server	1/1	Running	5	5d15h
kube-controller-manager-server	1/1	Running	10 (53m ago)	5d15h
kube-proxy-agent	1/1	Running	0	50m
kube-proxy-server	1/1	Running	0	53m
kube-scheduler-server	1/1	Running	5 (53m ago)	5d15h
rke2-canal-72vks	2/2	Running	10 (50m ago)	5d15h
rke2-canal-bbnsp	2/2	Running	10 (53m ago)	5d15h
rke2-coredns-rke2-coredns-76994ddb78-l4j8c	1/1	Running	5 (53m ago)	5d15h
rke2-coredns-rke2-coredns-76994ddb78-rg4c4	1/1	Running	0	50m
rke2-coredns-rke2-coredns-autoscaler-67944c44cc-wq97h	1/1	Running	5 (53m ago)	5d15h
rke2-metrics-server-5699475d99-l92zw	1/1	Running	5 (53m ago)	5d15h
rke2-snapshot-controller-696989ffdd-6b5cf	1/1	Running	5 (53m ago)	5d15h
rke2-traefik-mn9f6	1/1	Running	5 (53m ago)	5d15h
rke2-traefik-xv8kp	1/1	Running	5 (50m ago)	5d15h

Main Kubernetes components run in the **kube-system** namespace, which is selected in the **kubectl get** command with the **-n kube-system** flag.

The Pod list in the output displays CoreDNS Pods, the built-in DNS service for the RKE2 Kubernetes cluster.

Step 2

Create a temporary Pod using the busybox image and the flag **--rm**, and access a shell within it by running the following command:

command

```
kubectl run temporary-pod --rm -it --image=busybox -- sh
```

Explanation

- **kubectl run temporary-pod**: Imperative command for creating a Pod named **temporary-pod**.
- **--rm**: Flag for deleting the Pod after it exits.
- **-it**: Interactive when opening a shell in the Pod.

- **--image=busybox**: Container image for the Pod. This image has network tools for testing DNS resolution.
- **--sh**: Command to be executed on the Pod. It opens a new shell.

Step 3

Test DSN resolution from the **temporary-pod** by running the following commands:

commands

```
> nslookup web-service.default.svc.cluster.local
> nslookup 10-42-0-65.default.pod.cluster.local
```

Explanation**Example output:**

```
/ # nslookup web-service.default.svc.cluster.local
Server:      10.43.0.10
Address:     10.43.0.10:53

Name: web-service.default.svc.cluster.local
Address: 10.43.56.252

/ # nslookup 10-42-0-65.default.pod.cluster.local
Server:      10.43.0.10
Address:     10.43.0.10:53

Name: 10-42-0-65.default.pod.cluster.local
Address: 10.42.0.65
```

DNS resolution is working internally for FQDN the Service previously created (web-service.default.svc.cluster.local), and one of the Pods in the Deployment (10-42-0-65.default.pod.cluster.local).

In both examples, the DNS server **10.43.0.10** provides the DNS resolution.

Note

IPs for the Service and Pods may be different in your cluster.

Step 4

Run **exit** to leave the temporary Pod and run the following command from the terminal in the **RKE2-management VM**:

command

```
kubectl get svc -A
```

Explanation ▾

Example output:

```
tux@management:~> kubectl get svc -A
NAMESPACE      NAME                      TYPE        CLUSTER-IP      EXTERNAL-IP
default        kubernetes               ClusterIP    10.43.0.1        <none>
default        web-service              ClusterIP    10.43.56.252     <none>
kube-system    rke2-coredns-rke2-coredns ClusterIP    10.43.0.10       <none>
kube-system    rke2-metrics-server      ClusterIP    10.43.148.33     <none>
kube-system    rke2-traefik             ClusterIP    10.43.128.52     <none>
```

10.43.0.10 is an IP address assigned to the CoreDNS service in the cluster. This Service is internal to the cluster (ClusterIP) and is configured as the default DNS server for all Pods.

Warning ▾

nslookup web-service.default.svc.cluster.local and *nslookup*

10-42-0-65.default.pod.cluster.local fail if executed from the **RKE2-management** terminal, as this machine is outside the cluster and uses a different DNS server.

Step 5 ▾

Use the following command to remove the Deployment and the Service, and clean the cluster:

commands

```
> kubectl delete svc web-service
> kubectl delete deployment web-server
```

Task 5: Create a NodePort Service for a Deployment

Step 1 ▾

ClusterIP Services are internal Services that can only be reached from within the Kubernetes cluster. They are useful for internal Kubernetes services, such as the CoreDNS presented earlier, or for Services you don't want to expose to the external world, such as backend databases. However, you may need to expose your application to the external world. That is possible using several methods, such as NodePort services.

Create the Deployment that will act as the backend for the Service by running the following command:

command

```
kubectl create deployment nginx-deployment --image=nginx --replicas=2
```

Explanation ▾

The command deploys an application with three replicas and uses the **http:2.2** container image, which is an Apache web server image.

Step 2 ▾

Verify that the Deployment and the Pods have been successfully created by running the following command:

command

```
kubectl get all
```

Explanation ▾**Example output:**

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-deployment-6cfb98644c-6ptm6	1/1	Running	0	4m21s
pod/nginx-deployment-6cfb98644c-pb2gk	1/1	Running	0	4m21s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	5d17h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx-deployment	2/2	2	2	4m21s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nginx-deployment-6cfb98644c	2	2	2	4m21s

Step 3 ▾

Run the following command to create a YAML manifest for a Service for the **nginx-deployment** deployment.

command

```
kubectl expose deployment nginx-deployment --port=80 \
--target-port=80 --type=NodePort --name=nginx-service \
--dry-run=client -o yaml > nginx-service.yaml
```

Explanation ▾

- **kubectl expose:** Imperative command to create a Service.
- **deployment nginx-deployment:** Sets the target workloads for the Pod. In this example, the target workload is a Deployment, but you can also choose among others, such as Pods and ReplicaSets.
- **--port=80:** Port offered by the Service. Traffic arriving at this Port is routed to the targetPort in the containers.

- **--target-port=80:** Port in the Pods to access the web server application. If this attribute is missing, the value equals the **--port** flag value.
- **--type=NodePort:** Sets the value for the Service Type
- **--name=nginx-service:** Sets the name for the service..
- **--dry-run=client:** When used, the **--dry-run** flag with the **client** value, the command prints the desired object without sending any request to the kube-apiserver. No object is created.
- **-o yaml > nginx-service.yaml:** Sets YAML as the output format and saves the output as a Service manifest in the web-service.yaml file.

Step 4 ▾

Review the previously created YAML manifest for the Service by running the following command:

command

```
cat nginx-service.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: nginx-deployment
    name: nginx-service
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: nginx-deployment
    type: NodePort
status:
  loadBalancer: {}
```

Step 5 ▾

Create the Service by running the following command:

command

```
kubectl apply -f nginx-service.yaml
```

Step 6

Retrieve information about the Service:

command

```
kubectl get service
```

Explanation**Example output:**

```
tux@management:~> kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	5d17h
nginx-service	NodePort	10.43.40.81	<none>	80:32560/TCP	2m20s

NodePort services are built on top of ClusterIP services, so when you create a NodePort Service, Kubernetes also assigns an internal IP to it. This internal IP displays in the **CLUSTER-IP** column. What is new for a NodePort Service is the content in the **PORTS** column. Since the Service is a NodePort service, port 32560 is now open on all cluster Nodes, and Kubernetes redirects the traffic to that port to port 80 in the Service.

Step 7

Verify you can reach the web server by running the following commands:

commands

```
> curl http://server:32560
> curl http://agent:32560
```

Explanation**Example output:**

```
tux@management:~> curl http://server:32560
<!DOCTYPE html>
<html>
...
<body>
...
<h1>Welcome to nginx!</h1>
</body>
</html>
tux@management:~> curl http://agent:32560
<!DOCTYPE html>
<html>
...
<body>
...
<h1>Welcome to nginx!</h1>
</body>
```

```
</html>
```

Since the curl HTTP request is made from the **RKE2-management** VM, which is outside the cluster, the CLUSTER-IP internal address can not be used to reach the web application. However, NodePort services expose applications to the external world using a port in the 30000-32767 range. The application can now be reached using a Node name or IP address and the port. The port is randomly assigned, but can be set in the YAML file using the nodePort property in the **ports** section.

Step 8 ▾

Use the following command to remove the Deployment and the Service, and clean the cluster:

commands

```
> kubectl delete svc nginx-service  
> kubectl delete deployment nginx-deployment
```

Exercise Summary

In this exercise, you have practically demonstrated why Services, which provide a stable network interface for applications in Kubernetes, are needed. You have also created and tested a ClusterIP Service and a NodePort Service, and also tested DNS resolution within the Kubernetes cluster.

(End of Exercise)

Exercise 10.2 - Expose an application using Ingress

Description

Kubernetes Services have some inconveniences when you expose your Kubernetes-deployed applications to the external world. Services don't provide smart-routing, so you can not redirect different HTTP/HTTPS routes to different services, and you need to manage SSL/TLS certificates manually for each Service offering HTTPs. To address this scenario, Kubernetes provides Ingress and the Gateway API, which is intended to replace Ingress.

In this exercise, you will create Ingress objects for HTTP and HTTPS services.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Understand why Services are needed for the Kubernetes Networking Model.
 - Expose an HTTP and HTTPS Services to the external world using Ingress.
-

Task 1: Configure Ingress

Step 1 ▾

To make Ingress resources work in your cluster, you must first install an Ingress controller. The RKE2 Kubernetes cluster was installed with [Traefik](#) as the Ingress controller. Run the following command to display all the Pods in the **kube-system** namespace:

command

```
kubectl get pods -n kube-system
```

Explanation ▾

Example output:

```
tux@management:~$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
cloud-controller-manager-server	1/1	Running	11 (7h13m ago)	5d22h
etcd-server	1/1	Running	5	5d22h
helm-install-rke2-canal-46t9p	0/1	Completed	0	5d22h
helm-install-rke2-coredns-dtw9w	0/1	Completed	0	5d22h
helm-install-rke2-metrics-server-fh4q7	0/1	Completed	0	5d22h
helm-install-rke2-runtimeclasses-gn92t	0/1	Completed	0	5d22h
helm-install-rke2-snapshot-controller-crd-8p5v8	0/1	Completed	0	5d22h
helm-install-rke2-snapshot-controller-wjftq	0/1	Completed	2	5d22h
helm-install-rke2-traefik-crd-gp4kl	0/1	Completed	0	5d22h
helm-install-rke2-traefik-gw2fv	0/1	Completed	2	5d22h
kube-apiserver-server	1/1	Running	5	5d22h
kube-controller-manager-server	1/1	Running	10 (7h13m ago)	5d22h
kube-proxy-agent	1/1	Running	0	7h10m
kube-proxy-server	1/1	Running	0	7h13m
kube-scheduler-server	1/1	Running	5 (7h13m ago)	5d22h
rke2-canal-72vks	2/2	Running	10 (7h10m ago)	5d22h
rke2-canal-bbnsf	2/2	Running	10 (7h13m ago)	5d22h
rke2-coredns-rke2-coredns-76994ddb78-l4j8c	1/1	Running	5 (7h13m ago)	5d22h
rke2-coredns-rke2-coredns-76994ddb78-rg4c4	1/1	Running	0	7h10m
rke2-coredns-rke2-coredns-autoscaler-67944c44cc-wq97h	1/1	Running	5 (7h13m ago)	5d22h
rke2-metrics-server-5699475d99-l92zw	1/1	Running	5 (7h13m ago)	5d22h
rke2-snapshot-controller-696989ffdd-6b5cf	1/1	Running	5 (7h13m ago)	5d22h
rke2-traefik-mn9f6	1/1	Running	5 (7h13m ago)	5d22h
rke2-traefik-xv8kp	1/1	Running	5 (7h10m ago)	5d22h

Traefik Pods are running and ready for using Ingress resources. Traefik deploys the Ingress controller as a DaemonSet (one Pod per cluster Node).

Step 2 ▾

Create two Deployments to be externally exposed using the Ingress resource by running the following commands:

Application A (nginx web server)

```
kubectl create deployment nginx-deployment --image=nginx --replicas=3
```

Application B (apache web server)

```
kubectl create deployment apache-deployment --image=httpd --replicas=4
```

Step 3

Create two Services for the Deployment by running the following commands:

command: Service for Application A (nginx-service)

```
kubectl expose deployment nginx-deployment --port=80 \
  --type=ClusterIP --name=nginx-service
```

command: Service for Application B (apache-service)

```
kubectl expose deployment apache-deployment --port=80 \
  --type=ClusterIP --name=apache-service
```

Step 4

Verify that Pods, Deployments, and Services have been successfully created by running the following command:

command:

```
kubectl get all
```

Example output:

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/apache-deployment-84d5d98865-8wqz2	1/1	Running	0	16m
pod/apache-deployment-84d5d98865-f98jb	1/1	Running	0	16m
pod/apache-deployment-84d5d98865-jlk9v	1/1	Running	0	16m
pod/apache-deployment-84d5d98865-q7fbp	1/1	Running	0	16m
pod/nginx-deployment-6cfb98644c-pmjzj	1/1	Running	0	17m
pod/nginx-deployment-6cfb98644c-tmjj7	1/1	Running	0	17m
pod/nginx-deployment-6cfb98644c-x84r9	1/1	Running	0	17m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/apache-service	ClusterIP	10.43.30.42	<none>	80/TCP
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP
service/nginx-service	ClusterIP	10.43.181.87	<none>	80/TCP

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/apache-deployment	4/4	4	4	16m
deployment.apps/nginx-deployment	3/3	3	3	17m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/apache-deployment-84d5d98865	4	4	4	16m
replicaset.apps/nginx-deployment-6cfb98644c	3	3	3	17m

Note

Feel free to create a YAML file from scratch or using the `--dry-run=client` flag. For the sake of speed and simplicity, Deployments and Services are created using CLI commands, the imperative approach.

Step 5 ▾

Review the YAML manifest for an Ingress object for the ***nginx-service*** by running the following commands

commands:

```
> cd ~/course_files/RKE201v2/labs/manifests/networking/
> cat single-service-ingress.yaml
```

Explanation ▾

Example output:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: single-service-ingress
spec:
  ingressClassName: traefik
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nginx-service
            port:
              number: 80
```

- **apiVersion:** [networking.k8s.io/v1](#):
- **kind: Ingress:** Type of Kubernetes object to create.
- **ingressClassName: traefik:** Since Traefik is the Ingress controller in the Labs' cluster.
- **Rules:** Information for redirecting HTTP/HTTPS requests
 - **path: /:** URL path to be routed
 - **pathType: Prefix:** Stands for any URL starting with /. All the URLs for the Service.
 - **backend:** Requests to the / path in the are redirected to port 80 in the ***nginx-service***.

Additional Information ▾

Review the [official Kubernetes documentation](#) for additional options and information.

Step 6 ▾

Create the Ingress resource by running the following command:

command:

```
kubectl apply -f single-service-ingress.yaml
```

Step 7 ▾

Retrieve information about the Ingress resource by running one of the following commands:

commands

```
> kubectl get ingress single-service-ingress
> kubectl describe ingress single-service-ingress
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/networking> kubectl describe
ingress single-service-ingress
Name:          single-service-ingress
Labels:        <none>
Namespace:     default
Address:
Ingress Class: traefik
Default backend: <default>
Rules:
  Host      Path  Backends
  ----      -
  *         /    nginx-service:80 (10.42.0.80:80,10.42.1.109:80,10.42.1.108:80)
Annotations: <none>
Events:      <none>
```

Note that the Ingress path is correctly tied to the Service and its Pods.

Step 8 ▾

Verify that Ingress routes request to the `/` Path to the **nginx-service** by running the following command from the **RKE2-management** VM, which does not belong to the cluster:

command

```
curl http://server.example.com/
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/networking> curl
http://server.example.com/
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</head>
<body>
<h1>Welcome to nginx!</h1>
...
</body>
</html>
```

Step 9 ▾

Use the following command to remove the Ingress object and clean the cluster for the next task:

command

```
kubectl delete ingress single-service-ingress
```

Task 2: Configure Ingress for multiple Services**Step 1** ▾

Different vendors have different Ingress controller implementations. Versions from vendors, even when they follow the same Kubernetes Ingress API specification, these implementations also provide custom capabilities.

Traefik is the Ingress controller in the Lab, and using this Ingress implementation to create an Ingress resource with two paths that route traffic to two Services requires a “special” Middleware Kubernetes object provided by Traefik. Review the Middleware by running the following commands:

commands:

```
> cd ~/course_files/RKE201v2/labs/manifests/networking/
> cat multi-service-traefik-middleware.yaml
```

Explanation ▾**Example output:**

```
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: clean-routes
spec:
```

```
stripPrefix:
  prefixes:
    - /serviceA
    - /serviceB
```

This Traefik Middleware cleans URL prefixes from the HTTP request.

- Requests to **/serviceA** will be routed to the root path **/** in the target Service.
- Requests to **/serviceB** will be routed to the root path **/** in the target Service.

If paths are not cleaned, requests are sent to the **/serviceA** path in the Service.

Note ▾

Review your Ingress controller information for specific configuration options.

Step 2 ▾

Create the Middleware by running the following command:

command

```
kubectl apply -f multi-service-traefik-middleware.yaml
```

Step 3 ▾

Review the Ingress object, which defines two paths for two services, by running the following commands:

commands:

```
> cd ~/course_files/RKE201v2/labs/manifests/networking/
> cat multi-service-ingress.yaml
```

Explanation ▾

Example output:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-service-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.middlewares: default-clean-routes@kubernetescrd
spec:
  ingressClassName: traefik
  rules:
    - http:
        paths:
          - path: /serviceA
            pathType: Prefix
            backend:
              service:
                name: nginx-service
                port:
                  number: 80
```

```
- path: /serviceB
  pathType: Prefix
  backend:
    service:
      name: apache-service
      port:
        number: 80
```

The Ingress object definition is pretty similar to the **single-service** Ingress but with some important differences:

- It has an annotation ***traefik.ingress.kubernetes.io/router.middlewares: default-clean-routes@kubernetescrd*** that refers to the previously created Middleware. Note the value: ***default*** (namespace) ***clean-routes*** (Middleware name) ***kubernetescrd*** (as Middleware is a Kubernetes extension object defined by Traefik. CRDs are covered later in this course).
- It has two paths instead of one:
 - ***/serviceA*** HTTP requests are redirected, via the Middleware that cleans the prefix, to the path ***/*** for the ***nginx-service***.
 - ***/serviceB*** HTTP requests are redirected via the Middleware that cleans the prefix to the path ***/*** for the ***apache-service***.

Step 4 ▾

Create the Ingress object by running the following command:

command

```
kubectl apply -f multi-service-ingress.yaml
```

Step 5 ▾

Run the following commands and verify that you can access both Services from the ***RKE2-management*** VM using the Kubernetes server node name :

commands:

```
> curl http://server.example.com/serviceA
> curl http://server.example.com/serviceB
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/networking> curl
http://server.example.com/serviceA
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</head>
<body>
```

```

<h1>Welcome to nginx!</h1>
...
</body>
</html>
tux@management:~/course_files/RKE201v2/labs/manifests/networking> curl
http://server.example.com/serviceB
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>

```

Task 3: Configure Ingress using TLS

Step 1 ▾

Run the following commands and verify that you can access both Services from the **RKE2-management** VM using the Kubernetes server node name :

commands:

```

> curl --insecure https://server.example.com/serviceA
> curl --insecure https://server.example.com/serviceB

```

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/networking> curl
--insecure https://server.example.com/serviceA
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</head>
<body>
<h1>Welcome to nginx!</h1>
...
</body>
</html>
tux@management:~/course_files/RKE201v2/labs/manifests/networking> curl
--insecure https://server.example.com/serviceB
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>

```



```
</head>
<body>
<p>It works!</p>
</body>
</html>
```

Explanation ▾

HTTPS requests routed from Ingress to the Services work because the Traefik Ingress controller installed in your RKE2 Kubernetes cluster provides a default certificate. If you plan to use your personal or your company's certificate, you must:

- Create a TLS secret.
- Modify the Ingress object and the ***tls*** property under the ***spec*** section to select the TLS secret, as in the example below:

Example output:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-service-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.middlewares:
default-clean-routes@kubernetescrd
spec:
  tls:
  - secretName: your-tls-secret
  ingressClassName: traefik
  rules:
  - http:
    paths:
    - path: /serviceA
      pathType: Prefix
      backend:
        service:
          name: nginx-service
          port:
            number: 80
    - path: /serviceB
      pathType: Prefix
      backend:
        service:
          name: apache-service
          port:
            number: 80
```

Step 2 ▾

Use the following command to remove the Ingress and Middleware objects, and clean the cluster for the next task:

commands:

```
> kubectl delete ingress multi-service-ingress
> kubectl delete middleware clean-routes
```

Warning

Do not remove Services or Deployments. They will be used in the following exercise.

Exercise Summary

In this exercise, you have verified that the Kubernetes cluster had the Traefik Ingress controller installed. Once this was confirmed, you created and tested an Ingress resource for a single service, you did the same for a multi-service Ingress resource with two paths, previously defining the required Middleware for the Traefik Ingress controller, and finally, you learned how to accept HTTPS requests in your Ingress resources.

(End of Exercise)

Exercise 10.3 - Expose an application using Gateway API

Description

Ingress resources have some limitations: they only work at the application Layer, support only HTTP and HTTPS protocols, and do not support advanced traffic routing. Since October 2023, Gateway API, a Kubernetes add-on installed as Custom Resource Definitions (CRDs), has been GA. Gateway API addresses Ingress limitations and provides additional capabilities.

In this exercise, you will verify whether the Gateway API is available on your system, install it if it is not, and then create Gateway API routes.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Verify if the Gateway API is already installed on your Kubernetes cluster.
 - Find out information on installing the Gateway API
 - Create Gateway API routes for Services in your Kubernetes cluster.
-

Task 1: Verify if the Gateway API is available in the Kubernetes cluster

Step 1 ▾

In RKE2, because we are using Traefik, the Gateway API is installed by default, but not enabled. You can verify that the Gateway API is installed by running the following command:

command

```
kubectl get crds | grep gateway.networking.k8s.io
```

Explanation ▾

Example output:

backendtlspolicies.gateway.networking.k8s.io	2026-01-17T19:50:10Z
gatewayclasses.gateway.networking.k8s.io	2026-01-17T19:50:10Z
gateways.gateway.networking.k8s.io	2026-01-17T19:50:10Z
gRPCroutes.gateway.networking.k8s.io	2026-01-17T19:50:10Z
httproutes.gateway.networking.k8s.io	2026-01-17T19:50:10Z
referencegrants.gateway.networking.k8s.io	2026-01-17T19:50:10Z

In RKE2, the Gateway API is installed as a part of the Traefik Helm chart (Kubernetes package) during the installation process. During installation, Kubernetes creates the extension objects, known as Custom Resource Definitions (CRDs), and the operators needed to make those extensions work. Since the output for the ***kubectl get crds*** command displays the required GatewayAPI CRS, it also confirms that the Gateway API was installed during the default RKE2 installation process.

Helm and CRDs are covered in detail later in this course.

Additional Information ▾

The Kubernetes API can also be manually installed in your Kubernetes cluster. Refer to the [Gateway API documentation](#) site for more information.

Step 2 ▾

Use the following command to verify if the GatewayAPI is enabled:

command

```
helm get values rke2-traefik -n kube-system
```

Explanation ▾

Example output:

```
USER-SUPPLIED VALUES:
global:
  clusterCIDR: 10.42.0.0/16
```

```
clusterCIDRv4: 10.42.0.0/16
clusterDNS: 10.43.0.10
clusterDomain: cluster.local
rke2DataDir: /var/lib/rancher/rke2
serviceCIDR: 10.43.0.0/16
systemDefaultIngressClass: traefik
```

Since the Helm chart was deployed during the RKE2 installation process in the **kube-system** namespace, you can retrieve its configuration values using **helm get values**. No information about the Gateway API displays; therefore, it is not enabled.

Step 3 ▾

Enable the GatewayAPI by running the following commands:

command

```
> cd ~/course_files/RKE201v2/labs/manifests/networking/
> scp enable-gateway-api.yaml root@server:/var/lib/rancher/rke2/server/manifests/
```

Explanation ▾

The **enable-gateway-api.yaml** contains the definition for a HelmChartConfig Kubernetes object that overrides the Traefik configuration. HelmChartConfig is a Traefik Custom Resource Definition (CRD) and is out of scope for this course. Once this file is installed in the server's directory, Traefik is reinstalled again using the new configuration that enables GatewayAPI. RKE2 periodically scans the manifests directory for new manifests for the Kubernetes components.

Step 4 ▾

Use the following command to verify again if the GatewayAPI is enabled:

command

```
helm get values rke2-traefik -n kube-system
```

Explanation ▾

Example output:

```
USER-SUPPLIED VALUES:
global:
  clusterCIDR: 10.42.0.0/16
  clusterCIDRv4: 10.42.0.0/16
  clusterDNS: 10.43.0.10
  clusterDomain: cluster.local
  rke2DataDir: /var/lib/rancher/rke2
  serviceCIDR: 10.43.0.0/16
  systemDefaultIngressClass: traefik
```

```

providers:
  kubernetesGateway:
    enabled: true

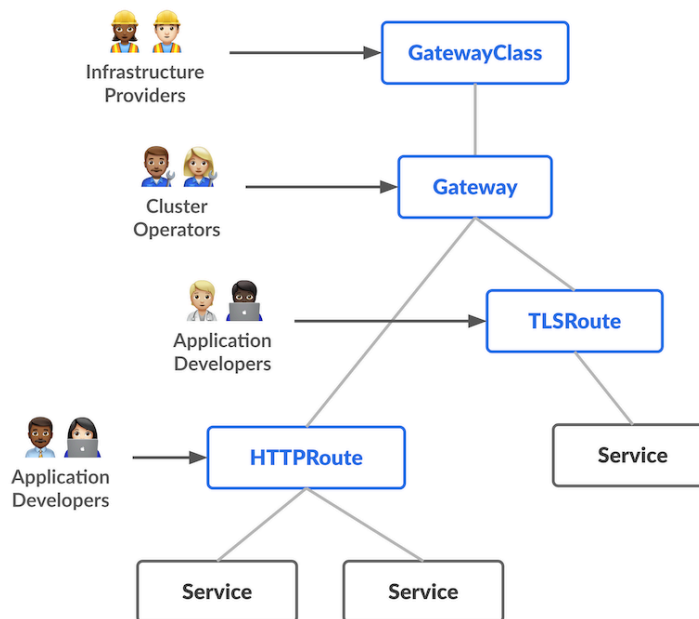
```

The GatewayAPI is now enabled in the **providers** section at the end.

Task 2: Create the required Kubernetes Gateway API objects for the HTTP and HTTPS services

Step 1 ▾

Before exposing HTTP and HTTPS services within your cluster to the external world using the Kubernetes Gateway API, several Kubernetes objects must be created, as shown in the following diagram:



Source: <https://gateway-api.sigs.k8s.io/>

The **GatewayClass**, supposed to be deployed by infrastructure providers, was deployed during the RKE2 installation process. You can verify that the GatewayClass is ready by running the following command:

command

```
kubectl describe gatewayclass traefik
```

Explanation ▾

Example output:

```

Name:      traefik
Namespace:

```

```

Labels:      app.kubernetes.io/instance=rke2-traefik-kube-system
             app.kubernetes.io/managed-by=Helm
             app.kubernetes.io/name=rke2-traefik
             helm.sh/chart=rke2-traefik-37.4.001
Annotations: meta.helm.sh/release-name: rke2-traefik
             meta.helm.sh/release-namespace: kube-system
API Version: gateway.networking.k8s.io/v1
Kind:        GatewayClass
Metadata:
  Creation Timestamp:  2026-01-19T09:41:48Z
  Generation:         1
  Resource Version:    760355
  UID:                143ad12e-a4a2-4d83-90de-8adbfb39a15a
Spec:
  Controller Name:    traefik.io/gateway-controller
Status:
  Conditions:
    Last Transition Time:  2026-01-19T09:41:49Z
    Message:              Handled by Traefik controller
    Observed Generation:  1
    Reason:               Handled
    Status:               True
    Type:                 Accepted
  Supported Features:
    Name: BackendTLPolicy
    Name: GRPCRoute
    Name: Gateway
    Name: GatewayPort8080
    Name: HTTPRoute
    Name: HTTPRouteBackendProtocolH2C
    Name: HTTPRouteBackendProtocolWebSocket
    Name: HTTPRouteDestinationPortMatching
    Name: HTTPRouteHostRewrite
    Name: HTTPRouteMethodMatching
    Name: HTTPRoutePathRedirect
    Name: HTTPRoutePathRewrite
    Name: HTTPRoutePortRedirect
    Name: HTTPRouteQueryParamMatching
    Name: HTTPRouteResponseHeaderModification
    Name: HTTPRouteSchemeRedirect
    Name: ReferenceGrant
    Name: TLSRoute
Events:  <none>

```

Note the supported features in the end.

Step 2 ▾

Since you plan to expose applications using HTTPS, you must create the required certificate and a Kubernetes TLS secret. First, create the certificate by running the following command:

command

```

openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key \
-out tls.crt -subj "/CN=server"

```

Create the Kubernetes TLS secret, required later for the Gateway definition, by running the following command:

command

```
kubectl create secret tls tls-secret --cert=tls.crt --key=tls.key
```

Step 3 ▾

Review the YAML manifest for the Gateway Kubernetes object, which refers to the *traefik* GatewayClass, by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/networking
> cat lab-gateway.yaml
```

Explanation ▾

Example output:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: lab-gateway
spec:
  gatewayClassName: traefik
  listeners:
    - name: web
      protocol: HTTP
      port: 8000
      allowedRoutes:
        namespaces:
          from: All
    - name: websecure
      protocol: HTTPS
      port: 8443
      allowedRoutes:
        namespaces:
          from: All
  tls:
    mode: Terminate
    certificateRefs:
      - name: tls-secret
```

In this YAML manifest:

- **apiVersion: gateway.networking.k8s.io/v1:** Sets the required API for the Gateway API objects.
- **kind: Gateway:** Sets the type of object to be created.

- **gatewayClassName: traefik**: Bind the Gateway to the **traefik** GatewayClass, which was previously described.
- Describe two listeners:
 - The **web** listener for external HTTP requests. Traffic from the external port 80 will be redirected to port 8000 in the Traefik system. These values, web and 8000, are default values in the Helm chart.
 - The **websecure** listener for external HTTPS requests. Traffic from the external port 443 will be redirected to port 8443 in the Traefik system. These values, websecure and 443, are default values in the Helm chart.
- For both listeners, the **allowedRoutes** section limits requests for this Gateway for **HTTP** routes defined in the **Same** namespace as the Gateway, the default namespace for us.
- The **tls** section for the **http** listener holds two important configuration settings:
 - **mode: Terminate**: This means Traefik terminates the encryption here. It receives the encrypted traffic from the client, decrypts it using the certificate, and passes it as plain HTTP to the internal services (Pods).
 - **certificateRefs**: It refers to the TLS secret containing the certificate and the key by name.

Step 4 ▾

Create the Gateway by running the following command:

command

```
kubectl apply -f lab-gateway.yaml
```

Step 5 ▾

Verify that the Gateway object has been successfully created by running the following command:

command

```
kubectl get gateway lab-gateway
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/networking>
kubectl get gateway lab-gateway
NAME          CLASS    ADDRESS    PROGRAMMED    AGE
lab-gateway   traefik                True          41m
```

If the **PROGRAMMED** column displays **TRUE**, the Gateway has been successfully created.

Step 6 ▾

Review the YAML manifest for the HTTPRoute by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/networking
> cat http-and-https-routes.yaml
```

Explanation ▾**Example output:**

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: multi-service-http-and-https-route
spec:
  parentRefs:
    - name: lab-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /serviceA
      filters:
        - type: URLRewrite
          urlRewrite:
            path:
              type: ReplacePrefixMatch
              replacePrefixMatch: /
      backendRefs:
        - name: nginx-service
          port: 80
    - matches:
        - path:
            type: PathPrefix
            value: /serviceB
      filters:
        - type: URLRewrite
          urlRewrite:
            path:
              type: ReplacePrefixMatch
              replacePrefixMatch: /
      backendRefs:
        - name: apache-service
          port: 80
```

In the YAML manifest:

- **parentRefs:** This binds the HTTPRoute to the **lab-gateway** Gateway, attaching all the compatible listeners defined in it (web-HTTP, websecure-HTTPS).

- **rules:** Defines two different traffic paths based on the URL prefix:
 - Rules of URLs starting with **/serviceA**
 - Rules of URLs starting with **/serviceB**
- Both rules use the **URLRewrite** filter:
 - **type: ReplacePrefixMatch:** This tells the controller to look for the matched part of the path.
 - **replacePrefixMatch: /:** It strips the prefix (e.g., /serviceA) and replaces it with a forward slash. Example: A request for https://<IP>/serviceA/login is sent to the backend as https://<IP>/login.
- Both rules redirect the traffic to a backend:
 - **nginx-service:** Traffic matched by the first rule is forwarded to the **nginx-service** service on port 80.
 - **apache-service:** Traffic matched by the second rule is forwarded to the **apache-service** service on port 80.

Step 7 ▾

Create the HTTPRoute object by creating the following command:

command

```
kubectl apply -f http-and-https-routes.yaml
```

Step 8 ▾

Verify that the HTTPRoute object has been successfully created by running the following command:

command

```
kubectl get httproute multi-service-http-and-https-route
```

Verify that the HTTPRoute object has been successfully created by running the following commands:

commands

```
> curl http://server.example.com/serviceA
> curl http://server.example.com/serviceB
> curl --insecure https://server.example.com/serviceA
> curl --insecure https://server.example.com/serviceB
```

Step 9 ▾

Use the following commands to remove everything and leave the RKE Kubernetes cluster in a clean status for the next section:

commands:

```
> kubectl delete svc nginx-service
> kubectl delete svc apache-service
> kubectl delete deployment nginx-deployment
> kubectl delete deployment apache-deployment
> kubectl delete -f http-and-https-routes.yaml
> kubectl delete -f lab-gateway.yaml
```

Exercise Summary

In this exercise, you started by enabling Gateway API in your RKE2 Kubernetes cluster. Later, you defined a Gateway object with dual listeners for HTTP (port 80) and HTTPS (port 443), and finally, an HTTPRoute was deployed to implement path-based routing for multiple services, using URLRewrite filters to strip prefixes before forwarding traffic to the backend services.

(End of Exercise)

Section 11: Storage in Kubernetes

Description

Pods, which are ephemeral, also use a temporary filesystem, which represents a significant challenge for deploying real-world applications in a Kubernetes cluster. If the filesystem is also ephemeral, data is lost if the Pod shuts down, and applications, such as Databases or Content Managers, need persistence. Moreover, Pods can have multiple containers, and containers within the same Pod cannot share files by default.

Kubernetes provides several methods and abstractions to address these challenges and persist applications' data.

Section Learning Objectives

- Justify persistent storage: Explain why Volumes are needed to overcome the ephemeral nature of container filesystems and the Pod lifecycle.
- Understand core storage objects: Define and relate Volumes, PVs, PVCs, and StorageClasses.
- Implement static provisioning: Understand the process of manually pre-creating a PV for a PVC to bind to (used for existing volumes).
- Implement dynamic provisioning: Understand the process of manually pre-creating a PV for a PVC to bind to (used for existing volumes).
- Analyze CSI: Describe how the Container Storage Interface (CSI) decouples Kubernetes storage logic from specific vendor implementations.

Section Topics

- The need for persistent storage.
- Storage abstractions (Volumes, PVs, PVCs, SCs)
- Provision storage.
- The Containers Storage Interface (CSI).

Exercise 11.1 - Use an emptyDir volume to share files between containers

Description

Pods can have multiple containers. By default, these containers have a separate filesystem and cannot share any files. Kubernetes solves this problem with Volumes, which can be ephemeral (existing as long as the Pod exists) or persistent (keeping data alive even when the Pod no longer exists). EmptyDir, ConfigMap, and Secrets are examples of an ephemeral Volume. ConfigMap and Secrets were covered in previous sections. EmptyDirs are typically used to collect logs, share files as checkpoints, or share configuration files between containers.

In this exercise, you demonstrate that, by default, containers in a multicontainer Pod have separate filesystems, and later you will create and mount an emptyDir volume to share logs between an application and a sidecar container that processes or sends them to a third-party log collector.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Define a multicontainer Pod
 - Access to a single container in a multicontainer Pod.
 - Define and mount an emptyDir volume in a multicontainer Pod to share files between containers.
-

Task 1: Separate file systems in multicontainer Pods.**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and review the YAML manifest for a simple multicontainer Pod by running the following commands:

command

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat multi-container-pod.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
    - name: busybox-container
```

```
image: busybox
command:
- "sleep"
- "3600"
```

The YAML manifest defines a multicontainer Pod with two containers:

- An application container running the **nginx** web server and exposing the web service on port 80.
- A sidecar container using the **busybox** image and running a **sleep** command on startup.

Step 4 ▾

Run the following command to create the multicontainer Pod:

command

```
kubectl apply -f multi-container-pod.yaml
```

Step 5 ▾

Verify that the Pod was successfully created by running the following command:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl
get pods multi-container-pod
NAME                READY    STATUS    RESTARTS   AGE
multi-container-pod  2/2      Running   0           57s
```

The Pod is **Running**. Note that the **READY** column shows that 2 out of 2 containers are ready.

Step 6 ▾

Create an empty file in the **nginx-container** container by running the following command:

command

```
kubectl exec multi-container-pod -c nginx-container -- touch /tmp/hello.txt
```

Explanation ▾

The **kubectl exec** command runs the **touch /tmp/hello.txt** in the **multi-container-pod**. Since this is a multicontainer Pod, you must specify which container runs the command using the **-c nginx-container** flag. This is the first time you have used this flag in the course.

Using **kubectl exec**, you can run the **ls -la /tmp** in the same container to verify that the file was successfully created:

command

```
kubectl exec multi-container-pod -c nginx-container -- ls -la /tmp
```

Example output:

```
total 0
drwxrwxrwt 1 root root 18 Jan 20 15:59 .
drwxr-xr-x 1 root root 24 Jan 20 15:49 ..
-rw-r--r-- 1 root root 0 Jan 20 15:59 hello.txt
```

Step 7 ▾

Use the following command to run **ls -la /tmp** in the **busybox-container** container:

command

```
kubectl exec multi-container-pod -c busybox-container -- ls -la /tmp
```

Explanation ▾

Even when you run the same **ls -la /tmp** in both containers, because containers in Pods have separate filesystems, the **hello.txt** file is only displayed in the container where it was created, **nginx-container**.

Step 8 ▾

Run the following command to remove the Pod and leave the cluster in a clean state:

command

```
kubectl delete pod multi-container-pod
```

Task 2: Use an emptyDir Volume to share log information between Pods

Step 1 ▾

Review the YAML manifest for a multicontainer Pod using an emptyDir Volume by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat emptydir-multi-container-pod.yaml
```

Explanation ▾

Example output:

```

apiVersion: v1
kind: Pod
metadata:
  name: emptydir-multi-container-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/nginx
    - name: busybox-container
      image: busybox
      command: ["/bin/sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: shared-logs
          mountPath: /shared/logs
  volumes:
    - name: shared-logs
      emptyDir: {}

```

The YAML manifest defines a multicontainer Pod with two containers:

- An application container running the **nginx** web server and exposing the web service on port 80.
- A sidecar container using the **busybox** image and running a **sleep** command on startup.
- Under the **spec** section, the YAML manifest has the **volumes** section. This section contains an **emptyDir** Volume named **shared-logs**. You can define more than one Volume if needed.
- The **shared-logs** Volume is referenced by name and mounted in both containers using the **VolumeMounts** property in the container definition.
 - The Volume is mounted at the **/var/log/nginx** folder in the **nginx-container**. This is the default log folder for the Nginx web server.
 - The Volume is mounted at **/shared/logs** in the **busybox-container**. The sidecar container can now access the Nginx logs and perform any required task.

Step 2 ▾

Run the following command to create the multicontainer Pod:

command

```
kubectl apply -f emptydir-multi-container-pod.yaml
```

Step 3 ▾

Verify that the Pod was successfully created by running the following command:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl
get pods
NAME                                READY   STATUS    RESTARTS   AGE
emptydir-multi-container-pod        2/2     Running   0           7s
```

The Pod is **Running**. Note that the **READY** column shows that 2 out of 2 containers are ready.

Step 4 ▾

Run the following commands to verify that the containers in the Pod are sharing a folder:

commands

```
> kubectl exec emptydir-multi-container-pod -c nginx-container \
-- cat /var/log/nginx/error.log
> kubectl exec emptydir-multi-container-pod -c busybox-container \
-- cat /shared/logs/error.log
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl exec
emptydir-multi-container-pod -c nginx-container -- cat /var/log/nginx/error.log
2026/01/20 16:38:24 [notice] l#1: using the "epoll" event method
2026/01/20 16:38:24 [notice] l#1: nginx/1.29.4
2026/01/20 16:38:24 [notice] l#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/01/20 16:38:24 [notice] l#1: OS: Linux 6.12.0-160000.5-default
2026/01/20 16:38:24 [notice] l#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/01/20 16:38:24 [notice] l#1: start worker processes
2026/01/20 16:38:24 [notice] l#1: start worker process 30
2026/01/20 16:38:24 [notice] l#1: start worker process 31
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl exec
emptydir-multi-container-pod -c busybox-container -- cat /shared/logs/error.log
2026/01/20 16:38:24 [notice] l#1: using the "epoll" event method
```

```

2026/01/20 16:38:24 [notice] 1#1: nginx/1.29.4
2026/01/20 16:38:24 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/01/20 16:38:24 [notice] 1#1: OS: Linux 6.12.0-160000.5-default
2026/01/20 16:38:24 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/01/20 16:38:24 [notice] 1#1: start worker processes
2026/01/20 16:38:24 [notice] 1#1: start worker process 30
2026/01/20 16:38:24 [notice] 1#1: start worker process 31

```

Since the containers share the folder, the error.log is the same for both.

Step 4 ▾

Run the following commands to delete the Pod and clean the cluster:

command

```
kubectl delete pod emptydir-multi-container-pod
```

Exercise Summary

In this exercise, you created a multicontainer Pod and verified that its containers have separate filesystems. Later, you added an emptyDir Volume to the Pod definition and mounted it in both containers of a multicontainer Pod, so the containers could share a folder.

(End of Exercise)

Exercise 11.2 - Provide Persistent Storage Using an NFS Volume (static provisioning)

Description

Real-world applications require persistence, and Kubernetes provides it through Volumes. In the previous exercise, you used an `emptyDir` Volume. This type is ephemeral; even when it allows sharing information among containers in a Pod, once the Pod is deleted, the information in the `emptyDir` is lost. Kubernetes provides additional Volume types for persistent storage. In this exercise, you will use an NFS Volume for providing persistent storage to Pods. First, using the Volume directly in the Pod, and later using a `PersistentVolume` instead.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Provide persistent storage to Pods directly using an NFS Volume.
 - Provide persistent storage to Pods using `PersistentVolumes` and static provisioning.
-

Task 1: Directly use an NFS Volume in the Pod definition:**Step 1** ▾

Review the YAML manifest for a Pod that directly uses an NFS Volume to get persistent storage by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat pod-with-nfs-volume.yaml
```

Explanation ▾**Example output:**

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-nfs-volume
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: nfs-vol
          mountPath: /usr/share/nginx/html
  volumes:
    - name: nfs-vol
      nfs:
        server: 172.30.170.2
        path: /srv/nfs-shares/data
        readOnly: false
```

The YAML manifest:

- Defines a Volume (**nfs-vol**) to be used by the Pod, being the Volume:
 - An NFS server running on the **172.30.170.2** machine, which is the **RKE2-management** machine.
 - Exporting the **/srv/nfs-share/data** folder.
 - Allowing writing operations from the container as **readOnly** is set to **false**.
- This Volume is mounted on the container's **/user/share/nginx/html (mountPath)** folder using the **volumeMounts** property.

Additional Information ▾

NFS volumes and other Volume types have specific properties. Review the [official Kubernetes documentation site](#) for further information.

Step 2 ▾

Create the Pod by running the following command:

command

```
kubectl apply -f pod-with-nfs-volume.yaml
```

Step 3 ▾

Open a shell in the recently created Pod by running the following command:

command

```
kubectl exec -it pod-with-nfs-volume -- sh
```

Step 4 ▾

From the Pod's shell, create a new file in the `/usr/share/nginx/html` folder and leave the container by running the following commands:

commands

```
> echo "HELLO WORLD FROM THE RKE2 COURSE" \  
> /usr/share/nginx/html/new_index.html  
> exit
```

Step 5 ▾

Back to the ***RKE2-management*** terminal, verify that the file created from the Pod's shell appears in the NFS Server root path by running the following command:

command

```
cat /srv/nfs-shares/data/new_index.html
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> cat  
/srv/nfs-shares/data/new_index.html  
HELLO WORLD FROM THE RKE2 COURSE
```

Since the NFS Volume was successfully mounted in the container, files created in the container are persisted in the NFS Server path.

Step 6 ▾

Delete the Pod by running the following command:

command

```
kubectl delete -f pod-with-nfs-volume.yaml
```

Step 7 ▾

Recreate the Pod again by running the following command:

command

```
kubectl apply -f pod-with-nfs-volume.yaml
```

Step 8 ▾

Verify that, as the file was persisted using an NFS Volume, new Pods can reuse it, by running the following command:

command

```
kubectl exec -it pod-with-nfs-volume \
-- cat /usr/share/nginx/html/new_index.html
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl
exec -it pod-with-nfs-volume -- cat
/usr/share/nginx/html/new_index.html
HELLO WORLD FROM THE RKE2 COURSE
```

Step 9 ▾

Finally, leave the cluster in its previous state by running the following commands:

commands

```
> kubectl delete -f pod-with-nfs-volume.yaml
> sudo rm /srv/nfs-shares/data/new_index.html
```

Task 2: Provide persistent storage to a Pod using a PersistentVolume (PV) and a PersistentVolumeClaim (PVC)

Step 1 ▾

Review the YAML manifest for a PersistentVolume (PV) by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat nfs-pv.yaml
```

Explanation ▾

Example output:


```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: 172.30.170.2
    path: /srv/nfs-shares/data

```

In this YAML manifest:

- The ***apiVersion*** and the ***kind*** properties have the required values to create a PersistentVolume.
- Within the ***spec*** section:
 - The ***capacity*** property requests ***5 Gi*** of storage from the physical storage device backing the PV.
 - The ***accessModes*** property allows the PV to be read and written from different Nodes as it sets the ***ReadWriteMany*** value.
 - Since we are referring to an NFS server, we use the ***nfs*** property with the same values as in the previous exercise: the server IP address and the path exported by the server.

Step 2 ▾

Run the following command to create the PersistentVolume(PV):

command

```
kubectl apply -f nfs-pv.yaml
```

Explanation ▾

When completed successfully, retrieve information about the PersistentVolume by running this command:

command

```
kubectl get pv nfs-pv
```

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/storage> k get pv
NAME      CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS   CLAIM
nfs-pv    5Gi       RWX           Retain          Available

```

The **kubectl get** command shows two interesting attributes in the output:

- **RECLAIM POLICY** is set to **Retain**, meaning that when a PersistentVolumeClaim (PVC) using the PV is deleted, the data in the physical storage and the NFS server remain, so it can be used later. This is the default value.
- **STATUS** is set to **Available**; no PVC is using or claiming the PV.

Note ▾

pv in the **kubectl get** command is the shortname for **persistentvolume**.

Step 3 ▾

Review the YAML manifest for a PersistentVolumeClaim (PVC) by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat nfs-pvc.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
```

In this YAML manifest:

- The **apiVersion** and the **kind** properties have the required values to create a PVC.
- Within the **spec** section:
 - The **accessModes** property is set to **ReadWriteMany**, and the PVC definition **requests 1Gi** of **storage** under the **resources** section. Kubernetes tries to find a PV with a compatible accessMode and enough capacity for the claim. Our **nfs-pv** PV fulfills both requirements.

Step 4 ▾

Run the following command to create the PersistentVolumeClaim(PV):

command

```
kubectl apply -f nfs-pvc.yaml
```

Explanation ▾

When completed successfully, retrieve information about the PVC by running this command:

command

```
kubectl get pvc nfs-pvc
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get pvc
nfs-pvc
NAME      STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS
nfs-pvc   Bound     nfs-pv   5Gi        RWX            ...
```

The ***kubectl get*** command shows an interesting attribute in the output:

- ***STATUS*** is set to ***Bound***: The PVC claim for storage has been fulfilled, and the ***nfs-pv*** PV provides the requested storage. If you run the ***kubectl get pv*** command again, the output shows the PV in a ***Bound*** status, bound to ***nfs-pvc***.

Note ▾

pvc in the ***kubectl get*** command is the shortname for ***persistentvolumeclaim***.

Step 5 ▾

Review the YAML manifest for a Pod using persistent storage via a PersistentVolumeClaim (PVC) by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat pod-nfs-pvc.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nfs-pvc
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
```

```

      - name: nfs-storage
        mountPath: /data
    volumes:
      - name: nfs-storage
        persistentVolumeClaim:
          claimName: nfs-pvc

```

In this YAML manifest:

- The **volumes** section contains a single Volume definition, the **nfs-storage** Volume. This Volume refers to the **nfs-pvc persistentVolumeClaim**.
- The **nfs-storage** Volume is mounted in the **/data** folder within the container definition using the **volumeMounts** property.

Step 6 ▾

Create the Pod using the PersistentVolumeClaim by running the following command:

command

```
kubectl apply -f pod-nfs-pvc.yaml
```

Step 7 ▾

Create a file in the folder within the Pod where the PVC was mounted by running the following command:

command

```
kubectl exec -it pod-nfs-pvc -- touch /data/hello.txt
```

Explanation ▾

Since the NFS Server runs on the **RKE2-management** VM, you can verify that the **hello.txt** was successfully persisted by running the following command:

command

```
ls -la /srv/nfs-shares/data/
```

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/storage> ls -la
/srv/nfs-shares/data/
total 20
drwxr-xr-x. 3 root root 4096 Jan 24 12:54 .
drwxr-xr-x. 1 root root   8 Nov 25 11:52 ..
-rw-r--r--. 1 root root   0 Jan 24 12:54 hello.txt
drwx-----. 2 root root 16384 Nov 25 11:52 lost+found

```

In the end, the Pod uses the PVC, which is bound to the PV, which is backed by a physical storage, the NFS server. The persistent storage was statically provisioned as you, as the Kubernetes administrator, created the PV.

Step 8 ▾

Finally, leave the cluster in its previous state by running the following commands:

commands

```
> kubectl delete -f pod-nfs-pvc.yaml
> kubectl delete -f nfs-pvc.yaml
> kubectl delete -f nfs-pv.yaml
> sudo rm /srv/nfs-shares/data/hello.txt
> sudo rm /srv/nfs-shares/data/new_index.html
```

Exercise Summary

In this exercise, first, you provided persistent storage to a Pod directly using an NFS volume in the Pod definition. Next, you performed the same operation; however, you started by creating a PersistentVolume (PV) tied to the NFS server, bound a PersistentVolumeClaim (PVC) to the PV, and finally mounted the PVC as a Volume in the Pod definition.

(End of Exercise)

Exercise 11.3 - Deploy a Container Storage Interface (CSI) Driver

Description

Static provisioning requires Kubernetes administrators to create PersistentVolumes beforehand. Dynamic provisioning relies on StorageClasses to dynamically create PersistentVolumes when a PersistentVolumeClaim requires them. The RKE2 cluster used in the Labs does not provide any StorageClass or CSI driver for providing advanced storage capabilities.

In this exercise, you will install an NFS CSI driver in your Kubernetes cluster to continue working with the NFS server available in the Labs, and later create a StorageClass using the NFS provider deployed with the NFS CSI driver.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Install an NFS Container Storage Interface (CSI)
 - Verify that the CSI driver has been installed.
 - Create a StorageClass using the providers given by the NFS driver.
-

Task 1: Install the NFS Container Storage Interface (CSI) driver

Step 1 ▾

The [Kubernetes CSI Developer Documentation](https://kubernetes.io/docs/concepts/storage/container-storage-interface/) website lists Kubernetes CSI drivers. The NFS CSI driver appears on the list and links to the driver site at <https://github.com/kubernetes-csi/csi-driver-nfs>, which includes instructions for installing it. We will use **kubectl** to install the driver on the Lab's cluster. Run the following command:

command

```
curl -skSL
https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.12.1/
deploy/install-driver.sh | bash -s v4.12.1 --
```

Explanation ▾

The **curl** command downloads the installation script and executes it using **v4.12.1** as the version for the NFS CSI Driver.

Step 2 ▾

Verify that the NFS CSI driver has been successfully installed by running the following commands:

commands

```
> kubectl -n kube-system get pod -o wide -l app=csi-nfs-controller
> kubectl -n kube-system get pod -o wide -l app=csi-nfs-node
```

Explanation ▾

Example output:

```
tux@management:~/> kubectl -n kube-system get pod -o wide -l app=csi-nfs-controller
NAME                                READY   STATUS    RESTARTS   AGE   IP             
csi-nfs-controller-6f78747487-zkddv 5/5     Running   0           173m  172.30.170.4

tux@management:~/> kubectl -n kube-system get pod -o wide -l app=csi-nfs-node
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE
csi-nfs-node-n8wht                 3/3     Running   0           173m  172.30.170.4    agent ...
csi-nfs-node-ztxlz                 3/3     Running   0           173m  172.30.170.3    server ...
```

CSI Drivers have two main components, controller plugins and node plugins. The **kubectl get** command uses the **-l** flag to filter results and list only Pods with a specific label.

- The controller plugin Pod has the **app=csi-nfs-controller** label and runs only on the server (control-plane) Node.
- Controller node Pods have the **app=csi-nfs-node** label and run on every Node.

Step 3 ▾

You can also list CSI drivers in your Kubernetes cluster by running the following command:

```
kubectl get csidriver
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get csidriver
NAME                ATTACHREQUIRED  PODINFOONMOUNT  STORAGECAPACITY  TOKENREQUESTS
nfs.csi.k8s.io      false           false           false            <unset> ...
```

Task 2: Create a StorageClass (SC) for dynamic storage provisioning**Step 1** ▾

Review the YAML manifest for a StorageClass by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat sc-nfs.yaml
```

Explanation ▾

Example output:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sc-nfs
provisioner: nfs.csi.k8s.io
parameters:
  server: 172.30.170.2
  share: /srv/nfs-shares/data
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

In this YAML manifest:

- The ***apiVersion*** and ***kind*** properties have the required values for StorageClass objects.
- ***provisioner: nfs.csi.k8s.io*** refers to the provisioner, which dynamically creates PersistentVolumes. This provisioner was made available during the NFS CSI Driver installation.
- The ***parameters*** section contains the information about the NFS Server.
- In this example, the ***ReclaimPolicy*** value is ***Delete*** instead of the default ***Retain***. Data in the NFS server is deleted if the PersistentVolumeClaim(PVC) using the StorageClass(SC) is deleted.

- Because the ***volumeBindingMode*** is set to ***WaitForFirstCustomer***, no PersistentVolume is created until a storage request is made.

Step 2 ▾

Run the following command to create the StorageClass(SC):

command

```
kubectl apply -f sc-nfs.yaml
```

Step 3 ▾

Verify that StorageClass(SC) was successfully created by running the following command:

command

```
kubectl get sc sc-nfs
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl
get sc sc-nfs
NAME          PROVISIONER          RECLAIMPOLICY    VOLUMEBINDINGMODE
sc-nfs        nfs.csi.k8s.io       Delete           WaitForFirstConsumer ...
```

Exercise Summary

In this exercise, you have installed the NFS CSI Driver in your Kubernetes cluster and created a StorageClass using the ***nfs.csi.k8s.io*** provider, which the CSI Driver made available. This StorageClass is used in the following exercise to dynamically provide persistent storage for Pods.

(End of Exercise)

Exercise 11.4 - Provide Persistent Storage Using a Dynamically Provisioned NFS Persistent Volume (PV) (dynamic provisioning).

Description

In the previous exercise, you installed an NFS CSI Driver and left a StorageClass(SC) ready to provide PersistentVolumes using the NFS provider installed with the driver. In this exercise, you will provision a PersistentVolume dynamically by referencing a StorageClass (SC) from a PersistentVolumeClaim(PVC). Later, you will provide persistent storage to a Pod using this PVC.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a PersistentVolumeClaim (PVC) that references a SC to provide a PV.
-

Task 1: Create a PersistentVolumeClaim(PVC) using a StorageClass(SC)**Step 1** ▾

Review the YAML manifest that defines a PVC using an SC by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat nfs-pvc-sc.yaml
```

Explanation ▾**Example output:**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc-sc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: sc-nfs
  resources:
    requests:
      storage: 3Gi
```

Compared with the PVC you created in a previous exercise for static provisioning, the only difference is the reference to the StorageClass (SC) using ***storageClassName: sc-nfs***.

Step 2 ▾

Create the PVC by running the following command:

commands

```
kubectl apply -f nfs-pvc-sc.yaml
```

Step 3 ▾

Verify that the PVC was successfully created by running the following command:

command

```
kubectl get pvc nfs-pvc-sc
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get pvc
nfs-pvc-sc
NAME          STATUS    VOLUME    CAPACITY   ACCESS MODES   STORAGECLASS
```

nfs-pvc-sc	Pending	sc-nfs	...
------------	---------	--------	-----

Since you used ***volumeBindingMode: WaitForFirstConsumer*** in the SC, the ***Pending*** status is not an error. Kubernetes does not create a PV until a Pod requests persistent storage using this PVC.

Task 2: Create a Pod using the PersistentVolumeClaim(PVC) that refers to a StorageClass(SC)

Step 1 ▾

Review the YAML manifest that defines a Pod mounting the previously created PVC as a Volume by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat pod-nfs-sc.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-nfs-sc
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: nfs-sc
          mountPath: /usr/share/nginx/html
  volumes:
    - name: nfs-sc
      persistentVolumeClaim:
        claimName: nfs-pvc-sc
```

Compared with the Pod you created in a previous exercise for static provisioning, the only differences are that you are referencing the nfs-pvc-sc PVC and mounting the Volume in the ***/usr/share/nginx/html*** folder within the container.

Step 2 ▾

Create the Pod by running the following command:

command

```
kubectl apply -f pod-nfs-sc.yaml
```

Step 3 ▾

Review the current status of the PVC by running the following command:

command

```
kubectl get pvc nfs-pvc-sc
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get pvc
nfs-pvc-sc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS
MODES        STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
nfs-pvc-sc    Bound       pvc-54be49c9-efc1-46de-8344-0ba16a479326  3Gi        RWX
sc-nfs        <unset>      23m
```

The PVC status is now **Bound**. The **pvc-54be49c9-efc1-46de-8344-0ba16a479326** has been dynamically created, and the SC used is **sc-nfs**.

Warning ▾

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 4 ▾

Create a file in the folder within the Pod where the PVC was mounted by running the following command:

command

```
kubectl exec -it pod-nfs-sc -- touch /usr/share/nginx/html/lab.html
```

Step 5 ▾

Verify if the file was persistent in the NFS server running in the **RKE2-management** VM by running the following command:

command

```
ls -R /srv/nfs-shares/data/
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> ls -R
/srv/nfs-shares/data/
/srv/nfs-shares/data/:
lost+found  pvc-54be49c9-efc1-46de-8344-0ba16a479326
```

```
ls: cannot open directory '/srv/nfs-shares/data/lost+found':  
Permission denied
```

```
/srv/nfs-shares/data/pvc-54be49c9-efc1-46de-8344-0ba16a479326:  
lab.html
```

The file **lab.html** was created in a folder with the same name as the dynamically provisioned PersistentVolume (PV).

Step 6 ▾

Finally, clean the cluster by running the following commands:

commands

```
> kubectl delete pod pod-nfs-sc  
> kubectl delete pvc nfs-pvc-sc
```

Warning ▾

Do not remove the StorageClass (SC); it will be used in the next exercise. Since you created the SC with the **reclaimPolicy: Delete** property, the dynamically provisioned PV's folder and the file **lab.html** within it are automatically deleted.

Exercise Summary

In this exercise, you used the previously created StorageClass (SC) to automatically provide persistent storage to a Pod via a PVC referencing that SC. You also verified that a new folder with the same name as the dynamically provided PV was created within the physical storage.

(End of Exercise)

Exercise 11.5 - Advanced Storage Operations: Clone

Description

CSI Drivers in Kubernetes enhance storage operations and make it easier to perform advanced storage operations, such as cloning, snapshotting, and restoring. Not all CSI Drivers provide the same set of advanced operations, and you should review the driver documentation to see which operations are allowed before installing it. In this exercise, you will clone a dynamically provisioned PersistentVolume (PV) using the StorageClass that uses the provider installed with the NFS CSI Driver.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Clone a PersistentVolume using the NFS CSI Driver
-

Task 1: Dynamically create a PersistentVolume(PV) using a StorageClass(SC)**Step 1**

Recreate the Pod and the PersistentVolumeClaim(PVC) used in the previous exercise by running the following commands:

commands

```
> cd ~course_files/RKE201v2/labs/manifests/storage/
> kubectl apply -f nfs-pvc-sc.yaml
> kubectl apply -f pod-nfs-sc.yaml
```

Step 2

Review the PVC status by running the following command:

command

```
kubectl get pvc nfs-pvc-sc
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> k get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS
MODES         STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
nfs-pvc-sc    Bound       pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a  3Gi        RWX
sc-nfs        <unset>      53s
```

STATUS displays as **Bound**; therefore, the PV has successfully been created and attached to the Pod to provide persistent storage. The **VOLUME** column displays the name for the dynamically created PV. In the example: **pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a**

Step 3

Create a file in the folder within the Pod where the PVC was mounted by running the following command:

command

```
kubectl exec -it pod-nfs-sc -- touch /usr/share/nginx/html/clone_lab.html
```

Step 4

Verify if the file was persisted in the NFS server running in the **RKE2-management** VM by running the following command:

command

```
ls -R /srv/nfs-shares/data/
```

Explanation

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> ls -R
/srv/nfs-shares/data/
/srv/nfs-shares/data/:
lost+found  pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a
ls: cannot open directory '/srv/nfs-shares/data/lost+found':
Permission denied

/srv/nfs-shares/data/pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a:
clone_lab.html
```

The file ***clone_lab.html*** was created in a folder with the same name as the dynamically provisioned PersistentVolume (PV).

Task 2: Clone the PersistentVolumeClaim (PVC)**Step 1**

Review the YAML manifest that defines a PersistentVolumeClaim that clones the ***nfs-pvc-sc*** PVC by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat clone-nfs-pvc-sc.yaml
```

Explanation**Example output:**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: clone-nfs-pvc-sc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 4Gi
  storageClassName: sc-nfs
  dataSource:
    kind: PersistentVolumeClaim
    name: nfs-pvc-sc
```

Compared with the manifest for the ***nfs-pvc-sc***, there are only two differences:

- This PVC requires more storage capacity: 4 Gi.

- There is a new property: **dataSource**. This property refers to the **nfs-pvc-sc** PVC as the source. The content from **nfs-pvc-sc** is cloned into the new PersistentVolume when it is attached to a Pod.

Step 2 ▾

Create the PVC by running the following command:

command

```
kubectl apply -f clone-nfs-pvc-sc.yaml
```

Step 3 ▾

Review the PVC status by running the following command:

command

```
kubectl get pvc
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get pvc
NAME                STATUS    VOLUME                                     CAPACITY
ACCESS MODES        STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
clone-nfs-pvc-sc    Pending
nfs-sc              <unset>      4m10s
nfs-pvc-sc          Bound       pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a  3Gi      RWX
sc-nfs              <unset>      28m
```

Even when the previous PVC appears as **Bound** and the new PVC was created using **nfs-pvc-sc** as the source, the new PVC still appears as **Pending**. Why is that? Since you set **volumeBindingMode: WaitForFirstConsumer** in the SC, Kubernetes does not create a PV until a Pod requests persistent storage using this PVC.

Step 4 ▾

Review the YAML manifest for a Pod that uses the new **clone-nfs-pvc-sc** by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/storage/
> cat clone-pod-nfs-sc.yaml
```

Explanation ▾

Example output:

```
kind: Pod
metadata:
  name: clone-pod-nfs-sc
```

```
spec:
  containers:
    - name: busybox
      image: busybox
      command:
        - "sleep"
        - "3600"
      volumeMounts:
        - name: nfs-sc
          mountPath: /data
  volumes:
    - name: nfs-sc
      persistentVolumeClaim:
        claimName: clone-nfs-pvc-sc
```

This YAML defines a Pod which uses the new **clone-nfs-pvc-sc** PVC and mounts it in the container's **/data** folder.

Step 5 ▾

Create the Pod by running the following command:

command

```
kubectl apply -f clone-pod-nfs-sc.yaml
```

Step 6 ▾

Review the PVC status by running the following command:

command

```
kubectl get pvc
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> kubectl get pvc
NAME                STATUS    VOLUME                                     CAPACITY   ACCESS
clone-nfs-pvc-sc    Bound     pvc-12b9d0ce-030e-4647-9bdf-9564b6c79845  4Gi        RWX
nfs-pvc-sc          Bound     pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a  3Gi        RWX
```

Both PVCs appear as **Bound**, using different dynamically provisioned volumes.

Step 7 ▾

Verify if the Volume was clones by running the following command:

command

```
ls -R /srv/nfs-shares/data/
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/storage> ls -lR
/srv/nfs-shares/data/
/srv/nfs-shares/data/:
total 24
drwx----- . 2 root root 16384 Nov 25 11:52 lost+found
drwxr-xr-x. 2 root root 4096 Jan 24 18:41
pvc-12b9d0ce-030e-4647-9bdf-9564b6c79845
drwxr-xr-x. 2 root root 4096 Jan 24 18:41
pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a
ls: cannot open directory '/srv/nfs-shares/data/lost+found':
Permission denied

/srv/nfs-shares/data/pvc-12b9d0ce-030e-4647-9bdf-9564b6c79845:
total 0
-rw-r--r--. 1 root root 0 Jan 24 18:41 clone_lab.html

/srv/nfs-shares/data/pvc-2d8c939f-f297-4c59-9a94-ff9bc242842a:
total 0
-rw-r--r--. 1 root root 0 Jan 24 18:41 clone_lab.html
```

The newly created volume contains the same data as the source volume; it is a successful clone.

Step 8 ▾

Run the following commands to clean the cluster:

commands

```
> kubectl delete pod clone-pod-nfs-sc pod-nfs-sc
> kubectl delete pvc clone-nfs-pvc-sc nfs-pvc-sc
> kubectl delete sc sc-nfs
```

Exercise Summary

In this exercise, you used one of the advanced storage capabilities of the NFS CSI Driver to clone the contents of a PersistentVolume (PV) that was dynamically created in response to a PersistentVolumeClaim (PVC) request to a StorageClass (SC).

(End of Exercise)

Section 12: Security in Kubernetes

Description

Add the overall section description here. Should be several sentences long and may be multiple paragraphs. It should include an overview of all of the exercises included and what the student will learn.

Section Learning Objectives

- Understand the multi-faceted nature of Kubernetes security: Identify the different layers that require security in a Kubernetes cluster.
- Differentiate between Authentication and Authorization.
- Master RBAC components: Define and create Roles (ClusterRoles) and RoleBindings(ClusterRoleBindings) to grant specific permissions.
- Understand the purpose of ServiceAccounts and how they provide identity and permissions to pods.
- Implement Network Policies: Use NetworkPolicy objects to control allowed traffic to or from Pods.
- Discover additional security elements in Kubernetes.

Section Topics

- Kubernetes security: A Multi-layered approach.
- API Access Control.
- Network Segmentation with NetworkPolicies.
- Other Kubernetes Key Security Features

Exercise 12.1 - Create a new Kubernetes user and ServiceAccount

Description

Kubernetes has two types of accounts: User and Group accounts represent human users, whereas ServiceAccounts are used by applications or system services to interact with the Kubernetes cluster. While ServiceAccounts are native Kubernetes objects, Users and Groups are managed by external identity providers.

In this exercise, you will manually create a user account using an X.509 certificate for authentication and a ServiceAccount. In a later exercise, you will assign different permissions to them.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a user account using an X.509 certificate for authentication.
 - Create a ServiceAccount in your RKE2 Kubernetes cluster.
 - Retrieve information about the user and the ServiceAccount.
-

Task 1: Create a new user for the RKE2 Kubernetes cluster**Step 1** ▾

Add Explanation, including command syntax, options, output. You should have an explanation in most steps.

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and generate a new private key for the john-doe user by running the following command:

command

```
openssl genrsa -out john.doe.key 4096
```

Explanation ▾

The previous command generates a **4096** bit RSA private key and stores the result in the **john.doe.key** file.

Step 4 ▾

Generate a Certificate Signing Request (CSR) using the previously created private key by running the following command:

command

```
openssl req -new -key john.doe.key -out john.doe.csr \
  -subj "/CN=john.doe/O=developers"
```


Explanation ▾

The previous command generates a CSR using the previously generated key and stores it in *john.doe.csr*. The **-subj** flag is used to include identity information for the CSR:

- **CN** is used for the user's Common Name: *john.doe*
- **O** is used for the user's Organization: *developers*. This field can later be used as the group field when defining Role-Based Access Control (RBAC) Kubernetes objects.

Note ▾

A CSR file contains information about the user, such as the name and the organization, and, more importantly, the user's public key and the digital signature that ties it to the user's private key.

Step 5 ▾

Certificate Sign Requests in Kubernetes require requests to be base64-encoded. Base64-encode the csr by running the following command and copying the result for later use:

command

```
cat john.doe.csr | base64 | tr -d '\n'
```

Explanation ▾

Because Kubernetes CertificateSigningRequests objects require base64-encoded values, the previous command reads the CSR file, base64-encodes its contents, and removes newlines.

Step 6 ▾

Review the YAML manifest for the CSR by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> vim john-doe-csr.yaml
```

Explanation ▾**Exam**

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: john.doe-request
spec:
  request: <BASE64-ENCODED-VALUE>
  signerName: kubernetes.io/kube-apiserver-client
usages:
```

- client auth

In this YAML manifest:

- The **apiVersion** and the **kind** properties have the required values for a Kubernetes CertificateSigningRequest object.
- The signer for the request is set using the **signerName** property. Kubernetes provides different signers. The one used in the file, **kubernetes.io/kube-apiserver-client**, signs certificates that the API server will honor as client certificates.
- The **usages** section defines the allowed uses for the certificate once it is signed. The value **client auth** allows the certificate to be used for authentication with the client **kubectl** tool.
- The **request** property contains the base64-encoded value for the previously generated CSR file.

Step 7 ▾

Edit the YAML manifest and replace the **<BASE64-ENCODED-VALUE>** with the result of the previous command, **cat john.doe.csr | base64 | tr -d '\n'**, which you copied in the previous step.

Step 8 ▾

Create the CertificateSigningRequest (CSR) by running the following command:

command

```
kubectl apply -f john-doe-csr.yaml
```

Step 9 ▾

Run the following command to verify that the CSR was created successfully:

command

```
kubectl get csr
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get
csr
NAME                AGE    SIGNERNAME                                REQUESTOR
REQUESTEDDURATION   CONDITION
john.doe-request    25s    kubernetes.io/kube-apiserver-client      system:admin
<none>              Pending
```

The CSR appears in a **Pending** condition, which means that the request is already in the cluster but not allowed until approved by a Kubernetes administrator or a user with the appropriate permissions.

Note ▾

csr stands as the shortname for **certificatesigningrequest**

Step 10 ▾

Assuming that the request is valid, approve the request by running the following command:

command

```
kubectl certificate approve john.doe-request
```

Explanation ▾

What does approving a CSR mean?

- The Kubernetes administrator has verified the user's identity.
- The Kubernetes cluster trusts the user's public key.
- The Kubernetes cluster has issued an X.509 certificate for the user signed by the cluster's Certificate Authority (CA).

Step 11 ▾

Run the following command to review the current CSR status:

command

```
kubectl get csr
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get
csr
NAME                AGE    SIGNERNAME                                REQUESTOR
REQUESTEDDURATION   CONDITION
john.doe-request    10m    kubernetes.io/kube-apiserver-client      system:admin
<none>              Approved, Issued
```

The CSR now appears as **Approved** and **Issued**.

Step 12 ▾

Extract the newly issued certificate from the request by running the following command:

command

```
kubectl get csr john.doe-request -o jsonpath='{.status.certificate}' \
| base64 --decode > john.doe.crt
```

Explanation ▾

This command retrieves the issued certificate from the CSR by using the *.status.certificate* JSON path, base64-decodes the content, and stores the result in the crt file.

Step 13 ▾

Review the certificate information by running the following command :

command

```
openssl x509 -in john.doe.crt -text -noout
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> openssl x509
-in john.doe.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            ae:70:7e:3e:fc:2f:57:82:f1:b3:97:73:39:40:aa:43
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=rke2-client-ca@1768834467
        Validity
            Not Before: Jan 26 07:46:08 2026 GMT
            Not After : Jan 26 07:46:08 2027 GMT
        Subject: O=developers, CN=john.doe
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (4096 bit)
            Modulus:
                00:91:8b:e1:41:03:33:9a:1d:db:c7:4f:e3:8a:14:
                ...
                85:ee:9a:04:1e:87:39:b6:bd:73:b9:86:2b:70:fe:
                c5:25:7f
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Extended Key Usage:
                TLS Web Client Authentication
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Authority Key Identifier:
                1A:ED:5D:F3:D4:72:40:C4:17:AF:4E:CD:DB:8D:77:B7:BD:FB:E6:3B
        Signature Algorithm: ecdsa-with-SHA256
        Signature Value:
            30:45:02:20:38:d2:04:25:41:f8:cf:a9:e2:04:04:f2:ea:02:
            e2:51:be:45:12:e3:fa:ee:1a:33:71:e8:fb:38:f8:80:18:b5:
            02:21:00:ad:4b:eb:9e:01:43:9c:00:25:c5:74:30:cd:0a:c4:
```

```
cb:f0:14:f7:16:26:a6:ef:b6:0a:33:13:7d:44:36:30:9f
```

Step 14 ▾

Add the user to the user's kubeconfig file by running the following command:

command

```
kubectl config set-credentials john.doe \
  --client-certificate=john.doe.crt \
  --client-key=john.doe.key \
  --embed-certs=true
```

Explanation ▾

The previous command adds the user's key and certificate to the default kubeconfig file at `~/kube/config`.

Step 15 ▾

Create a new context for the user by running the following command:

command

```
kubectl config set-context john-context \
  --cluster=default \
  --user=john.doe
```

Explanation ▾

The previous command adds a new context (***john-context***) for the ***john-doe*** user and the ***default*** cluster to the default kubeconfig file at `~/kube/config`.

Step 16 ▾

Verify that the user can authenticate on the server by running the following command:

command

```
kubectl version --context=john-context
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl version --context=john-context
Client Version: v1.33.7+rke2r1
Kustomize Version: v5.6.0
Server Version: v1.33.7+rke2r1
```

Using **--context=john-context**, you can select a specific authentication context from your kubeconfig file. The value **john-context** is the name of the context that holds the authentication information for **john.doe** user. Since the server version is displayed in the output, the user can authenticate to the cluster.

Step 17 ▾

Verify that the user still does not have any permissions by running the following command:

command

```
kubectl get pods --context=john-context
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get
pods --context=john-context
Error from server (Forbidden): pods is forbidden: User "john.doe" cannot list
resource "pods" in API group "" in the namespace "default"
```

Since you have not assigned any permission to the user, the request for the pods running in the default namespace fails. User permissions will be created in a following exercise.

Task 2: Create a new ServiceAccount

Step 1 ▾

Create a new ServiceAccount using the imperative approach by running the following command:

command

```
kubectl create sa my-lab-sa
```

Note ▾

sa stands as the shortname for **serviceaccount**

Step 2 ▾

Run the following command to verify that the ServiceAccount was successfully created::

command

```
kubectl get sa my-lab-sa
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>  
kubectl get sa my-lab-sa  
NAME          SECRETS  AGE  
my-lab-sa     0        80s
```

Exercise Summary

In this exercise, you manually created a new user for your Kubernetes cluster and a ServiceAccount. For the user, you started by creating the user. Then you generated and approved a CertificateSigningRequest for the user, and finally, you added the user's authentication information to your default kubeconfig file at ~/.kube/config.

(End of Exercise)

Exercise 12.2 - Define and test RBAC permissions

Description

Previously, you created a user and a ServiceAccount for your Kubernetes cluster. However, no permissions have been granted to them yet. So far, they are not allowed to perform any operation on the cluster.

In this exercise, you will assign a Role to the user and a ClusterRole to the ServiceAccount, and verify that both have been successfully assigned using commands.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Role and bind the Role to a user using a RoleBinding object.
 - Create a ClusterRole and bind it to the ServiceAccount using a ClusterRoleBinding object.
 - Use the right commands to verify that the Role and the ClusterRole have been successfully assigned.
-

Task 1: Create a Role and assign it to the john-doe user**Step 1** ▾

Use the following imperative command to create the Role:

command

```
kubectl create role john-role --verb=get,list,watch,create \
--resource=pod
```

Explanation ▾

The previous command creates a Role named **john-role** and grants permissions to **get**, **list**, **watch**, and **create** Pods in the working namespace.

Note ▾

Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings can be created imperatively or declaratively. For assigning permissions to the user, you will use the imperative command; for the ServiceAccount, you will create the YAML manifest first and apply it later using the declarative approach.

Step 2 ▾

Retrieve detailed information about the Role by running the following command:

command

```
kubectl describe role john-role
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl
describe role john-role
Name:          john-role
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  pods      []                 []              [get list watch create]
```

Step 3 ▾

Bind the user **john.doe** with the **john-role** Role by running the following command, which creates a RoleBinding:

command

```
kubectl create rolebinding john-rb --user=john.doe --role=john-role
```

Explanation ▾

The previous command creates a rolebinding named *john-rb* that assigns the *john-role* role to the *john.doe* user.

Step 4 ▾

Retrieve detailed information about the RoleBinding by running the following command:

command

```
kubectl describe rolebinding john-rb
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl describe rolebinding john-rb
Name:          john-rb
Labels:        <none>
Annotations:   <none>
Role:
  Kind: Role
  Name: john-role
Subjects:
  Kind  Name      Namespace
  ----  ----      -
  User  john.doe
```

Note the role in the **Role** section and the user within the **Subjects** section.

Step 5 ▾

Verify the user permissions by running the following commands:

commands

```
> kubectl get pods --context=john-context
> kubectl get pods -A --context=john-context
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get pods --context=john-context
No resources found in default namespace.
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get pods -A --context=john-context
Error from server (Forbidden): pods is forbidden: User "john.doe"
cannot list resource "pods" in API group "" at the cluster scope
```

The user can now get the Pod list from the default namespace, the namespace where the Role and RoleBinding were created, but it still has no permissions to get the global Pod list.

Step 6 ▾

Verify the user permissions by using the **kubectl auth can-i** command. Try with these commands:

commands

```
> kubectl auth can-i create pods --as=john.doe
> kubectl auth can-i delete pods --as=john.doe
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl auth can-i create pods --as=john.doe
yes
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl auth can-i delete pods --as=john.doe
no
```

The **kubectl auth can-i** command is a powerful tool that lets you check whether an action is allowed. You need to define the action and the subject using the **--as** flag. With the previous commands, you have checked whether the user **john.doe** had permission to create or delete pods. According to the **john-role** previously assigned to the user, the user is able to create Pods, but is not allowed to delete Pods.

Additional Information ▾

Run **kubectl auth can-i --help** to get more information and examples on that command.

Task 2: Create a ClusterRole and assign it to the my-lab-sa ServiceAccount

Step 1 ▾

Generate the YAML manifest for the ClusterRole by running the following command:

command

```
kubectl create clusterrole lab-cluster-role --verb=get,list,watch \
--resource=pods --dry-run=client -o yaml > lab-cluster-role.yaml
```

Step 2 ▾

Review the YAML manifest by running the following command:

command

```
cat lab-cluster-role.yaml
```

Explanation ▾

Example output:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  creationTimestamp: null
  name: lab-cluster-role
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - get
  - list
  - watch
```

Note the **resources** section, which includes **pods**, and the **verbs** section, which lists the allowed actions: **get**, **list**, **watch**. Since this is a ClusterRole, no namespace is required

Step 3 ▾

Generate the YAML manifest for the ClusterRoleBinding by running the following command:

command

```
kubectl create clusterrolebinding lab-cluster-rb \
  --clusterrole=lab-cluster-role --serviceaccount=default:my-lab-sa \
  --dry-run=client -o yaml > lab-cluster-rb.yaml
```

Step 4 ▾

Review the YAML manifest by running the following command:

command

```
cat lab-cluster-rb.yaml
```

Explanation ▾

Example output:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  creationTimestamp: null
```

```

    name: lab-cluster-rb
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: lab-cluster-role
  subjects:
  - kind: ServiceAccount
    name: my-lab-sa
    namespace: default

```

Note the **roleRef** section, which references the ClusterRole **lab-cluster-role**, and the **subjects** section that refers to the ServiceAccount **my-lab-sa**.

Step 5 ▾

Create the ClusterRole and the ClusterRoleBinding by running the following commands:

commands

```

> kubectl apply -f lab-cluster-role.yaml
> kubectl apply -f lab-cluster-rb.yaml

```

Explanation ▾

Example output:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  creationTimestamp: null
  name: lab-cluster-rb
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: lab-cluster-role
subjects:
- kind: ServiceAccount
  name: my-lab-sa
  namespace: default

```

Note the **roleRef** section, which references the ClusterRole **lab-cluster-role**, and the **subjects** section that refers to the ServiceAccount **my-lab-sa**.

Step 6 ▾

Run the following **kubectl describe** commands to retrieve detailed information about the ClusterRole and the ClusterRoleBinding:

commands

```
> kubectl describe clusterrole lab-cluster-role
> kubectl describe clusterrolebinding lab-cluster-rb
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl describe clusterrole lab-cluster-role
Name:          lab-cluster-role
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
  pods          []                []              [get list watch]
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl describe clusterrolebinding lab-cluster-rb
Name:          lab-cluster-rb
Labels:        <none>
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: lab-cluster-role
Subjects:
  Kind      Name      Namespace
  ----
  ServiceAccount my-lab-sa default
```

Note how the permissions are properly defined and how the ClusterRole and the ServiceAccount are bound using the ClusterRoleBinding.

Step 7 ▾

Verify the ServiceAccount permissions by using the ***kubectl auth can-i*** command. Try with these commands:

commands

```
> kubectl auth can-i list pods \
  --as=system:serviceaccount:default:my-lab-sa -n kube-system
> kubectl auth can-i create pods \
  --as=system:serviceaccount:default:my-lab-sa -n default
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
```

```
kubectl auth can-i list pods
--as=system:serviceaccount:default:my-lab-sa -n kube-system
yes
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl auth can-i create pods
--as=system:serviceaccount:default:my-lab-sa -n default
no
```

Note how to set the ServiceAccount for the ***kubectl auth can-i*** command.

--as=system:serviceaccount:default:my-lab-sa, where ***default*** is the namespace where the ServiceAccount was created. The ServiceAccount is permitted to list Pods in the ***kube-system*** namespace but is restricted from creating Pods in the ***default*** namespace.

Step 8 ▾

Review the YAML manifest for a Pod using the ServiceAccount by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat pod-with-sa.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-sa
  namespace: default
spec:
  serviceAccountName: my-lab-sa
  containers:
  - name: nginx
    image: nginx
```

This is a regular Pod manifest that includes the ServiceAccount information in the Pod using the ***serviceAccountName*** property, with the ServiceAccount name as the value. Applications running in the container can interact with the Kubernetes cluster using this information.

Step 9 ▾

Create the Pod using the ServiceAccount by running the following command:

command

```
kubectl apply -f pod-with-sa
```

Step 10 ▾

List the ServiceAccount authentication information mounted on the Pod by running the following command:

command

```
kubectl exec -it pod-with-sa -- ls -la \
/var/run/secrets/kubernetes.io/serviceaccount/
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec -it pod-with-sa -- ls -la
/var/run/secrets/kubernetes.io/serviceaccount/
total 0
drwxrwxrwt 3 root root 140 Jan 26 11:29 .
drwxr-xr-x 1 root root 28 Jan 26 11:29 ..
drwxr-xr-x 2 root root 100 Jan 26 11:29
..2026_01_26_11_29_36.1517394510
lrwxrwxrwx 1 root root 32 Jan 26 11:29 ..data →
..2026_01_26_11_29_36.1517394510
lrwxrwxrwx 1 root root 13 Jan 26 11:29 ca.crt → ..data/ca.crt
lrwxrwxrwx 1 root root 16 Jan 26 11:29 namespace → ..data/namespace
lrwxrwxrwx 1 root root 12 Jan 26 11:29 token → ..data/token
```

The **token** file contains a JWT token, and the ca.crt file contains the Certificate Authority (CA) certificate for the cluster. Applications running in the container can use this information to send requests to the RKE2 API server.

Step 11 ▾

Clean the cluster by running the following commands:

commands

```
> kubectl delete pod pod-with-sa
> kubectl delete clusterrolebindings lab-cluster-rb
> kubectl delete clusterrole lab-cluster-role
> kubectl delete role john-role
> kubectl delete rolebinding john-rb
```


Exercise Summary

In this exercise, you assigned permissions to a user using a Role and a RoleBinding object, assigned global permissions to a ServiceAccount using a Cluster and a ClusterRoleBinding object, and tested those permissions using the ***kubectl auth can-i*** command, which was used for the first time in the course. Later, you also run a Pod using the ServiceAccount and checked where the authentication information is mounted in the Pod.

(End of Exercise)

Exercise 12.3 - Secure a multi-layer application using a NetworkPolicy

Description

In a Kubernetes cluster, by default, all Pods can reach each other. This default scenario presents many security challenges. NetworkPolicies are the Kubernetes security mechanism for segmenting network traffic and isolating Pod communication. In this exercise, you will deploy a multi-layer application, running a frontend (web server) and a backend (MySQL), and later ensure the application's security by using NetworkPolicies to constrain network traffic to the application.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Understand the different elements of a NetworkPolicy.
 - Create a NetworkPolicy to ensure network traffic for a multi-layer application.
 - Verify Pod isolation once NetworkPolicies have been applied.
-

Task 1: Create the multi-layer application**Step 1** ▾

Review the YAML file for the multi-layer application by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat multi-layer-application.yaml
```

Explanation ▾**Example output:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-backend
  labels:
    app: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "linux"
---
apiVersion: v1
kind: Service
metadata:
  name: db-backend-service
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 3306
---
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: web-frontend
  labels:
    app: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-frontend-service
spec:
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80

```

This YAML file contains four manifests, which are separated by a new line using `---` :

- A Deployment, ***db-backend***, with 1 replica of a container using the
- ***mysql:8.0*** image.
- A ClusterIP Service, ***db-backend-service***, exposing the previous Deployment on port ***3306***.
- A Deployment, ***web-frontend***, with 1 replica of a container using the ***nginx*** image.
- A ClusterIP Service, ***web-frontend-service***, exposing the previous Deployment on port ***80***.

Note ▾

You must be familiar with all the Kubernetes objects in the YAML file. For agility, we've used a single replica Deployment for the database application. In production environments, databases usually require StatefulSets.

Step 2 ▾

Deploy the multi-layer application by running the following command:

command

```
kubectl apply -f multi-layer-application.yaml
```

Step 3 ▾

Verify that the application was successfully deployed by running:

command

```
kubectl get all
```

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/db-backend-587d95f44c-5sfgg	1/1	Running	0	3s
pod/web-frontend-d8c9f8976-xjdbb	1/1	Running	0	3s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/db-backend-service	ClusterIP	10.43.101.105	<none>
service/kubernetes	ClusterIP	10.43.0.1	<none>
service/web-frontend-service	ClusterIP	10.43.155.187	<none>

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/db-backend	1/1	1	1	3s
deployment.apps/web-frontend	1/1	1	1	3s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/db-backend-587d95f44c	1	1	1	3s
replicaset.apps/web-frontend-d8c9f8976	1	1	1	3s

Warning ▾

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 4 ▾

Use the following commands to verify how the traffic flows within the network after deploying the multi-layer application:

commands

```
> kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com
> kubectl run -it --rm test --image=busybox --restart=Never \
```

```
-- wget -O - http://web-frontend-service
> kubectl run -it --rm test --image=busybox --restart=Never \
-- nc -w 10 -zv db-backend-service 3306
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com
<HTML><HEAD><meta http-equiv="content-type"
...
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl run -it --rm tester --image=busybox --restart=Never -- wget -O
- http://web-frontend-service
Connecting to web-frontend-service (10.43.155.187:80)
writing to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
<body>
<h1>Welcome to nginx!</h1>
...
</html>
-                               100% |*****| 615
0:00:00 ETA
written to stdout
pod "tester" deleted
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl run -it --rm tester --image=busybox --restart=Never -- nc -w
10 -zv db-backend-service 3306
db-backend-service (10.43.101.105:3306) open
pod "tester" deleted
```

- **`kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com`:** Opens a shell in the only Pod in the **`web-frontend`** deployment, and sends an HTTP request to **`google.com`**. Since it ends successfully, it shows that the Pod can resolve **`google.es`** using a DNS server.
- **`kubectl run -it --rm tester --image=busybox --restart=Never -- wget -O - http://web-frontend-service`:** Creates a temporary Pod using the **`busybox`** image and sends an HTTP request to the **`http://web-frontend-service`** using **`wget`**. Since it succeeds, it shows that the Pods in the **`web-frontend-service`** Service can be reached on port 80. Once the request is completed, as you used **`--rm`** in the **`kubectl run`** command, the Pod is deleted.

- **`kubectrl run -it --rm tester --image=busybox --restart=Never -- nc -w 10 -zv db-backend-service 3306`**: Creates a temporary Pod using the **`busybox`** image and sends uses the netcat (**`nc`**) command to check if port 3306 is open for the **`db-backend-service`**. The command succeeds, and once it finishes, since you used **`--rm`** in the **`kubectrl run`** command, the Pod is deleted.

Task 2: Create the NetworkPolicy objects

Step 1 ▾

Review the YAML manifest for the NetworkPolicy to be applied to frontend Pods:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat frontend-network-policy.yaml
```

Example output:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - ports:
        - protocol: TCP
          port: 80
  egress:
    - to:
        ports:
          - protocol: UDP
            port: 53
          - protocol: TCP
            port: 53
    - to:
        - podSelector:
            matchLabels:
              app: backend
        ports:
          - protocol: TCP
            port: 3306
```

This is a Dual-Purpose NetworkPolicy managing inbound and outbound traffic. By default, it blocks all incoming or outgoing traffic, and later makes two exceptions:

- It ensures the web server remains reachable by allowing TCP traffic on port 80 from any source (*ingress*)
- It allows two types of outbound traffic (*egress*):
 - DNS resolution using port **53** and **TCP** or **UDP** protocols.
 - Connections to the backend, to Pods labeled with **app:backend**, targeting port 3306 using TCP. No other outbound traffic is allowed.

Step 2 ▾

Review the YAML manifest for the NetworkPolicy to be applied to backend Pods:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat backend-network-policy.yaml
```

Example output:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 3306
```

This NetworkPolicy blocks by default all incoming traffic to Pods labeled **app: backend**. Then it makes an exception for Pods labeled **app: frontend** and specifically limits this access to TCP port 3306 (the MySQL port). No other ports or protocols are allowed.

Step 3 ▾

Create the NetworkPolicy objects by running the following commands:

commands


```
> kubectl apply -f frontend-network-policy.yaml
> kubectl apply -f backend-network-policy.yaml
```

Step 4

Verify that both NetworkPolicies were successfully created by running the following command:

commands

```
kubectl get networkpolicies
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get networkpolicies
```

NAME	POD-SELECTOR	AGE
backend-network-policy	app=backend	14s
frontend-network-policy	app=frontend	9s

Step 5

Verify that both NetworkPolicies were successfully created by running the following command:

commands

```
> kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com
> kubectl run -it --rm test --image=busybox --restart=Never \
-- wget -O - http://web-frontend-service
> kubectl run -it --rm test --image=busybox --restart=Never \
-- nc -w 10 -zv db-backend-service 3306
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com
* Host google.com:80 was resolved.
* IPv6: 2a00:1450:4003:803::200e
* IPv4: 142.250.185.14
* Trying [2a00:1450:4003:803::200e]:80...
* Immediate connect fail for 2a00:1450:4003:803::200e: Network is
unreachable
* Trying 142.250.185.14:80...
* Connection timed out after 10001 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10001 milliseconds
command terminated with exit code 28
```

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl run -it --rm test --image=busybox --restart=Never -- wget -O -
http://web-frontend-service
Connecting to web-frontend-service (10.43.155.187:80)
writing to stdout
<!DOCTYPE html>
...
<body>
<h1>Welcome to nginx!</h1>
...
</html>
-                               100% |*****|          615
0:00:00 ETA
written to stdout
pod "test" deleted
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl run -it --rm test --image=busybox --restart=Never -- nc -w 10
-zv db-backend-service 3306
If you don't see a command prompt, try pressing enter.
nc: db-backend-service (10.43.101.105:3306): Connection timed out
pod "test" deleted
pod default/test terminated (Error)
```

Due to the NetworkPolicies, now:

- **`kubectl exec -it deployment/web-frontend -- curl -v -m 10 google.com`:** Outgoing DNS traffic for DNS resolution works, but as other **egress** traffic was blocked, the HTTP request times out.
- **`kubectl run -it --rm tester --image=busybox --restart=Never -- wget -O - http://web-frontend-service`:** Since **ingress** traffic to port 80 is allowed, this command still gets a valid HTTP response.
- **`kubectl run -it --rm tester --image=busybox --restart=Never -- nc -w 10 -zv db-backend-service 3306`:** Since the only allowed source for requests to port 3306 are Pods labeled **`app: frontend`**, the request from the temporary Pod fails.

Step 6 ▾

Finally, clean your RKE2 cluster by running the following commands:

commands

```
> kubectl delete -f frontend-network-policy.yaml
> kubectl delete -f backend-network-policy.yaml
> kubectl delete -f multi-layer-application.yaml
```

Exercise Summary

In this exercise, you deployed a multi-layer application consisting of a frontend (web server) and a backend (database), and secured that deployment using NetworkPolicies to control network traffic to and from the application components.

(End of Exercise)

Exercise 12.4 - Secure a Pod using a securityContext

Description

Containers are enabled by Linux kernel features, such as namespaces and cgroups, but, in the end, they interact with the host system, which represents another potential security attack surface. A compromised container can affect the host's integrity and, therefore, the Kubernetes cluster security. A SecurityContext in Kubernetes is a set of privileges and access control settings for a Pod or a container. It defines a security identity for them and determines how they can interact with the host system running Kubernetes.

In this exercise, you will first run a Pod without a SecurityContext and understand the associated risks. Later, you will address those potential risks by adding a SecurityContext to the Pod.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Understand the relationship between containers in Pods and the underlying host running the Kubernetes cluster.
 - Know the risks associated with not using SecurityContexts
 - Apply SecurityContext to Pods to increase your RKE Kubernetes cluster security.
-

Task 1: Understand the risks of using Pods without a SecurityContext**Step 1** ▾

Review the YAML manifest for a Pod without a SecurityContext by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat pod-without-context.yaml
```

Explanation ▾**Example output**

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-without-context
spec:
  containers:
  - name: ubuntu
    image: ubuntu
    command: ["sleep", "3600"]
```

The YAML manifest describes a simple Pod using the **ubuntu** container image and running the **sleep 3600** command on start.

Step 2 ▾

Create the Pod by running the following command:

command

```
kubectl apply -f pod-without-context.yaml
```

Step 3 ▾

Verify if the Pod has been successfully created by running the following command:

command

```
kubectl get pod pod-without-context
```

Explanation ▾**Example output**

```
pod/pod-without-context created
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get pod pod-without-context
```

NAME	READY	STATUS	RESTARTS	AGE
pod-without-context	1/1	Running	0	2m33s

The pod-without-context Pod displays as up and **Running** without any problem.

Step 4 ▾

Display the current user for the single container in the Pod by running the following command:

command

```
kubectl exec pod-without-context -- whoami
```

Explanation ▾

Example output

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec pod-without-context -- whoami
root
```

While a Linux namespace logically isolates a container's root user (UID 0) from the host's administrative space, both environments still share the same underlying host kernel. If an attacker breaks namespace isolation, for example, due to a "Zero-Day" vulnerability, they would become root on the host system running the Kubernetes cluster. Additionally, root users often inherit default administrative capabilities that may allow them to manipulate sensitive network settings or host-mounted files if security configurations are slightly misaligned.

Step 5 ▾

Create a file within the container's root filesystem by running the following command:

command

```
kubectl exec pod-without-context -- sh -c \
"touch /usr/bin/my-ssh && chmod +x /usr/bin/my-ssh && \
ls -la /usr/bin/my-ssh"
```

Explanation ▾

Example output

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec pod-without-context -- sh -c "touch /usr/bin/my-ssh &&
chmod +x /usr/bin/my-ssh && ls -la /usr/bin/my-ssh"
-rwxr-xr-x 1 root root 0 Jan 27 09:02 /usr/bin/my-ssh
```

As you have access to the container's root file system, if attackers gain access to the Pod, they can replace legitimate system binaries with malicious ones.

Step 6 ▾

Install and run the **ping** network utility in the container by running the following command:

command

```
kubectrl exec pod-without-context -- sh -c "apt update && apt install \
-y iputils-ping && ping -c 1 8.8.8.8"
```

Explanation ▾**Example output**

```
...
Setting up iputils-ping (3:20240117-lubuntu0.1) ...
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=14.2 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 14.234/14.234/14.234/0.000 ms
```

Since you installed packages, the container's root filesystem was modified, and the **ping** command requires some root capabilities to work.

Step 7 ▾

Delete the Pod by running the following command:

command

```
kubectrl delete -f pod-without-context.yaml
```

Task 2: Harden your Pod using a SecurityContext**Step 1** ▾

Review the YAML manifest for a Pod without a SecurityContext by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> cat pod-with-context.yaml
```

Explanation ▾**Example output**

```
apiVersion: v1
```

```

kind: Pod
metadata:
  name: pod-with-context
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]

```

This YAML manifest defines a Pod-level SecurityContext and a container-level SecurityContext.

In the Pod-level SecurityContext:

- ***runAsUser: 1000***: Sets a non-privileged User ID to the main container's process. Even if an attacker gains access to the Pod, the process runs as a non-privileged user, with no power to modify system configurations.
- ***runAsNonRoot: true***: Forces Kubernetes to check whether the image is running as root and prevents it from starting if it is.

In the container-level SecurityContext:

- ***allowPrivilegeEscalation: false***: This ensures that a child process can never gain more privileges than its parent. In Linux, some programs (such as `sudo` or `passwd`) have a special bit that allows a normal user to temporarily act as root. A hacker can exploit bugs in these programs to permanently gain root access.
- ***readOnlyRootFilesystem: true***: Mounts the container's entire root file system as Read-Only.
- ***drop: ["ALL"]*** within the ***capabilities*** section: This is a "Zero Trust" approach. It strips away every single kernel-level privilege from processes.

Step 2 ▾

Create the Pod by running the following command:

command

```
kubectl apply -f pod-with-context.yaml
```


Step 3 ▾

Verify if the Pod has been successfully created by running the following command:

command

```
kubectl get pod pod-with-context
```

Explanation ▾**Example output**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get pod pod-with-context
NAME                READY   STATUS    RESTARTS   AGE
pod-with-context    1/1     Running   0           11s
```

The pod-without-context Pod displays as up and **Running** without any problem.

Step 4 ▾

Try to run the command you ran for the **pod-without-context** in the new **pod-with-context** Pod:

commands

```
> kubectl exec pod-with-context -- whoami
> kubectl exec pod-with-context -- sh -c "touch /usr/bin/my-ssh && \
  chmod +x /usr/bin/my-ssh && ls -la /usr/bin/my-ssh"
> kubectl exec pod-with-context -- sh -c "apt update && apt \
  install -y iputils-ping && ping -c 1 8.8.8.8"
```

Explanation ▾**Example output**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec pod-with-context -- whoami
ubuntu
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec pod-with-context -- sh -c "touch /usr/bin/my-ssh && chmod
+x /usr/bin/my-ssh && ls -la /usr/bin/my-ssh"
touch: cannot touch '/usr/bin/my-ssh': Read-only file system
command terminated with exit code 1
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec pod-with-context -- sh -c "apt update && apt install -y
iputils-ping && ping -c 1 8.8.8.8"
```

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

Reading package lists...

E: List directory /var/lib/apt/lists/partial is missing. - Acquire
(30: Read-only file system)

```
command terminated with exit code 100
```

Since the Pod has a SecurityContext, the current user in the container is ***ubuntu***, and the other commands failed.

Step 5 ▾

Delete the Pod by running the following command:

command

```
kubectl delete -f pod-with-context.yaml
```

Exercise Summary

In this exercise, you created a Pod without a SecurityContext and ran several commands to identify security risks. Later, you created a new Pod with a SecurityContext to address the previously identified potential risk.

(End of Exercise)

Section 13: Monitoring and Troubleshooting in Kubernetes

Description

In Kubernetes production environments, applications or clusters may fail or become unresponsive. Applications might start failing, become unresponsive, or even be unreachable. The cluster itself can misbehave, or its nodes can crash. Administrators must monitor cluster health to detect issues early and troubleshoot failures promptly when they occur. This section presents basic tools, commands to monitor and troubleshoot a Kubernetes cluster, and lists some of the most typical issues you can find while being a Kubernetes administrator.

Section Learning Objectives

- Understand the three pillars of Observability: Metrics, Logs, and Traces.
- Verify resource health: Use `kubectl get` and `kubectl describe` to review the status of Pods, Nodes, and other Kubernetes objects.
- Analyze Logs and Events: Access container logs and inspect cluster-wide events to diagnose changes or failures.
- Debug running containers: Use `kubectl exec` to troubleshoot running container processes.
- Identify common Pod failure statuses, including `CrashLoopBackOff`, `ImagePullBackOff`, and `OOMKilled`, and their causes.
- Troubleshooting networking issues.

Section Topics

- Troubleshooting and Monitoring Tools.
- Diagnosing Application Issues.
- Cluster Problems.
- Network and Service Troubleshooting.

Exercise 13.1 - Use the Kubernetes Troubleshooting Toolbox

Description

Kubernetes provides a useful native toolbox for diagnosing, troubleshooting, and monitoring a cluster. In this exercise, before using the toolbox to fix problems in the cluster, we will practice with some of the tools and highlight those output values that are valuable for troubleshooting.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use the native Kubernetes Toolbox to retrieve information about applications, services, and the Kubernetes cluster.
 - Discover the most important information points for troubleshooting.
-

Task 1: Deploy a multi-layer application**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and deploy a multi-layer application by running the following commands:

command

```
> cd ~/course_files/RKE201v2/labs/manifests/security/
> kubectl apply -f multi-layer-application.yaml
```

Explanation ▾

To introduce the Kubernetes toolbox for troubleshooting, we will reuse an application from another section.

Task 2: Check the health status of the application Pods**Step 1** ▾

Run the following command to retrieve basic information about the Pods in the application:

command

```
kubectl get pods -o wide
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	
db-backend-587d95f44c-2lhvs	1/1	Running	0	5m12s	10.42.1.155	...
web-frontend-d8c9f8976-5b7rc	1/1	Running	0	5m12s	10.42.1.156	...

Note the values for the **READY**, **STATUS**, **RESTARTS**, and **IP** columns:

- In the **READY** column, the value is **1/1**: all the containers in the Pod are up and running.
- The **STATUS** column displays Pods are running. Everything seems to be in a normal state.
- None of the Pods have **RESTARTS**. Restarts may point to potential problems.
- Kubernetes has assigned an **IP** to both Pods.

Warning ▾

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 2 ▾

Run the following command to retrieve detailed information about the Pods in the application:

command

```
kubectl describe pods
```

Explanation ▾

Example output:

```
Name: db-backend-587d95f44c-2lhvs
Namespace: default
Priority: 0
...
Events:
```

Type	Reason	Age	From	Message
Normal	Scheduled	12m	default-scheduler	Successfully assigned
Normal	Pulled	12m	kubelet	Container image "mysql:8.0" already present on machine
Normal	Created	12m	kubelet	Created container: mysql
Normal	Started	12m	kubelet	Started container mysql

```

Name:          web-frontend-d8c9f8976-5b7rc
Namespace:     default
Priority:       0
...
...
Events:
  Type    Reason      Age   From          Message
  ----    -
Normal    Scheduled   12m   default-scheduler Successfully assigned
default/web-frontend-d8c9f8976-5b7rc to agent
Normal    Pulling     12m   kubelet        Pulling image "nginx"
Normal    Pulled      12m   kubelet        Successfully pulled image "nginx" in
1.575s (1.575s including waiting). Image size: 62870438 bytes.
Normal    Created     12m   kubelet        Created container: nginx
Normal    Started     12m   kubelet        Started container nginx

```

Review the **Events** section for both Pods. Everything appears as **Normal** and successful. If something goes wrong with the Pods or the application startup process, this is a good place to start troubleshooting.

Step 3 ▾

Run the following commands to view logs for the web server and database server:

commands

```

> kubectl logs db-backend-587d95f44c-2lhvs
> kubectl logs web-frontend-d8c9f8976-5b7rc

```

Explanation ▾

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl logs
db-backend-587d95f44c-2lhvs
2026-01-29 09:02:09+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server
8.0.45-1.el9 started.
2026-01-29 09:02:09+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-01-29 09:02:09+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server
8.0.45-1.el9 started.
2026-01-29 09:02:09+00:00 [Note] [Entrypoint]: Initializing database files
...
2026-01-29T09:02:18.980811Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for
connections. Version: '8.0.45' socket: '/var/run/mysqld/mysqld.sock' port: 3306
MySQL Community Server - GPL.

tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl logs
web-frontend-d8c9f8976-5b7rc
...
/docker-entrpoint.sh: Configuration complete; ready for start up
2026/01/29 09:02:10 [notice] 1#1: using the "epoll" event method
2026/01/29 09:02:10 [notice] 1#1: nginx/1.29.4
...
2026/01/29 09:02:10 [notice] 1#1: start worker process 30

```

Logs show applications running in Pods as ready. Startup errors caused by configuration issues will be visible in these logs.

Step 4 ▾

Review the resource usage for the Pods by running:

command

```
kubectl top pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl top pods
NAME                                CPU(cores)   MEMORY(bytes)
db-backend-587d95f44c-2lhvs         6m           456Mi
web-frontend-d8c9f8976-5b7rc        0m           3Mi
```

CPU and Memory usage are two parameters to check whether Pods are displaying unexpected behavior. Moreover, if you defined requests and limits for them.

Step 5 ▾

Verify that DNS resolution is working on the Pods by running the following commands:

commands

```
> kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl google.es
> kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl -v \
db-backend-service:3306
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl google.es
<HTML><HEAD><meta http-equiv="content-type"
content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.es/">here</A>.
</BODY></HTML>
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl -v
db-backend-service:3306
* Host db-backend-service:3306 was resolved.
* IPv6: (none)
```



```
* IPv4: 10.43.239.47
* Trying 10.43.239.47:3306...
* Connected to db-backend-service (10.43.239.47) port 3306
* using HTTP/1.x
> GET / HTTP/1.1
> Host: db-backend-service:3306
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
* Received HTTP/0.9 when not allowed
* closing connection #0
curl: (1) Received HTTP/0.9 when not allowed
command terminated with exit code 1
```

- ***kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl google.es*** : Shows that DNS resolution for external sites is working within the cluster.
- ***kubectl exec -it web-frontend-d8c9f8976-5b7rc -- curl -v db-backend-service:3306*** Shows that internal DNS resolution is working in the cluster.

Note ▾

Since we are working from the default namespace, ***db-backend-service*** is the shortname for ***db-backend-service.default.svc.cluster.local***.

Task 3: Review the Kubernetes cluster health

Step 1 ▾

Run the following command to retrieve basic information about the Nodes in your cluster:
command

```
kubectl get nodes
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
agent	Ready	<none>	9d	v1.33.7+rke2r1
server	Ready	control-plane,etcd,master	9d	v1.33.7+rke2r1

Both Nodes appear as ***READY*** in the ***STATUS*** column and are using the same version. The status indicates both nodes are healthy.

M

Step 2

Run the following command to retrieve information about resource usage on the Nodes:
command

```
kubectl top nodes
```

Explanation

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl top nodes
```

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
agent	261m	13%	1651Mi	42%
server	120m	6%	2197Mi	56%

Nodes are not under resource pressure; both have sufficient CPU and memory available.

Step 3

Verify the status of your cluster components by running the following command:
command

```
kubectl get pods -n kube-system
```

Explanation

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
cloud-controller-manager-server	1/1	Running	25 (3h2m ago)	9d
csi-nfs-controller-6f78747487-zkddv	5/5	Running	25 (3h1m ago)	4d21h
csi-nfs-node-n8wht	3/3	Running	15 (3h1m ago)	4d21h
csi-nfs-node-ztxlz	3/3	Running	15 (3h2m ago)	4d21h
etcd-server	1/1	Running	16	9d
helm-install-rke2-canal-7zlhf	0/1	Completed	0	9d
helm-install-rke2-coredns-z8pwg	0/1	Completed	0	9d
helm-install-rke2-metrics-server-jc8mg	0/1	Completed	0	9d
helm-install-rke2-runtimeclasses-nvrz9	0/1	Completed	0	9d
helm-install-rke2-snapshot-controller-99bcf	0/1	Completed	0	9d
helm-install-rke2-snapshot-controller-crd-djk6f	0/1	Completed	0	9d
helm-install-rke2-traefik-crd-pgq69	0/1	Completed	0	9d
helm-install-rke2-traefik-lmj5r	0/1	Completed	1	3h1m
kube-apiserver-server	1/1	Running	12	9d
kube-controller-manager-server	1/1	Running	24 (3h2m ago)	9d
kube-proxy-agent	1/1	Running	0	3h1m
kube-proxy-server	1/1	Running	0	3h2m
kube-scheduler-server	1/1	Running	12 (3h2m ago)	9d
rke2-canal-tgk7p	2/2	Running	22 (3h2m ago)	9d
rke2-canal-zhhm2	2/2	Running	22 (3h1m ago)	9d
rke2-coredns-rke2-coredns-76994ddb78-ct6th	1/1	Running	11 (3h2m ago)	9d
rke2-coredns-rke2-coredns-76994ddb78-qdrsf	1/1	Running	1 (3h1m ago)	25h
rke2-coredns-rke2-coredns-autoscaler-67944c44cc-c4j5s	1/1	Running	11 (3h2m ago)	9d
rke2-metrics-server-5699475d99-mwdgk	1/1	Running	11 (3h2m ago)	9d
rke2-snapshot-controller-696989ffdd-58qz9	1/1	Running	13 (3h2m ago)	9d
rke2-traefik-8sz4t	1/1	Running	10 (3h2m ago)	9d
rke2-traefik-x7r6j	1/1	Running	10 (3h1m ago)	9d

Pods for the Kubernetes components appear as **Running**. If you find any issues with them in the **STATUS**, use **kubectl get**, **kubectl describe**, or **kubectl logs**.

Step 4 ▾

Open an SSH session on the server (control-plane) Node by running the following command. Use **linux** as the password when prompted::

command

```
ssh root@server
```

Step 5 ▾

Verify the RKE2 services status (kubelet for standard Kubernetes installation) by running one the following commands:

commands

```
> systemctl status rke2-server
> journalctl -u rke2-server
```

Explanation ▾

Example output:

```
...
rke2-server.service - Rancher Kubernetes Engine v2 (server)
   Loaded: loaded (/etc/systemd/system/rke2-server.service; enabled; preset:
disabled)
   Active: active (running) since Thu 2026-01-29 08:18:27 CET; 3h 8min ago
 Invocation: 05d4a573f36e4d49812cd75d23fdb0a8
    Docs: https://github.com/rancher/rke2#readme
   Process: 1062 ExecStartPre=/sbin/modprobe br_netfilter (code=exited,
status=0/SUCCESS)
   Process: 1073 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
    Main PID: 1087 (rke2)
       Tasks: 227
          CPU: 8min 54.383s
...

```

If the first command returns an error, use the second to dig deeper into the RKE2 Server logs. The same operations should also be performed in the agent (worker) Node if you are having infrastructure issues.

Step 6 ▾

Type **exit** to close the session in the server and return to the **RKE2-management** terminal.

Task 4: Review the Services in the application deployment**Step 1**

Run the following command recover the IP addresses of the Pods:

command

```
kubectl get pods -o wide
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE
db-backend-587d95f44c-srdck         1/1    Running   0           25m   10.42.1.157    ...
web-frontend-d8c9f8976-zrj74        1/1    Running   0           25m   10.42.1.158    ...
```

Step 2

Review the information about the application Services by running the following command:

command

```
kubectl describe svc web-frontend-service db-backend-service
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/security>
kubectl describe svc web-frontend-service db-backend-service
Name:                                web-frontend-service
Namespace:                           default
Labels:                              <none>
Annotations:                         <none>
Selector:                            app=frontend
...
Endpoints:                           10.42.1.158:80
Session Affinity:                     None
Internal Traffic Policy:              Cluster
Events:                              <none>

Name:                                db-backend-service
Namespace:                           default
Labels:                              <none>
Annotations:                         <none>
Selector:                            app=backend
...
Endpoints:                           10.42.1.157:3306
Session Affinity:                     None
Internal Traffic Policy:              Cluster
```

Events:	<none>
---------	--------

IP addresses in the **Endpoints** fields match those of the Pods. Services have properly selected their targeted pods using the right selectors: **app: frontend** and **app: backend**.

Step 3 ▾

Clean the cluster by running the following command:

command

```
kubectll delete -f multi-layer-application.yaml
```

Exercise Summary

In this exercise, you deployed a multi-layer application and used Kubernetes commands to monitor and troubleshoot, paying special attention to the parts of the output that should be reviewed first when issues arise.

(End of Exercise)

Exercise 13.2 - Deploy and diagnose a failing application

Description

In this exercise, you will diagnose and troubleshoot a failing application using the commands you practiced with during the previous exercise.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Diagnose a failing application.
 - Understand some of the most common errors for running Pods.
 - Troubleshoot the issues of the failing application.
-

Task 1: Deploy a failing application

Step 1 ▾

Deploy the failing application, which creates a Kubernetes Deployment, by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/troubleshooting/
> kubectl apply -f failing-application.yaml
```

Step 2 ▾

Run the following command to retrieve the current status for the Pods:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl
get pods
NAME                                READY   STATUS             RESTARTS   AGE
failing-app-6dd87cddb-nqv4l         0/2     CrashLoopBackOff   1 (2s ago)  7s
failing-app-6dd87cddb-qbrp2         0/2     CrashLoopBackOff   1 (2s ago)  7s
failing-app-6dd87cddb-w2ncb         0/2     CrashLoopBackOff   1 (2s ago)  7s
```

Pods repeatedly fail to deploy and restart. As the value for the **STATUS** column changes over time, you should get additional information from the Pods. As the **READY** column displays a 0/2 value, none of the containers in the Pod are ready.

Step 3 ▾

Run the following command to view detailed Pod information:

command

```
kubectl describe pod failing-app-6dd87cddb-w2ncb
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl describe
pod failing-app-6dd87cddb-w2ncb
Name:                failing-app-6dd87cddb-w2ncb
Namespace:           default
Priority:              0
...
...
Events:
  Type     Reason      Age           From              Message
  ---     -
```

```

-----
Normal    Scheduled    6m15s          default-scheduler    Successfully assigned
default/failing-app-6dd87cddb-w2ncb to server
Normal    Pulled       6m12s          kubelet              Successfully pulled
image "busybox" in 950ms (950ms including waiting). Image size: 2222260 bytes.
Normal    Pulled       6m11s          kubelet              Successfully pulled
image "busybox" in 865ms (865ms including waiting). Image size: 2222260 bytes.
Normal    Pulled       5m55s          kubelet              Successfully pulled
image "busybox" in 890ms (890ms including waiting). Image size: 2222260 bytes.
Normal    Pulling      5m31s (x3 over 6m14s) kubelet              Pulling image "nginx"
Normal    Pulling      5m30s (x4 over 6m13s) kubelet              Pulling image "busybox"
Normal    Pulled       5m29s          kubelet              Successfully pulled
image "busybox" in 878ms (878ms including waiting). Image size: 2222260 bytes.
Normal    Created      5m29s (x4 over 6m12s) kubelet              Created container:
sidecar-app
Normal    Started      5m29s (x4 over 6m12s) kubelet              Started container
sidecar-app
Warning   BackOff      4m53s (x8 over 6m10s) kubelet              Back-off restarting
failed container sidecar-app in pod
failing-app-6dd87cddb-w2ncb_default(90930f09-d803-47cf-8f00-f6f992e678c8)
Warning   Failed       4m38s (x4 over 6m13s) kubelet              Error: ErrImagePull
Warning   Failed       4m38s (x4 over 6m13s) kubelet              Failed to pull image
"nginx": failed to pull and unpack image "docker.io/library/nginx:latest": failed to
resolve reference "docker.io/library/nginx:latest": pull access denied, repository
does not exist or may require authorization: server message: insufficient_scope:
authorization failed
Warning   Failed       62s (x25 over 6m11s) kubelet              Error: ImagePullBackOff
Normal    BackOff      62s (x25 over 6m11s) kubelet              Back-off pulling image
"nginx"

```

Many problems appear within the **Events** section. Let's start with **ImagePullBackOff**. Correct the image name typo in the YAML manifest by changing 'nginx' to 'nginx'.

Use your default editor, replace the image name with the correct one, and run **kubectl apply -f failing-application.yaml** again to apply the change.

Step 4 ▾

Run the following command to retrieve the status for the Pods after fixing the problem with the image:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> k get
pods
NAME                                READY   STATUS              RESTARTS   AGE
failing-app-6dd87cddb-nqv4l         0/2     CrashLoopBackOff    7 (4m48s ago)  15m
failing-app-6dd87cddb-qbrp2         0/2     CrashLoopBackOff    7 (4m54s ago)  15m
failing-app-6dd87cddb-w2ncb         0/2     CrashLoopBackOff    7 (4m53s ago)  15m
failing-app-74f558445b-w4ktd       0/2     CrashLoopBackOff    2 (2s ago)     7s

```


Because the Deployment uses a Rolling Update strategy, Kubernetes creates a new Pod to replace the failed ones, **failing-app-74f558445b-w4ktk**, but that Pod is also failing. Since the **READY** column still shows **0/2**, none of the containers in the Pod are ready.

Step 5 ▾

Retrieve detailed information about the new Pod by running the following command :

command

```
kubectl describe pod failing-app-74f558445b-w4ktk
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl describe
pod failing-app-74f558445b-w4ktk
Name:          failing-app-74f558445b-w4ktk
Namespace:     default
Priority:       0
...
Containers:
  web-server:
    State:          Waiting
    Reason:         CrashLoopBackOff
    Last State:     Terminated
    Reason:         StartError
    Message:        failed to create containerd task: failed to create shim task: OCI
runtime create failed: runc create failed: unable to start container process: container
init was OOM-killed (memory limit too low?)
    Exit Code:      128
    Started:        Thu, 01 Jan 1970 01:00:00 +0100
    Finished:       Sun, 01 Feb 2026 10:23:57 +0100
    Ready:         False
...
...
Events:
  Type     Reason      Age           From          Message
  ----     -
  Normal   Scheduled   3m24s         default-scheduler   Successfully assigned
default/failing-app-74f558445b-w4ktk to agent
  Normal   Pulled      3m22s         kubelet        Successfully pulled
image "nginx" in 1.088s (1.088s including waiting). Image size: 62870438 bytes.
  Warning   Failed      3m22s         kubelet        Error: failed to create
containerd task: failed to create shim task: OCI runtime create failed: runc create
failed: unable to start container process: container init was OOM-killed (memory limit
too low?)
  Normal   Pulled      3m21s         kubelet        Successfully pulled
image "busybox" in 882ms (882ms including waiting). Image size: 2222260 bytes.
  Normal   Pulled      3m20s         kubelet        Successfully pulled
image "nginx" in 913ms (913ms including waiting). Image size: 62870438 bytes.
  Warning   Failed      3m20s         kubelet        Error: failed to create
containerd task: failed to create shim task: OCI runtime create failed: runc create
failed: unable to start container process: error during container init: error setting
cgroup config for procHooks process: failed to call BPF_PROG_ATTACH (BPF_CGROUP_DEVICE,
BPF_F_ALLOW_MULTI): attach program: no such file or directory
  Normal   Pulled      3m19s         kubelet        Successfully pulled
image "busybox" in 890ms (890ms including waiting). Image size: 2222260 bytes.
```

```

Warning Failed      3m3s                kubelet                Error: failed to create
containerd task: failed to create shim task: OCI runtime create failed: runc create
failed: unable to start container process: error during container init: error setting
cgroup config for procHooks process: openat2
/sys/fs/cgroup/kubepods.slice/kubepods-burstable.slice/kubepods-burstable-pod5e339f86_b
87d_4907_8fcb_d2d2210642c3.slice/cri-containerd-web-server.scope/cgroup.controllers: no
such file or directory
Normal Pulled      3m3s                kubelet                Successfully pulled
image "nginx" in 912ms (912ms including waiting). Image size: 62870438 bytes.
Normal Started    3m2s (x3 over 3m21s) kubelet                Started container
sidecar-app
Normal Created    3m2s (x3 over 3m21s) kubelet                Created container:
sidecar-app
Normal Pulled     3m2s                kubelet                Successfully pulled
image "busybox" in 959ms (959ms including waiting). Image size: 2222260 bytes.
Normal Pulling    2m36s (x4 over 3m23s) kubelet                Pulling image "nginx"
Normal Pulling    2m35s (x4 over 3m22s) kubelet                Pulling image "busybox"
Normal Created    2m35s (x4 over 3m22s) kubelet                Created container:
web-server
Normal Pulled     2m35s                kubelet                Successfully pulled
image "nginx" in 1.025s (1.025s including waiting). Image size: 62870438 bytes.
Warning BackOff    115s (x10 over 3m19s) kubelet                Back-off restarting
failed container web-server in pod
failing-app-74f558445b-w4tkk_default(5e339f86-b87d-4907-8fcb-d2d2210642c3)
Warning BackOff    115s (x10 over 3m19s) kubelet                Back-off restarting
failed container sidecar-app in pod
failing-app-74f558445b-w4tkk_default(5e339f86-b87d-4907-8fcb-d2d2210642c3)
Warning Failed     99s (x2 over 2m35s)  kubelet                Error: failed to create
containerd task: failed to create shim task: OCI runtime create failed: runc create
failed: unable to start container process: error during container init: reading from
parent failed: fetch packet length from socket: recvfrom: connection reset by peer

```

The previous issue with the container image name is gone, but additional problems remain. Two error messages display for the **web-server** container:

- **“Back-off restarting failed container web-server in pod”** at the end.
- **OOMKilled** for the **web-server** container in the **LastState** section and also in the **Events** section.

The **OOMKilled** reason means that the container is running out of memory. A quick review of the YAML manifest reveals low memory values for the **web-server** requests and limits:

failing-application.yaml

```

...
spec:
  containers:
    - name: web-server
      image: nginx
      resources:
        limits:
          memory: "1Mi"
        requests:
          memory: "1Mi"

```

...

Use the editor and replace the current values with the following ones:

- **"32Mi"** as the memory limit.
- **"16Mi"** as the memory request.

Run **kubectl apply -f failing-application.yaml** again to apply the change.

Step 6 ▾

Run the following command to retrieve the status for the Pods after fixing the problem with the command name:

command

```
kubectl get pods
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> k get
pods
NAME                                READY   STATUS             RESTARTS   AGE
failing-app-6dd87cddb-qbrp2         0/2     CrashLoopBackOff   11 (13s ago)  31m
failing-app-6dd87cddb-w2ncb         0/2     CrashLoopBackOff   11 (14s ago)  31m
failing-app-74f558445b-w4ktk        0/2     CrashLoopBackOff   14 (4m53s ago)  16m
failing-app-78d895f8d4-sdb9v        1/2     CrashLoopBackOff   5 (2m22s ago)  22s
```

The new Pod, **failing-app-78d895f8d4-sdb9v**, continues to fail. But now, at least, one of the containers displays as ready.

Step 7 ▾

Retrieve detailed information about the new Pod by running the following command :

command

```
kubectl describe pod failing-app-78d895f8d4-sdb9v
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl describe
pod failing-app-78d895f8d4-sdb9v
Name:          failing-app-78d895f8d4-sdb9v
Namespace:     default
Priority:       0
...
...
```

```
Containers:
...
...
  sidecar-app:
...
    State:      Waiting
    Reason:     CrashLoopBackOff
    Last State: Terminated
    Reason:     Error
...
Events:
  Type        Reason         Age           From          Message
  ----        -
  Normal      Scheduled       7m28s         default-scheduler   Successfully assigned
default/failing-app-78d895f8d4-sdb9v to agent
  Normal      Pulling         7m27s         kubelet         Pulling image "nginx"
  Normal      Pulled          7m26s         kubelet         Successfully pulled
image "nginx" in 929ms (929ms including waiting). Image size: 62870438 bytes.
  Normal      Created         7m26s         kubelet         Created container:
web-server
  Normal      Started         7m26s         kubelet         Started container
web-server
  Normal      Pulled          7m25s         kubelet         Successfully pulled
image "busybox" in 959ms (959ms including waiting). Image size: 2222260 bytes.
  Normal      Pulled          7m24s         kubelet         Successfully pulled
image "busybox" in 888ms (888ms including waiting). Image size: 2222260 bytes.
  Normal      Pulled          7m11s         kubelet         Successfully pulled
image "busybox" in 944ms (945ms including waiting). Image size: 2222260 bytes.
  Normal      Pulled          6m43s         kubelet         Successfully pulled
image "busybox" in 907ms (907ms including waiting). Image size: 2222260 bytes.
  Normal      Created         5m47s (x5 over 7m25s)   kubelet         Created container:
sidecar-app
  Normal      Started         5m47s (x5 over 7m25s)   kubelet         Started container
sidecar-app
  Normal      Pulled          5m47s         kubelet         Successfully pulled
image "busybox" in 1.429s (1.429s including waiting). Image size: 2222260 bytes.
  Warning     BackOff         2m10s (x26 over 7m24s)   kubelet         Back-off restarting
failed container sidecar-app in pod
failing-app-78d895f8d4-sdb9v_default(99e5e98c-8881-4943-b9e6-d57d0804f017)
  Normal      Pulling         104s (x7 over 7m26s)   kubelet         Pulling image
"busybox"
```

The *kubectl describe* output now only displays errors for the *sidecar-app* container. However, it provides little detail about the root cause of the error with the **“Warning BackOff 2m10s (x26 over 7m24s) kubelet Back-off restarting failed container sidecar-app in pod”** message.

Step 8 ▾

Run the following command to retrieve the *sidecar-app* container logs of the fresh Pod.
command

```
kubectl logs failing-app-78d895f8d4-sdb9v -c sidecar-app
```

Explanation ▾

When using multi-container Pods, such as the one in the application, use the `-c` flag to specify a container for the ***kubectl logs*** command. To get the container name, you can use the information in the ***kubectl describe*** command or review the YAML manifest.

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl
logs failing-app-78d895f8d4-sdb9v -c sidecar-app
ls: /my-shared-dir: No such file or directory
```

The container fails because a wrong command was used during the start process. The ***ls*** command attempts to list the contents of a nonexistent directory: ***/my-shared-dir***.

Step 9 ▾

Use the editor to fix the problem with the container's start command. Replace command: ***["sh","-c","ls -la /my-shared-dir && sleep 3600"]*** with ***["sh","-c","ls -la / && sleep 3600"]***.

Save the manifest and apply the changes by running and retrieve the Pods by running the following commands:

commands

```
> kubectl apply -f failing-application.yaml
> kubectl get pods
```

Explanation ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
failing-app-c748f7bbb-8pl7n	2/2	Running	0	5s
failing-app-c748f7bbb-j6ljt	2/2	Running	0	8s
failing-app-c748f7bbb-z6z27	2/2	Running	0	11s

After
a few

seconds, Kubernetes replaces failing old Pods and all the Pods in the Deployment appear as running. The application no longer presents any problems.

Task 2: Test the web server application**Step 1** ▾

Since you fixed all the problems with the Pod definition and the application container runs a web server, you should test whether the web server runs correctly. Run the following command to get the IP addresses of the Pods:

command

```
kubectl get pods -o custom-columns=IP:.status.podIP
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl get pods
-o custom-columns=IP:.status.podIP
IP
10.42.0.120
10.42.1.237
10.42.1.238
```

As explained in previous sections, the **custom-columns** allow filtering the output based on its structure. In this example, the filter only retrieves PodIP information from the Pods' **status** property.

Step 2 ▾

Use a busybox container image and reach the web server using the **wget** command from the busybox container and the IP address from any of the Deployment Pods:

command

```
kubectl run busy -i --rm --image=busybox \
-- sh -c "sleep 2;wget -O - http://10.42.0.120"
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl run busy
-i --rm --image=busybox -- sh -c "sleep 2;wget -O - http://10.42.0.120"
If you don't see a command prompt, try pressing enter.
Connecting to 10.42.0.120 (10.42.0.120:80)
writing to stdout
- 100% |*****| 615 0:00:00 ETA
written to stdout
<!DOCTYPE html>
<html>
...
...
<h1>Welcome to nginx!</h1>
...
...
</html>
pod "busy" deleted
```

Pods are healthy, and the **nginx** web server application runs as expected.

Step 3 ▾

Clean the cluster by running the following command:

command

```
kubectl delete -f failing-application.yaml
```

Exercise Summary

In this exercise, you deployed a failing application and troubleshooted three different problems: a mismatched container image (ImagePullBackOff), a typo in a container command, and a low memory request value (OOMKilled) that prevented the Pod from starting. Later, you used a busybox container to reach the web server.

(End of Exercise)

Exercise 13.3 - Diagnose and troubleshoot a Kubernetes Service.

Description

In this exercise, you will diagnose and troubleshoot a failing Kubernetes service using the commands from the Kubernetes Troubleshooting Toolbox.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Diagnose a failing application. Service.
 - Understand some of the most common errors that cause Service failures.
 - Troubleshoot the issues of the failing Service.
-

Task 1: Deploy the failing Service in the Kubernetes cluster

Step 1 ▾

Deploy the failing Service, which creates a Pod running an nginx web server, and a Service for the Pod, by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/troubleshooting/
> kubectl apply -f failing-service.yaml
```

Step 2 ▾

Run the following command to retrieve the current status for the Pod and the Service:

command

```
kubectl get all
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting> kubectl get all
NAME          READY   STATUS    RESTARTS   AGE
pod/backend    1/1     Running   0           8s

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/backend-service             ClusterIP      10.43.252.27  <none>        8080/TCP   8s
service/kubernetes                   ClusterIP      10.43.0.1     <none>        443/TCP    12d
```

Note the **TYPE (ClusterIP)**, the **CLUSTER-IP(10.43.214.194)**, and the **PORT(8080/tcp)** for the Service.

Step 3 ▾

Run the following command to test the Service using a busybox container and the Service data:

command

```
kubectl run busy -i --rm --image=busybox -- sh -c \
"sleep 2;wget -T 10 -O - http://10.43.252.27:8080"
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl run busy -i --rm --image=busybox -- sh -c "sleep 2;wget -T 10
-O - http://10.43.252.27"
If you don't see a command prompt, try pressing enter.
Connecting to 10.43.252.27 (10.43.252.27)
wget: download timed out
```

```
pod "busy" deleted
```

Since this is a course exercise and we already know the Service fails, we added a 10-second timeout (**-T 10**) to the **wget** command. It timed out, therefore something is wrong with the Service.

Task 2: Fix the empty Endpoint problem

Step 1 ▾

Run the following command to retrieve detailed information about the Service:

command

```
kubectl describe svc backend-service
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl describe svc backend-service
Name:                backend-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:             app=web-server
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.43.252.27
IPs:                 10.43.252.27
Port:                <unset> 8080/TCP
TargetPort:          8080/TCP
Endpoints:           <none>
Session Affinity:    None
Internal Traffic Policy: Cluster
Events:              <none>
```

Since **Endpoints** is empty, there must be a problem with the Service's selector that targets the Pods. Note that the current value is **app=web-server**

Note ▾

svc in the **kubectl get** command stands as the shortname for **service**

Step 2 ▾

Review the labels used in the Pod by running the following command:

command

```
kubectl get pod backend -o custom-columns=LABELS:.metadata.labels
```

Explanation ▾

We used **-o custom-columns=LABELS:.metadata.labels** to only retrieve the Labels.

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl get pod backend -o custom-columns=LABELS:.metadata.labels
LABELS
map[app: backend]
```

The Pod display a **app: backend** labeled, which does not match the selector defined for the Pod: **app: web-server**

Step 3 ▾

Edit the **failing-service.yaml** and replace the selector for the service with the content below to match the label in the Pod definition:

failing-service.yaml

```
spec
  selector:
    app: backend
```

Save the changes and recreate the Service again by running **kubectl apply -f failing-service.yaml**

Step 4 ▾

Run the following command to retrieve detailed information about the Service:

command

```
kubectl describe svc backend-service
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl describe svc backend-service
Name:                backend-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:             app=backend
Type:                ClusterIP
```

```

IP Family Policy:      SingleStack
IP Families:           IPv4
IP:                    10.43.252.27
IPs:                   10.43.252.27
Port:                  <unset> 8080/TCP
TargetPort:            8080/TCP
Endpoints:             10.42.1.240:8080
Session Affinity:      None
Internal Traffic Policy: Cluster
Events:                <none>

```

*The **Endpoints*** section shows a Pod with IP 10.42.1.240 associated with the Service.

Step 5 ▾

Run the following command to retrieve the Pod IP address:

command

```
kubectl get pods -o wide
```

Explanation ▾

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE
backend   1/1     Running   0           33m   10.42.1.240   agent

```

The Service targets the right Pod because the Pod's IP address matches the IP in the ***Endpoint*** section.

Step 6 ▾

Run the following command to test the Service using a busybox container and the Service data:

command

```
kubectl run busy -i --rm --image=busybox \
  -- sh -c "sleep 2;wget -T 10 -O - http://10.43.252.27"
```

Explanation ▾

Example output:

```

tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl run busy -i --rm --image=busybox -- sh -c "sleep 2;wget -T 10
-O - http://10.43.252.27"
If you don't see a command prompt, try pressing enter.

```

```
Connecting to 10.43.252.27 (10.43.252.27:80)
wget: download timed out
pod "busy" deleted
```

The request times out again.

Task 3: Fix the port problem

Step 1 ▾

Run the following command to retrieve detailed information about the Service:

command

```
kubectl describe svc backend-service
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl describe svc backend-service
Name:                backend-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:             app=backend
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.43.252.27
IPs:                 10.43.252.27
Port:                <unset> 8080/TCP
TargetPort:          8080/TCP
Endpoints:           10.42.1.240:8080
Session Affinity:    None
Internal Traffic Policy: Cluster
Events:              <none>
```

The Service opens the **8080** port and targets the same port in the Pod definition. Requests to the Service must use port **8080**.

Step 2 ▾

Run the following command to test the Service using port **8080**:

command

```
kubectl run busy -i --rm --image=busybox \
  -- sh -c "sleep 2; wget -T 10 -O - http://10.43.252.27:8080"
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl run busy -i --rm --image=busybox -- sh -c "sleep 2;wget -T 10
-O - http://10.43.252.27:8080"
If you don't see a command prompt, try pressing enter.
Connecting to 10.43.252.27:8080 (10.43.252.27:8080)
wget: can't connect to remote host (10.43.252.27): Connection refused
pod "busy" deleted
```

This time the connection is refused.

Step 3 ▾

If the `wget` request continues to fail, verify that the Service is targeting the correct Pod port:

command

```
kubectl get pod backend -o
custom-columns=NAME:.metadata.name,PORT:.spec.containers[0].ports[0].c
ontainerPort
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl get pod backend -o
custom-columns=NAME:.metadata.name,PORT:.spec.containers[0].ports[0].c
ontainerPort
NAME      PORT
backend   80
```

The **containerPort** is informative, but an **80** value makes sense, as the Pod is running an `nginx` server that uses port 80 by default. The Service should target port 80 rather than 8080 in the Pod.

Step 4 ▾

Edit the *failing-service.yaml* and set the correct value for the `targetPort`:

failing-service.yaml

```
ports:
  - protocol: TCP
    port: 8080
    targetPort: 80
```

Save the changes and recreate the Service again by running ***kubectl apply -f failing-service.yaml***

Step 5 ▾

Run the following command to test the Service using port **8080**:

command

```
kubectl run busy -i --rm --image=busybox \
  -- sh -c "sleep 2;wget -T 10 -O - http://10.43.252.27:8080"
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/troubleshooting>
kubectl run busy -i --rm --image=busybox -- sh -c "sleep 2;wget -T 10
-O - http://10.43.252.27:8080"
If you don't see a command prompt, try pressing enter.
Connecting to 10.43.252.27:8080 (10.43.252.27:8080)
writing to stdout
-                               100% |*****|          615
0:00:00 ETA
written to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
...
</body>
</html>
pod "busy" deleted
```

The Service is now correctly exposing the Pod via its ClusterIP.

Step 6 ▾

Clean the cluster by running the following command:

command

```
kubectl delete -f failing-service.yaml
```

Exercise Summary

In this exercise, you addressed and solved two different problems while exposing a Pod with a ClusterIP service. First, you noted an empty Endpoints section, indicating an incorrect use of the selector property, and you fixed it. Later, you detected and fixed a mismatch between the targetPort in the Service and the port in which the Pod was offering the web server service.

(End of Exercise)

Section 14: Cluster Operations in Kubernetes

Description

In addition to creating Kubernetes objects, deploying applications, and taking security measures, Kubernetes administrators are also responsible for the cluster's daily operations. This section introduces them and provides guidelines for performing the most important cluster operations in Upstream Kubernetes and in RKE2, the cluster used for the course Labs.

Section Learning Objectives

- Understand cluster day-to-day challenges: identify the tasks and risks associated with cluster maintenance and upgrades.
- Perform Node maintenance: Use `kubect`l cordon and `kubect`l drain to move Pods from a Node before maintenance.
- Execute cluster upgrades: Describe and follow the steps for upgrading the control plane (server) and worker nodes (agents).
- Back up the cluster: Create and manage snapshots.
- Restore cluster state: Use a previously created backup to restore your cluster to a previous state.
- Decommission nodes: Safely remove a worker node from the cluster.

Section Topics

- Cluster operations
- Node lifecycle management
- Cluster upgrades
- Backups and disaster recovery

Exercise 14.1 - Node maintenance using Kubernetes tools

Description

In addition to creating Kubernetes objects, deploying applications, and taking security measures, Kubernetes administrators are also responsible for the cluster's daily operations. Some of these operations affect the cluster and may require maintenance on Nodes. In this exercise, you will discover Kubernetes maintenance commands and understand the impact those commands have on Nodes and in the cluster itself.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Cordon a Node to make it unschedulable.
 - Uncordon a Node to bring it back in service.
 - Drain
-

Task 1: Cordon the agent Node and understand the effect of cordoning a Node**Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open a terminal in the management VM and run the following command to cordon the **agent** (worker) Node:

command

```
kubectl cordon agent
```

Step 4 ▾

Run the following command to get information about the Nodes and their labels:

command

```
kubectl get nodes --show-labels
```

Explanation ▾

Example output:

```
kubectl get node --show-labels
NAME          STATUS          ROLES    AGE    VERSION
agent         Ready,SchedulingDisabled  <none>   13d    v1.33.7+rke2r1
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=rke2,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=agent,kubernetes.io/os=linux,node.kubernetes.io/instance-type=rke2
server        Ready           control-plane,etcd,master  13d    v1.33.7+rke2r1
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=rke2,beta.kubernetes.io/
```

```
os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=server,kubernetes.io/os=linux,
node-role.kubernetes.io/control-plane=true,node-role.kubernetes.io/etcd=true,node-role.
kubernetes.io/master=true,node.kubernetes.io/instance-type=rke2
```

After cordoning a Node, it displays as **SchedulingDisable** in the **STATUS** column. The Kubernetes scheduler cannot assign any Pod to run on the Node. Note that the label **kubernetes.io/hostname=agent** identifies the **agent** Node.

Step 5 ▾

Review the YAML manifest for a Pod by running the following commands:

commands

```
> cd ~/course_files/RKE201v2/labs/manifests/cluster-operations/
> vim pod-in-agent-node.yaml
```

Explanation ▾

Example output:

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: pod-in-agent-node
    name: pod-in-agent-node
spec:
  nodeSelector:
    kubernetes.io/hostname: agent
  containers:
  - image: busybox
    name: pod-in-agent-node
    command: ["sleep", "3600"]
```

The Pod YAML definition includes a nodeSelector property that instructs the scheduler to select Nodes with the **kubernetes.io/hostname: agent** label to assign one of them to the Pod.

Step 6 ▾

Run the following command to create the Pod:

command

```
kubectl apply -f pod-in-agent-node.yaml
```

Step 7 ▾

Run the following command to retrieve detailed information about the Pod:

command

```
kubectl get describe pod pod-in-agent-node
```

Explanation ▾

Example output:

```
Name:          pod-in-agent-node
Namespace:     default
Priority:       0
Service Account: default
Node:          <none>
Labels:        run=pod-in-agent-node
Annotations:    <none>
Status:        Pending
...
Conditions:
  Type           Status
  PodScheduled   False
...
Events:
  Type       Reason              Age   From          Message
  ----       -
  Warning    FailedScheduling    7s    default-scheduler  0/2 nodes are available:
1 node(s) didn't match Pod's node affinity/selector, 1 node(s) were
unschedulable. preemption: 0/2 nodes are available: 2 Preemption is not
helpful for scheduling.
```

As expected, because the agent Node is cordoned, the Pod remains in a Pending status, and the Events section indicates that no Node is available for the Pod's selection.

Step 8 ▾

Uncordon the **agent** Node by running the following command:

command

```
kubectl uncordon agent
```

Step 9 ▾

Run the following command to get the current status for the Nodes:

command

```
kubectl get nodes
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operation
ns> kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
```

agent	Ready	<none>	13d	v1.33.7+rke2r1
server	Ready	control-plane,etcd,master	13d	v1.33.7+rke2r1

The agent node status returns to **Ready**.

Step 10 ▾

Retrieve the information about the Pods in the working namespace by running the following command to

command

```
kubectl get pods -o wide
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operation
ns> kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP
NODE    NOMINATED NODE   READINESS GATES
pod-in-agent-node   1/1     Running    0          35m   10.42.1.247
agent    <none>    <none>
```

Since the Node is ready after being uncordoned, the Kubernetes scheduler found a destination Node for the Pod, and the Pod could run in the **agent** Node.

Task 2: Drain the agent Node and understand the effects of draining a Node

Step 1 ▾

Create a Deployment interactively by running the following command:

command

```
kubectl create deployment new-deployment --image=nginx --replicas=3
```

Step 2 ▾

Retrieve information about the Pods by running the following command:

command

```
kubectl get pods -o wide
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operations> k get pods -o
wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE
```

NOMINATED NODE	READINESS GATES							
new-deployment-6c84b9b6cc-5xr82	1/1	Running	0	3s	10.42.1.249	agent		
new-deployment-6c84b9b6cc-hwzcd	1/1	Running	0	3s	10.42.0.123	server		
new-deployment-6c84b9b6cc-n8plr	1/1	Running	0	3s	10.42.1.248	agent		
pod-in-agent-node	1/1	Running	0	72m	10.42.1.247	agent		

The **pod-in-agent-node** Pod runs on the agent node as requested, and the Kubernetes scheduler has distributed the Deployment Pods between the server and agent Nodes.

Warning

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 3

Try to drain the **agent** Node by running the following command:

command

```
kubectl drain agent
```

Explanation

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operation
ns> kubectl drain agent
node/agent cordoned
error: unable to drain node "agent" due to error: [cannot delete
cannot delete Pods that declare no controller (use --force to
override): default/pod-in-agent-node, cannot delete Pods with local
storage (use --delete-emptydir-data to override):
kube-system/csi-nfs-controller-6f78747487-zkddv, cannot delete
DaemonSet-managed Pods (use --ignore-daemonsets to ignore):
kube-system/csi-nfs-node-n8wht, kube-system/rke2-canal-zhbm2,
kube-system/rke2-traefik-x7r6j], continuing command...
There are pending nodes to be drained:
agent
cannot delete cannot delete Pods that declare no controller (use
--force to override): default/pod-in-agent-node
cannot delete Pods with local storage (use --delete-emptydir-data to
override): kube-system/csi-nfs-controller-6f78747487-zkddv
cannot delete DaemonSet-managed Pods (use --ignore-daemonsets to
ignore): kube-system/csi-nfs-node-n8wht, kube-system/rke2-canal-zhbm2,
kube-system/rke2-traefik-x7r6j
```

The Node was cordoned, but the drain failed for three reasons:

- The standalone Pod **pod-in-agent-node** is not managed. No controller (Deployment, ReplicaSet, StatefulSet, etc) is responsible for it. To drain the Node with this kind of Pod, use the **--force** flag. If you use this flag, this Pod will be lost.
- The **csi-nfs-controller-6f78747487-zkddv** uses an emptyDir Volume. To drain a Node with Pods using an emptyDir Volume, use the **--delete-emptydir-data** flag. Kubernetes can reschedule the Pod to another Node, but the contents of the Volume will be lost.
- The **kube-system/csi-nfs-node-n8wht**, **kube-system/rke2-canal-zhbm2**, and **kube-system/rke2-traefik-x7r6j** Pods are part of DaemonSets, which deploy a Pod in every Node. Use the **--ignore-daemonsets** flag to drain a Node with Pods managed by a DaemonSet.

Step 4 ▾

Try to drain the **agent** Node by running the following command:

command

```
kubectl drain agent --force --delete-emptydir-data --ignore-daemonsets
```

Explanation ▾

Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operation
ns> kubectl drain agent --force --delete-emptydir-data
--ignore-daemonsets
node/agent already cordoned
Warning: ignoring DaemonSet-managed Pods:
kube-system/csi-nfs-node-n8wht, kube-system/rke2-canal-zhbm2,
kube-system/rke2-traefik-x7r6j; deleting Pods that declare no
controller: default/pod-in-agent-node
evicting pod kube-system/rke2-coredns-rke2-coredns-76994ddb78-4dpjd
evicting pod default/pod-in-agent-node
evicting pod default/new-deployment-6c84b9b6cc-5xr82
evicting pod default/new-deployment-6c84b9b6cc-n8plr
evicting pod kube-system/csi-nfs-controller-6f78747487-zkddv
evicting pod kube-system/helm-install-rke2-traefik-n757q
pod/helm-install-rke2-traefik-n757q evicted
pod/new-deployment-6c84b9b6cc-n8plr evicted
pod/csi-nfs-controller-6f78747487-zkddv evicted
pod/new-deployment-6c84b9b6cc-5xr82 evicted
pod/rke2-coredns-rke2-coredns-76994ddb78-4dpjd evicted
pod/pod-in-agent-node evicted
node/agent drained
```

Step 5 ▾

Retrieve information about all the Pods in the cluster by running the following command:

command

```
kubectl get pods -o wide
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operations> kubectl get pods -o wide
```

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP
new-deployment-6c84b9b6cc-hwzcd	server	1/1	Running	0	25m	10.42.0.123
new-deployment-6c84b9b6cc-qx8jl	server	1/1	Running	0	5m12s	10.42.0.124
new-deployment-6c84b9b6cc-v5lmf	server	1/1	Running	0	5m12s	10.42.0.125

The **pod-in-agent-node** has disappeared. All the Deployment Pods are now running on the **server** Node, and Kubernetes has removed or recreated the system Pods from the cluster, depending on whether they were part of a DaemonSet or used an emptyDir Volume.

Step 6

Uncordon the Node and enable scheduling:

command

```
kubectl uncordon agent
```

Step 7

Verify that the **agent** Node is back in service:

command

```
kubectl get nodes
```

Explanation**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/cluster-operations> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
agent	Ready	<none>	13d	v1.33.7+rke2r1
server	Ready	control-plane,etcd,master	13d	v1.33.7+rke2r1

Step 8

Delete the deployment to return the cluster to its original state:

command

```
kubect! delete deployment new-deployment
```

Exercise Summary

In this exercise, you cordoned a cluster Node, verified that it was immediately unavailable to the Kubernetes scheduler, and confirmed that it was back in service once uncordoned. Later, you identified issues while trying to drain a Node, resolved them, and verified how Pods, depending on their nature and the cluster resource availability, were rescheduled or removed.

(End of Exercise)

Exercise 14.2 - Update the RKE2 cluster to Kubernetes 1.34.XX

Description

The Cloud-Native Computing Foundation (CNCF) releases new Kubernetes versions three times a year. Latest versions include new features and security fixes. Upgrading a Kubernetes cluster is one of the most crucial cluster operations. In this exercise, you use RKE2 native tools to upgrade the Lab RKE2 Kubernetes cluster from version 1.33.7 to version 1.34.XX.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Understand the upgrade process for RKE2 Kubernetes clusters.
 - Upgrade an RKE2 Kubernetes cluster.
-

Task 1: Upgrade the RKE2 server node

Step 1 ▾

According to the [official RKE documentation](#), to manually upgrade an RKE2 Kubernetes cluster, upgrade the server (control-plane) Nodes one at a time. Once all servers have been upgraded, you may then upgrade agent nodes.

Run the following command to start the upgrade process by opening a shell in the server virtual machine (VM) (Use linux as the password when prompted):

command

```
ssh root@server
```

Step 2 ▾

Because the cluster was installed using the Tarball method, use the same installation script to upgrade the cluster by running:

command

```
INSTALL_RKE2_TYPE=server INSTALL_RKE2_CHANNEL=v1.34 ./install.sh
```

Explanation ▾

- **INSTALL_RKE2_TYPE=server:** Selects the Node type for the upgrade. Server is the default type.
- **INSTALL_RKE2_CHANNEL=v1.34:** Select the latest 1.34.X version for the upgrade process.

Example output:

```
server:~ # INSTALL_RKE2_TYPE=server INSTALL_RKE2_CHANNEL=v1.34
./install.sh
[WARN] /usr/local is read-only or a mount point; installing to
/opt/rke2
[INFO] finding release for channel stable
[INFO] using v1.34.3+rke2r1 as release
[INFO] downloading checksums at
https://github.com/rancher/rke2/releases/download/v1.34.3%2Brke2r1/sha
256sum-amd64.txt
[INFO] downloading tarball at
https://github.com/rancher/rke2/releases/download/v1.34.3%2Brke2r1/rke
2.linux-amd64.tar.gz
[INFO] verifying tarball
[INFO] unpacking tarball file to /opt/rke2
[INFO] updating tarball contents to reflect install path
[INFO] moving systemd units to /etc/systemd/system
[INFO] install complete; you may want to run: export
PATH=$PATH:/opt/rke2/bin
```

Note

Note that version **1.34.3** has been selected by the installation script as the latest **1.34.X** version. This value might change in the future.

Step 4

Restart the server service by running the following command:

command

```
systemctl restart rke2-server
```

Warning

Restarting the server (control-plane) node may take several minutes and causes RKE2 downtime. To avoid downtime, you must have a High Availability (HA) configuration with more than one server (control-plane) Node. Review the [official RKE documentation](#) for more information about HA clusters.

Note

The RKE2 upgrade process performs maintenance cluster operations (cordon, drain, uncordon) under the hood.

Step 5

Run the following command to review the Node status:

command

```
kubectl get nodes
```

Explanation**Example output:**

```
server:~ # kubectl get nodes
NAME      STATUS   ROLES                                AGE   VERSION
agent     Ready    <none>                               13d   v1.33.7+rke2r1
server    Ready    control-plane,etcd,master           13d   v1.34.3+rke2r1
```

The **server** Node displays the new version, **1.34.3**, while the **agent** Node displays the current version. Since the minor version differs by only one, the upgrade process complies with the Kubernetes Version Skew Policy, and no issues are expected.

Step 6

Run **exit** in the shell to return to the **RKE2-management** terminal.

Task 2: Upgrade the RKE2 agent node**Step 1** ▾

Run the following command to open a shell in the agent virtual machine (VM) (Use linux as the password when prompted):

command

```
ssh root@agent
```

Step 2 ▾

Since we used the Tarball method and its installation script to install the cluster, use the same installation script to upgrade the cluster by running:

command

```
INSTALL_RKE2_TYPE=agent INSTALL_RKE2_CHANNEL=v1.34 ./install.sh
```

Explanation ▾

- ***INSTALL_RKE2_TYPE=agent***: Selects the Node type for the upgrade. Server is the default type.
- ***INSTALL_RKE2_CHANNEL=v1.34***: Select the latest 1.34.X version for the upgrade process.

Example output:

```
agent:~ # INSTALL_RKE2_TYPE=agent INSTALL_RKE2_CHANNEL=v1.34
./install.sh
[WARN] /usr/local is read-only or a mount point; installing to
/opt/rke2
[INFO] finding release for channel stable
[INFO] using v1.34.3+rke2r1 as release
[INFO] downloading checksums at
https://github.com/rancher/rke2/releases/download/v1.34.3%2Brke2r1/sha
256sum-amd64.txt
[INFO] downloading tarball at
https://github.com/rancher/rke2/releases/download/v1.34.3%2Brke2r1/rke
2.linux-amd64.tar.gz
[INFO] verifying tarball
[INFO] unpacking tarball file to /opt/rke2
[INFO] updating tarball contents to reflect install path
[INFO] moving systemd units to /etc/systemd/system
[INFO] install complete; you may want to run: export
PATH=$PATH:/opt/rke2/bin
```

Note ▾

Note that version **1.34.3** has been selected by the installation script as the latest **1.34.X** version. This value might change in the future.

Step 4 ▾

Restart the agent service by running the following command:

command

```
systemctl restart rke2-agent
```

Step 5 ▾

Run **exit** in the shell to return to the **RKE2-management** terminal.

Step 6 ▾

Run the following command to review the Node status:

command

```
kubectl get nodes
```

Explanation ▾

Example output:

```
tux@management:~> kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
agent	Ready	<none>	13d	v1.34.3+rke2r1
server	Ready	control-plane,etcd,master	13d	v1.34.3+rke2r1

The **agent** Node now displays the same version as the **server** Node, version, **1.34.3**.

Step 7 ▾

Review the version for the different components in the cluster by running the following command:

command

```
kubectl version
```

Explanation ▾

Example output:

```
tux@management:~> kubectl version
```

Client Version: v1.33.7+rke2r1
Kustomize Version: v5.6.0
Server Version: v1.34.3+rke2r1

Since you copied the server kubectl client to the management VM during installation, kubectl on the management VM is still the old version.

Step 8 ▾

Copy the new kubectl binary from the server VM (use 'linux' as the password when prompted):

command

```
scp root@server:/usr/local/bin/kubectl ~/bin/
```

Step 9 ▾

Review the version again:

command

```
kubectl version
```

Explanation ▾

Example output:

```
Client Version: v1.34.3+rke2r1
Kustomize Version: v5.7.1
Server Version: v1.34.3+rke2r1
```

Now that the upgrade process has finished, the nodes are running the new version, and so is the client binary used on the management VM.

Exercise Summary

In this exercise, you use the installation script, downloaded for the Tarball installation method, to upgrade a server (control-plane) Node, and an agent (worker) Node in the Lab's Kubernetes cluster.

(End of Exercise)

Exercise 14.3 - Back up and restore your RKE2 cluster

Description

Disaster recovery is a critical cluster operation. Kubernetes administrators must have cluster backups to restore the cluster to a previous state in case of an issue. In this exercise, you will learn how to back up and recover an RKE2 Kubernetes cluster using the native tools provided by this Kubernetes distribution.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a backup of an RKE2 Kubernetes cluster using the RKE2 native tools.
 - Discover the default destination directory for automated and on-demand backups (snapshots).
 - Restore an RKE2 cluster to a stable state using RKE2 native tools.
-

Task 1: Create an RKE cluster backup**Step 1** ▾

Create a new Deployment imperatively by running the following command:

command

```
kubectl create deployment deployment-to-backup --image=redis \
  --replicas=10
```

Step 2 ▾

Verify that the Deployment was successfully created by running the following command:

command

```
kubectl get all
```

Explanation ▾**Example output:**

```
tux@management:~> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-to-backup-7b8db7c6c8-4565q	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-4l895	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-d75xl	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-kk47z	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-mslpv	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-mv5sd	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-ncnb6	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-qv8bs	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-ibxdf	1/1	Running	0	3m32s
pod/deployment-to-backup-7b8db7c6c8-t7gfh	1/1	Running	0	3m32s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	13d

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment-to-backup	10/10	10	10	3m32s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-to-backup-7b8db7c6c8	10	10	10	3m32s

The Deployment, the corresponding ReplicaSet, and the Pods appear to be healthy and running.

Warning ▾

For some objects (e.g., Pods in a Deployment or a ReplicaSet), Kubernetes generates random suffixes. Always verify the exact names in your environment and replace the example names in this guide with your actual names.

Step 3 ▾

As part of the installation process, RKE2 Native tools are on the server Node. Run the following command to open a shell in the server virtual machine (VM) (Use linux as the password when prompted):

command

```
ssh root@server
```

Step 4

List the automated backups (snapshots) created by RKE2 by running the following command:

command

```
server:~ # ls -l /var/lib/rancher/rke2/server/db/snapshots/
```

Explanation

Example output:

```
server:~ # ls -l /var/lib/rancher/rke2/server/db/snapshots/
total 72192
-rw----- 1 root root 14835744 Jan 28 12:00 etcd-snapshot-server-1769598023
-rw----- 1 root root 14835744 Jan 29 00:00 etcd-snapshot-server-1769641200
-rw----- 1 root root 14028832 Jan 29 12:00 etcd-snapshot-server-1769684402
-rw----- 1 root root 15360032 Feb  1 00:00 etcd-snapshot-server-1769900423
-rw----- 1 root root 14843936 Feb  1 12:00 etcd-snapshot-server-1769943603
```

`/var/lib/rancher/rke2/server/db/snapshots/` is the RKE2 default directory for the snapshots. By default, RKE2 keeps up to 5 automated backups, made twice a day at 00:00 and 12:00.

Step 5

Create an on-demand snapshot by running the following commands:

commands

```
> cd /opt/rke2/bin
> ./rke2 etcd-snapshot save
```

Explanation

Example output:

```
server:/opt/rke2/bin # ./rke2 etcd-snapshot save
WARN[0000] Unknown flag --token found in config.yaml, skipping
WARN[0000] Unknown flag --tls-san found in config.yaml, skipping
WARN[0000] Unknown flag --cni found in config.yaml, skipping
WARN[0000] Unknown flag --ingress-controller found in config.yaml, skipping
INFO[0000] Snapshot on-demand-server-1770017799 saved.
```

The `/opt/rke2/bin` directory contains some RKE2 native tools, such as the ***rke2*** command, which allows us to create snapshots of our RKE2 Kubernetes cluster using the ***etcd-snapshot*** save subcommand.

The name for the new snapshot appears at the end: ***on-demand-server-1770017799***.

Warning ▾

The name for the snapshot varies depending on the current timestamp.

Additional Information ▾

Review the [official RKE documentation](#) for more information about the *etcd-snapshot* subcommand and its options.

Step 6 ▾

List the backups (snapshots) again by running the following command:

command

```
server:~ # ls -l /var/lib/rancher/rke2/server/db/snapshots/
```

Explanation ▾

Example output:

```
server:/opt/rke2/bin # ls -l /var/lib/rancher/rke2/server/db/snapshots/
total 89468
-rw----- 1 root root 14835744 Jan 28 12:00 etcd-snapshot-server-1769598023
-rw----- 1 root root 14835744 Jan 29 00:00 etcd-snapshot-server-1769641200
-rw----- 1 root root 14028832 Jan 29 12:00 etcd-snapshot-server-1769684402
-rw----- 1 root root 15360032 Feb  1 00:00 etcd-snapshot-server-1769900423
-rw----- 1 root root 14843936 Feb  1 12:00 etcd-snapshot-server-1769943603
-rw----- 1 root root 17686560 Feb  2 08:36 on-demand-server-1770017799
```

The name for the new snapshot appears at the end: ***on-demand-server-1770017799***.

Step 7 ▾

Delete the Deployment created at the beginning of the exercise by running the following command:

command

```
kubectl delete deployments deployment-to-backup
```

Step 8 ▾

Verify that the Deployment was successfully created by running the following command:

command

```
kubectl get all
```

Explanation ▾

Example output:

```
server:/opt/rke2/bin # kubectl get all
NAME                                TYPE             CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP         10.43.0.1     <none>         443/TCP    13d
```

The Deployment, the corresponding ReplicaSet, and the Pods have disappeared, but are still part of the previous backup.

Task 2: Restore an RKE2 cluster to a previous stable state

Step 1 ▾

Restoring an RKE2 cluster from an available snapshot requires stopping all server (control-plane) nodes in the cluster. Since the Lab only has one, and we must still be on a shell running on it, stop the RKE2 service by running the following command:

command

```
systemctl stop rke2-server
```

Warning ▾

Restoring a cluster requires stopping all the RKE2 server nodes, which ultimately implies a certain amount of downtime.

Step 2 ▾

Restore the cluster by running the following commands:

commands

```
> cd /opt/rke2/bin
> ./rke2 server --cluster-reset \
--cluster-reset-restore-path=/var/lib/rancher/rke2/server/db/snapshots
/on-demand-server-1770017799
```

Step 3 ▾

Restart the RKE2 cluster:

command

```
systemctl restart rke2-server
```

Step 4 ▾

Check if the Deployment was successfully restored by running the following command:

command

```
kubectl get all
```

Explanation**Example output:**

```
server:/opt/rke2/bin # kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/deployment-to-backup-7b8db7c6c8-4565q	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-4l895	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-d75xl	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-kk47z	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-mslpv	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-mv5sd	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-ncnb6	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-qv8bs	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-ibxdf	1/1	Running	0	69m
pod/deployment-to-backup-7b8db7c6c8-t7gfv	1/1	Running	0	69m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	13d

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/deployment-to-backup	10/10	10	10	69m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/deployment-to-backup-7b8db7c6c8	10	10	10	69m

All the Deployment elements are back as part of the recovery process.

Step 5

Leave the server VM and return to the **RKE2-management** VM by running **exit**. Delete the Deployment to clean the cluster for upcoming exercises:

command

```
kubectl delete deployments deployment-to-backup
```

Exercise Summary

In this exercise, you used RKE2 native tools to perform a backup of an RKE2 Kubernetes cluster, discovered the default destination for snapshots, and automated backups. Later, you used the snapshot after a “supposed” accidental deletion to restore the cluster to a previous and stable state.

(End of Exercise)

Section 15: Kubernetes Application Management with Helm

Description

Deploying applications in Kubernetes is not easy. Kubernetes administrators often manage dozens or hundreds of YAML files to create the Kubernetes objects required for each application, must keep track of all the changes in the YAML files, and must repeat this process across every environment where the application is deployed.

Helm, a Kubernetes application management tool, helps Kubernetes administrators address all those issues.

Section Learning Objectives

- Define main Helm components and concepts: Understand the structure and function of a Helm Chart (templates, values, metadata).
- Manage Helm repositories: Add and search Helm repositories, including Artifacthub.io, to discover stable Helm Charts.
- Manage Helm releases: Use Helm commands, such as `helm install`, `helm upgrade`, `helm rollback`, `helm list`, or `helm history` to manage the application lifecycle.
- Use custom values for deployments with Helm: Override default configuration parameters using custom values.yaml files or the `-set` flag.
- Understand templates: Use the Go template language to generate Kubernetes manifests.

Section Topics

- Why Helm?
- Helm Charts and Repositories
- Release Management (Install, Upgrade, List, History)

Exercise 15.1 - Add a repository to Helm

Description

Helm charts are distributed through Helm repositories. Repositories can be public or private, with ArtifactHub (artifact.io) being the most well-known website for finding repositories for deploying applications. In this exercise, you will get familiar with ArtifactHub, find a repository there, and add it to your Helm installation. Later, you will use Helm capabilities to search for charts in the repository and retrieve information about them.

Learning Outcomes

Upon completing this exercise, you will be able to:

- Use ArtifactHub to
 - Add new repositories to Helm.
 - Search for Helm charts within a repository.
 - Search for Helm charts across the entire ArtifactHub site.
 - Retrieve information
-

Task 1: Add a repository to Helm

Step 1 ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

Step 2 ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

Username: tux

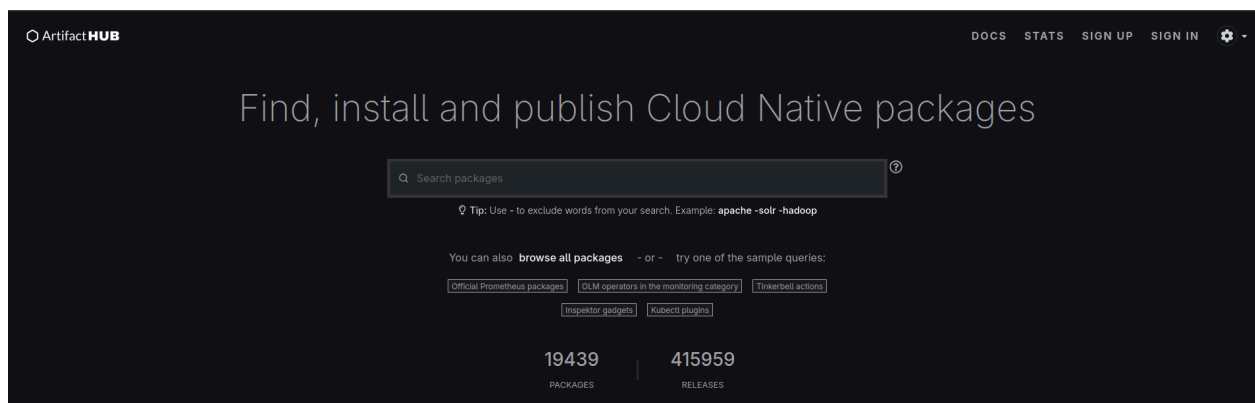
Password: linux

Warning ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

Step 3 ▾

Open the Firefox browser in your management VM and navigate to ArtifactHub using the following URL: <http://artifacthub.io>

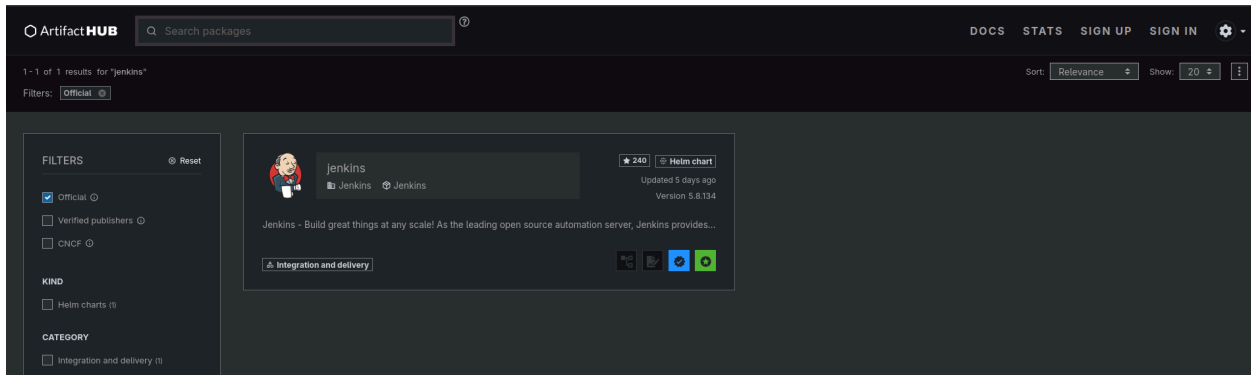


Step 4 ▾

Enter **jenkins** in the search box to find Helm repositories and charts for the famous open-source CI/CD application.

Step 5

Select **Official** on the FILTER section on the right and click on the only result that displays:

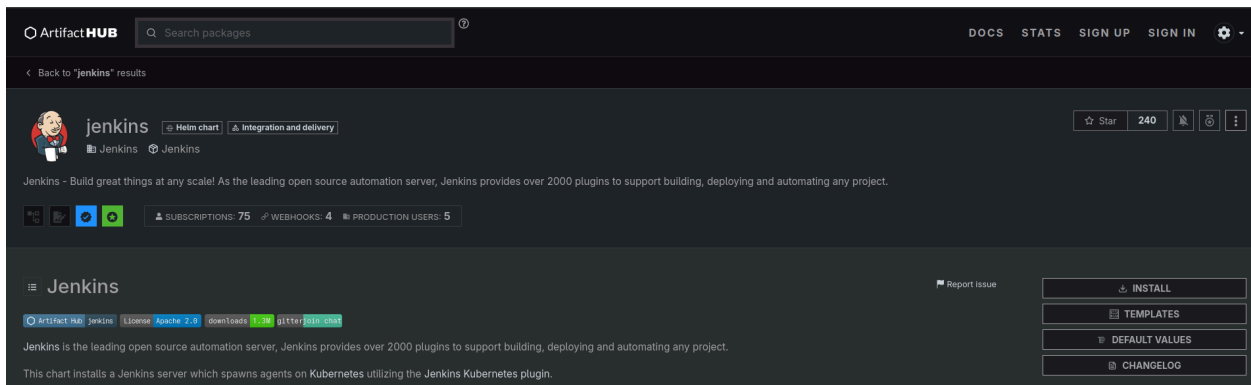
**Explanation**

Official images are preferred over unofficial images. They are trustworthy and less likely to present security issues.

Step 6

Browse the page for the official Jenkins Helm chart and review the following elements:

- The main area: It displays the content of the README.md file for the latest version of the chart. It contains general information and instructions.
- The **INSTALL** button on the right displays a pop-up with general instructions for installing the chart.
- The **TEMPLATES** button, also on the right, displays in a pop-up window the files in the chart's templates folder.
- The **DEFAULT VALUES** button, also on the right, opens a pop-up window that displays the values.yaml file in the chart.



Step 7 ▾

Find the **Get Repository Info** section in the main area and copy the two commands for later use.

Step 8 ▾

Open a terminal in your management VM and add the Helm repository for Jenkins using the following commands (previously copied from ArtifactHub):

commands

```
> helm repo add jenkins https://charts.jenkins.io
> helm repo update
```

Explanation ▾**Example output:**

```
tux@management:~> helm repo add jenkins https://charts.jenkins.io
"jenkins" has been added to your repositories
tux@management:~> helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "jenkins" chart repository
...Successfully got an update from the "bitnami" chart repository
Update Complete. \E2\8E\88Happy Helming!\E2\8E\88
```

- **helm repo add jenkins https://charts.jenkins.io:** This command adds the Jenkins repository to Helm, using **jenkins** as the repository name.
- **helm repo update:** This command updates chart information for all repositories added to Helm.

Step 9 ▾

Verify that the repository was successfully added by running the following command:

command

```
helm repo list
```

Explanation ▾**Example output:**

```
tux@management:~> helm repo list
NAME          URL
jenkins       https://charts.jenkins.io
```

Task 2: Search for Charts

Step 1 ▾

Display the latest stable version for all the charts in the local repositories:

command

```
helm search repo
```

Explanation ▾

Example output:

```
tux@management:~> helm search repo
NAME          CHART VERSION   APP VERSION DESCRIPTION
jenkins/jenkins 5.8.134         2.541.1   Jenkins - Build great
```

Without any keyword, the **helm search repo** command returns all the latest versions of the charts in the locally added Helm repositories. Because only one repository was added and it contains only one chart, only a single result is displayed.

Step 2 ▾

Add a keyword to the search operation by running the following command:

command

```
helm search repo SCALE
```

Explanation ▾

Example output:

```
tux@management:~> helm search repo
NAME          CHART VERSION   APP VERSION DESCRIPTION
jenkins/jenkins 5.8.134         2.541.1   Jenkins - Build great
```

Searches are case-insensitive and include several chart fields, such as NAME, DESCRIPTION, and the keywords property on the **charts.yaml** file. Since we have only a single repository with a single chart, the output remains the same.

Step 3 ▾

Search for a Helm chart at ArtifactHub by running the following command:

command

```
helm search hub jenkins
```

Explanation ▾

Example output:

```
tux@management:~> helm search hub jenkins
```

URL	CHART VERSION	APP
VERSION	DESCRIPTION	
https://artifacthub.io/packages/helm/jenkinsci/...	5.8.134	
2.541.1	Jenkins - Build great things at any scale! As t...	
https://artifacthub.io/packages/helm/test-jenki...	9.1.0	
2.426.1	Jenkins - Build great things at any scale! The ...	
https://artifacthub.io/packages/helm/aditisingh...	1.0.0	
Setup of jenkins manifests		
https://artifacthub.io/packages/helm/fatihtepe-...	1.0.0	
Setup of jenkins manifests		
https://artifacthub.io/packages/helm/openshift/...	0.0.3	
1.16.0	Jenkins is an open source automation server whi...	
https://artifacthub.io/packages/helm/jenkins-x/...	0.6.1	5.4
Flexible and powerful open source, distributed ...		
...		
...		
...		
...		
...		
https://artifacthub.io/packages/helm/jenkins-x/...	0.0.63	
A Helm chart for Tekton Pipelines		
https://artifacthub.io/packages/helm/jenkins-x/...	0.0.10	1.0
A Helm chart for Kubernetes		
https://artifacthub.io/packages/helm/jenkins-x/...	0.3.3	
0.4.16	A Helm chart for bank vault operator	
https://artifacthub.io/packages/helm/jenkins-x/...	0.0.3	
A Helm chart for velero-backup		
https://artifacthub.io/packages/helm/webhookrel...	0.4.1	0.6.0
Webhook Relay Operator provides an easy way to ...		

Since the ***helm search hub*** command searched the entire ArtifactHub site, there are many more results.

Note ▾

Note that the output fields differ and include the URL for the repository with the matching result. Also note that CLI results do not distinguish between community, official, or verified charts.

Step 4 ▾

Display all the versions for a chart across all the locally added repositories by running the following command:

command

```
helm search repo jenkins -l
```

Explanation ▾

Example output:

```
tux@management:~> helm search repo jenkins -l
```

NAME	CHART VERSION	APP VERSION	DESCRIPTION
------	---------------	-------------	-------------

```
jenkins/jenkins5.8.134      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.133      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.132      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.131      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.130      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.129      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.128      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.127      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.125      2.541.1      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.124      2.528.3      Jenkins - Build great things at any scale!
jenkins/jenkins5.8.123      2.528.3      Jenkins - Build great things at any scale!
...
jenkins/jenkins0.1.10       Open source continuous integration server.
jenkins/jenkins0.1.9        Open source continuous integration server.
jenkins/jenkins0.1.8        Open source continuous integration server.
jenkins/jenkins0.1.7        A Jenkins Helm chart for Kubernetes.
jenkins/jenkins0.1.6        A Jenkins Helm chart for Kubernetes.
jenkins/jenkins0.1.5        A Jenkins Helm chart for Kubernetes.
jenkins/jenkins0.1.4        A Jenkins Helm chart for Kubernetes.
jenkins/jenkins0.1.1        A Jenkins Helm chart for Kubernetes.
jenkins/jenkins0.1.0        A Jenkins Helm chart for Kubernetes.
```

When searching with the **-l** flag, the output displays all versions of the charts that match the search keywords.

Task 3: Retrieve information about Helm Charts

Step 1 ▾

Display the **chart.yaml** file for the latest stable version of the **jenkins** chart in the **jenkins** repository by running the following command:

command

```
helm show chart jenkins/jenkins
```

Explanation ▾

Example output:

```
tux@management:~> helm show chart jenkins/jenkins
annotations:
  artifacthub.io/category: integration-delivery
  artifacthub.io/changes: |
    - Update `configuration-as-code` to version `2037.v8e5349845172`
  artifacthub.io/images: |
    - name: jenkins
      image: docker.io/jenkins/jenkins:2.541.1-jdk21
    - name: k8s-sidecar
      image: docker.io/kiwigrd/k8s-sidecar:2.5.0
    - name: inbound-agent
      image: jenkins/inbound-agent:3355.v388858a_47b_33-7
  artifacthub.io/license: Apache-2.0
  artifacthub.io/links: |
    - name: Chart Source
      url: https://github.com/jenkinsci/helm-charts/tree/main/charts/jenkins
    - name: Jenkins
      url: https://www.jenkins.io/
```

```

- name: support
  url: https://github.com/jenkinsci/helm-charts/issues
apiVersion: v2
appVersion: 2.541.1
description: 'Jenkins - Build great things at any scale! As the leading open source
  automation server, Jenkins provides over 2000 plugins to support building, deploying
  and automating any project. '
home: https://www.jenkins.io/
icon: https://get.jenkins.io/art/jenkins-logo/logo.svg
keywords:
- jenkins
- ci
- devops
maintainers:
- email: maor.friedman@redhat.com
  name: maorfr
- email: mail@torstenwalter.de
  name: torstenwalter
- email: garriDOMOTA@gmail.com
  name: mogaal
- email: wmcDONA89@gmail.com
  name: wmcDONA89
- email: timJACOBL@gmail.com
  name: timja
name: jenkins
sources:
- https://github.com/jenkinsci/jenkins
- https://github.com/jenkinsci/docker-agent
- https://github.com/maorfr/kube-tasks
- https://github.com/jenkinsci/configuration-as-code-plugin
type: application
version: 5.8.134

```

The **chart.yaml** file contains metadata describing the chart. **jenkins/jenkins** refers to the **jenkins** chart in the **jenkins** repository.

Step 2 ▾

Display the **values.yaml** file for the latest stable version of the **jenkins** chart in the **jenkins** repository by running the following command:

command

```
helm show values jenkins/jenkins
```

Explanation ▾

Example output:

```

tux@management:~> helm show values jenkins/jenkins
# Default values for jenkins.
# This is a YAML-formatted file.
# Declare name/value pairs to be passed into your templates.
# name: value

## Overrides for generated resource names
# See templates/_helpers.tpl
# -- Override the resource name prefix

```

```

# @default -- `Chart.Name`
nameOverride:
# -- Override the full resource names
# @default -- `jenkins-(release-name)` or `jenkins` if the release-name is `jenkins`
fullnameOverride:
# -- Override the deployment namespace
# @default -- `Release.Namespace`
namespaceOverride:

# For FQDN resolving of the controller service. Change this value to match your
existing configuration.
# ref: https://github.com/kubernetes/dns/blob/master/docs/specification.md
# -- Override the cluster name for FQDN resolving
clusterZone: "cluster.local"
...
...
...
serviceAccountAgent:
# -- Configures if an agent ServiceAccount should be created
create: false

# If not set and create is true, a name is generated using the fullname template
# -- The name of the agent ServiceAccount to be used by access-controlled resources
name:
# -- Configures annotations for the agent ServiceAccount
annotations: {}
# -- Configures extra labels for the agent ServiceAccount
extraLabels: {}
# -- Agent ServiceAccount image pull secret
imagePullSecretName:
# -- Auto-mount ServiceAccount token
automountServiceAccountToken: true

# -- Checks if any deprecated values are used
checkDeprecation: true

awsSecurityGroupPolicies:
enabled: false
policies:
- name: ""
  securityGroupIds: []
  podSelector: {}

# Here you can configure unit tests values when executing the helm unittest in the
CONTRIBUTING.md
helmtest:
# A testing framework for bash
bats:
# Bash Automated Testing System (BATS)
image:
# -- Registry of the image used to test the framework
registry: "docker.io"
# -- Repository of the image used to test the framework
repository: "bats/bats"
# -- Tag of the image to test the framework
tag: "1.13.0"

```

The **values.yaml** file contains default values used when deploying the application in the Helm chart. Kubernetes administrators can overwrite these values during the installation using **-f <file-name>** or **--set key=value**.

Step 3 ▾

Display the **README.md** file for the latest stable version of the **jenkins** chart in the **jenkins** repository by running the following command:

command

```
helm show readme jenkins/jenkins
```

Explanation ▾**Example output:**

```
$ helm show values jenkins/jenkins
...

For a summary of all configurable options, see
[VALUES.md] (https://github.com/jenkinsci/helm-charts/blob/main/charts/jenkins/VALUES.md
).

### Configure Security Realm and Authorization Strategy

This chart configured a `securityRealm` and `authorizationStrategy` as shown below:

```yaml
controller:
 JCasC:
 securityRealm: |-
 local:
 allowsSignup: false
 enableCaptcha: false
 users:
 - id: "${chart-admin-username}"
 name: "Jenkins Admin"
 password: "${chart-admin-password}"
 authorizationStrategy: |-
 loggedInUsersCanDoAnything:
 allowAnonymousRead: false
...

...

...

Migration Guide

From stable repository

Upgrade an existing release from `stable/jenkins` to `jenkins/jenkins` seamlessly by
ensuring you have the latest [repository info](#get-repository-info) and running the
[upgrade commands](#upgrade-chart) specifying the `jenkins/jenkins` chart.

Major Version Upgrades

Chart release versions follow [SemVer](../CONTRIBUTING.md#versioning), where a MAJOR
version change (example `1.0.0` → `2.0.0`) indicates an incompatible breaking change
needing manual actions.

See [UPGRADING.md](../UPGRADING.md) for a list of breaking changes
```

The **REAMD.md** file contains the Markdown content that ArtifactHub displays on the chart website.

**Step 4** ▾

Display all together all the information in the previous steps by running the following command:

**command**

```
helm show all jenkins/jenkins
```

**Explanation** ▾

The **all** option in the **helm show** command adds to the output the **chart.yaml**, **values.yaml**, and **README.md** files in the selected chart.

**Note** ▾

The information retrieved from the chart using the helm search command is exactly the same as the information displayed in ArtifactHub in web format.

---

**Exercise Summary**

In this exercise, you browsed ArtifactHub to find a Helm repository. Later, you added this repository to your local Helm installation, searched for charts within this repository and across ArtifactHub, and used the **helm show** command to retrieve valuable information from a chart.

(End of Exercise)

## Exercise 15.2 - Deploy an application using Helm

---

### Description

Once local Helm installations include Helm repositories and charts, Kubernetes administrators can deploy applications packaged in those charts with the ***helm install*** command. In this exercise, you will create a Helm release by installing a Helm chart and learn how to retrieve the list of deployed releases in your cluster.

### Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Helm release.
  - Retrieve information about deployed releases.
-

## Task 1: Deploy Jenkins using Helm

### Step 1 ▾

Display the latest versions for the charts in the local repositories that match a search using *jenkins* as the keyword:

**command**

```
helm search repo jenkins -l | head -n 20
```

### Explanation ▾

**Example output:**

```
tux@management:~> helm search repo jenkins -l | head -n 20
NAME CHART VERSION APP VERSION DESCRIPTION
jenkins/jenkins5.9.22 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.21 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.20 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.19 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.18 2.555.1 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.17 2.555.1 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.16 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.15 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.14 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.13 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.12 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.11 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.10 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.9 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.8 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.7 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.6 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.5 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.4 2.541.2 Jenkins - Build great things at any scale!
...
```

The previous command filtered the *helm search repo jenkins -l*, which displays all versions of the *jenkins* chart, and selected the first 20 lines of the output (19 versions) using the *head -n 20* command.

### Note ▾

This lab exercise will install version 5.9.16, which corresponds to Jenkins 2.541.3. Later, the application will be upgraded to version 5.9.17.

### Warning ▾

Helm charts for older versions might fail. If Jenkins does not start properly try using a newer version according to the output displayed in the previous command.

**Step 2** ▾

Use the editor and create a ***my-values.yaml*** with the content below:

**my-values.yaml**

```
controller:
 ingress:
 enabled: true
 path: /
 hostName: "server.example.com"
persistence:
 enabled: false
```

**Explanation** ▾

These properties in the ***my-values.yaml*** file:

- Enable Ingress to expose the application to the external world, and set the desired host and path values.
- Disable persistence for Jenkins, since this is only a Lab. For real environments, the application requires a PersistentVolumeClaim and a StorageClass.

Review the ***values.yaml*** file of the chart for additional information about all the chart default values and parameters.

**Step 3** ▾

Install the Helm chart by running the following command:

**command**

```
helm install jenkins-release jenkins/jenkins --version=5.9.10 \
-f my-values.yaml --namespace jenkins --create-namespace
```

**Explanation** ▾

**Example output:**

```
tux@management:~> helm install jenkins-release jenkins/jenkins --version=5.9.16 -f
my-values.yaml --namespace jenkins --create-namespace
NAME: jenkins-release
LAST DEPLOYED: Thu Jun 4 17:53:58 2026
NAMESPACE: jenkins
STATUS: deployed
REVISION: 1
NOTES:
1. Get your 'admin' user password by running:
 kubectl exec --namespace jenkins -it svc/jenkins-release -c jenkins -- /bin/cat
 /run/secrets/additional/chart-admin-password && echo
2. Visit http://server.example.com

3. Login with the password from step 1 and the username: admin
4. Configure security realm and authorization strategy
5. Use Jenkins Configuration as Code by specifying configScripts in your values.yaml
file, see documentation: http://server.example.com/configuration-as-code and examples:
```

<https://github.com/jenkinsci/configuration-as-code-plugin/tree/master/demos>

For more information on running Jenkins on Kubernetes, visit:  
<https://cloud.google.com/solutions/jenkins-on-container-engine>

For more information about Jenkins Configuration as Code, visit:  
<https://jenkins.io/projects/jcasc/>

NOTE: Consider using a custom image with pre-installed plugins

```
#####
WARNING: Persistence is disabled!!! You will lose your data when
the Jenkins pod is terminated.
#####
```

In the **helm install** command:

- **jenkins-release** : It's the name of the release.
- **jenkins/jenkins**: Selects the **jenkins** chart in the **jenkins** repository as the one to be deployed in the cluster.
- **--version=5.8.124**: Selects the chart version.
- **-f my-values.yaml**: Value files to overwrite the chart's default values.
- **--namespace jenkins --create-namespace**: Set the namespace for the installation, and create the namespace if it does not previously exist.

#### Note ▾

Note that the output displays a command to retrieve the randomly generated password for the admin user. Copy this command for later use.

#### Step 4 ▾

Retrieve the admin password by running the following command. Copy the result for later use:

#### command

```
kubectl exec --namespace jenkins -it svc/jenkins-release -c jenkins \
-- /bin/cat /run/secrets/additional/chart-admin-password && echo
```

#### Explanation ▾

#### Example output:

```
tux@management:~> kubectl exec --namespace jenkins -it svc/jenkins-release -c jenkins
-- /bin/cat /run/secrets/additional/chart-admin-password && echo
cuUbdgLC4DdQnh88sNHU8j
```

#### Warning ▾

The password is randomly generated; values differ for each installation process.

**Step 5**

Display the list of running releases in the *jenkins* namespace using the following command:

**command**

```
helm list -n jenkins
```

**Explanation****Example output:**

```
tux@management:~> helm list -n jenkins
NAME NAMESPACE REVISION UPDATED
STATUS CHART APP VERSION
jenkins-releasejenkins 1 2026-06-04 17:53:58.282511525 +0200 CEST
deployed jenkins-5.9.16 2.541.3
```

The new release appears on the list.

**Step 6**

After a few seconds, retrieve the list of Kubernetes objects in the *jenkins* namespace by running the following commands:

**command**

```
> kubectl get all -n jenkins
> kubectl get ingress -n jenkins
```

**Explanation****Example output:**

```
tux@management:~> kubectl get all -n jenkins
NAME READY STATUS RESTARTS AGE
pod/jenkins-release-0 2/2 Running 0 7m37s

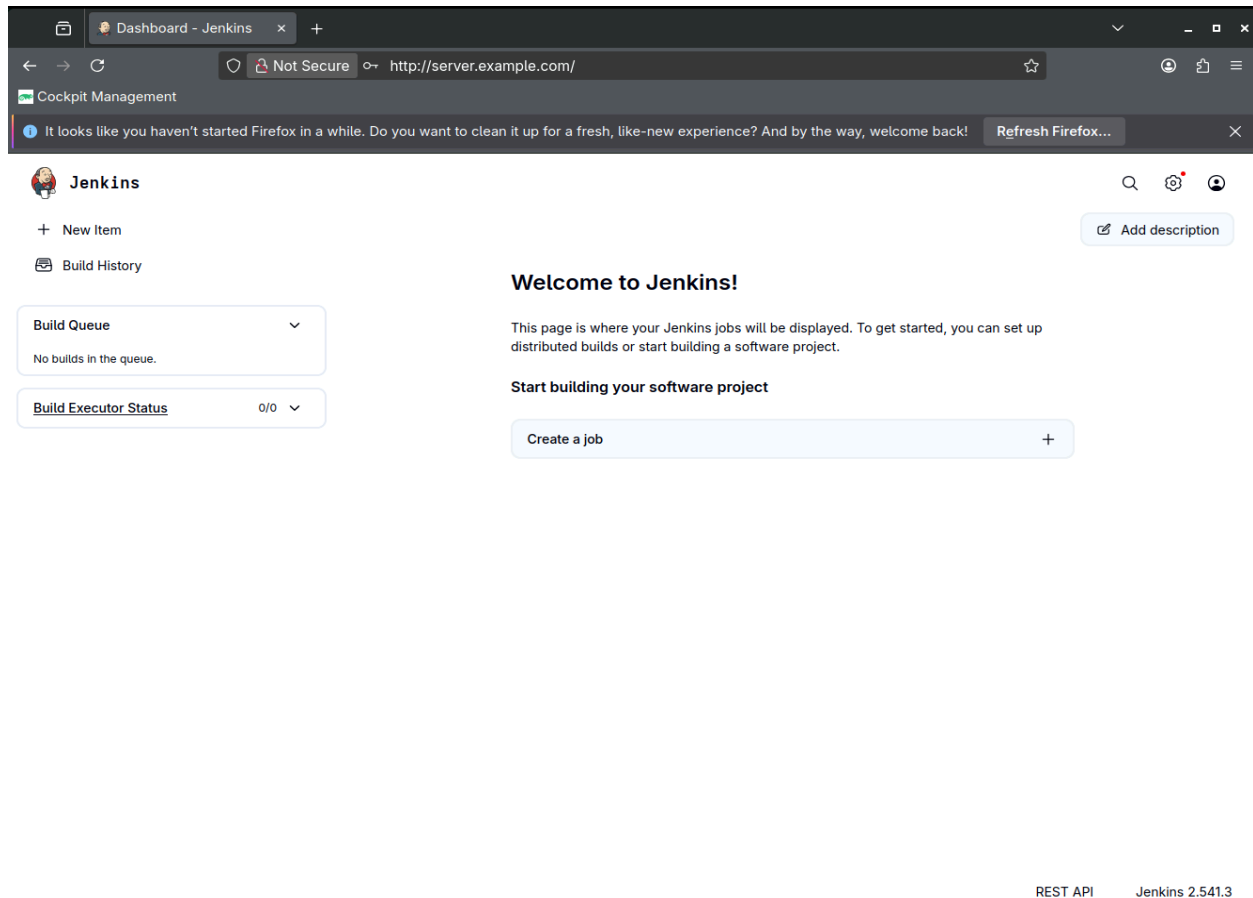
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
AGE
service/jenkins-release ClusterIP 10.43.104.115 <none> 8080/TCP
7m37s
service/jenkins-release-agent ClusterIP 10.43.29.216 <none> 50000/TCP
7m37s

NAME READY AGE
statefulset.apps/jenkins-release 1/1 7m37s
tux@management:~> kubectl get ingress -n jenkins
NAME CLASS HOSTS ADDRESS PORTS AGE
jenkins-release traefik server.example.com 80 7m42s
```

Installing a Helm chart creates all the Kubernetes objects needed for the application to work.

**Step 7**

Open a Firefox browser in the management VM, navigate to the ***http://server.example.com*** URL, and use the admin credentials to log into Jenkins. Once logged in, the browser must display something similar to the image below:

**Note**

Note that the Jenkins version in the bottom-right corner, 2.541.3, corresponds to the installed chart version 5.9.16.

**Exercise Summary**

In this exercise, you installed a chart version using custom values that override default values in the chart's values.yaml file. Later, you confirmed that the Helm release was deployed from the command line and that the application is accessible via a web browser on the management VM.

(End of Exercise)



## Exercise 15.3 - Manage a Helm release lifecycle: upgrades and rollback

---

### Description

Helm provides tools for upgrading applications and rolling back if a problem occurs during the upgrade. In this exercise, you will upgrade the Jenkins application installed in the previous exercise, roll back the upgrade, and use Helm tools to review the release history.

### Learning Outcomes

Upon completing this exercise, you will be able to:

- Upgrade a Helm release.
  - Rollback a Helm release to a previous version.
  - Review a Helm release history.
-

## Task 1: Upgrade a Helm chart

### Step 1

Display the versions for the charts in the local repositories that match a search using *jenkins* as the keyword:

**command**

```
helm search repo jenkins -l | head -n 20
```

### Explanation

**Example output:**

```
tux@management:~> helm search repo jenkins -l | head -n 20
NAME CHART VERSION APP VERSION DESCRIPTION
jenkins/jenkins5.9.22 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.21 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.20 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.19 2.555.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.18 2.555.1 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.17 2.555.1 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.16 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.15 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.14 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.13 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.12 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.11 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.10 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.9 2.541.3 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.8 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.7 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.6 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.5 2.541.2 Jenkins - Build great things at any scale!
jenkins/jenkins5.9.4 2.541.2 Jenkins - Build great things at any scale!
...
```

Version 2.541.3 is the version used in the previous exercise. In this version, you will upgrade the deployed application to version, which corresponds to Jenkins 2.541.1.

### Step 2

Upgrade the chart to the new version by running the following command:

**command**

```
helm upgrade jenkins-release jenkins/jenkins --version=5.9.17 \
-f my-values.yaml --namespace jenkins
```

### Explanation

**Example output:**

```
tux@management:~> helm upgrade jenkins-release jenkins/jenkins --version=5.9.17 \
> -f my-values.yaml --namespace jenkins
Release "jenkins-release" has been upgraded. Happy Helming!
NAME: jenkins-release
```

```

LAST DEPLOYED: Thu Jun 4 18:01:12 2026
NAMESPACE: jenkins
STATUS: deployed
REVISION: 2
NOTES:
1. Get your 'admin' user password by running:
 kubectl exec --namespace jenkins -it svc/jenkins-release -c jenkins -- /bin/cat
 /run/secrets/additional/chart-admin-password && echo
2. Visit http://server.example.com

3. Login with the password from step 1 and the username: admin
4. Configure security realm and authorization strategy
5. Use Jenkins Configuration as Code by specifying configScripts in your values.yaml
 file, see documentation: http://server.example.com/configuration-as-code and examples:
 https://github.com/jenkinsci/configuration-as-code-plugin/tree/master/demos

For more information on running Jenkins on Kubernetes, visit:
https://cloud.google.com/solutions/jenkins-on-container-engine

For more information about Jenkins Configuration as Code, visit:
https://jenkins.io/projects/jcasc/

NOTE: Consider using a custom image with pre-installed plugins
#####
WARNING: Persistence is disabled!!! You will lose your data when
the Jenkins pod is terminated.
#####

```

In the **helm upgrade** command:

- **jenkins-release** : It's the name of the release.
- **jenkins/jenkins**: Selects the **jenkins** chart in the **jenkins** repository as the one to be deployed in the cluster.
- **--version=5.8.125**: Selects the new chart version.
- **-f my-values.yaml**: Value files to overwrite the chart's default values.
- **--namespace jenkins**: Selects the namespace where the previous version of the release is running.

#### Note ▾

Note that the **REVISION** property value has changed to **2**.

### Step 3 ▾

Review the release history by running the following command:  
**command**

```
helm history jenkins-release -n jenkins
```

#### Explanation ▾

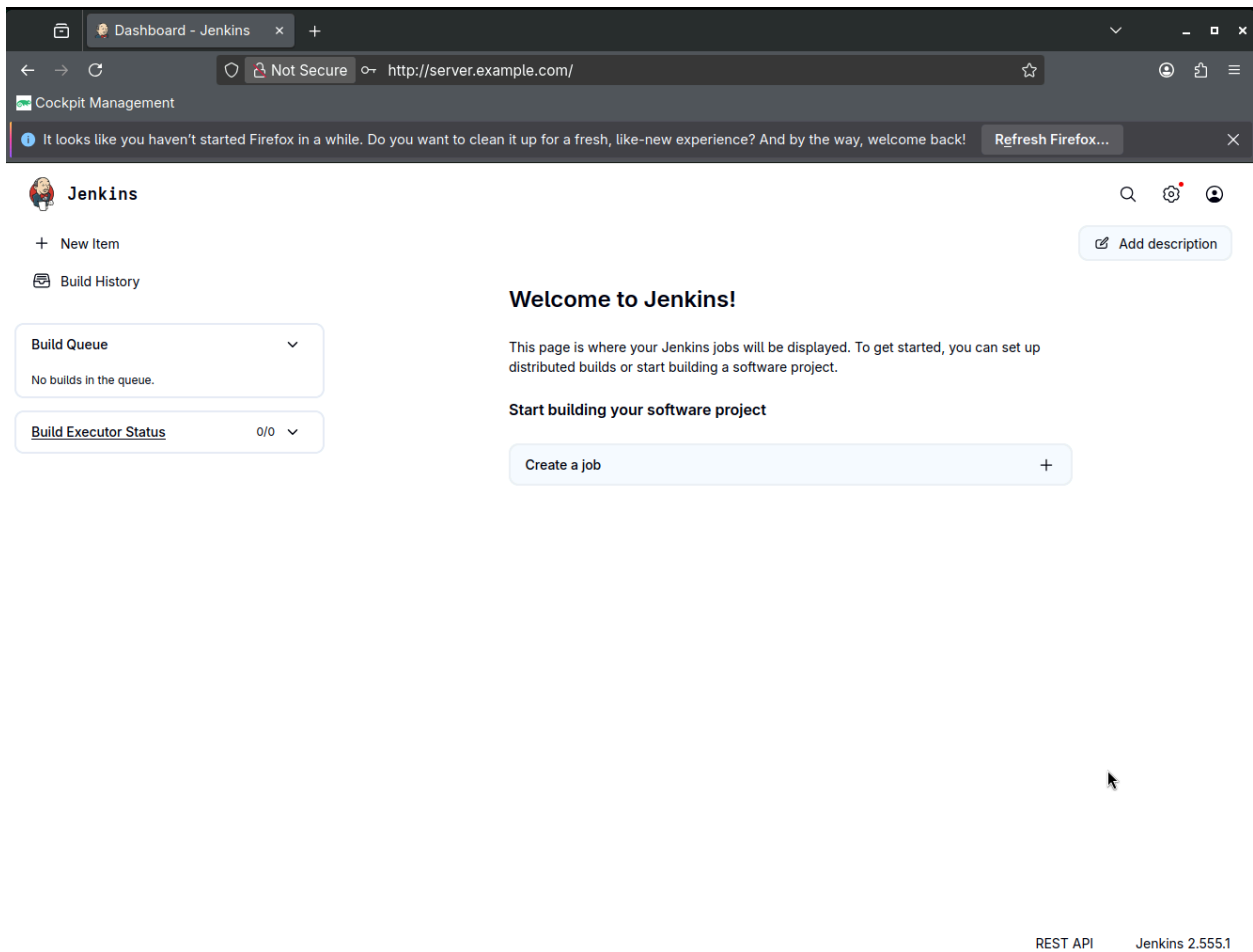
**Example output:**

```
tux@management:~> helm history -n jenkins jenkins-release
```

REVISION	UPDATED DESCRIPTION	STATUS	CHART	APP VERSION
1	Thu Jun 4 17:53:58 2026 Install complete	superseded	jenkins-5.9.16	2.541.3
2	Thu Jun 4 18:01:12 2026 Upgrade complete	deployed	jenkins-5.9.17	2.555.1

**Step 4**

Open a Firefox browser in the management VM, navigate to the ***http://server.example.com*** URL, and use the same admin credentials as before to log into Jenkins. Once logged in, the browser must display something similar to the image below:

**Explanation**

Note that the Jenkins version in the bottom-right corner, 2.555.1, corresponds to the new chart version 5.9.17.

## Task 2: Rollback the previous upgrade

### Step 1 ▾

Review the release history by running the following command:

**command**

```
helm history jenkins-release -n jenkins
```

#### Explanation ▾

**Example output:**

```
tux@management:~> helm history -n jenkins jenkins-release
REVISION UPDATED STATUS CHART APP VERSION
1 Thu Jun 4 17:53:58 2026 superseded jenkins-5.9.16 2.541.3
 Install complete
2 Thu Jun 4 18:01:12 2026 deployed jenkins-5.9.17 2.555.1
 Upgrade complete
```

### Step 2 ▾

Assuming that something went wrong, roll back the release to the previous revision by running the following command:

**command**

```
helm rollback jenkins-release 1 -n jenkins
```

#### Explanation ▾

**Example output:**

```
tux@management:~> helm rollback jenkins-release 1 -n jenkins
Rollback was a success! Happy Helming!
```

In the **helm rollback** command:

- **jenkins-release** : It's the name of the release.
- **1**: Set the target revision number for the rollback. If omitted, the target revision would be the previous one.
- **-n jenkins**: Selects the namespace where the previous version of the release is running.

### Step 3 ▾

Review the release history again by running the following command:

**command**

```
helm history jenkins-release -n jenkins
```

#### Explanation ▾

**Example output:**

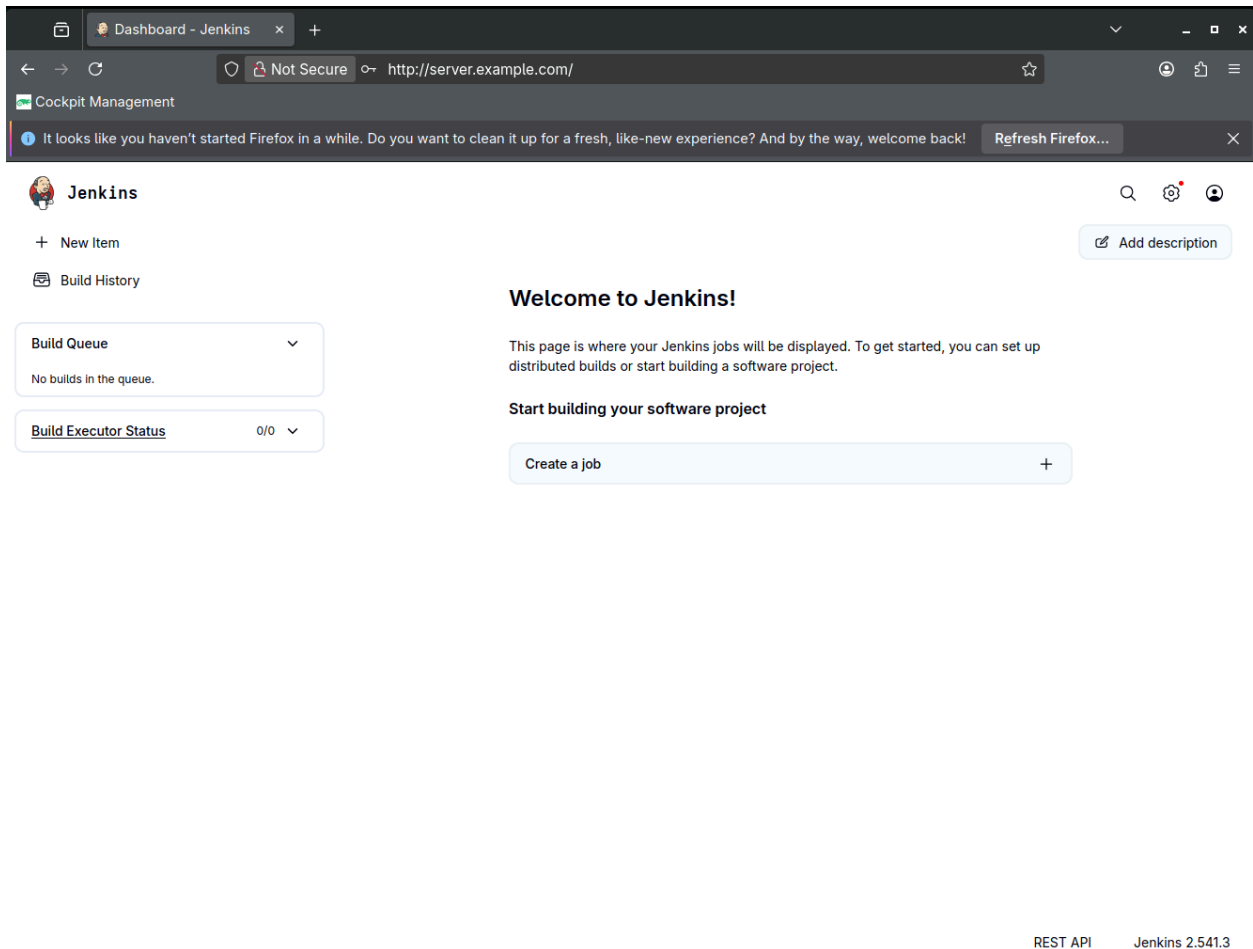
```
tux@management:~> helm history -n jenkins jenkins-release
```

REVISION	UPDATED	STATUS	CHART	APP VERSION
1	Thu Jun 4 17:53:58 2026 Install complete	superseded	jenkins-5.9.16	2.541.3
2	Thu Jun 4 18:01:12 2026 Upgrade complete	superseded	jenkins-5.9.17	2.555.1
3	Thu Jun 4 18:05:51 2026 Rollback to 1	deployed	jenkins-5.9.16	2.541.3

Note that the latest entry in the history shows the rollback pointing to revision 1.

**Step 4**

Open a Firefox browser in the management VM, navigate to the ***http://server.example.com*** URL, and use the same admin credentials as before to log into Jenkins. Once logged in, the browser must display something similar to the image below:

**Explanation**

Note that the Jenkins version in the bottom-right corner is back to 2.541.3, indicating that the rollback completed successfully.

### Task 3: Uninstall the Helm chart

#### Step 1 ▾

Uninstall the Helm chart and remove the release by running the following command:

**command**

```
helm uninstall jenkins-release -n jenkins
```

#### Step 2 ▾

Review the running releases in the *jenkins* namespace by running the following command:

**command**

```
helm list -n jenkins
```

**Explanation ▾**

**Example output:**

```
tux@management:~> helm list -n jenkins
NAME NAMESPACE REVISION UPDATED STATUSCHART APP VERSION
```

---

### Exercise Summary

In this exercise, you installed Jenkins in your Kubernetes cluster using a Helm chart and custom values, and you verified that the application was working properly. Later, you upgraded the chart to a newer version, tested that everything was still working as expected, and rolled back Jenkins to the first version. Finally, you uninstalled the Helm chart.

(End of Exercise)

## Section 16: Extending Kubernetes with Operators and CRDs

### Description

Add the overall section description here. Should be several sentences long and may be multiple paragraphs. It should include an overview of all of the exercises included and what the student will learn.

### Section Learning Objectives

- Understand Kubernetes limitations when modelling domain-specific application concepts.
- Differentiate between built-in Kubernetes resources and custom resources.
- Understand the role of controllers and operators.
- Compare Helm and Operators capabilities.

### Section Topics

- The Need for Extensibility
- Extending the Kubernetes API (CRDs)
- Kubernetes Controllers and Operators



## Exercise 16.1 - Create a Customer Resource Definition (CRD)

---

### Description

Built-in Kubernetes objects cannot model application-domain-specific concepts, such as certificate expiration dates, primary nodes for replicated databases, or external infrastructure elements. However, the Kubernetes API can be extended by using Custom Resource Definitions (CRDs).

In this exercise, you will create a CRD for a static website, instantiate it, and verify that CRDs do not introduce inherent behavior, meaning no functional changes occur in the cluster upon instantiation alone.

### Learning Outcomes

Upon completing this exercise, you will be able to:

- Create a Custom Resource Definition (CRD).
  - Retrieve information about the CRDs in our cluster.
  - Instantiate the CRD and create a new custom resource..
  - Understand that CRDs don't provide any behavior.
-

**Task 1: Create a Custom Resource Definition (CRD) for a web server serving a static website****Step 1** ▾

Verify that all the Lab VMs, **RKE201-management**, **RKE201-server**, and **RKE201-agent** are up and running.

**Step 2** ▾

Select the **RKE201-management** machine from the VM list in the Virtual Machine Manager.

When the graphical login screen appears, use the following credentials to log in to the VM:

**Username:** tux

**Password:** linux

**Warning** ▾

Remember that the **RKE201-management** VM runs services for the rest of the VMs in the Lab, so it must remain running throughout the Lab exercises. This VM is also used as the entry point to the rest of the VMs via SSH connections.

**Step 3** ▾

Open a terminal in the management VM and retrieve the current Custom Resource Definition list by running the following command:

**command**

```
kubectl get crds
```

**Explanation** ▾

**Example output:**

```
tux@management:~> kubectl get crds
NAME CREATED AT
accesscontrolpolicies.hub.traefik.io 2026-01-19T14:56:00Z
addons.k3s.cattle.io 2026-01-19T14:55:22Z
adminnetworkpolicies.policy.networking.k8s.io 2026-01-19T14:55:35Z
aiservices.hub.traefik.io 2026-01-19T14:56:00Z
apiauths.hub.traefik.io 2026-01-19T14:56:00Z
apibundles.hub.traefik.io 2026-01-19T14:56:00Z
apicatalogitems.hub.traefik.io 2026-01-19T14:56:00Z
apiplans.hub.traefik.io 2026-01-19T14:56:00Z
apiportalauths.hub.traefik.io 2026-01-19T14:56:00Z
apiportals.hub.traefik.io 2026-01-19T14:56:00Z
apiratelimits.hub.traefik.io 2026-01-19T14:56:00Z
apis.hub.traefik.io 2026-01-19T14:56:00Z
apiversions.hub.traefik.io 2026-01-19T14:56:00Z
backendtlspolicies.gateway.networking.k8s.io 2026-01-19T14:56:00Z
baselineadminnetworkpolicies.policy.networking.k8s.io 2026-01-19T14:55:35Z
bgpconfigurations.crd.projectcalico.org 2026-01-19T14:55:35Z
```

bgpfilters.crd.projectcalico.org	2026-01-19T14:55:35Z
bgppeers.crd.projectcalico.org	2026-01-19T14:55:35Z
blockaffinities.crd.projectcalico.org	2026-01-19T14:55:35Z
caliconodestatuses.crd.projectcalico.org	2026-01-19T14:55:35Z
clusterinformations.crd.projectcalico.org	2026-01-19T14:55:35Z
etcdsnapshotfiles.k3s.cattle.io	2026-01-19T14:55:22Z
felixconfigurations.crd.projectcalico.org	2026-01-19T14:55:35Z
gatewayclasses.gateway.networking.k8s.io	2026-01-19T14:56:00Z
gateways.gateway.networking.k8s.io	2026-01-19T14:56:00Z
globalnetworkpolicies.crd.projectcalico.org	2026-01-19T14:55:35Z
globalnetworksets.crd.projectcalico.org	2026-01-19T14:55:35Z
grpcroutes.gateway.networking.k8s.io	2026-01-19T14:56:00Z
helmchartconfigs.helm.cattle.io	2026-01-19T14:55:22Z
helmcharts.helm.cattle.io	2026-01-19T14:55:22Z
hostendpoints.crd.projectcalico.org	2026-01-19T14:55:35Z
httproutes.gateway.networking.k8s.io	2026-01-19T14:56:00Z
ingressroutes.traefik.io	2026-01-19T14:56:00Z
ingressroutetcps.traefik.io	2026-01-19T14:56:00Z
ingressrouteudps.traefik.io	2026-01-19T14:56:00Z
ipamblocks.crd.projectcalico.org	2026-01-19T14:55:35Z
ipamconfigs.crd.projectcalico.org	2026-01-19T14:55:35Z
ipamhandles.crd.projectcalico.org	2026-01-19T14:55:35Z
ippools.crd.projectcalico.org	2026-01-19T14:55:35Z
ipreservations.crd.projectcalico.org	2026-01-19T14:55:35Z
kubecontrollersconfigurations.crd.projectcalico.org	2026-01-19T14:55:35Z
managedapplications.hub.traefik.io	2026-01-19T14:56:00Z
managedsubscriptions.hub.traefik.io	2026-01-19T14:56:00Z
middlewares.traefik.io	2026-01-19T14:56:00Z
middlewaretcps.traefik.io	2026-01-19T14:56:00Z
networkpolicies.crd.projectcalico.org	2026-01-19T14:55:35Z
networksets.crd.projectcalico.org	2026-01-19T14:55:35Z
referencegrants.gateway.networking.k8s.io	2026-01-19T14:56:00Z
serverstransports.traefik.io	2026-01-19T14:56:00Z
serverstransporttcps.traefik.io	2026-01-19T14:56:00Z
stagedglobalnetworkpolicies.crd.projectcalico.org	2026-01-19T14:55:35Z
stagedkubernetesnetworkpolicies.crd.projectcalico.org	2026-01-19T14:55:35Z
stagednetworkpolicies.crd.projectcalico.org	2026-01-19T14:55:35Z
tiers.crd.projectcalico.org	2026-01-19T14:55:35Z
tlsoptions.traefik.io	2026-01-19T14:56:00Z
tlsstores.traefik.io	2026-01-19T14:56:00Z
traefikservices.traefik.io	2026-01-19T14:56:00Z
volumegroupsnapshotclasses.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumegroupsnapshotcontents.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumegroupsnapshots.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshotclasses.snapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshotcontents.snapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshots.snapshot.storage.k8s.io	2026-01-19T14:55:59Z

The RKE2 installation process with the configuration provided for the Labs created several CRDs for domains such as networking and the installation process itself. Note all the CRDs related to **Traefik** and **Calico**, and all the **Helm** CRDs related to the RKE2 installation process itself.

#### Note ▾

**crds** is the shortname for customresourcedefinitions. Note that CRDs are cluster objects.

**Step 4** ▾

Review the YAML manifest for a Custom Resource Definition by running the following commands:

**commands**

```
> cd ~/course_files/RKE201v2/labs/manifests/crds/
> cat static-website-crd.yaml
```

**Explanation** ▾**Example output:**

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
 name: staticwebs.mycompany.com
spec:
 group: mycompany.com
 scope: Namespaced
 names:
 plural: staticwebs
 singular: staticweb
 kind: StaticWeb
 shortNames: ["site"]
 versions:
 - name: v1
 served: true
 storage: true
 schema:
 openAPIV3Schema:
 type: object
 required: ["spec"]
 properties:
 spec:
 type: object
 required: ["source-repository", "domain"]
 properties:
 source-repository:
 type: string
 pattern: '^git@github\.com:.*\.git$'
 domain:
 type: string
 enableSSL:
 type: boolean
 default: true
```

In the YAML manifest:

- The CRD is **Namespaced** and belongs to the **mycompany.com** API.

- Names are **StaticWeb** for kind, **staticweb** and **staticweb** as singular and plural, and **site** as the shortname.
- Only one version is provided, and for this section, which is served via API, the properties:
  - source-repository:** String with a pattern to match Git URLs for repositories.
  - domain:** String to contain the domain name for the website.
  - enableSSL:** Boolean to set whether the server uses HTTPS.

**Step 5**

Register the CRD in the Kubernetes cluster by running the following command:

**command**

```
kubectl apply -f static-website-crd.yaml
```

**Step 6**

Retrieve the Custom Resource Definition list again by running the following command:

**command**

```
kubectl get crds
```

**Explanation**

**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get crds
NAME CREATED AT
accesscontrolpolicies.hub.traefik.io 2026-01-19T14:56:00Z
addons.k3s.cattle.io 2026-01-19T14:55:22Z
adminnetworkpolicies.policy.networking.k8s.io 2026-01-19T14:55:35Z
aiservices.hub.traefik.io 2026-01-19T14:56:00Z
apiauths.hub.traefik.io 2026-01-19T14:56:00Z
...
...
stagedkubernetesnetworkpolicies.crd.projectcalico.org 2026-01-19T14:55:35Z
stagednetworkpolicies.crd.projectcalico.org 2026-01-19T14:55:35Z
staticwebs.mycompany.com 2026-02-06T15:15:12Z
tiers.crd.projectcalico.org 2026-01-19T14:55:35Z
...
...
volumegroupsnapshotcontents.groupsnapshot.storage.k8s.io 2026-01-19T14:55:59Z
volumegroupsnapshots.groupsnapshot.storage.k8s.io 2026-01-19T14:55:59Z
volumesnapshotclasses.snapshot.storage.k8s.io 2026-01-19T14:55:59Z
volumesnapshotcontents.snapshot.storage.k8s.io 2026-01-19T14:55:59Z
volumesnapshots.snapshot.storage.k8s.io 2026-01-19T14:55:59Z
```

Note that **staticwebs.mycompany.com** appears in the list.. Since the CRD is already registered, users can start creating custom resources using it.

**Step 7**

Review the YAML manifest for a StaticWeb resource that uses the previous CRD by running the following commands:

#### commands

```
> cd ~/course_files/RKE201v2/labs/manifests/crds/
> cat landing-staticweb.yaml
```

#### Explanation ▾

#### Example output:

```
apiVersion: mycompany.com/v1
kind: StaticWeb
metadata:
 name: landing-page
 namespace: landing
spec:
 source-repository: "git@github.com:mycompnay/landing-page.git"
 domain: "www.mycompany.com"
 enableSSL: true
```

This YAML:

- Since the value for the **kind** property is **StaticWeb**, the YAML is used to instantiate the **staticwebs.mycompany.com** CRD.
- Set **landing-page** as the name for the new custom resource, and **landing** as the namespace where the **StaticWeb** object will be created.
- Define values for the three properties defined in the CRD: **source-repository**, **domain**, and **enableSSL**.

#### Step 8 ▾

Create the **landing** namespace and the StaticWeb Kubernetes object by running the following commands:

#### commands

```
> kubectl create ns landing
> kubectl apply -f landing-staticweb.yaml
```

#### Step 7 ▾

Since the custom resource defined the **landing** namespace, run the following commands to check if something happened in that namespace:

#### commands

```
> kubectl get staticwebs -n landing
> kubectl get all -n landing
> kubectl get secrets -n landing
> kubectl get ingress -n landing
```

```
> kubectl get httproutes -n landing
```

### Explanation ▾

#### Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get staticwebs -n
landing
NAME AGE
landing-page 3m27s
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get all -n landing
No resources found in landing namespace.
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get secrets -n
landing
No resources found in landing namespace.
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get ingress -n
landing
No resources found in landing namespace.
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get httproutes -n
landing
No resources found in landing namespace.
```

No Pods, no Deployments, no Service, no Secrets, no resources exist in the **landing** namespace other than the **landing-page** StaticWeb resource.

Since the CRD defines a web server serving a static website, we could expect some Kubernetes objects in the namespace, objects such as:

- A Deployment running nginx.
- A PVC with the git repository content mounted in the Deployment's Pods.
- A ClusterIP Service exposing the Deployment.
- A TLS secret.
- An Ingress resource with TLS and with the host property set to **"www.mycompany.com"**

However, as the CRD has no Operator, there is no associated behavior.

### Step 7 ▾

Clean everything to return the cluster to its previous state by running the following commands:

#### commands

```
> kubectl delete staticwebs -n landing landing-page
> kubectl delete namespace landing
```

**Exercise Summary**

In this exercise, you have defined a CRD for a web server serving a static website, reviewed the YAML manifest for the CRD, and created a custom Kubernetes object that used the CRD. Later, you verified that the CRD by itself does not provide behavior.

(End of Exercise)



## Exercise 16.2 - Install and use a pre-built operator from operatorhub.io)

---

### Description

Companies can create their own operators from scratch, and developers can use [operatorhub.io](https://operatorhub.io) as the central point to search for companies' operators or community-based operators.

In this exercise, you will install the KEDA operator from [operatorhub.io](https://operatorhub.io) using an Operator Lifecycle Manager (OLM). KEDA (Kubernetes Event-Driven Autoscaling) is a lightweight component that provides event-driven autoscaling for your workloads. Events that trigger KEDA scaling can be time-based or originate from external systems (AWS SQS, Redis, Prometheus, Kafka, PostgreSQL, etc.). Kubernetes cannot autoscale natively for those types of events. The Horizontal Pod Autoscaler (HPA), presented in the course, can only scale based on CPU and memory. This exercise uses the KEDA operator to autoscale a Deployment based on the number of elements in a Redis list.

### Learning Outcomes

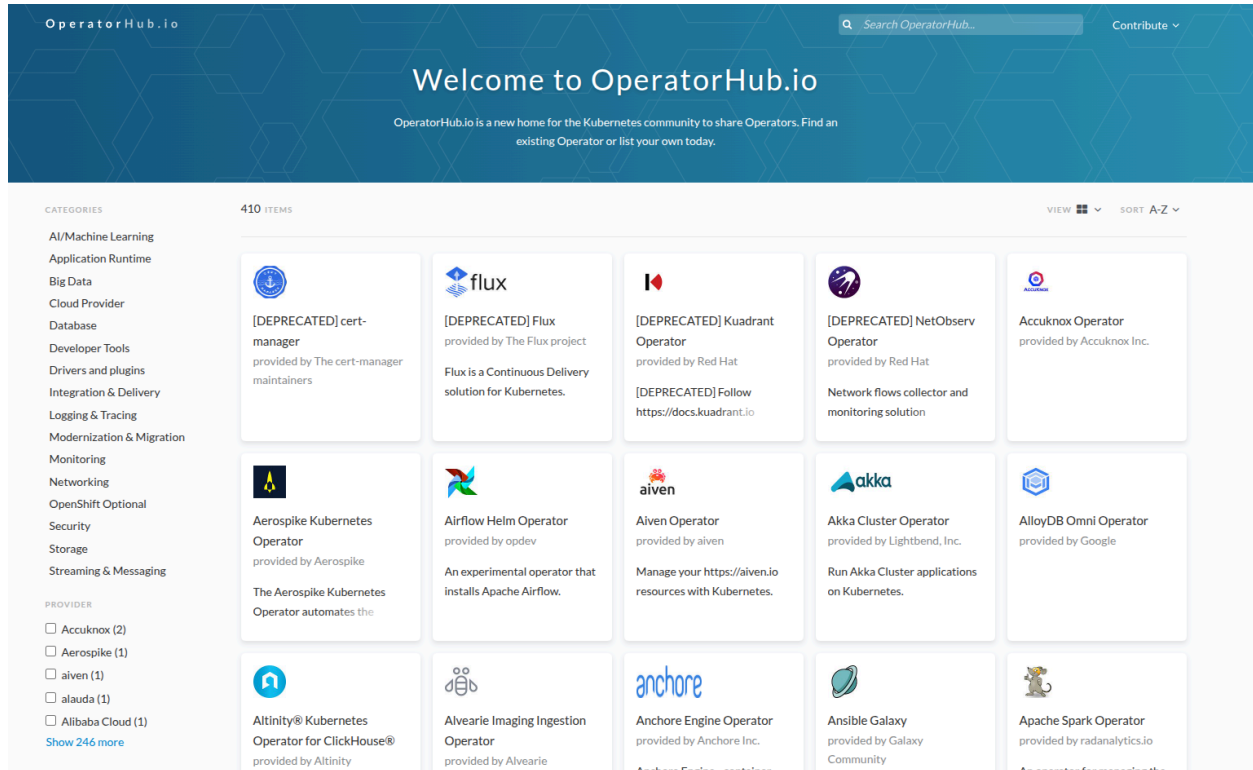
Upon completing this exercise, you will be able to:

- Find an operator on [operatorhub.io](https://operatorhub.io) and familiarize yourself with the website's structure and content.
  - Install an operator from [artifacthub.io](https://artifacthub.io) using an OLM.
  - Understand all the pieces associated with the operator
-

## Task 1: Install an operator from operatorHub ([operatorhub.io](https://operatorhub.io)) in the RKE2 cluster.

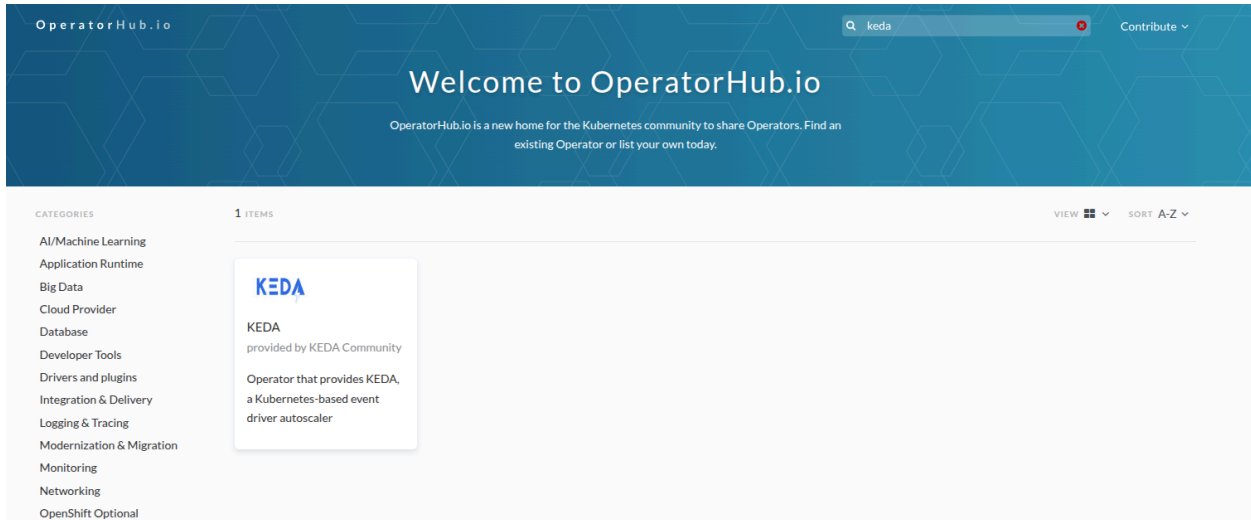
### Step 1 ▾

Open the Firefox browser in your management VM and navigate to ArtifactHub using the following URL: <http://operatorhub.io>



### Step 2 ▾

Enter **KEDA** in the search box at the top to find a KEDA operator. KEDA is a Kubernetes event-based autoscaler.

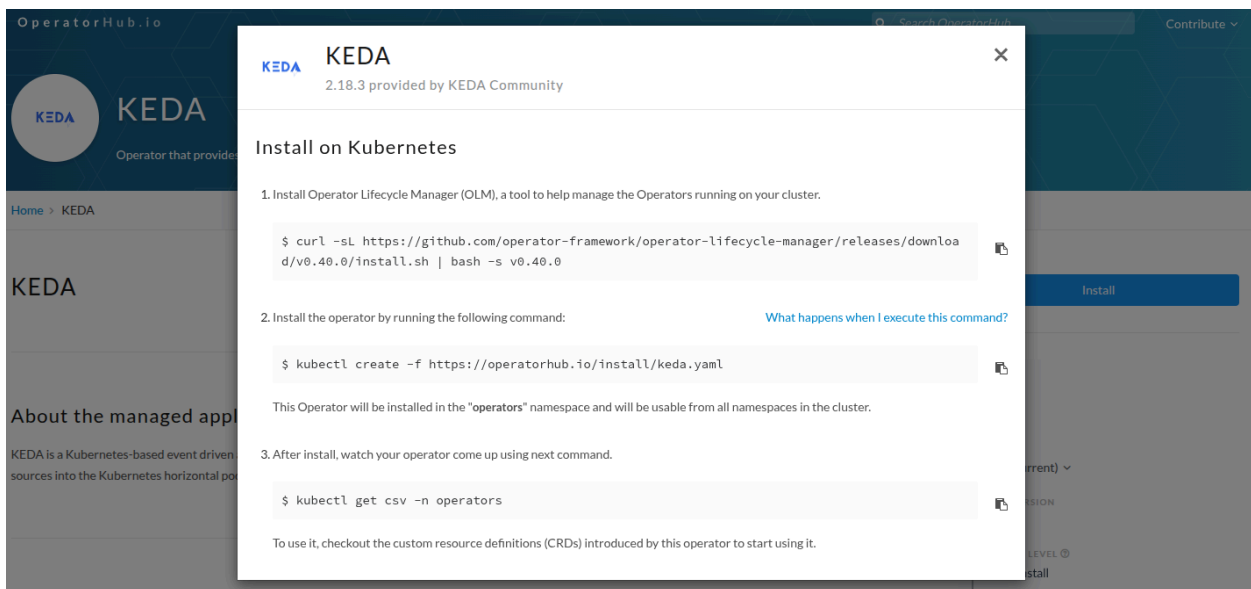


### Step 3 ▾

Select the **KEDA** operator *provided by KEDA community*

### Step 4 ▾

Browse the operator page and review all the information. When finished, select **Install** to display the installation instructions for the KEDA operator.



### Explanation ▾

The dialog window displays the steps for installing the Operator.

1. Install the Operator Lifecycle Manager (OLM). This operator will act as a meta-operator, an operator for operators. As part of the process, the script installs

the OLM operator itself, the Catalog operator, which acts as the operator librarian, and the Package server, which lists all the operators available for installation.

2. Install the KEDA Operator by running the **kubectl create** command from a remote URL.
3. Finally, check whether the operator has been created successfully.

#### Step 5 ▾

Run the commands in the instructions (the first one could take a while):

##### commands

```
> curl -sL
https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.40.0/install.sh | bash -s v0.40.0
> kubectl create -f https://operatorhub.io/install/keda.yaml
> kubectl get csv -n operators
```

#### Explanation ▾

##### Example output:

```
...
tux@management:~> kubectl get csv -n operators
NAME DISPLAY VERSION RELEASE REPLACES PHASE
keda.v2.18.3 KEDA 2.18.3 keda.v2.18.2 Succeeded
```

The output of the last command should show **Succeeded** for the KEDA operator, indicating the operator is ready to use.

**csv** is the shortname for ClusterServiceVersion. CSVs act as the manifest for an operator within the OLM. It contains all the metadata required to manage the operator's lifecycle, including its installation strategy, RBAC permissions, required CRDs, dependencies on other operators, and the container image. Once a CSV reaches the Succeeded phase, the OLM has successfully verified and deployed the operator into the cluster.

#### Step 6 ▾

Retrieve the Custom Resource Definition list again by running the following command:

##### command

```
kubectl get crds
```

#### Explanation ▾

##### Example output:

```
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get crds
NAME CREATED AT
```

accesscontrolpolicies.hub.traefik.io	2026-01-19T14:56:00Z
addons.k3s.cattle.io	2026-01-19T14:55:22Z
...	
...	
caliconodestatuses.crd.projectcalico.org	2026-01-19T14:55:35Z
catalogsources.operators.coreos.com	2026-02-06T16:23:41Z
cloudeventsources.eventing.keda.sh	2026-02-06T17:07:21Z
clustercloudeventsources.eventing.keda.sh	2026-02-06T17:07:21Z
clusterinformations.crd.projectcalico.org	2026-01-19T14:55:35Z
clusterserviceversions.operators.coreos.com	2026-02-06T16:23:48Z
clustertriggerauthentications.keda.sh	2026-02-06T17:07:21Z
etcdsnapshotfiles.k3s.cattle.io	2026-01-19T14:55:22Z
...	
...	
kedacontrollers.keda.sh	2026-02-06T17:07:22Z
...	
...	
scaledjobs.keda.sh	2026-02-06T17:07:22Z
scaledobjects.keda.sh	2026-02-06T17:07:22Z
...	
...	
triggerauthentications.keda.sh	2026-02-06T17:07:22Z
volumegroupsnapshotclasses.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumegroupsnapshotcontents.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumegroupsnapshots.groupsnapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshotclasses.snapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshotcontents.snapshot.storage.k8s.io	2026-01-19T14:55:59Z
volumesnapshots.snapshot.storage.k8s.io	2026-01-19T14:55:59Z

The KEDA operator installation created several KEDA CRDs. We will use the **scaledobjects.keda.sh** later. Review the KEDA homepage on operatorHub for examples of custom resources using this CRD, along with others.

## Step 7 ▾

Review the CRD for KEDA by running the following command:

**command**

```
kubectl get crds scaledobjects.keda.sh -n operators -o yaml
```

### Note ▾

Output might be overwhelming in length. Note the names (**ScaledObject**, **scaledobjects**, **scaledobject**, **so**) and properties such as **scaleTargetRef**, **minReplicaCount**, **maxReplicaCount**, or **triggers**.

## Task 2: Use KEDA CRD to trigger autoscaling according to a Redis list status

### Step 1 ▾

Create a Redis server and expose it as a Kubernetes service by running the following commands:

**commands**

```
> kubectl run redis --image=redis --port=6379
> kubectl expose pod redis --port=6379 --name redis-service
```

**Step 2** ▾

Review the application to be scaled with the KEDA operator by running the following commands:

**commands**

```
> cd ~/course_files/RKE201v2/labs/manifests/crds/
> cat order-processor.yaml
```

**Explanation** ▾**Example output:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: order-processor
spec:
 replicas: 0
 selector:
 matchLabels:
 app: worker
 template:
 metadata:
 labels:
 app: worker
 spec:
 containers:
 - name: processor
 image: busybox
 command: ["sleep", "3600"]
```

From the Kubernetes perspective, the YAML manifest creates a Deployment with 0 replicas initially, using a busybox container.

From a business perspective, let's assume that the **processor** container, which uses a **busybox** image, processes orders coming from the Redis queue. If there are no orders in the queue, the business earns \$0 from that process. Therefore, maintaining even a single running server is an unnecessary operational cost. Then, the Deployment must run 0 replicas.

Conversely, when, for example, Black Friday hits, many orders arrive at once at the Redis queue, and the order processor must react instantly and automatically scale to meet demand and process all orders.

**Step 3** ▾

Create the Deployment and verify that all the objects are ready for using the KEDA operator by running the following commands:

**commands**

```
> kubectl apply -f order-processor.yaml
> kubectl get all
```

**Explanation** ▾**Example output:**

```
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl apply -f
order-processor.yaml
deployment.apps/order-processor created
tux@management:~/course_files/RKE201v2/labs/manifests/crds> kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/redis	1/1	Running	0	24m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP
service/redis-service	ClusterIP	10.43.34.183	<none>	6379/TCP

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/order-processor	0/0	0	0	3s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/order-processor-86c66698f	0	0	0	3s

**Note** ▾

Note the ***order-processor*** Deployment with 0 replicas.

**Step 4** ▾

Kubernetes can not provide autoscaling capabilities according to Redis queues, but the KEDA operator does. Review the YAML manifest for KEDA's ScaledObject by running the following commands:

**commands**

```
> cd ~/course_files/RKE201v2/labs/manifests/crds/
> cat keda-processor-autoscaler.yaml
```

**Explanation** ▾**Example output:**

```

apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
 name: keda-processor-autoscaler
spec:
 scaleTargetRef:
 name: order-processor
 minReplicaCount: 0
 maxReplicaCount: 5
 pollingInterval: 5
 cooldownPeriod: 5
 triggers:
 - type: redis
 metadata:
 address: redis-service.default.svc.cluster.local:6379
 listName: order-queue
 listLength: "1"

```

The YAML manifest:

- Uses KEDA's CRD ***ScaledObject*** as the value for the ***kind*** property.
- Targets the ***order-processor*** Deployment for autoscaling using the ***scaleTargetRef*** property.
- Sets the maximum and minimum values for the replica count, as well as the 5-second cooling and polling intervals. The cooling period is used for scaling down after a change.
- Defines a ***redis*** trigger with:
  - The internal URL for the service.
  - The Redis queue to be observed: ***order-queue***
  - The scaling step length. KEDA will scale up the Deployment every time a new order arrives in the queue.

#### Additional Information ▾

Review the [KEDA documentation](#) for information on more KEDA triggers and CRDs.

#### Step 5 ▾

Create KEDA's ScaledObject by running the following command:

**command**

```
kubectl apply -f keda-processor-autoscaler.yaml
```

#### Step 6 ▾

Open a new terminal window and run the following to watch the autoscaling process by KEDA:



**command**

```
watch kubectl get all
```

**Step 6 ▾**

For the first terminal, add or remove items to the Redis queue using the following commands:

**commands**

```
#Add an item to the order-queue
> kubectl exec -it redis -- redis-cli RPush order-queue "item-name"
#Remove an item from the order-queue
> kubectl exec -it redis -- redis-cli LPop order-queue
#Empty the order-queue
kubectl exec -it redis -- redis-cli DEL order-queue
```

Since KEDA autoscaling watches Redis queue events, depending on your operations (adding or removing items), the number of replicas for the **order-processor** will change. Keep in mind that, due to the cooling period and HPA, scale-downs may take time.

If the queue returns to an empty state, the Deployment returns to 0 replicas, and you will not incur operational costs.

**Step 7 ▾**

Clean everything and leave the cluster in its previous state by running the following commands:

**commands**

```
> kubectl delete deployment order-processor
> kubectl delete pod redis
> kubectl delete svc redis-service
> kubectl delete scaledobject keda-processor-autoscaler
```

---

**Exercise Summary**

In this exercise, you installed KEDA from OperatorHub using the Operator Lifecycle Manager (OLM) meta-operator. Later, you use a KEDA custom resource, ScaledObject, to autoscale an application depending on external events coming from a Redis queue.

(End of Exercise)

# Thank you

---